Elemente de Grafică pe Calculator

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

UPB, Automatică și Calculatoare
2021-2022

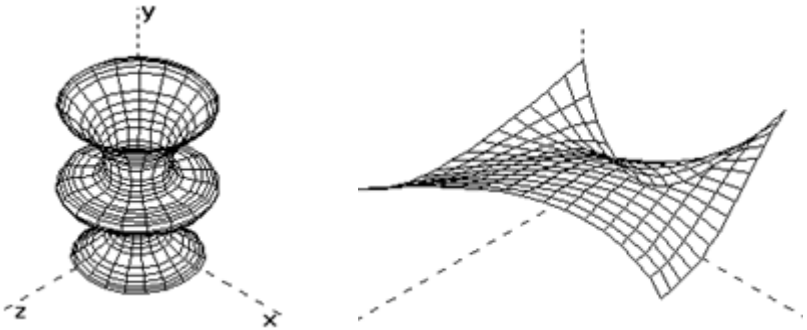
Evaluare la disciplina EGC

- **Laborator:**
  - 1.0 p – activitate în laborator
  - 5.0 p – teme (3 teme) prezentate în laborator
- **Examen**:
  - 4 puncte + 0.5 puncte suplimentar
- **Se acordă bonusuri pentru**:
  - Teme cu realizare deosebită (peste cerinţe) și activitate în laborator:
    - max 0.75 puncte (teme) + 0.25 puncte (laborator)
  - ~~Prezenţa la curs: max 1 punct~~
  - Prezentare susținută la cursurile aplicative:
    - Lista de subiecte disponibilă pe site-ul cursului
    - 5 date de prezentare
    - Maxim 15-20 de minute
    - Echipe de maxim 3 studenți
    - Maxim 1 punct bonus + 0.5 puncte suplimentar
- **Conditii de promovare:**
  - **Minim 50% punctaj pentru activitatea pe parcurs**
  - **Minim 50% punctaj în examen**

Sisteme Grafice: clasificare

- **De sinteză a imaginilor**: crearea de imagini ale unor obiecte / scene reale sau virtuale, statice sau dinamice (animate); domeniul de studiu corespunzator: **Computer Graphics (CG).**
  - Aplicații în: industria de jocuri și film, crearea de spații virtuale, arhitectură, design industrial și artistic, etc.
- **De prelucrare și analiza a imaginilor / Sisteme cu viziune**: extragerea de informații din imagini digitale produse de dispozitive de achiziție (camere video, scanner); domeniul de studiu: **Computer Vision (CV)**.
  - Aplicații în: analiza imaginilor medicale, recunoașterea obiectelor din mediul înconjurător (în robotică, conducerea automată a autovehiculelor, etc), a feţelor, a caracterelor de text, recunoașterea obiectelor din imagini satelitare, etc.
- **De reconstrucție și vizualizare a unor obiecte / fenomene reale pornind de la date volumetrice;** domeniul de studiu: **Volume Graphics (combină algoritmi de CG si CV)**.
  - Aplicații în: medicină (vizualizarea 3D a organelor interne pornind de la imagini tomografice), vizualizarea 3D a dinamicii moleculare, a datelor meteo, etc.

Computer graphics (1)

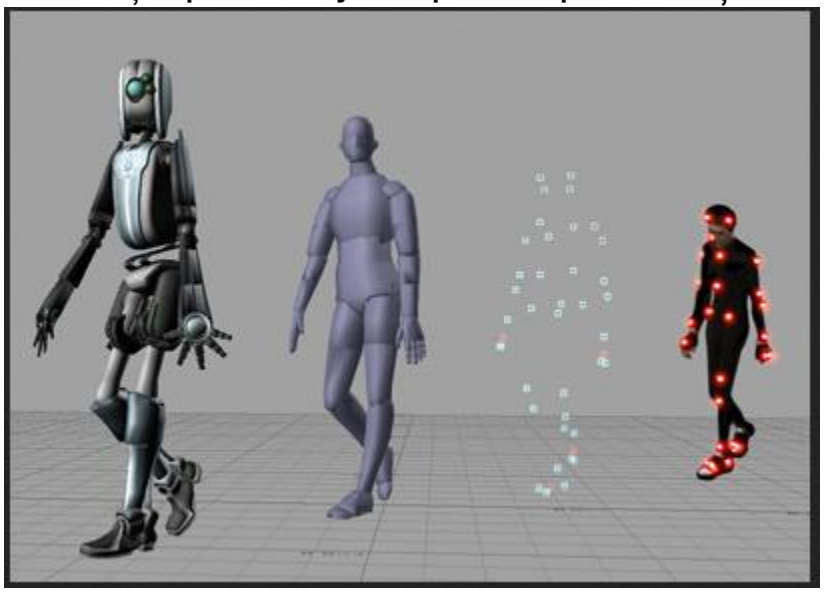


Modelarea suprafețelor 3D



Simularea detaliilor suprafețelor (texturi)

Animația personajelor prin captura mișcării

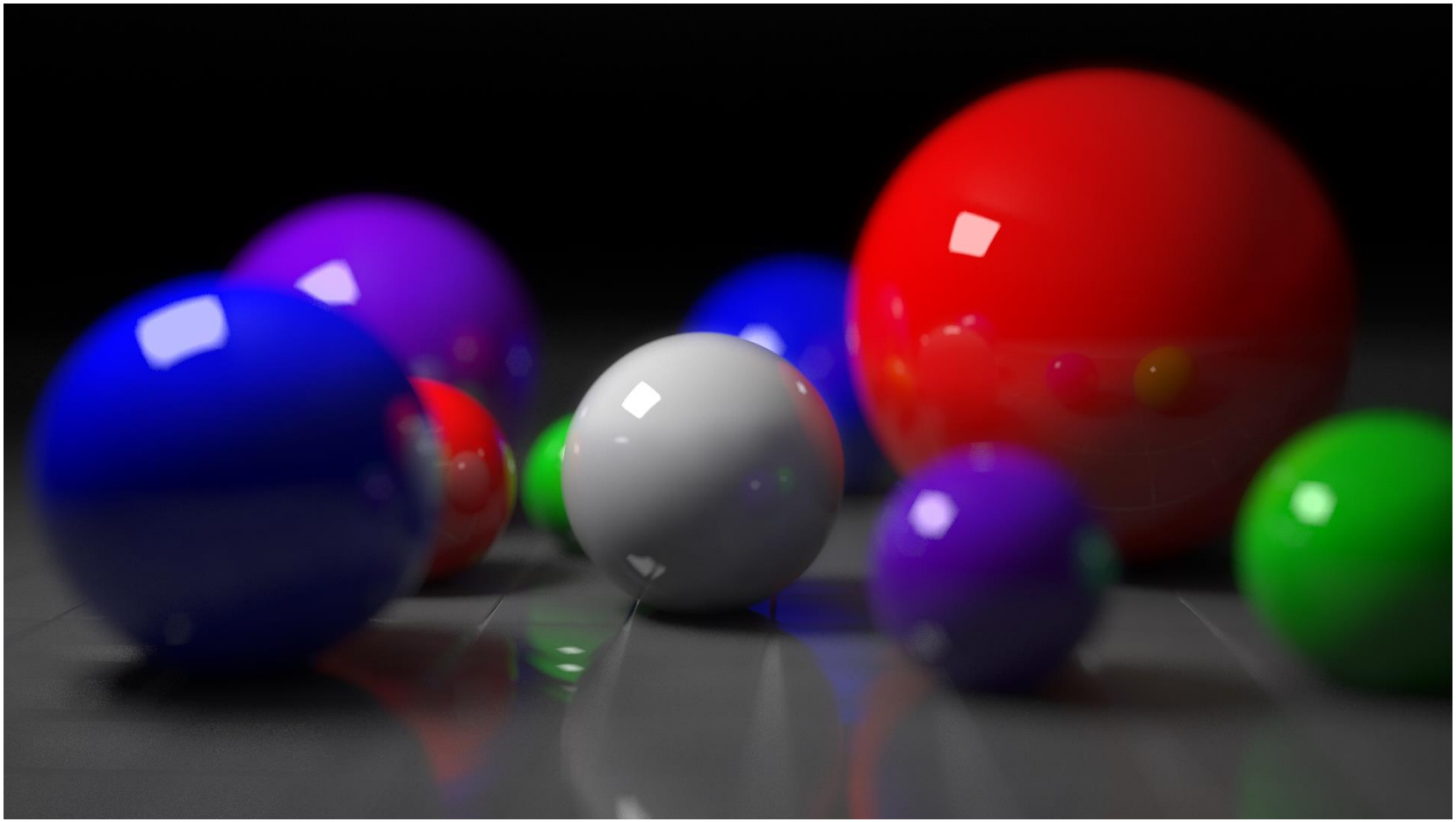


Simularea reflexiei și a refracției luminii



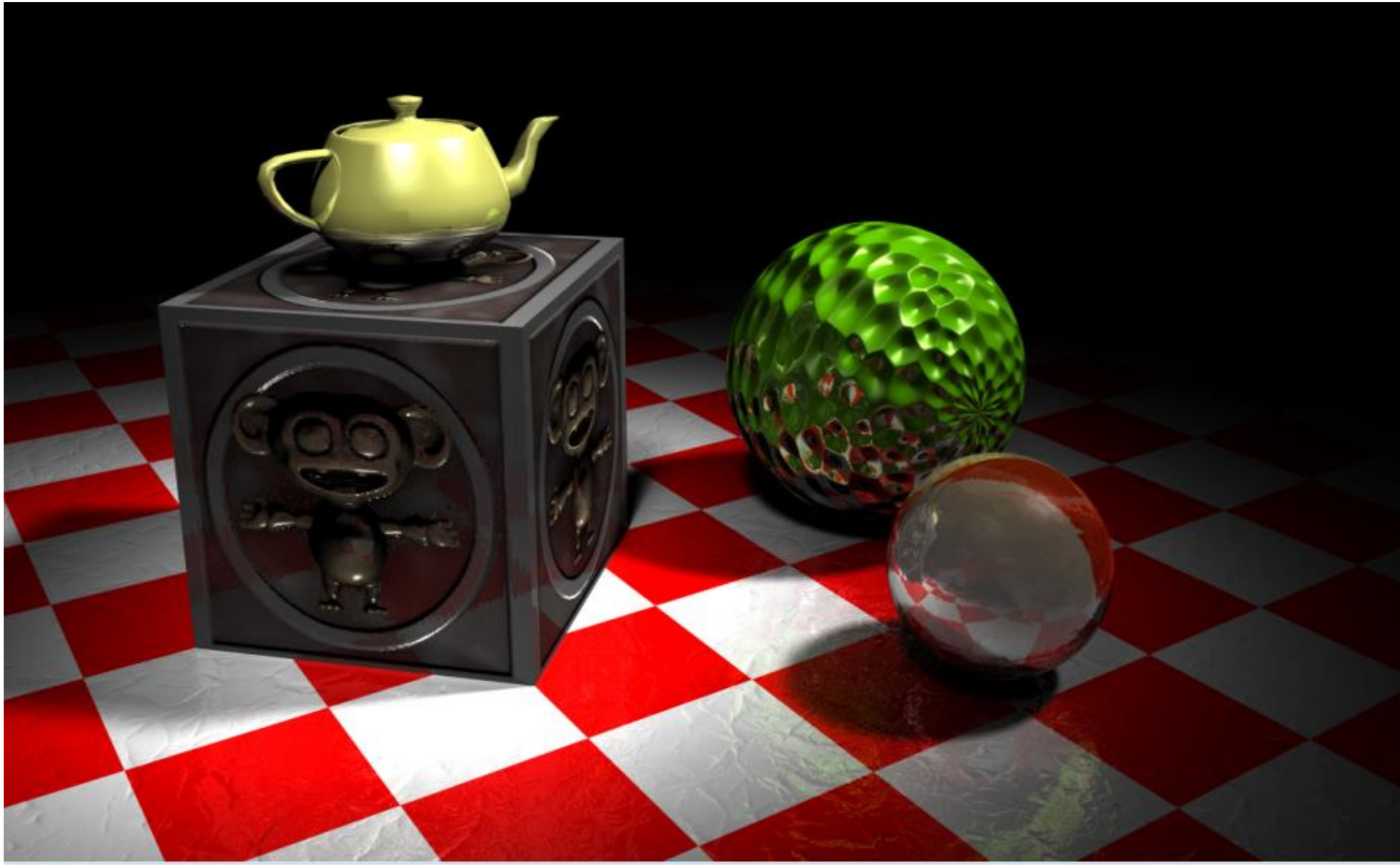
Computer graphics (2)

Imagine sintetizată prin algoritmul Ray-tracing



Computer graphics (3)

Imagine sintetizată prin Ray-tracing



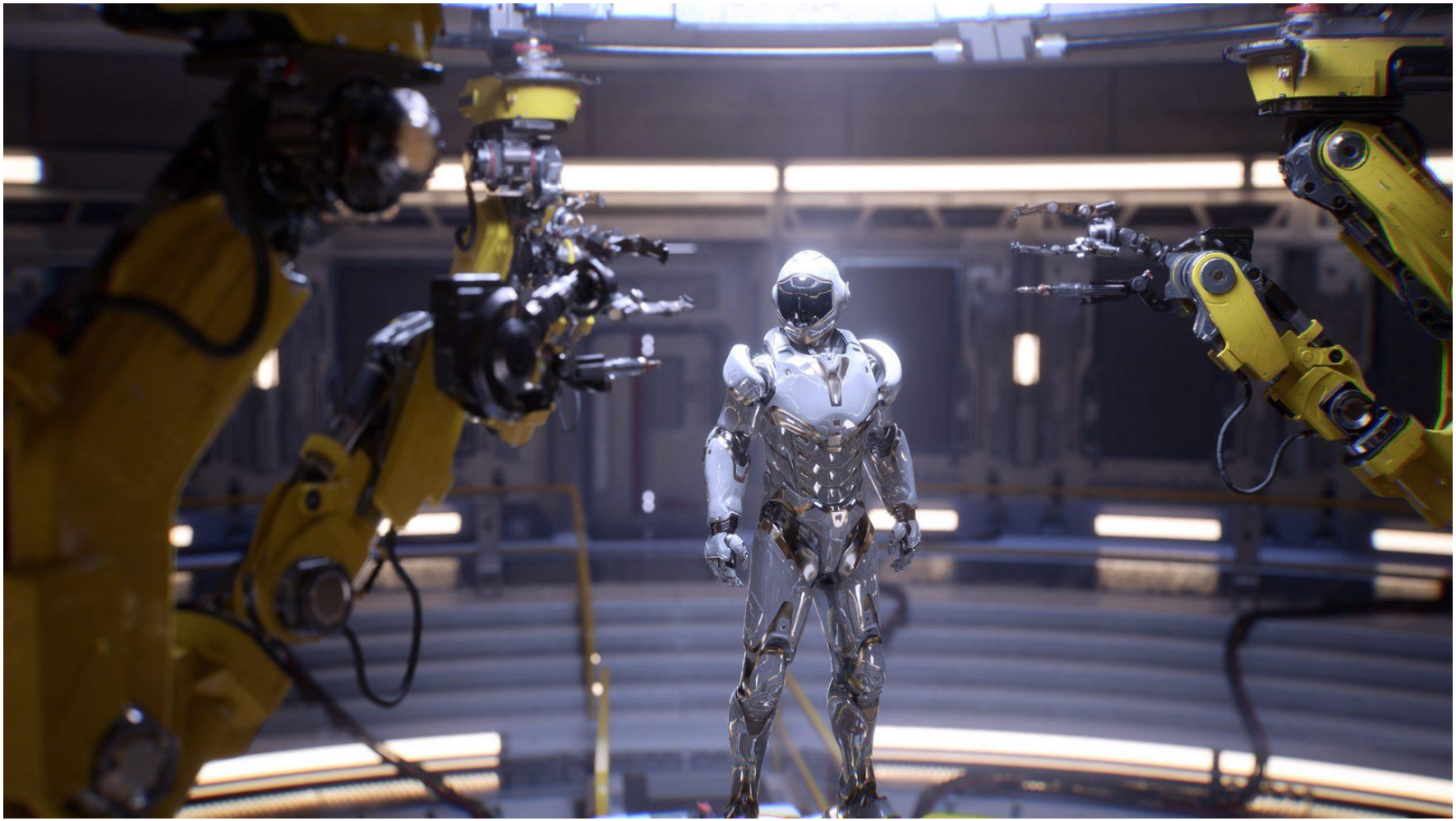
Computer graphics (4)

Imagine sintetizată prin algoritmul Ray-tracing



Computer graphics (5)

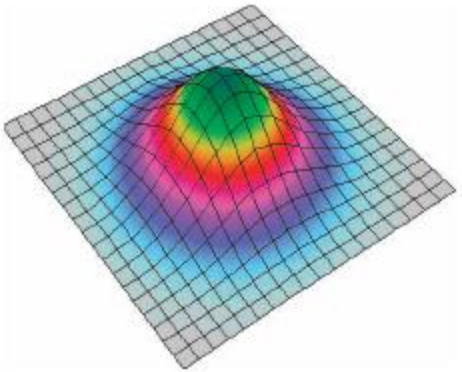
Imagine sintetizată prin algoritmul Ray-tracing



Prelucrarea și analiza imaginilor (1)



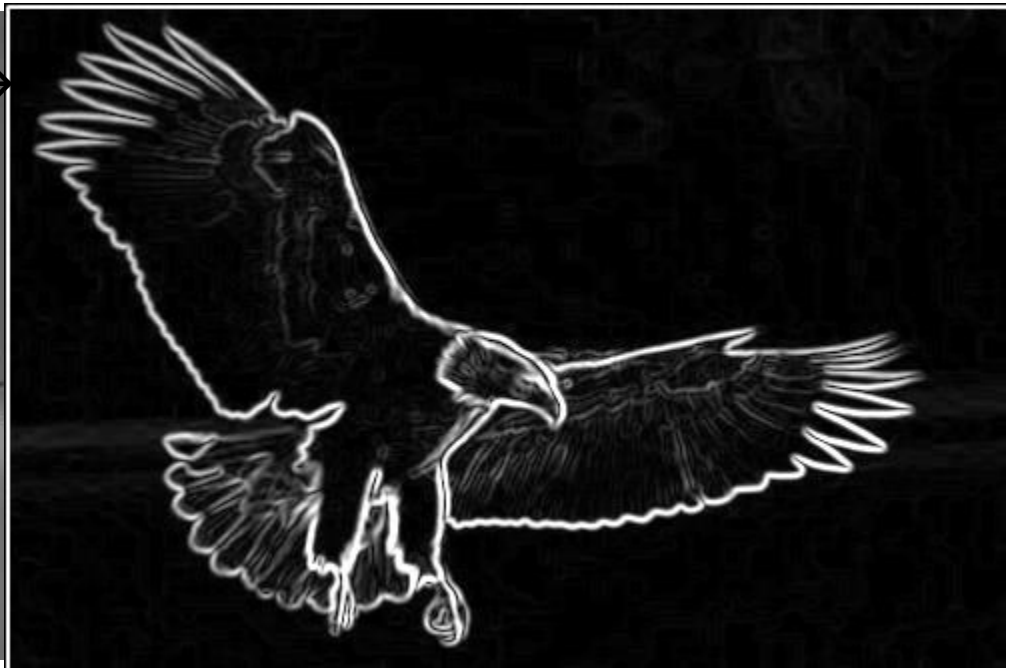




Eliminarea zgomotelor

Extragerea frontierelor →





Prelucrarea și analiza imaginilor (2)

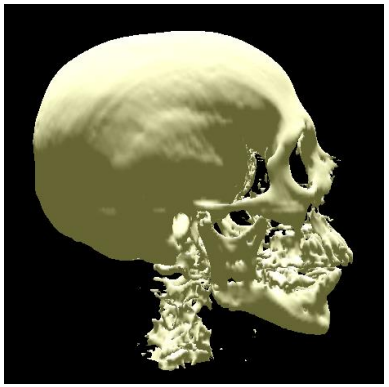




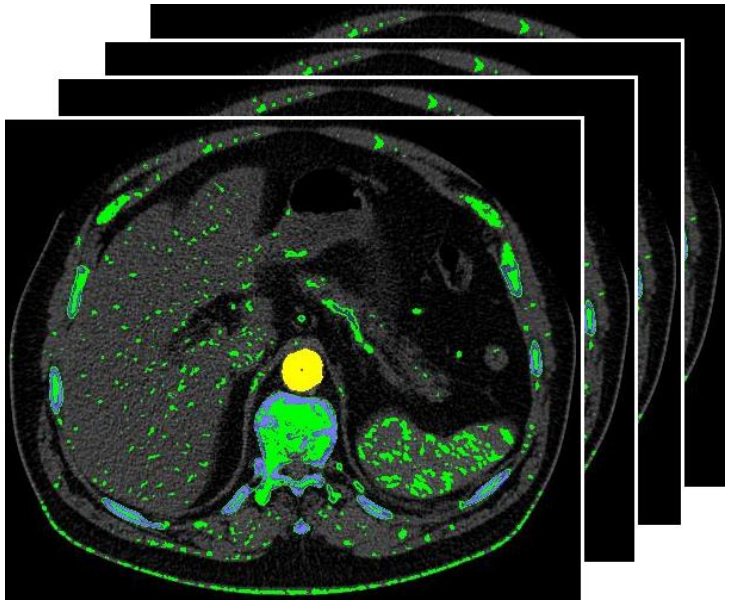
Analiza imaginilor medicale
(extragerea formei osului femural și efectuarea de măsurători automate)

Reconstrucția și vizualizarea volumelor

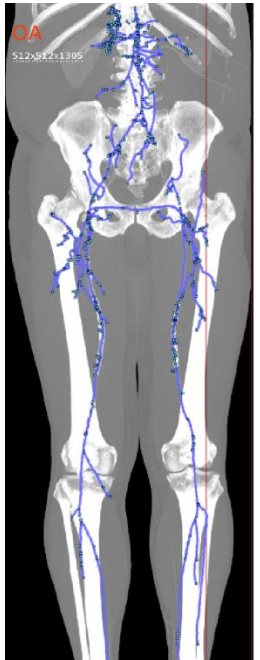




Reconstrucția / vizualizarea volumelor pornind de la imagini 3D



Vizualizarea vaselor de sânge



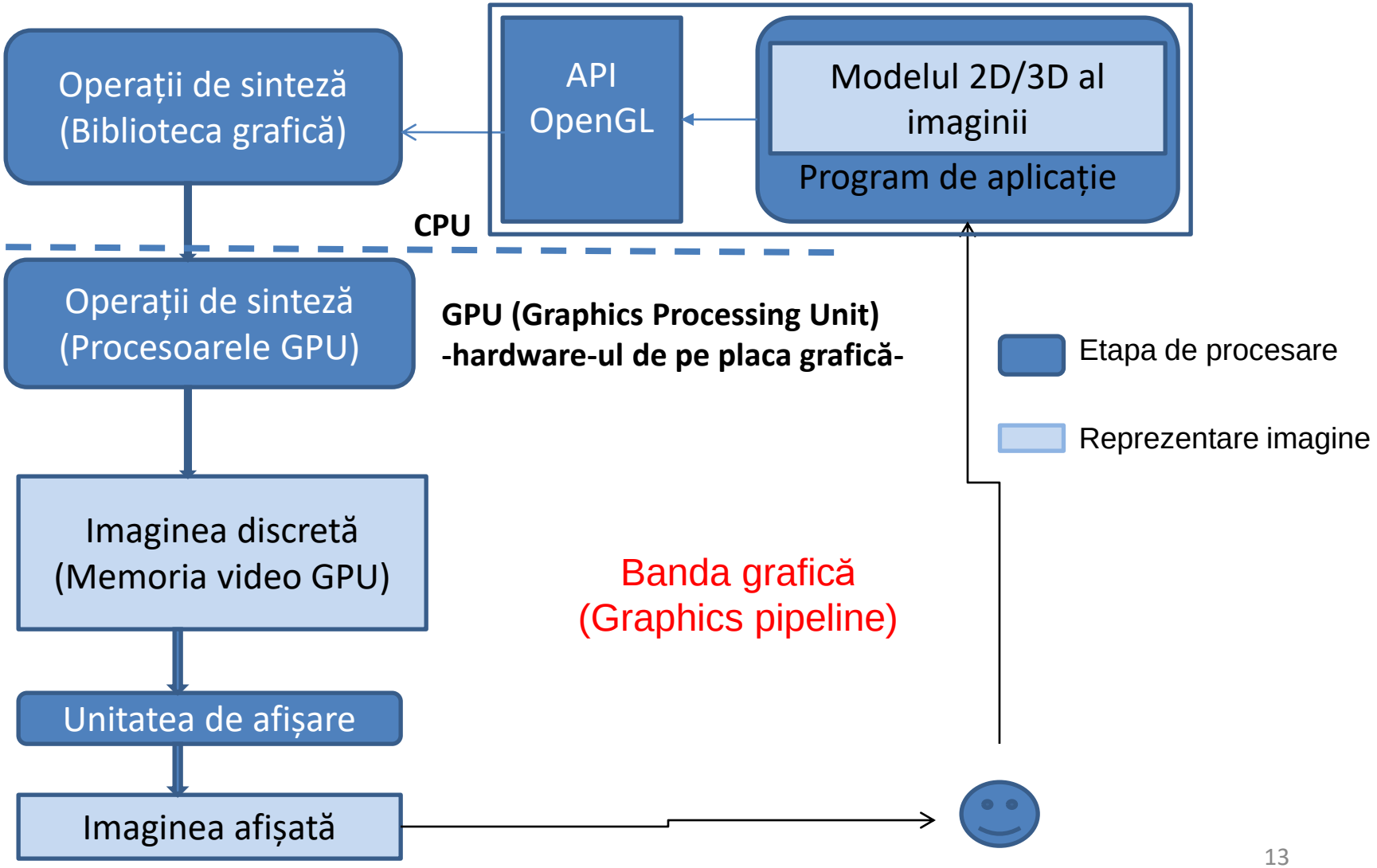
Sisteme grafice de sinteză - aplicaţii

Cursul Elemente de Grafică pe Calculator: curs de **Computer Graphics.**

Obiectivul: însuşirea de către studenţi a noţiunilor fundamentale de sinteză (creare) a imaginilor digitale, formarea deprinderilor studenţilor de a crea aplicaţii bazate pe sinteza imaginilor grafice.

- **Aplicații ale sistemelor grafice de sinteză**:
  - Proiectarea asistată de calculator: arhitectură, construcții, design industrial, confecţii, etc.
  - **Jocuri pe calculator şi pe dispozitive mobile**
  - Cartografie
  - Producția de filme
  - Spații virtuale interactive
  - Etc.

Sisteme grafice de sinteză-model conceptual

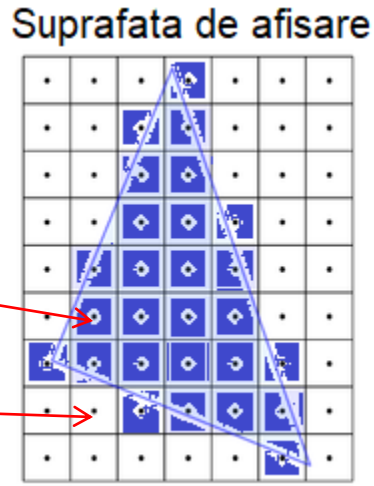


Modelul 2D/3D al imaginii

- Inclus în programul de aplicație
- Alcătuit din:
  - Primitive grafice 2D, 3D (linii, poligoane) sau obiecte complexe – alcătuiesc **scena 2D/3D redată într-un cadru imagine**
  - Proprietăți ale suprafetelor obiectelor din scena (culoare, textură, proprietăți de reflexie/refracție a luminii)
  - Transformări de aplicat obiectelor în scopul compunerii, vizualizării scenei și animării obiectelor scenei
  - Surse de lumină (poziție, caracteristici de culoare/luminozitate) – într-o scena 3D
  - Poziția observatorului virtual (a camerei virtuale) în scena 3D si direcția privirii
- Este modificat de aplicație conform interacțiunii cu utilizatorul aplicației
- Orice modificare trebuie să aibă ca efect modificarea în timp real a imaginii afișate

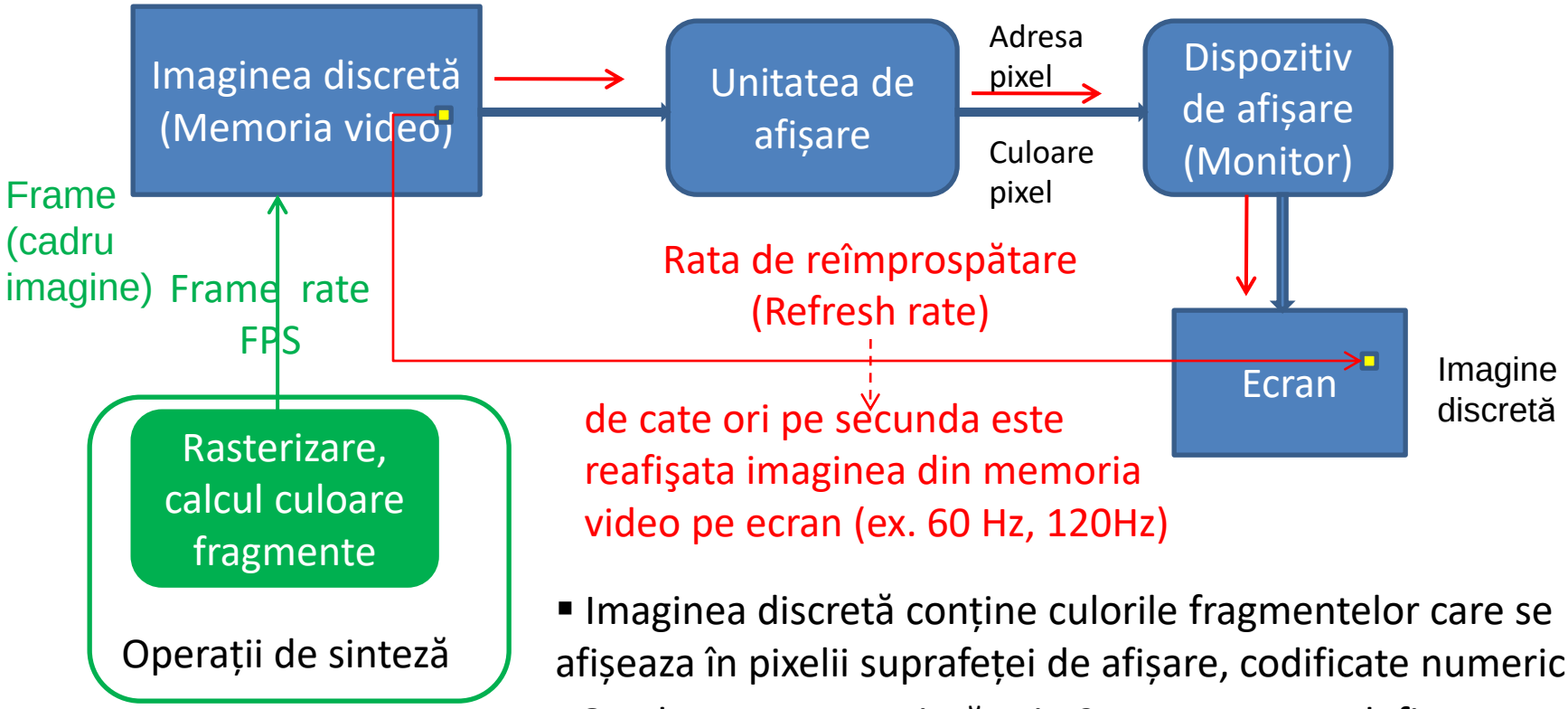
Operații de sinteză a imaginilor grafice

- Efectuarea transformărilor care se aplică obiectelor pentru modelarea scenei
- Efectuarea transformarilor de proiectie din spatiul 3D in spatiul 2D
- Eliminarea obiectelor nevizibile din imaginea unei scene 3D
- Eliminarea fețelor nevizibile (auto-obturate) ale obiectelor
- Decuparea primitivelor la marginile volumului vizual (field of view)
- Rasterizarea: descompunerea primitivelor grafice în fragmente care se afișează în pixeli



- Calculul culorii fiecarui fragment (în fragment shader) folosind: modele de reflexie și refracție a luminii, calcul de umbre, texturi, ș.a.
- Operaţii la nivel de pixel (operaţii raster): testul de vizibilitate a fragmentelor în imagine (testul de adâncime), combinarea culorii fragmentului cu cea a pixelului in care se afișează, ş. a.

Imaginea discretă şi afişarea sa (schema conceptuală)



- Imaginea discretă conține culorile fragmentelor care se afișeaza în pixelii suprafeței de afișare, codificate numeric.
- O culoare se reprezintă prin 3 numere, care definesc o combinație a celor 3 culori primare Red,Green,Blue (3x 8 biti).

FPS: Frames Per Second
Real time: 30 FPS

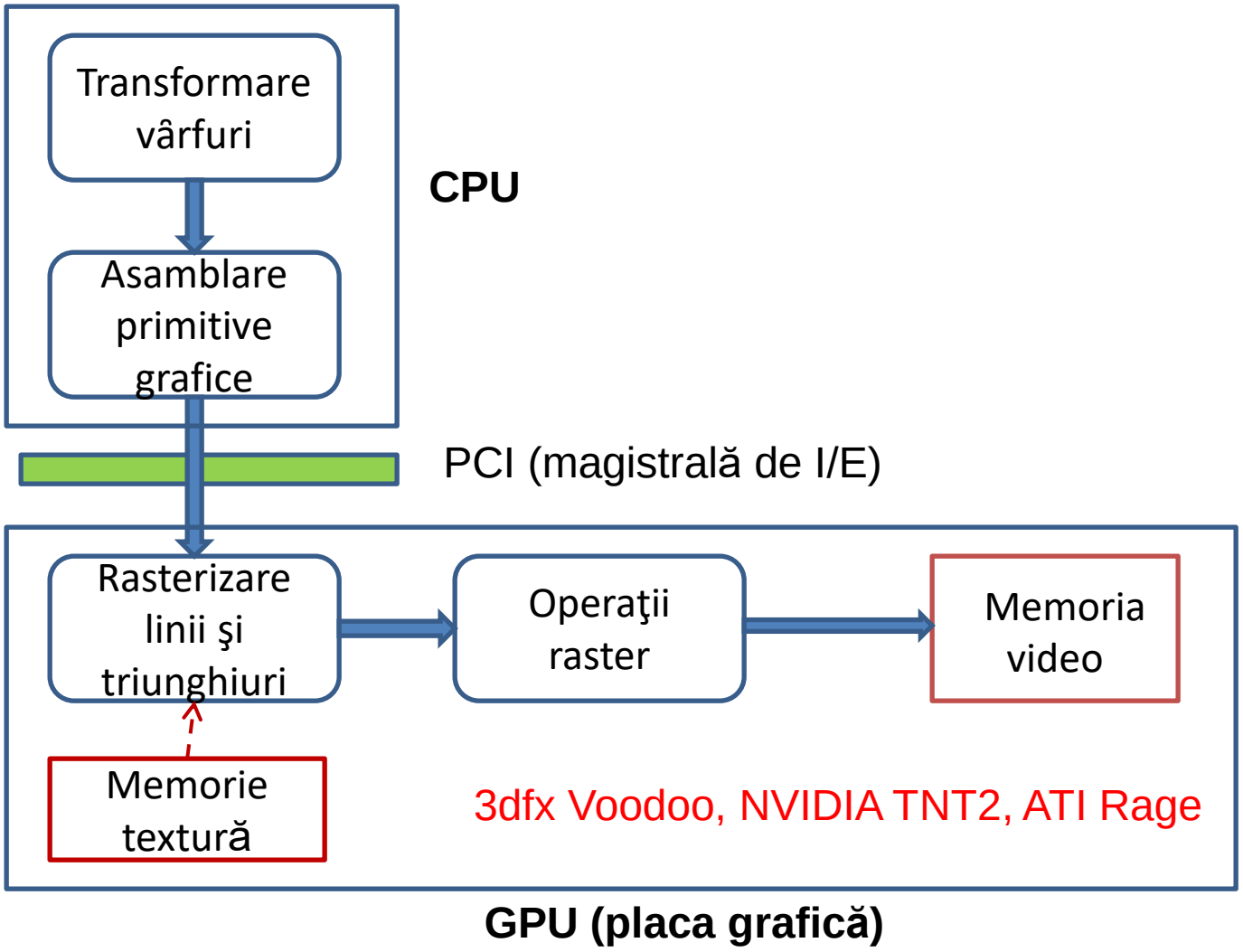
Scurt istoric al hardware-ului grafic de sinteză
Evoluţia benzii grafice

- **Mijlocul anilor ‘90**: cel mai complex hardw. grafic realiza numai rasterizarea primitivelor
- **Sfârșitul anilor ‘90**: NVIDIA a introdus termenul GPU, înlocuind termenul VGA – Video Graphics Adapter (introdus de IBM în 1987), devenit neadecvat pentru hardware-ul grafic dintr-un calculator
- **Hardware de accelerare grafică: hardware specializat pentru sinteza imaginilor, dar scump (ex. stațiile grafice SiliconGraphics** –SGI, Evans& Sutherland)

❖ **Prima generație GPU: până în 1998**

- 3dfx Voodoo, NVIDIA TNT2, ATI Rage
- Rasterizare linii și triunghiuri, cu vărfurile transformate pe CPU
- Aplicare 1-2 texturi

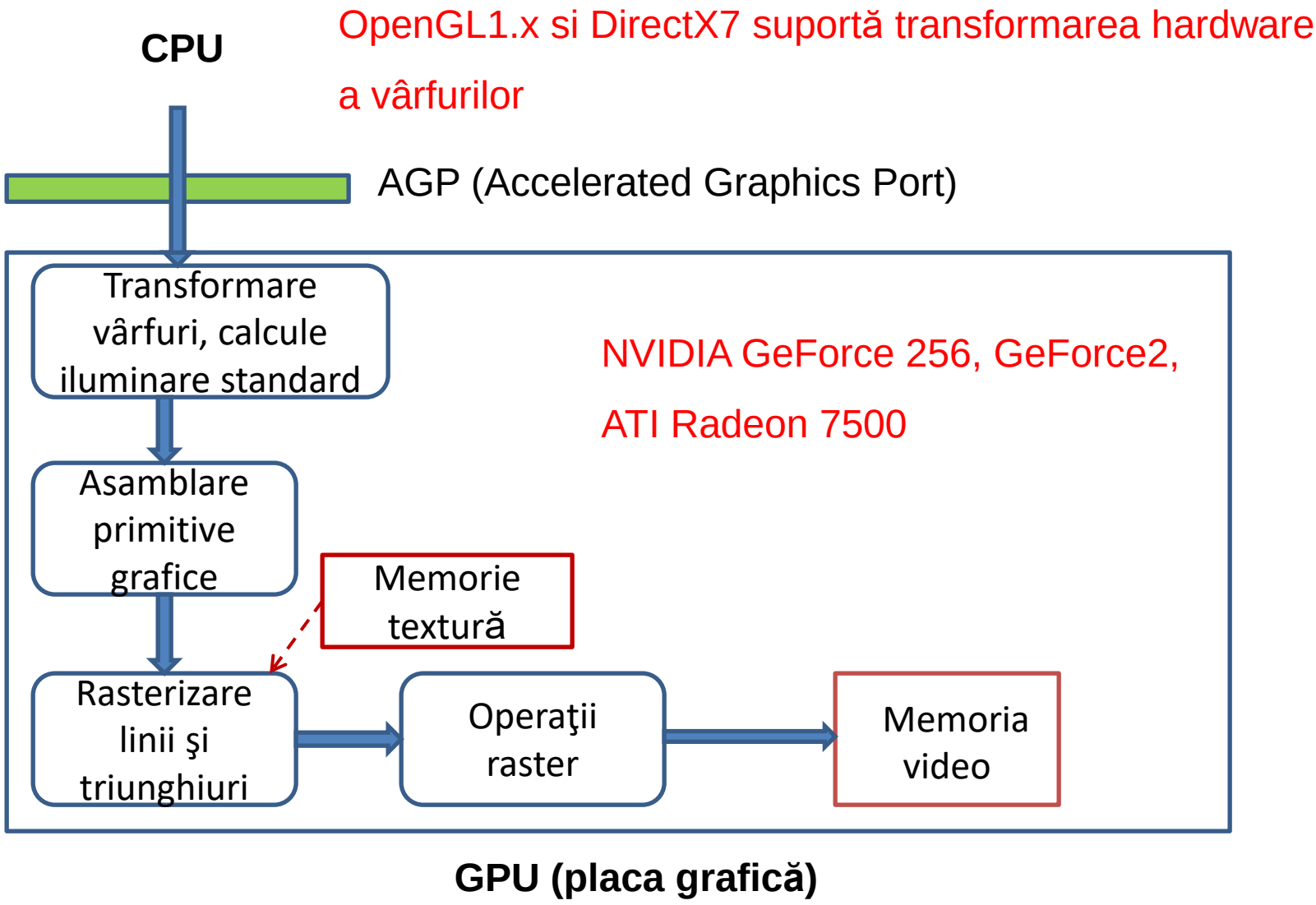
Banda grafică - prima generaţie GPU (până în 1998)



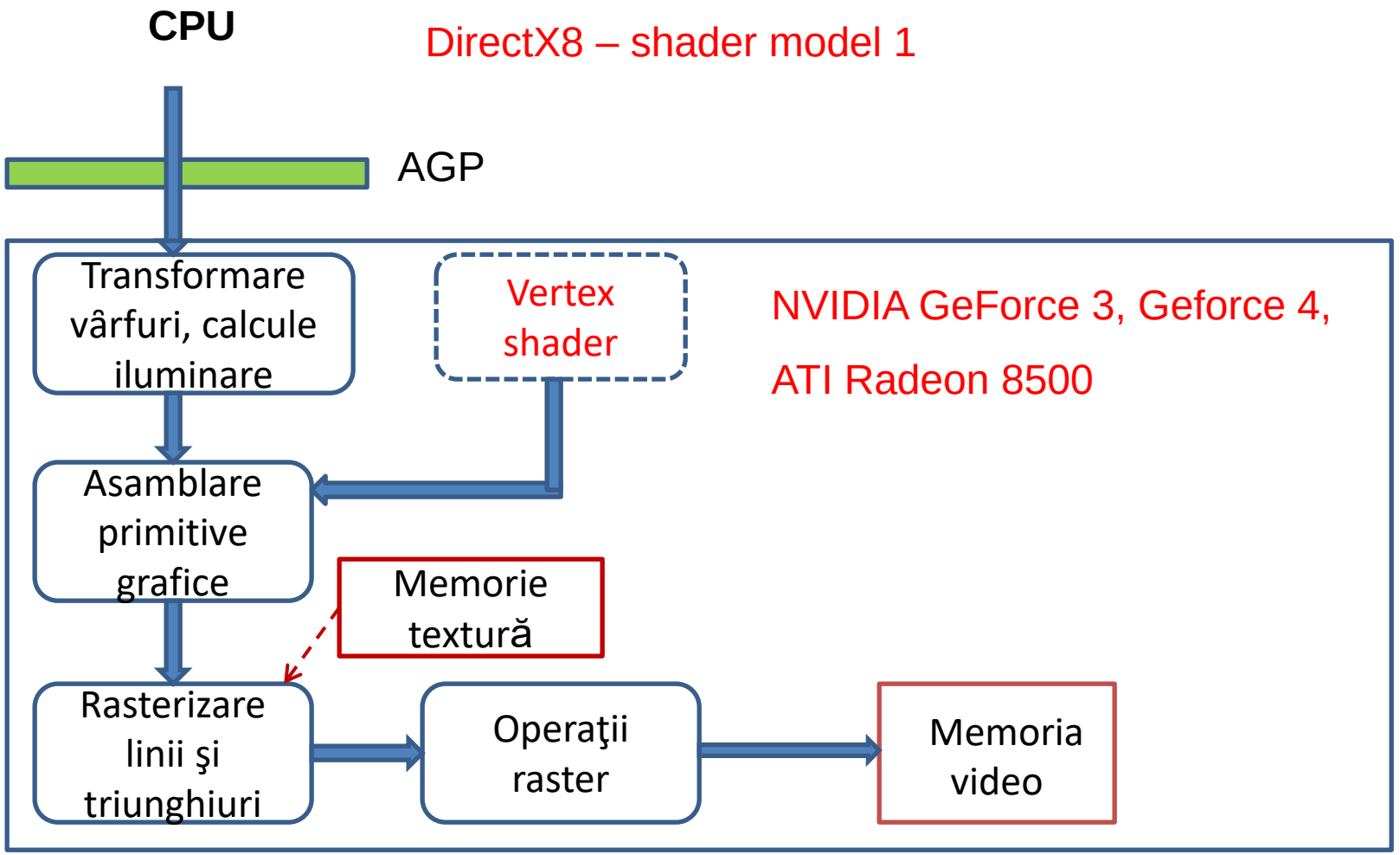
Scurt istoric al hardware-ului grafic de sinteză (2)

- **A 2-a generație GPU (1999-2000):**
  - NVIDIA GeForce 256, Geforce2, ATI Radeon 7500
  - Transformare varfuri 3D și calcule de iluminare standard la nivel de vârfuri
  - OpenGL1.x și DirectX7 suportă transformarea hardware a vărfurilor
- **A 3-a generație GPU (2001-2002)**
  - ~~3dfx Vodoo~~ (cumpărat de Nvidia)
  - NVIDIA GeForce 3, GeForce 4, ATI Radeon 8500
  - Transformare vârfuri 3D și calcule de iluminare standard la nivel de vârfuri
  - Posibilitatea programării calculelor de iluminare la nivel de vârfuri → VERTEX SHADER (program scris de programator și transferat la GPU; executat în paralel de un număr mare de procesoare GPU pentru vârfuri diferite)
  - DirectX8 – “shader model” 1

Banda grafică - a II-a generaţie GPU (1999-2000)



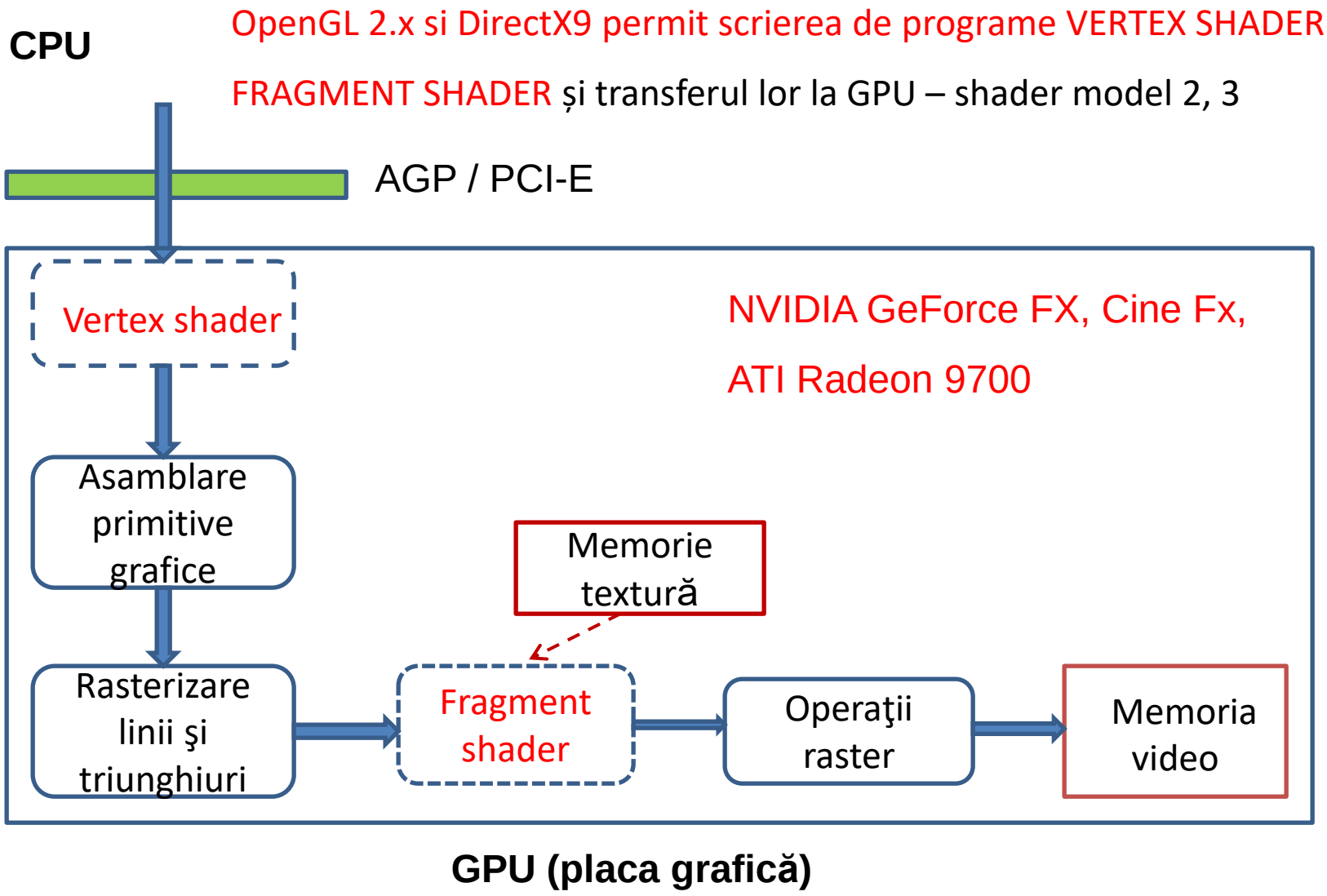
Banda grafică- a III-a generaţie GPU (2001-2002)



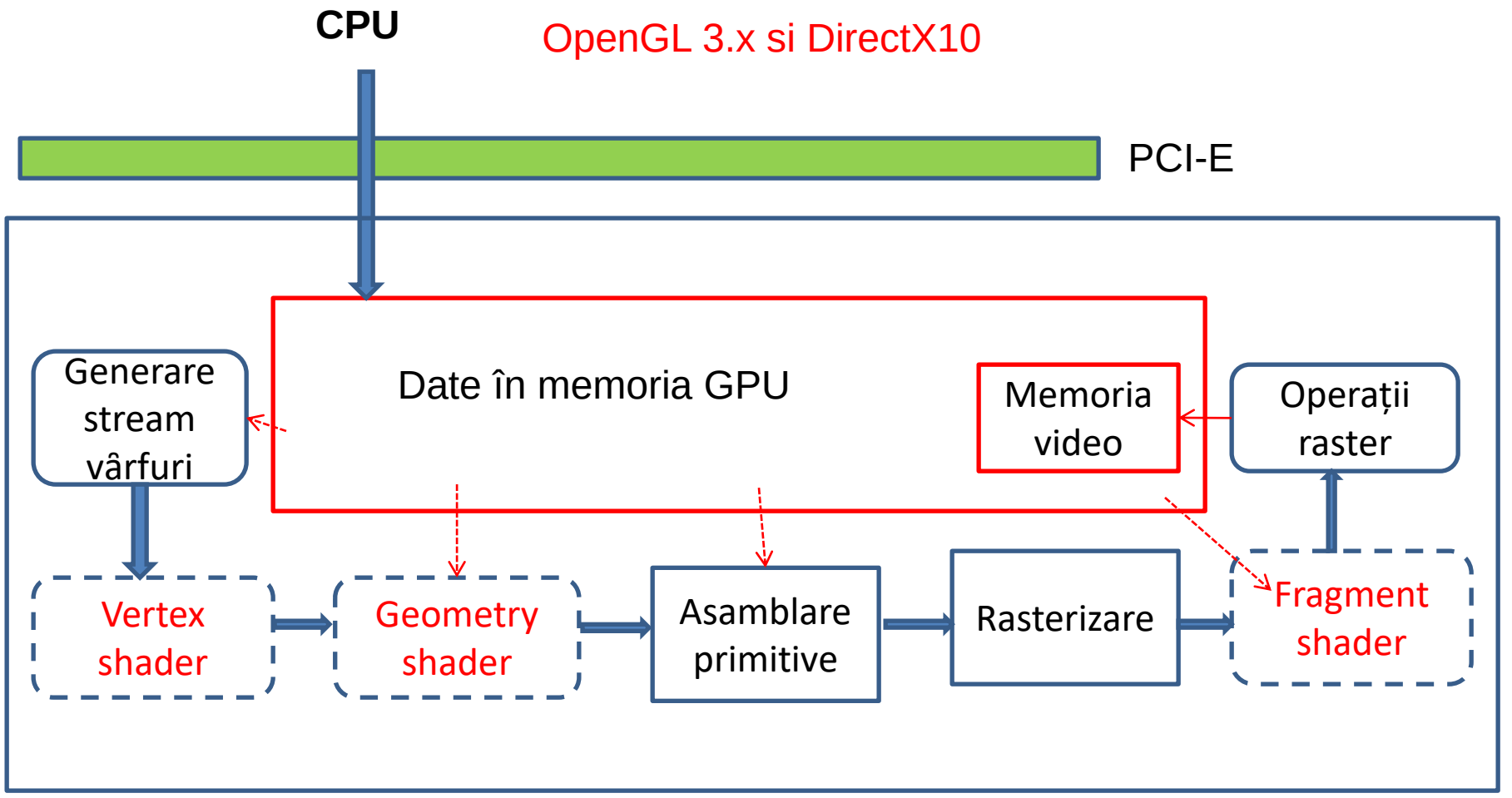
Scurt istoric al hardware-ului grafic de sinteza(3)

- **A 4-a generație GPU (2003 - 2005):**
  - NVIDIA GeForce FX, ATI Radeon 9700
  - Procesoare programabile la nivel de vârfuri și de fragment (pixel) → calculele la nivel de vârfuri și de fragmente sunt etape programabile.
  - Procesoare specializate pentru calcule cu vectori și matrici
  - Calcul paralel la nivel de vârfuri și fragmente de primitive
  - OpenGL 2.x si DirectX9 permit scrierea de programe VERTEX SHADER si FRAGMENT SHADER (denumite PIXEL SHADER în DX) și transferul lor la GPU – shader model 2, 3
- **A 5-a generație GPU (2006 -> 2010):**
  - NVIDIA GeForce G80, Fermi, Kepler, ATI – AMD Radeon începând cu seria 4000
  - Adaugă etapa programabilă GEOMETRY SHADER – în plus față de generația a 4-a
  - OpenGL 3.x si DirectX10 permit scrierea de programe GS → shader model 4

Banda grafică- a IV-a generaţie GPU (2003-2005)



Banda grafică - a V-a generaţie GPU (2006-2010)



GPU (placa grafică)

Scurt istoric al hardware-ului grafic de sinteză (4)

❖**A 6-a generație GPU (2011 ->):**

- ➢Procesoare programabile pentru teselare (marirea rezoluției geometrice) – TESSELLATION SHADER
- ➢OpenGL4.x, Direct3D 11, Direct3D 12 – Shader model 5,6
- ➢Procesoare programabile pentru calcule de uz general – COMPUTE SHADER (OpenGL 4.3, Direct3D 11)

❖**Performanța placii grafice (arhitectură multiprocessor, grad înalt de paralelism, procesoare dedicate, stream processing) → GPGPU (General Purpose Graphics Processing Unit):** programare de aplicații ne-grafice pentru executie pe placa grafică

➢ Limbaje de programare paralelă pentru execuție pe placa grafică: CUDA, OpenCL (disponibile începând cu generația a 5-a)

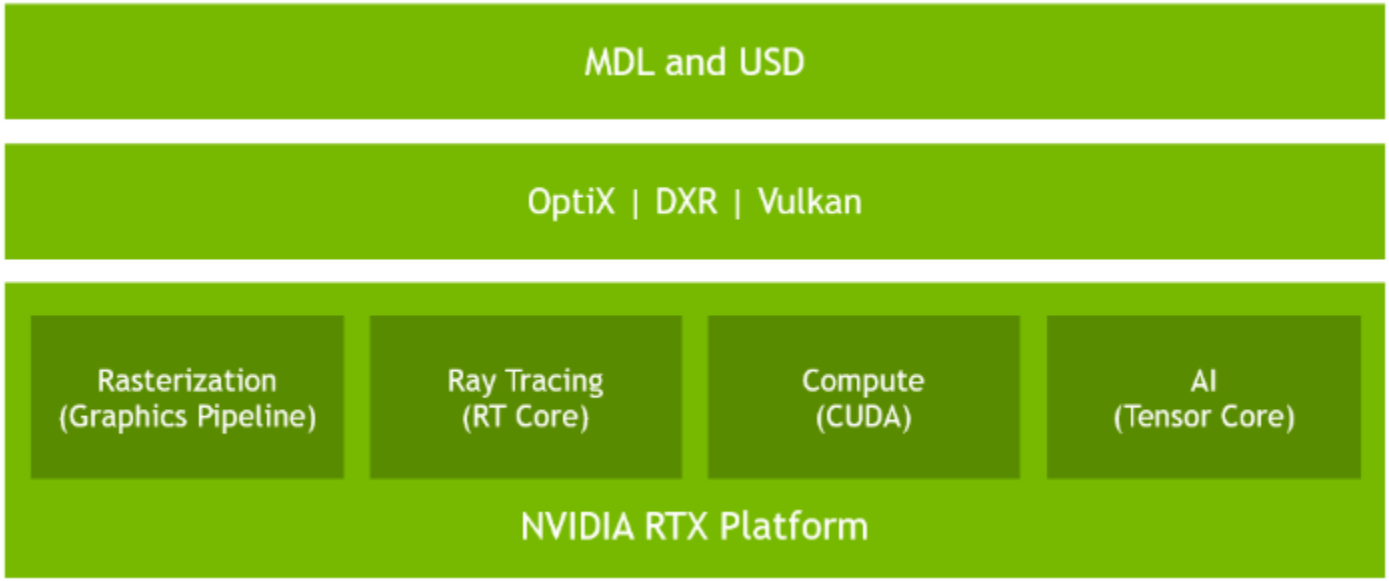
➢ Compute Shader (program scris în GLSL – limbajul pentru programare shader-e OpenGL, program scris în HLSL - limbajul pentru programare shader-e Direct3D).

Scurt istoric al hardware-ului grafic de sinteză (5)

NVIDIA RTX – cea mai recentă și mai avansată arhitectură GPU

- **Arhitecturile Pascal (parțial seria 10xx), Turing (seriile 16xx și 20xx), Ampere (seria 30xx)**

Platforma NVIDIA RTX permite implementarea algoritmului Ray-tracing folosind API-uri și kituri software de dezvoltare dedicate: OptiX, Microsoft DXR și Vulkan.



Transformări Geometrice 2D

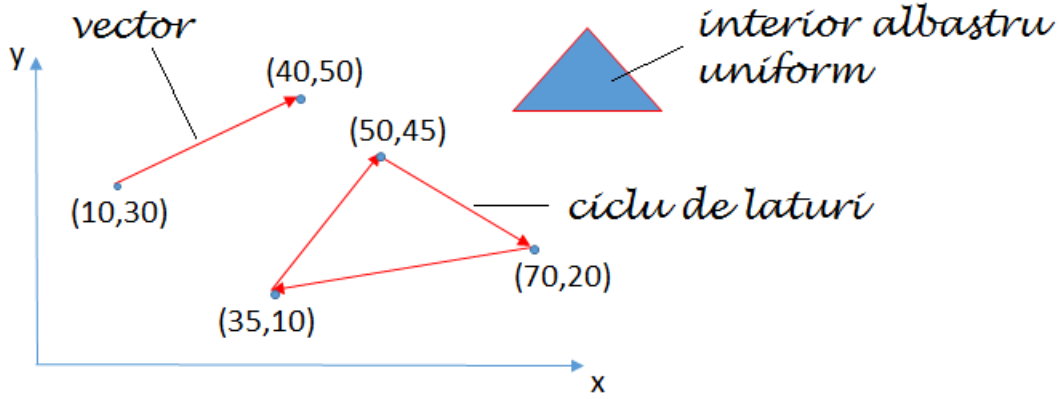
Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

Transformări geometrice (1)

Obiectele 2D/3D sunt reprezentate prin:

- Date geometrice: coordonatele vârfurilor, raportate la un sistem de coordonate carteziene 2D sau 3D;
- Atribute topologice, care definesc modul în care sunt conectate vârfurile (laturi, ciclul de laturi al unei fețe, ş.a.);
- Atribute de aspect: culoarea, tipul de interior pentru suprafețe 2D, atribute de material pentru suprafețe 3D (ex: reflexia/refracția luminii de către suprafață), ș.a.



Transformările geometrice se aplică (coordonatelor) vârfurilor obiectului și nu afectează atributele sale!

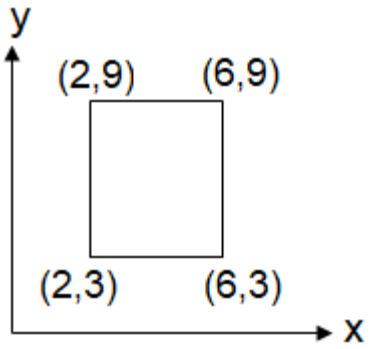
Transformări geometrice (2)

- Sunt operații fundamentale în sinteza imaginilor
- Folosite pentru:
  - Redarea desenelor la diferite mărimi
  - Compunerea desenelor sau a scenelor 3D
  - Realizarea animației
  - Transformarea obiectelor dintr-un spațiu de coordonate logic, în care sunt definite, în spațiul fizic de afișare
  - Etc.

Transformări geometrice 2D (1)

Sunt transformări ale obiectelor definite în sistemul de coordonate carteziene XOY, numite şi obiecte 2D. Fiecare vârf al unui obiect 2D este definit printr-o pereche de coordonate (x,y).

Transformări geometrice elementare



- Translaţia
- Scalarea faţă de origine
- Rotaţia faţă de origine
- Forfecarea faţă de origine
- Oglindirile faţă de axele principale şi faţă de origine

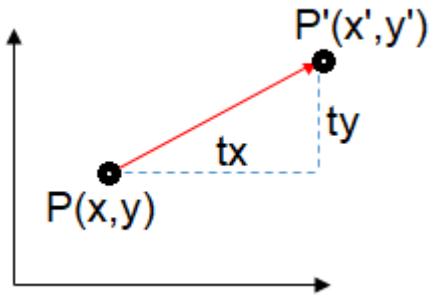
Orice alta transformare se obține prin compunerea a 2 sau mai multe transformări elementare

- Scalarea / rotaţia fata de un punct oarecare din plan
- Scalare compusa cu translație, etc

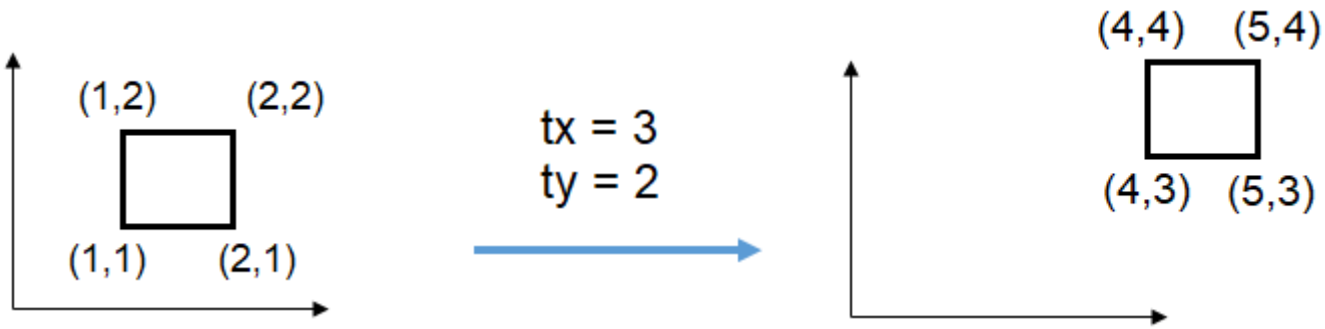
Transformări geometrice 2D elementare (1)

- Translația (1)

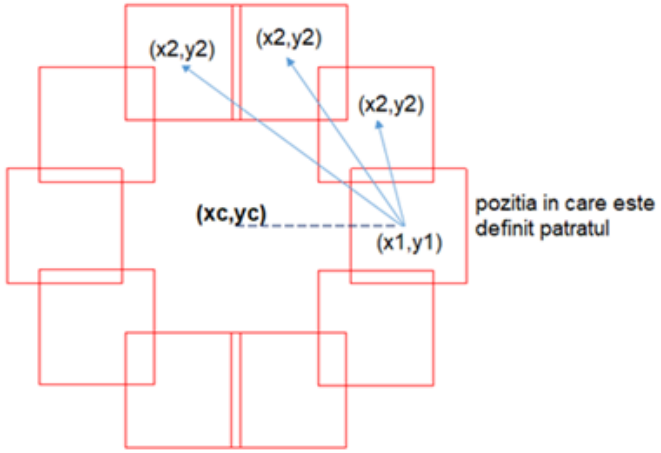
  Este definita printr-un vector, T[tx,ty].

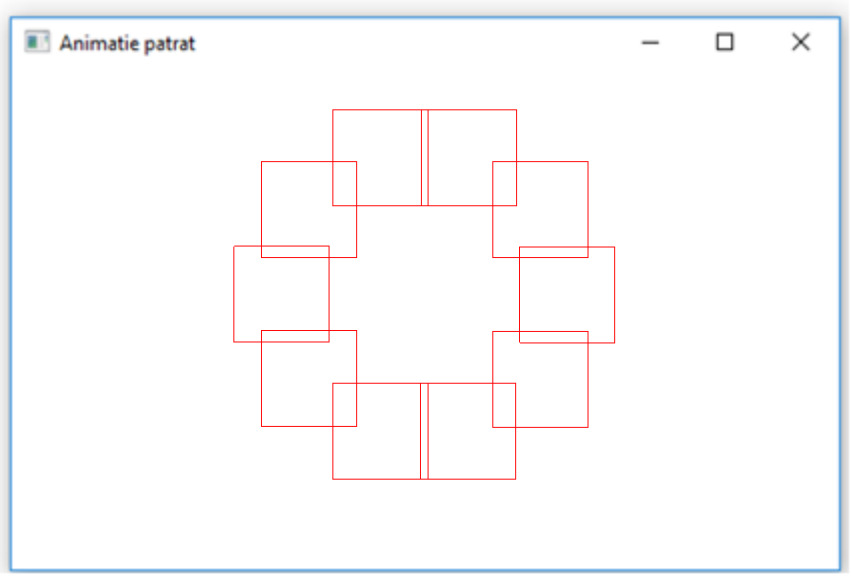


$$x' = x + tx$$
$$y' = y + ty$$



Translația unui pătrat pe circumferința unui cerc





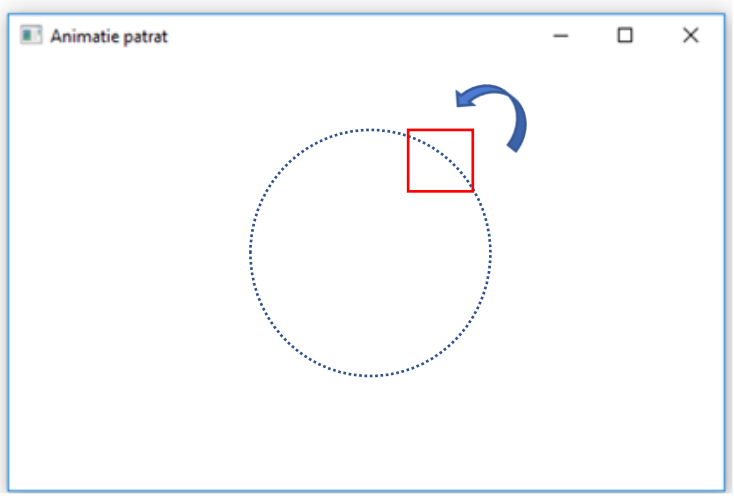

```
// campuri ale clasei DesenAnimatiePatrat
int raza, xc, yc, latura;

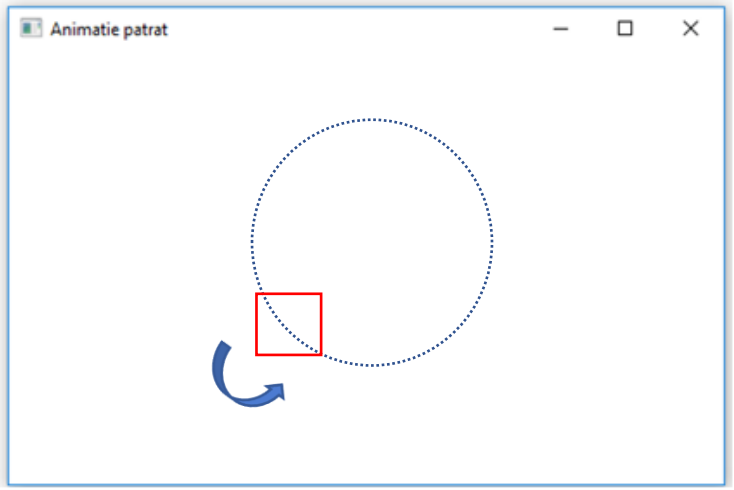
// afiseaza 10 patrate translatate pe
// circumferinta unui cerc
void C1_DesenAnimatiePatrat::UpdateScene1(float
deltaTimeSeconds)
{
   glm::ivec2 resolution = window->GetResolution();
   xc = resolution.x / 2;
   yc = resolution.y / 2;

   float x1 = xc + raza, y1 = yc, x2, y2;
   for (int i = 0; i < 10; i++)
   {
      // distanta unghiulara intre puncte: 6.28/10
      x2 = xc + raza * cos(0.628f*i);
      y2 = yc + raza * sin(0.628f*i);
      // seteaza matricea de transformare a varfurilor
      modelMatrix = Transform2D::Translate(x2 - x1, y2 - y1);

      RenderMesh2D(meshes["square"],
                  shaders["VertexColor"], modelMatrix);
   }
}
```

Animația translației pătratului






```
// campuri ale clasei DesenAnimatiePatrat
int raza, xc, yc, latura, angularStep;

// afiseaza animatie patrat translatat pe
// circumferinta unui cerc
void C1_DesenAnimatiePatrat::UpdateScene2(float deltaTimeSeconds)
{
    glm::ivec2 resolution = window->GetResolution();
    xc = resolution.x / 2;
    yc = resolution.y / 2;

    float x1 = xc + raza, y1 = yc, x2, y2;

    angularStep += 0.5 * deltaTimeSeconds;

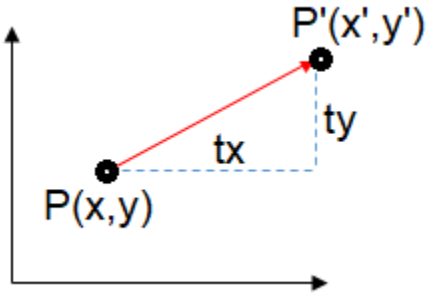
    x2 = xc + raza * cos(angularStep);
    y2 = yc + raza * sin(angularStep);

    // seteaza matricea de transformare a varfurilor
    modelMatrix = Transform2D::Translate(x2 - x1, y2 - y1);

    RenderMesh2D(meshes["square"],
                 shaders["VertexColor"], modelMatrix);
}
```

Transformări geometrice 2D elementare (2)

- Translația (2)



$$x' = x + tx$$
$$y' = y + ty$$

Se dorește o reprezentare matriceală a transformării, pentru compunerea sa cu alte transformări: compunerea transformărilor se realizează prin înmulțirea matricilor.

Un punct din plan, P(x,y), se reprezintă în coordonate carteziene printr-un vector [x, y] sau $\begin{bmatrix} x \\ y \end{bmatrix}$

→ Nu există o matrice M, de 2x2, astfel încât să se exprime translația prin:

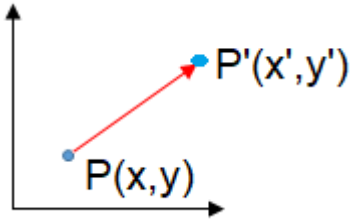
[x', y'] = [x, y] * M sau $\begin{bmatrix} x' \\ y' \end{bmatrix} = M * \begin{bmatrix} x \\ y \end{bmatrix}$

Transformări geometrice 2D elementare (3)

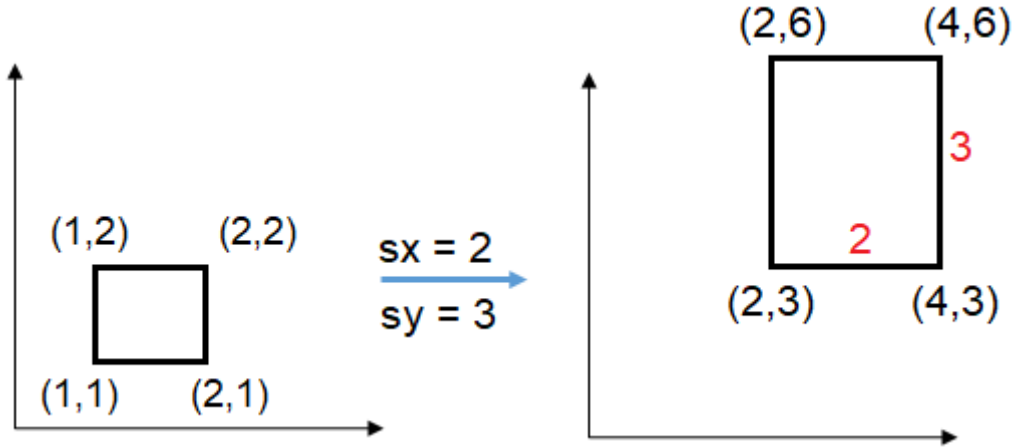
Scalarea față de origine

Este definită prin 2 numere reale, de regulă pozitive:

- sx - scalarea pe axa OX
- sy - scalarea pe axa OY



x' = x * sx
y' = y * sy



Efecte:

- Dacă sx sau sy sau ambii sunt >1:……..
- Dacă sx sau sy sau ambii sunt <1:…….
- ➢ sx = sy , scalare uniformă

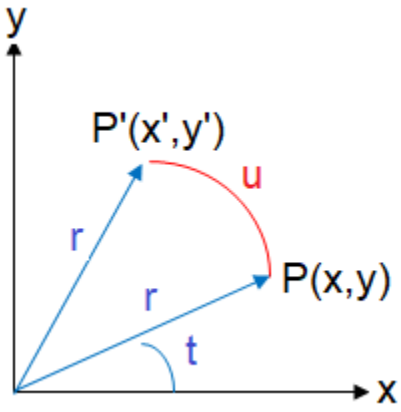
Reprezentarea matriceală

$$[x' \;\; y'] = [x \;\; y] * \begin{bmatrix} sx & 0 \\ 0 & sy \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} sx & 0 \\ 0 & sy \end{bmatrix} * \begin{bmatrix} x \\ y \end{bmatrix}$$

Transformări geometrice 2D elementare (3)

Rotaţia față de origine



x = r*cos(t)
y = r*sin(t)

Relația dintre coordonatele carteziene și coordonatele polare ale unui punct P(x,y)

x' = r*cos(t+u) = r*(cos(t)*cos(u) – sin(t)*sin(u)) = x*cos(u) – y*sin(u)
y' = r*sin(t+u) = r*(cos(t)*sin(u) + sin(t)*cos(u)) = x*sin(u) – y*cos(u)

Rotatia fata de origine

x' = x*cos(u) - y*sin(u)
y' = x*sin(u) + y*cos(u)

$$[x' \;\; y'] = [x \;\; y] * \begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(u) & -\sin(u) \\ \sin(u) & \cos(u) \end{bmatrix} * \begin{bmatrix} x \\ y \end{bmatrix}$$

Transformări geometrice 2D față de un punct oarecare din plan (1)

Scalarea fata de un punct oarecare din plan

- Punctul fix al transformării este un punct oarecare F(xf,yf) – coordonatele sale nu se modifică prin aplicarea transformării.
- P(x,y) este punctul transformat
- Scalarea se aplică vectorului FP [x-xf, y-yf]:

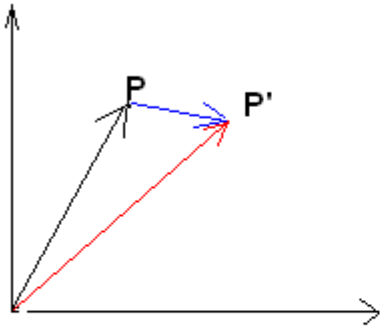
x’ – xf = sx*(x – xf)

y’ – yf = sy*(y – yf)

Rezultă:

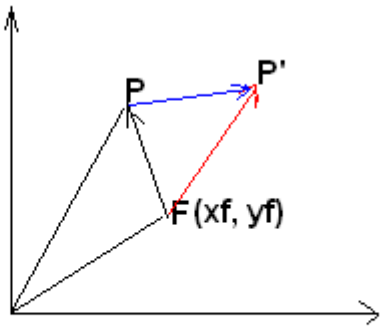
x’ = x*sx + xf –xf*sx

y’ = y*sy + yf –yf*sy



Scalarea fata de origine

x' - 0 = sx (x - 0)
y' - 0 = sy (y - 0)



Scalarea fata de un punct F din plan

x' - xf = sx (x - xf)
y' - yf = sy (y - yf)

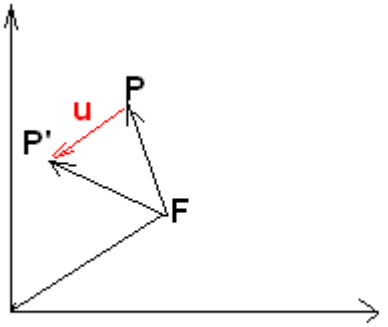
Nu poate fi exprimata în coordonate carteziene printr-o matrice de 2x2!

Transformări geometrice 2D față de un punct oarecare din plan(2)

Rotația față de un punct oarecare din plan

- Punctul fix al transformării este F(xf,yf)
- P(x,y) este punctul transformat
- Rotația se aplică vectorului FP [x-xf, y-yf], în jurul lui F:

$$x' - xf = (x-xf)* \cos(u) - (y - yf) * \sin(u)$$
$$y' - yf = (x-xf)* \sin(u) + (y - yf) * \cos(u)$$

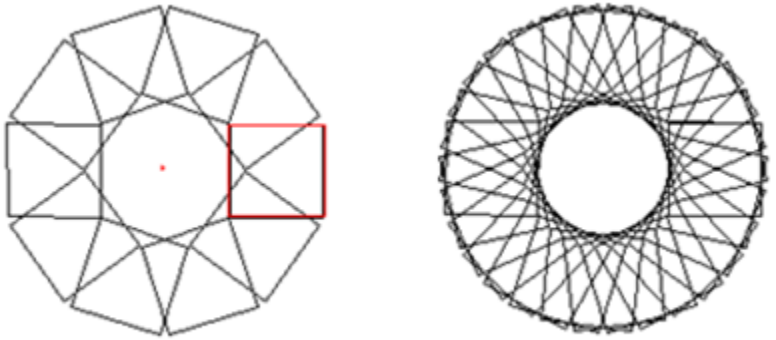


Rotatia fata de un punct F din plan

Rezultă:

$$x' = x*\cos(u) - y*\sin(u) + xf - xf*\cos(u) + yf*\sin(u)$$
$$y' = x*\sin(u) + y*\cos(u) + yf - xf*\sin(u) - yf*\cos(u)$$

Nu poate fi exprimată în coordonate carteziene printr-o matrice de 2x2!


Rotatia unui patrat in jurul unui punct din plan

Compunerea transformărilor geometrice 2D

De ce este necesară?

- Pentru a putea aplica o singură transformare care înglobează o secvenţă de transformări elementare, în locul aplicării unei succesiuni de transformări elementare; de ex., se aplică tuturor vârfurilor o transformare care înglobează scalare (S), rotație (R) și translație (T) în loc să se aplice fiecărui vârf secvența de transformări elementare

  Pt = M * P în loc de P' = S * P; P'' = R * P'; P''' = T * P'' unde M = T * R * S

- Matricea unei transformări compuse se obține prin înmulțirea matricilor transformărilor elementare. Exemplu:

RS = R*S = $\begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix} \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$

RS # SR

SR = S*R = $\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix}$

- **Translația nu poate fi reprezentată în coordonate carteziene printr-o matrice!**
- **Aceasta impune reprezentarea transformărilor în coordonate omogene.**

Reprezentarea transformărilor geometrice 2D în coordonate omogene (1)

Coordonate omogene:

Un punct din plan, P(x,y), se reprezinta in coordonate omogene printr-un vector

[xw, yw, w] sau $\begin{bmatrix} xw \\ yw \\ w \end{bmatrix}$ xw = x * w; yw = y * w; w – orice numar real

Exemplu: P(2, 0.5) → [2, 0.5, 1], [4, 1, 2], [20, 5, 10]

Transformarea din coordonate omogene in coordonate carteziene:

[xw yw w] → P(x, y), unde:

- pentru w #0, x = xw/w, y = yw/w
- pentru w =0, P este un punct la infinit

Reprezentarea transformărilor elementare în coordonate omogene

Translația

$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix} \quad \text{sau} \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

În OpenGL se folosesc vectori coloană pentru a reprezenta puncte din plan sau din spaţiu.

Scalarea față de origine

$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{sau} \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Rotația fața de origine

$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} \cos(u) & \sin(u) & 0 \\ -\sin(u) & \cos(u) & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{sau} \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(u) & -\sin(u) & 0 \\ \sin(u) & \cos(u) & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Transformările inverse ale transformărilor elementare

T(tx, ty): matricea translației, în coordonate omogene

S(0, 0, sx, sy): matricea scalării față de origine, în coordonate omogene

R(0,0,u): matricea rotației față de origine, în coordonate omogene

Transformările inverse sunt:

$T(tx, ty)^{-1} = T(-tx, -ty)$

$S(0, 0, sx, sy)^{-1} = S(0, 0, 1/sx, 1/sy)$

$R(0,0, u)^{-1} = R(0, 0, -u)$

Transformări geometrice 2D compuse (1)

Exemple de transformări compuse: scalarea/rotația față de un punct oarecare din plan.

- Expresiile matematice ale scalării şi rotaţiei faţă de un punct oarecare din plan se pot obţine prin compunerea următoarelor transformări:
  - Translaţia prin care punctul fix al transformării ajunge în origine: T(-xf, -yf);
  - Scalarea / rotaţia faţă de origine: S(0,0,sx,sy)/R(0,0,u);
  - Translaţia inversă celei de la punctul 1: T(xf, yf).

Transformări geometrice 2D compuse (2)

Scalarea faţă de punctul F(xf,yf)

$$[x'\ y'\ 1] = [x\ y\ 1] \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -x_f & -y_f & 1 \end{bmatrix}}_{T(-xf,\ -yf)} \underbrace{\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{S(0,\ 0,\ sx,\ sy)} \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ x_f & y_f & 1 \end{bmatrix}}_{T(xf,\ yf)}$$

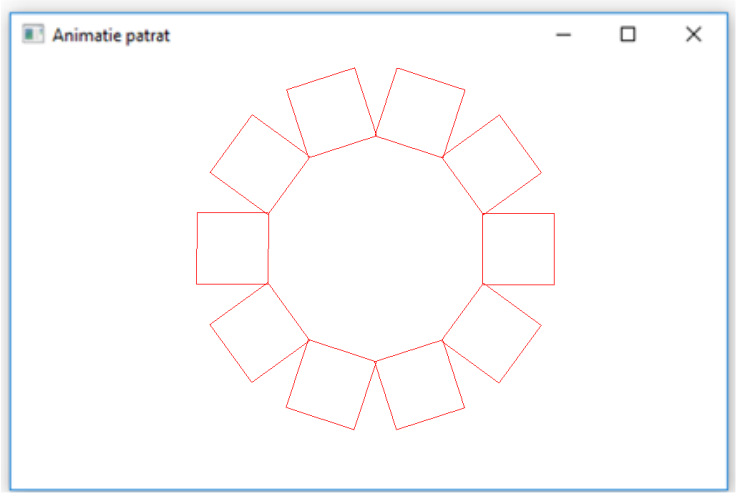
Matricea transformarii compuse:

M = T(-xf, -yf) * S(0, 0, sx, sy) * T(xf, yf)

Aplicarea transformarii compuse:

[x' y' 1] = [x y 1] * M

Rotația unui pătrat în jurul centrului ecranului




```
// campuri ale clasei DesenAnimatiePatrat
int raza, xc, yc, latura;

// afiseaza 10 patrate translatate in
// jurul centrului ecranului
void C1_DesenAnimatiePatrat::UpdateScene3(float deltaTimeSeconds)
{
   glm::ivec2 resolution = window->GetResolution();
   xc = resolution.x / 2;
   yc = resolution.y / 2;

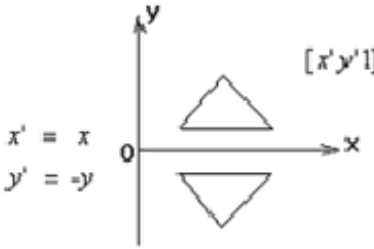
   for (int i = 0; i < 10; i++)
   {
      // seteaza matricea de transformare a varfurilor
      modelMatrix = Transform2D::Translate(xc, yc);
      modelMatrix *= Transform2D::Rotate(0.628f * i);
      modelMatrix *= Transform2D::Translate(-xc, -yc);

      RenderMesh2D(meshes["square2"],
                   shaders["VertexColor"], modelMatrix);
   }
}
```

Alte transformări geometrice 2D (1)

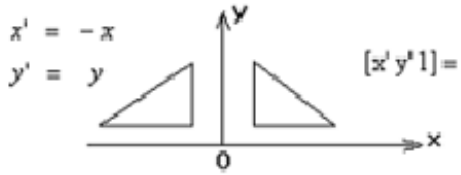
Oglindirea

Față de axa OX



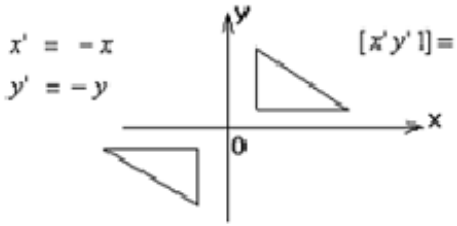
$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Față de axa OY



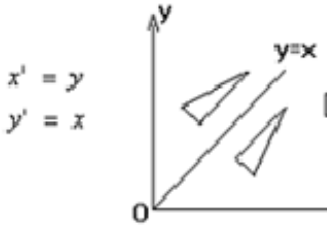
$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Față de origine



$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Față de dreapta x=y



$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Alte transformări geometrice 2D (2)

Oglindirea faţă de o dreaptă oarecare

Se exprimă ca transformare compusă prin înmulţirea matricilor care exprimă următoarele transformări:

1. O translaţie, astfel încât dreapta să treacă prin origine.

2. O rotaţie faţă de origine, a.î. dreapta să se suprapună peste una dintre axele principale.

3. Oglindirea faţă de axa principală peste care a fost suprapusă dreapta.

4. Rotaţia inversă celei de la punctul 2.

5. Translaţia inversă celei de la punctul 1.

În notaţie matriceală:

M = T * R* O* R^{-1} *T^{-1} (folosind vectori linie) **sau M = T^{-1} * R^{-1} *O *R *T** (folosind vectori coloană)

Deduceți elementele matricilor T, R, O, atunci când dreapta este dată printr-un punct, (xd, yd) și o direcție, D[a, b].

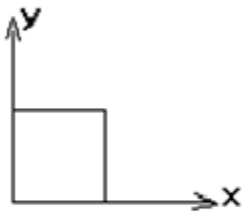
Alte transformări geometrice 2D(3)

Forfecarea

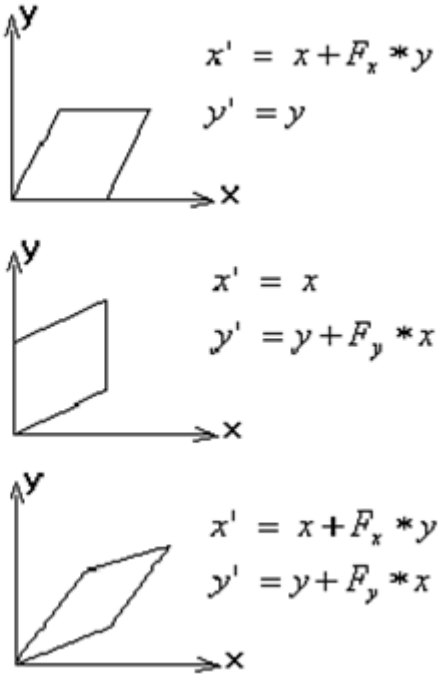
Este definită prin 2 numere reale:

- Fx: factorul de forfecare pe axa OX
- Fy: factorul de forfecare pe axa OY

Deduceți formele matriceale ale transformărilor de forfecare



Forfecarea fata de origine



Forfecarea față de un punct oarecare din plan, (xf,yf) , exprimată ca transformare compusă:

1. Translație prin care punctul (xf, yf) ajunge în origine
2. Forfecarea față de origine
3. Translația inversă celei de la pasul 1

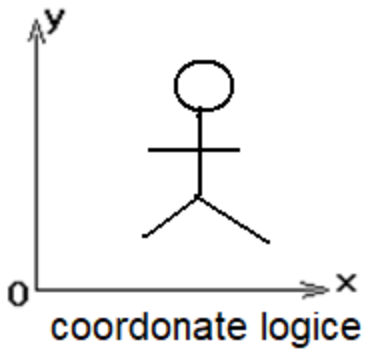
Transformarea de vizualizare 2D

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

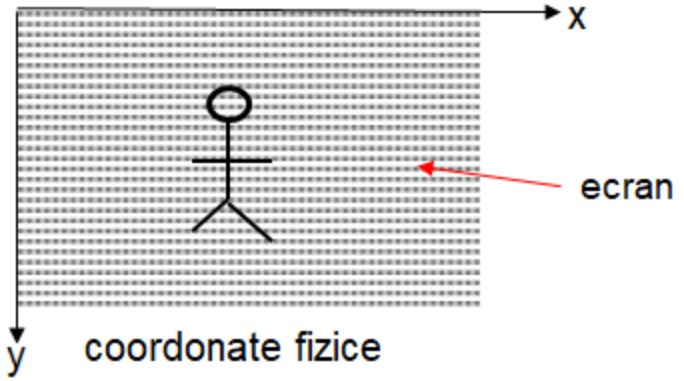
Coordonate logice şi coordonate fizice

- Desenele reprezentate într-un program de aplicație grafică 2D sunt, de regulă, raportate la un sistem de coordonate diferit de cel al suprafeței de afișare.
- Exemple: planul unei case, un desen tehnic, graficul unei funcții, etc.



(metri, milimetri, viteza, timp, etc.)

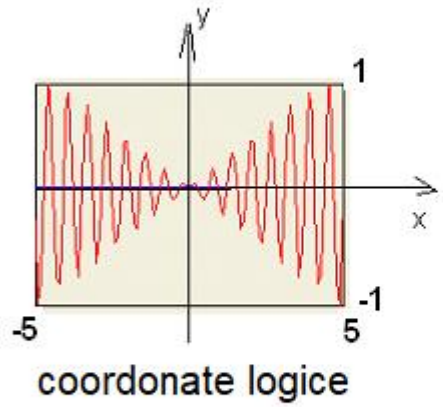
Sistemul de coordonate utilizator

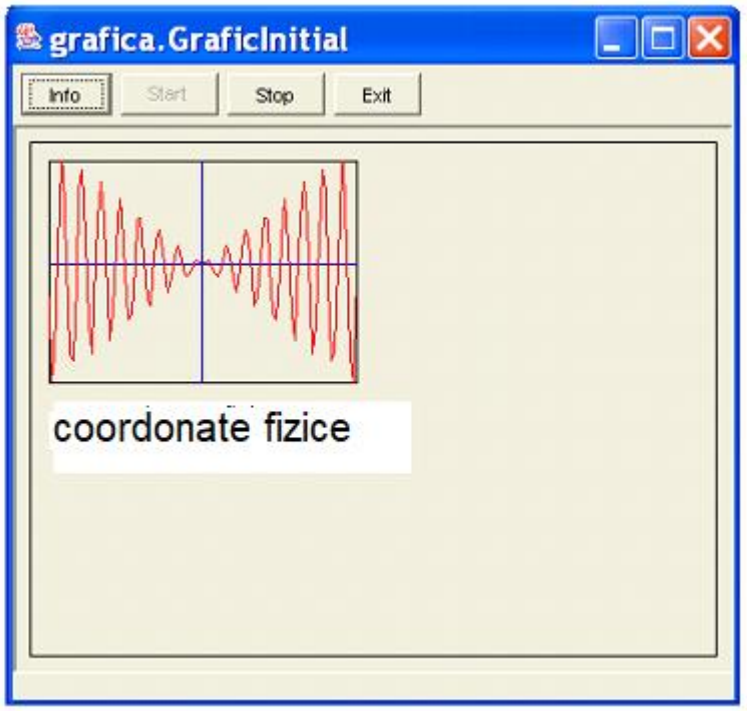


(pixeli, milimetri, s.a.)

Sistemul de coordonate dispozitiv

Transformarea coordonate logice - coordonate fizice



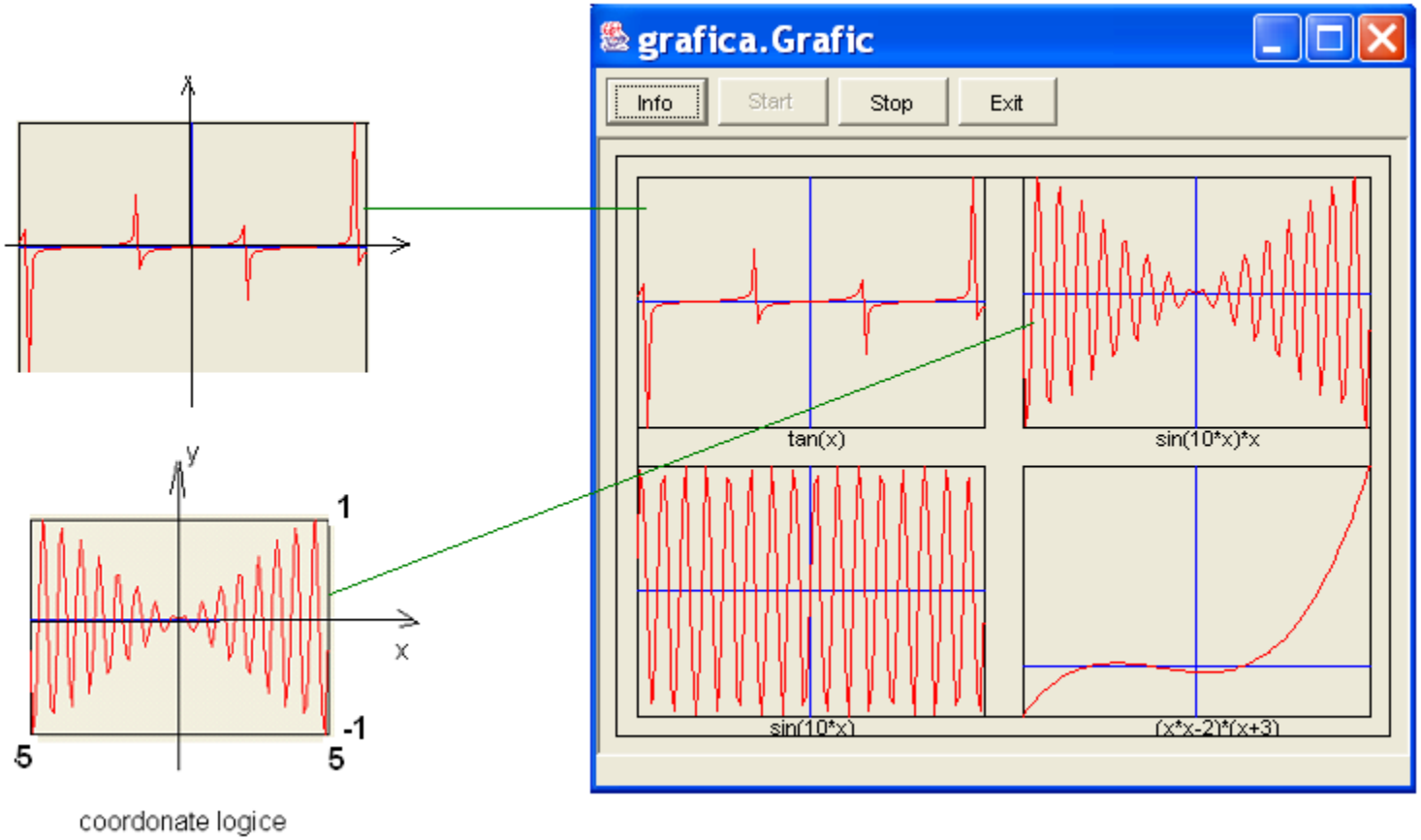


Aplicatia definește graficul funcției în sistemul de coordonate utilizator (logice).

Coordonatele punctelor de pe graficul afișat pe ecran sunt adrese de pixeli.

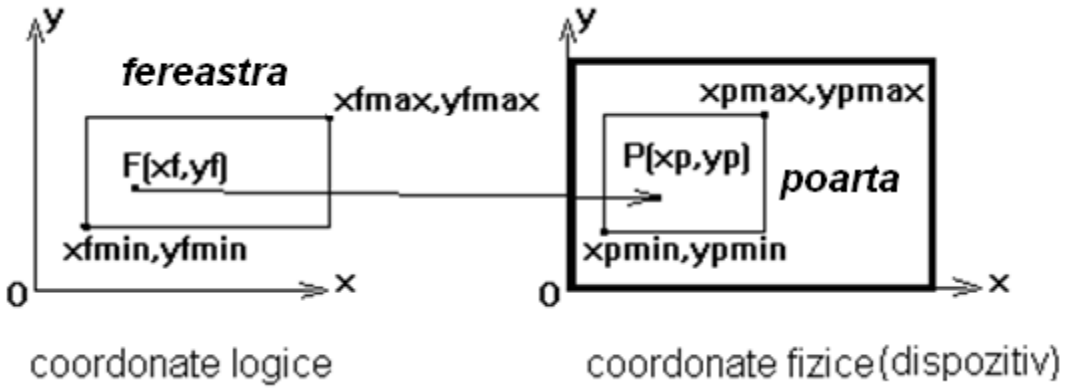
Coordonatele logice trebuie transformate în coordonate fizice.

Transformarea de vizualizare 2D (1)



Este necesară o transformare generală, prin care un desen definit în coordinate logice să poată fi afișat într-o zonă oarecare a ecranului.

Transformarea de vizualizare 2D (2)
- transformarea "fereastră-poartă"-



- fereastra: zona din spațiul coord. logice al cărei conținut se va afișa
- poarta: zona din spațiul coord. fizice în care se va afișa conținutul ferestrei

▪ Transformarea este definită folosind 2 dreptunghiuri, definite în cele 2 sisteme de coordonate, numite: ***fereastra (de vizualizare)*** și ***poarta (de afișare).***

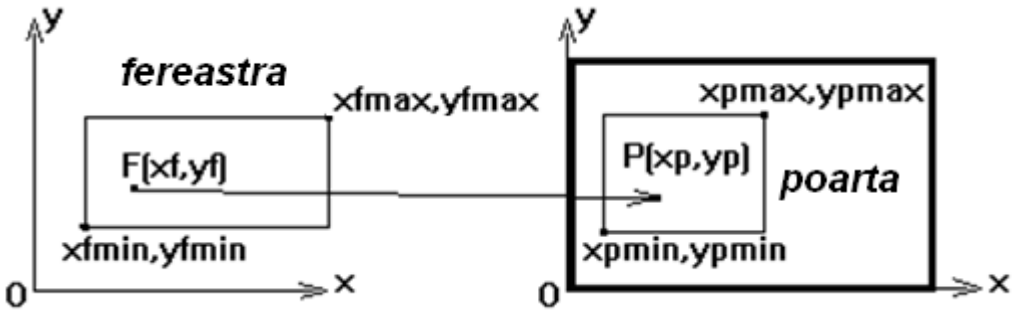
• Transformarea se mai numește ***transformarea fereastră-poartă.***

F: un punct din fereastră.
P: punctul în care se transformă F prin transformarea fereastră-poartă.

➢Transformarea fereastră-poartă trebuie să asigure că:

"poziția relativă a lui P în poarta de afișare este aceeași cu poziția relativă a lui F în fereastră".

Transformarea de vizualizare 2D (3)



$$\frac{xp - xp\,\min}{xp\,\max - xp\,\min} = \frac{xf - xf\,\min}{xf\,\max - xf\,\min}$$

$$\frac{yp - yp\,\min}{yp\,\max - yp\,\min} = \frac{yf - yf\,\min}{yf\,\max - yf\,\min}$$

$$sx = \frac{xp\,\max - xp\,\min}{xf\,\max - xf\,\min}$$

$$sy = \frac{yp\,\max - yp\,\min}{yf\,\max - yf\,\min}$$

- sx, sy reprezintă factorii de scalare ai transformării
- tx, ty depind de pozițiile celor 2 ferestre față de originea sistemului de coord. în care sunt definite

și $tx = xpmin - sx*xfmin$ $ty = ypmin - sy*yfmin$

$$xp = xf * sx + tx$$
$$yp = yf * sy + ty$$

← Exprimarea matematică a transformării fereastră – poartă, considerând o aceeași orientare a axelor celor 2 sisteme de coordonate.

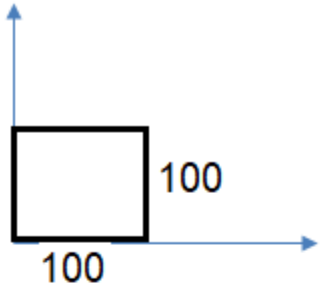
Transformarea de vizualizare 2D (4)

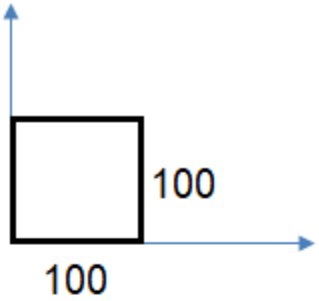
$$sx = \frac{xp\max - xp\min}{xf\max - xf\min}$$

$$sy = \frac{yp\max - yp\min}{yf\max - yf\min}$$

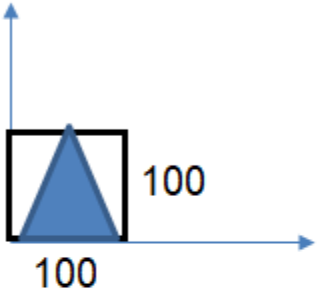
*tx = xpmin- sx*xfmin*

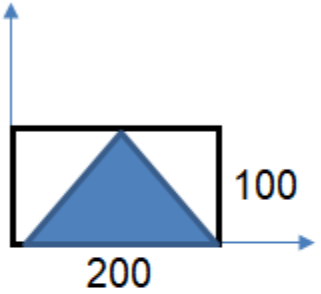
*ty= ypmin- sy*yfmin*



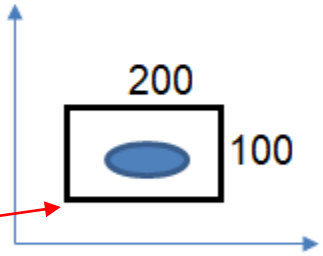


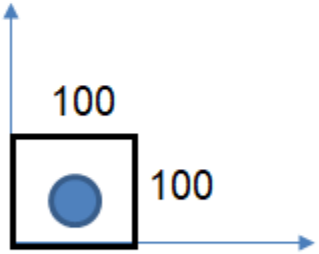
sx=1, sy = 1
tx = 0, ty =0



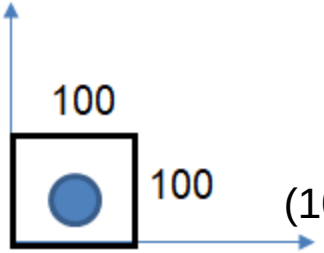


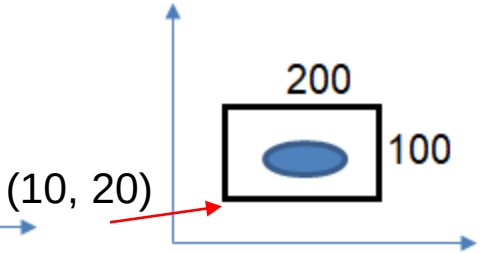
sx=2, sy = 1
tx = 0, ty =0





sx=0.5, sy = 1
tx = 0 - 0.5*10 = - 5
ty = 0 – 1*20 = -20

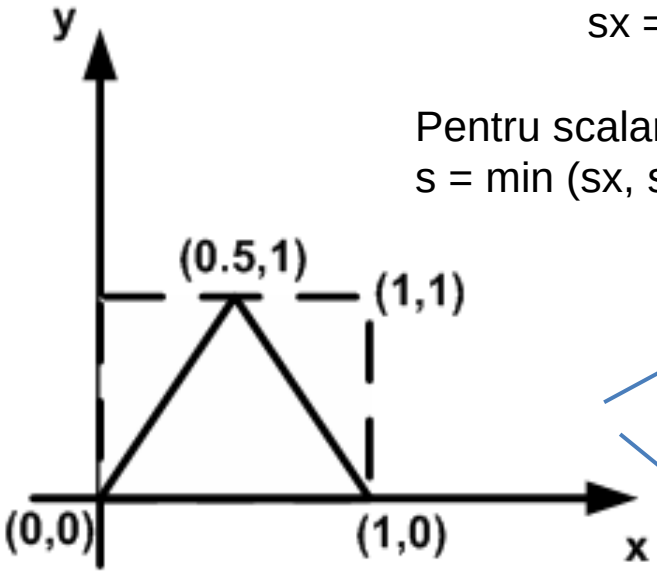




sx=2, sy = 1
tx = 10
ty = 20

Transformarea de vizualizare 2D (5)

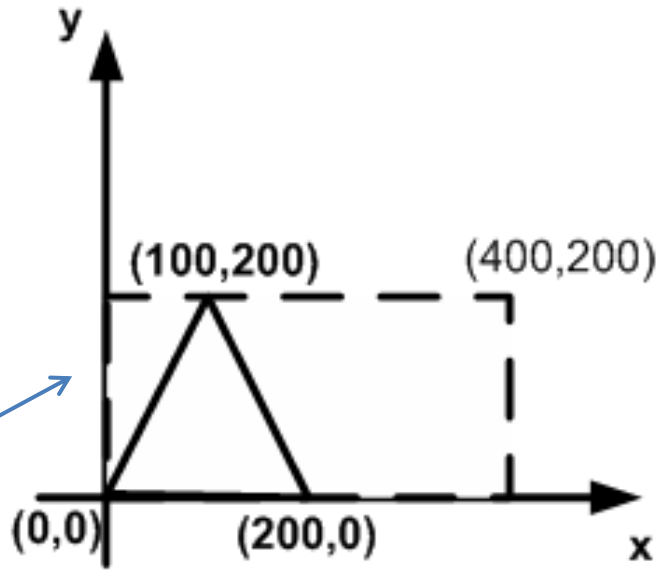
Scalare uniformă
cu afișare centrată în poarta de vizualizare



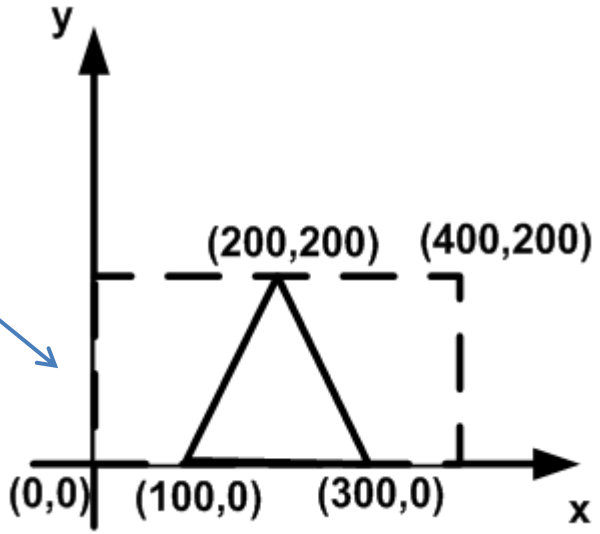
sx = 400, sy = 200

Pentru scalare uniformă:
s = min (sx, sy)

tx=0, ty =0



tsx = 100,
tsy =0



tsx =(xpmax – xpmin – s*(xfmax – xfmin))/2

= (400 – 200*1) /2 = 100

tsy =(ypmax – ypmin – s*(yfmax – yfmin))/2

= (200 – 200*1) /2 = 0

Transformarea de vizualizare 2D (7)

Efectele transformării

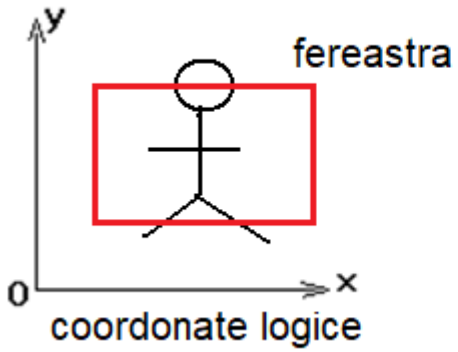
- Mărire / micșorare, în funcție de dimensiunile ferestrei și ale porții
- Deformare: fereastra și poarta nu sunt dreptunghiuri asemenea
- Pentru scalare uniformă: s = min(sx,sy)
- Afișare centrata in poartă: translatie suplimentara pe axa ox sau pe axa oy:

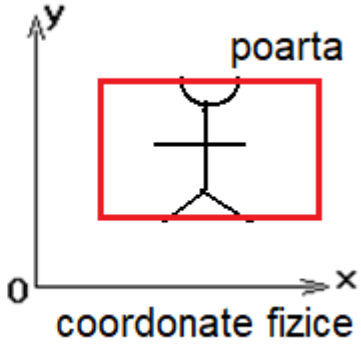
  tsx =(xpmax – xpmin – s*(xfmax – xfmin))/2

  tsy =(ypmax – ypmin – s*(yfmax – yfmin))/2

  Translația totală: tx+tsx, ty+tsy

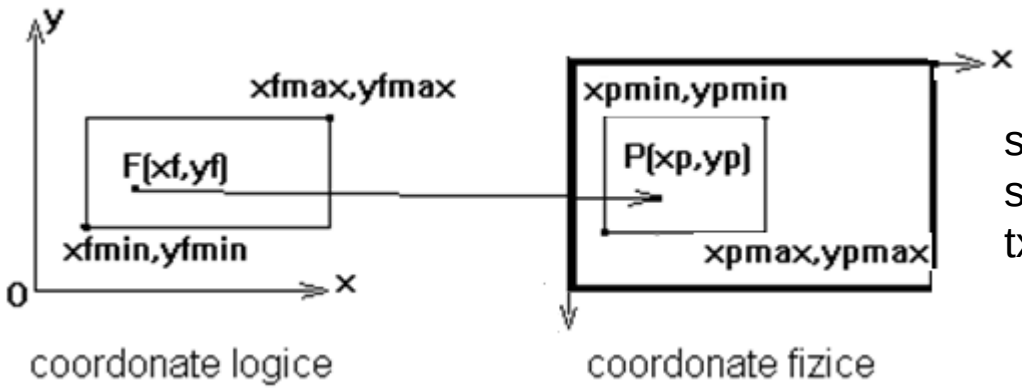
- Decuparea primitivelor aflate în afara ferestrei vizuale

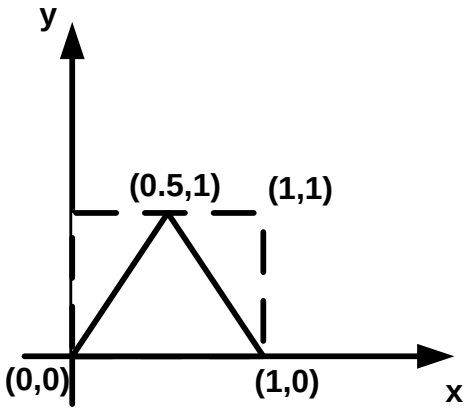




Transformarea de vizualizare 2D (8)

Corecția coordonatei yp
Ținând cont de orientarea axelor sist. coord fizice





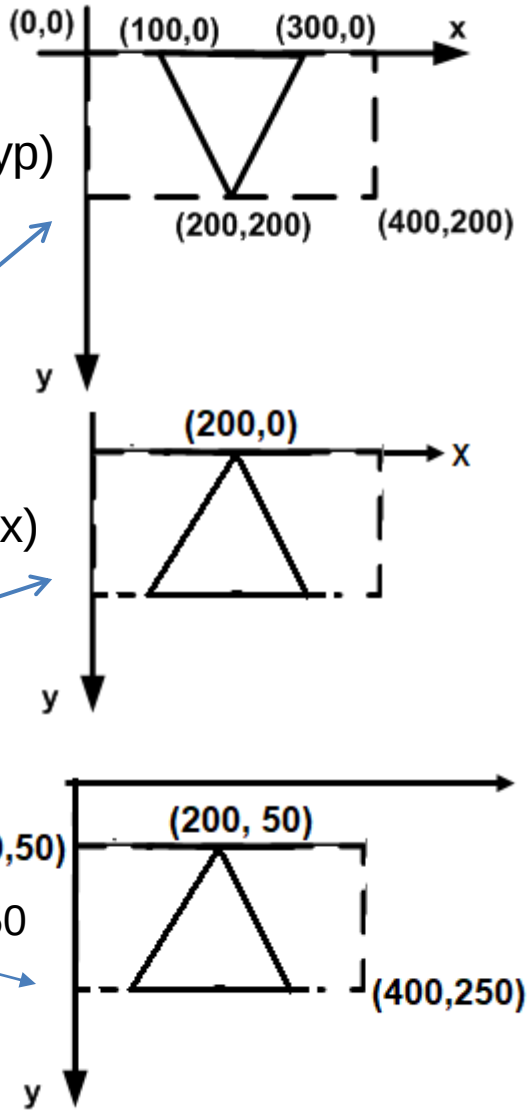
(xp, yp)

sx = 400, sy = 200
s=smin = 200
tx = tsx =100, ty =0

(xp, yp'= -yp+ypmax)

ty = 50

yp' = -yp + ypmax + ypmin



Transformarea de vizualizare 2D (9)

Formulele transformării fereastră – poartă atunci când axa OY a sistemului de coordonate fizice este orientata în jos:

xp = xf * sx + tx
yp = – (yf * sy + ty) + ypmin + ypmax = -yf * sy + ypmin + ypmax -ty

Exprimarea matriceală a transformării fereastră-poartă în cazul corecției coordonatei y

$$\begin{bmatrix} xp \\ yp \\ 1 \end{bmatrix} = M * \begin{bmatrix} xf \\ yf \\ 1 \end{bmatrix}$$

$$M = \begin{bmatrix} sx & 0 & tx \\ 0 & -sy & yp\min + yp\max - ty \\ 0 & 0 & 1 \end{bmatrix}$$

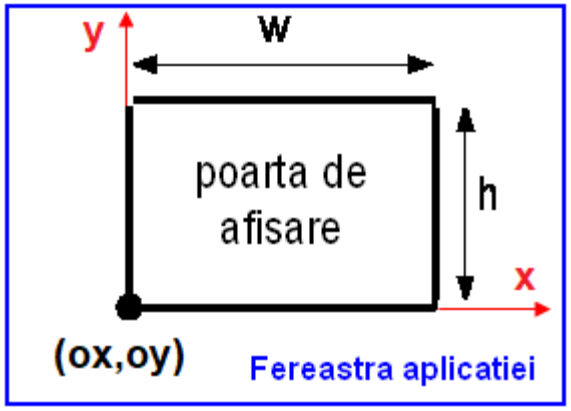
Implementarea transformării de vizualizare în OpenGL (1)

- Pentru setarea zonei de afisare, în OpenGL se apeleaza

  glViewport(ox, oy, width, height);

unde (ox,oy) sunt coord. colțului stânga jos al viewportului

- Orientarea axelor viewportului este aceeași cu a sistemului coordonatelor logice:
  - Nu este necesară corecția coordonatei y → matricea transformării de vizualizare este:



$$M = \begin{bmatrix} sx & 0 & tx \\ 0 & sy & ty \\ 0 & 0 & 1 \end{bmatrix}$$

- Se poate defini un desen în coordonate logice.
- Pentru obținerea desenului în coordonate ecran se folosește ca matrice de transformare a vârfurilor matricea transformării de vizualizare

Implementarea transformării de vizualizare în OpenGL (2)

```
class C2_TransformareaFereastraPoartaPatrate : public SimpleScene
{
 ..............................
protected:
  float length;
  ViewportSpace viewSpace;      // dreptunghiul ”poarta de afișare”
  LogicSpace logicSpace;        // dreptunghiul ”fereastră”
  glm::mat3 modelMatrix,
        visMatrix;              // matricea transformării de vizualizare
  bool uniform;                 // transformare uniformă / neuniformă

// figura desenată se obtine prin rotația unui pătrat în jurul centrului
ferestrei
  float raza;                 // raza cercului pe care se rotește pătratul
  float cx, cy;               // centrul cercului
};
```

Implementarea transformării de vizualizare în OpenGL (3)

```
void C2_TransformareaFereastraPoartaPatrate::Init()
{
    ..................................
    logicSpace.x = -5;      // logic x
    logicSpace.y = -5;      // logic y
    logicSpace.width = 10; // logic lățime
    logicSpace.height = 10;// logic înălțime
    length = 2.0f;          // latura patratului
    raza = 3.6f;

    cx = logicSpace.x + logicSpace.width / 2;
    cy = logicSpace.y +  logicSpace.height / 2;

    // Creare pătrat inițial
    glm::vec3 corner = glm::vec3(cx - length / 2 + raza , cy - length / 2, 0);
    Mesh* square = Object2D::CreateSquare("square", corner, length,
                                glm::vec3(1, 0, 0));
    AddMeshToList(square);

    // Creare cadru fereastră
    glm::vec3 cornerw = glm::vec3(logicSpace.x, logicSpace.y, 0);
    Mesh* window = Object2D::CreateSquare("window", cornerw, logicSpace.width,
                                      glm::vec3(0, 0, 0));
    AddMeshToList(window);

    uniform = true;
}
```

Implementarea transformării de vizualizare în OpenGL (4)
Calculul matricei transformării de vizualizare

```
// Implementarea transformării de vizualizare fereastră-poartă neuniformă
glm::mat3 C2_TransformareaFereastraPoartaPatrate::VisualizationTransf2D(const
LogicSpace & logicSpace, const ViewportSpace & viewSpace)
{
  float sx, sy, tx, ty;
  sx = viewSpace.width / logicSpace.width; sy = viewSpace.height / logicSpace.height;
  tx = viewSpace.x - sx * logicSpace.x; ty = viewSpace.y - sy * logicSpace.y;

  return glm::transpose(glm::mat3(sx, 0.0f, tx,0.0f, sy, ty,0.0f, 0.0f, 1.0f));
}

// Implementarea transformării de vizualizare fereastră-poartă uniformă (sx egal cu sy)
glm::mat3 C2_TransformareaFereastraPoartaPatrate::VisualizationTransf2DUnif(const
LogicSpace & logicSpace, const ViewportSpace & viewSpace)
{
  float sx, sy, tx, ty, smin;
  sx = viewSpace.width / logicSpace.width; sy = viewSpace.height / logicSpace.height;
  if (sx < sy)
    smin = sx;
  else
    smin = sy;
  tx = viewSpace.x - smin*logicSpace.x +(viewSpace.width - smin *logicSpace.width)/2;
  ty = viewSpace.y - smin*logicSpace.y +(viewSpace.height - smin *logicSpace.height)/2;

  return glm::transpose(glm::mat3(smin, 0.0f, tx,0.0f, smin, ty,0.0f, 0.0f, 1.0f));
}
```

Transformare uniformă/neuniformă - implementare

```
void C2_TransformareaFereastraPoartaPatrate::UpdateScene1(float deltaTimeSeconds)
{
  glm::ivec2 resolution = window->GetResolution();

  // Setează poarta de vizualizare
  // Parametrii:
  // - x,y coltul stânga jos al porții
  // - width, height lățimea și înălțimea porții
  //viewSpace = ViewportSpace(50, 50, resolution.x - 100, resolution.y - 100);
  viewSpace = ViewportSpace(200, 200, 800, 400);

  // Setează zona ecran (fereastra aplicației) în care se va desena
  SetViewportArea(viewSpace, glm::vec3(1), true);

  // Desenează cadrul ferestrei
  visMatrix = VisualizationTransf2D(logicSpace, viewSpace);
  RenderMesh2D(meshes["window"], visMatrix, glm::vec3(0, 0, 0));

  // Transformare uniformă / neuniformă
  if (uniform)
    visMatrix = VisualizationTransf2DUnif(logicSpace, viewSpace);
  else
    visMatrix = VisualizationTransf2D(logicSpace, viewSpace);

  // Desenează patratele rotite
  DrawRotatedSquares(visMatrix, glm::vec3(1, 0, 1), 10);
}
```

Transformare uniformă/neuniformă - implementare

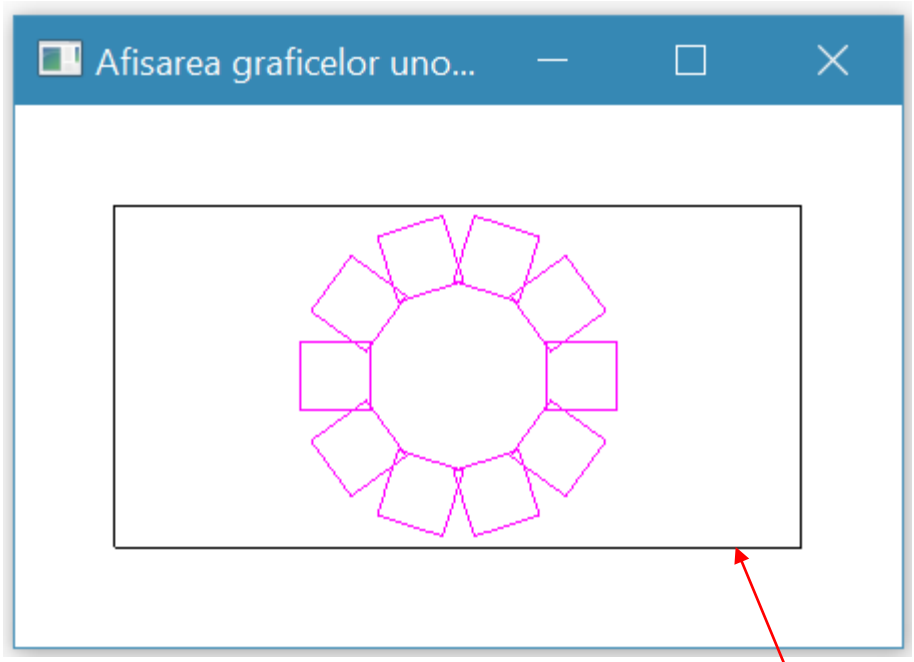
```
void C2_TransformareaFereastraPoartaPatrate::DrawRotatedSquares(glm::mat3
visMatrix, glm::vec3 color, int iterations)
{
  float delta = 2 * 3.14159 / iterations;

  for (int i = 0; i < iterations; i++)
  {

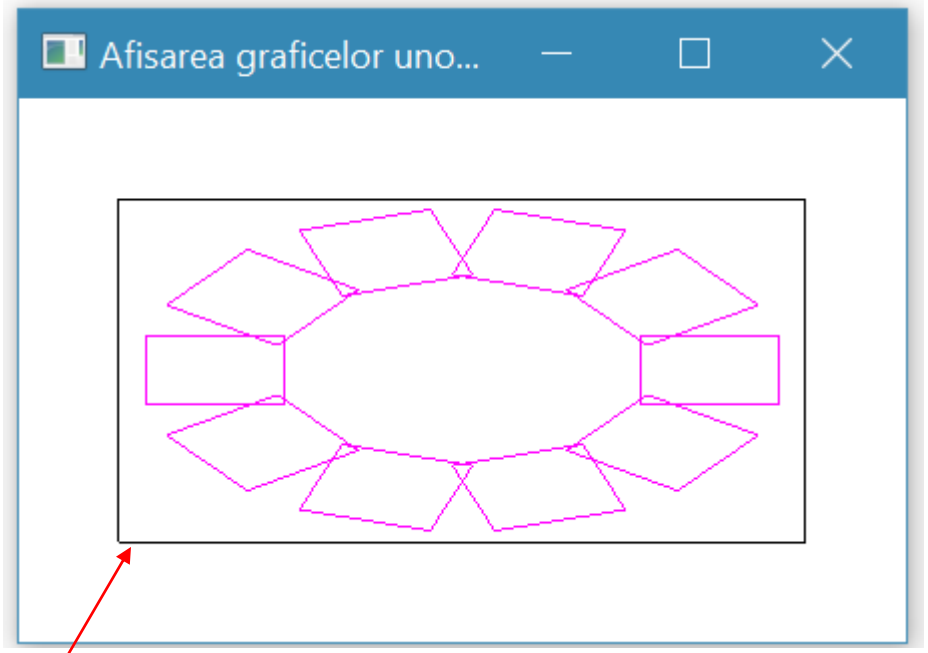
    modelMatrix = visMatrix       *Transform2D::Translate(cx,cy )
                                  *Transform2D::Rotate(delta * i)
                                  *Transform2D::Translate(-cx,-cy);

    RenderMesh2D(meshes["square"], modelMatrix, color);
  }
}
```

Transformare uniformă/neuniformă



Desenul cu transformare uniformă



Desenul cu transformare neuniformă

Cadrul ferestrei mapat pe poarta de afișare
(neafectat de transformarea uniformă)

Desenare în 4 porţi de afişare - implementare

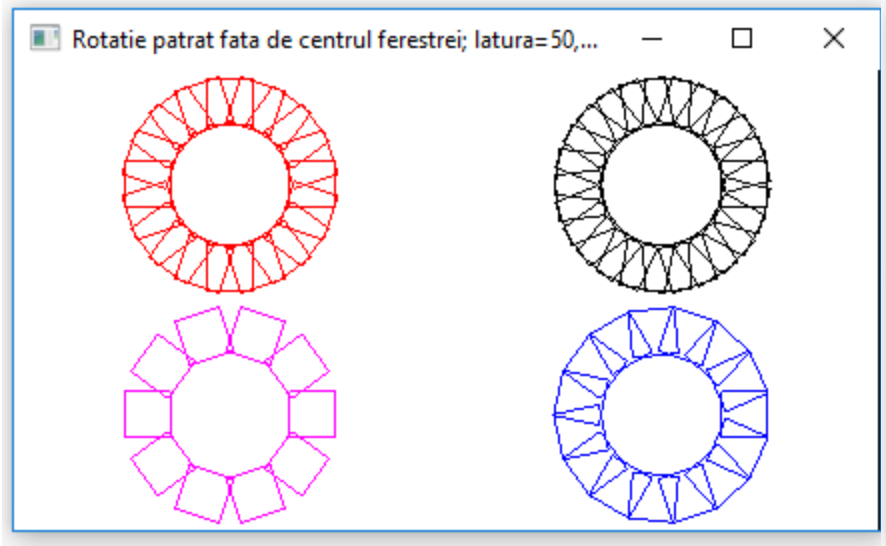
```
// Afișează 4 porți de vizualizare fiecare cu
// 10 patrate rotite în jurul centrului spațiului logic
void C2_TransformareaFereastraPoartaPatrate::UpdateScene2(float deltaTimeSeconds)
{
  glm::ivec2 resolution = window->GetResolution();

  // Partea stânga-jos a ferestrei aplicației
  viewSpace = ViewportSpace(0, 0, resolution.x / 2, resolution.y / 2);
  SetViewportArea(viewSpace, glm::vec3(1), true);
  if (uniform)
    visMatrix = VisualizationTransf2DUnif(logicSpace, viewSpace);
  else
    visMatrix = VisualizationTransf2D(logicSpace, viewSpace);
  DrawRotatedSquares(visMatrix, glm::vec3(1, 0, 1), 10);

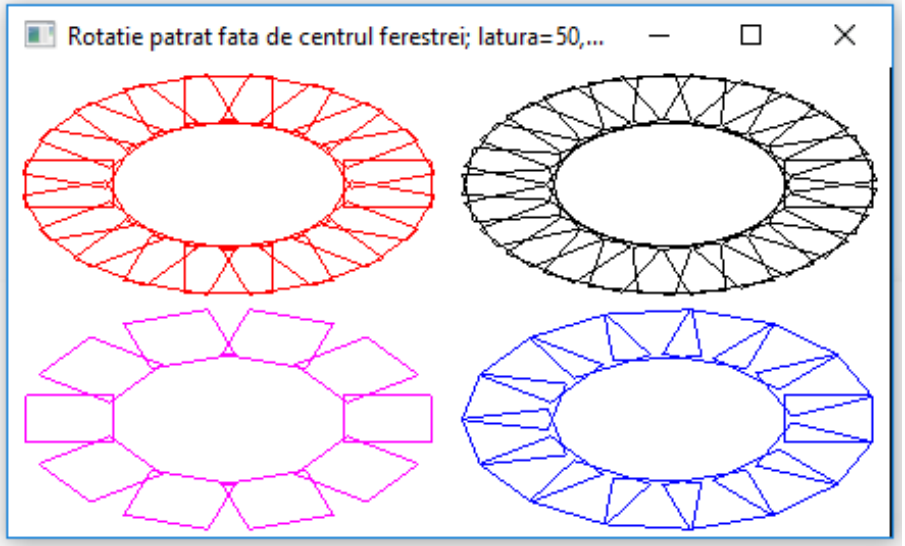
  // Partea dreapta-jos a ferestrei aplicației
  viewSpace = ViewportSpace(resolution.x/2, 0, resolution.x/2, resolution.y/2);
  SetViewportArea(viewSpace, glm::vec3(1), true);
  if (uniform)
    visMatrix = VisualizationTransf2DUnif(logicSpace, viewSpace);
  else
    visMatrix = VisualizationTransf2D(logicSpace, viewSpace);
  DrawRotatedSquares(visMatrix, glm::vec3(0, 0, 1), 15);

  // Analog pentru partea stânga-sus și dreapta-sus a ferestrei aplicației
}
```

Desenare în 4 porţi de afişare

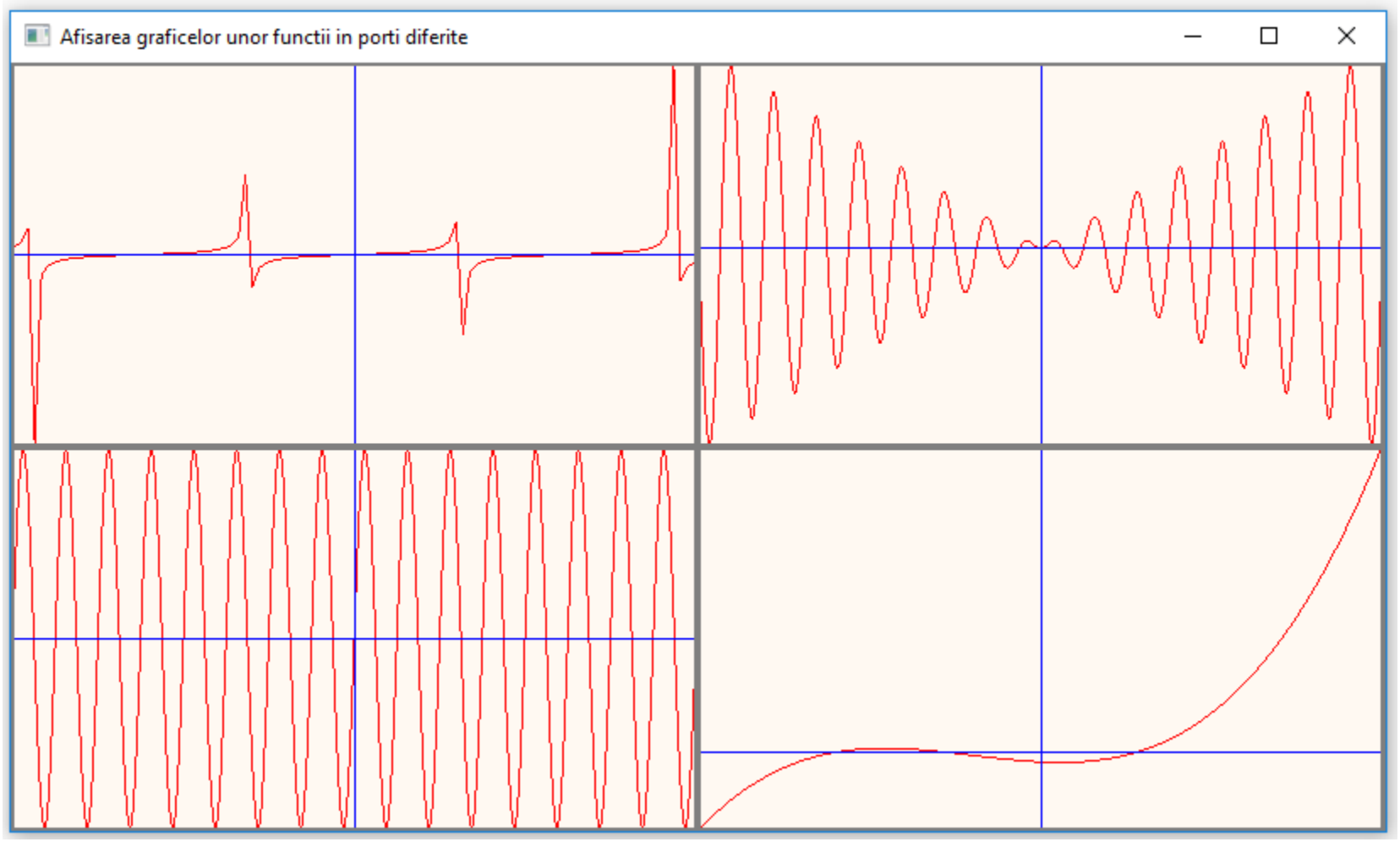


Transformare uniformă



Transformare neuniformă

Afişarea graficelor unor funcţii în porţi diferite



Afisarea graficelor unor funcții în porți diferite – implementare (4)

```
float f1(double x)
{
        return (float)(sin(10 * x)*x);
}

float f2(double x)
{
        return (float)(tan(x));
}

float f3(double x)
{
        return (float)(sin(10 * x));
}

float f4(double x)
{
        return (float)((x * x - 2) * (x + 3));
}
```

Afişarea graficelor unor funcţii în porţi diferite – implementare (1)

```
Mesh* Object2DGrafic::CreateFunctionPlot(std::string name,
                        float &x1, float &x2, float &y1, float &y2, float pas,
                        float(*f)(double f))
{
  // calculează punctele de pe graficul funcției matematice y = f(x) în
  // intervalul x1 – x2, în coordonate logice
  // întoarce în (x1,y1), (x2,y2) încadrarea graficului incluzând și axele
  // sistemului de coordonate
  float x, y, xmin=x1, xmax=x2;

  // calculeaza incadrarea graficului pe axa OY( [y1-y2]) in intervalul
  // xmin - xmax
  y1 = y2 = f(xmin);
  for (x = xmin + pas; x<xmax; x += pas)
  {
    if ((y = f(x))<y1) y1 = y;
    if (y>y2) y2 = y;
  }

  if ((y = f(xmax))<y1) y1 = y;
  if (y>y2) y2 = y;
```

Afisarea graficelor unor funcții în porți diferite – implementare (2)

```
// Include axele OX, OY în încadrarea graficului
if (0<xmin) x1 = 0; else x1 = xmin;
if (0<y1) y1 = 0;
if (0>xmax) x2 = 0; else x2 = xmax;
if (0>y2) y2 = 0;

Mesh* plot = new Mesh(name);
// Se vor uni câte 2 vârfuri succesive
plot->SetDrawMode(GL_LINES);
std::vector<VertexFormat> vertices;
// Memorează varfurile axei OX – se asociază vârfurilor culoarea albastru
vertices.push_back(VertexFormat(glm::vec3(x1, 0, 0), glm::vec3(0, 0, 1)));
vertices.push_back(VertexFormat(glm::vec3(x2, 0, 0), glm::vec3(0, 0, 1)));
// Memorează vârfurile axei OY – se asociază vârfurilor culoarea albastru
vertices.push_back(VertexFormat(glm::vec3(0, y1, 0), glm::vec3(0, 0, 1)));
vertices.push_back(VertexFormat(glm::vec3(0, y2, 0), glm::vec3(0, 0, 1)));
std::vector<unsigned short> indices;
indices.push_back(0);indices.push_back(1);
indices.push_back(2);indices.push_back(3);
```

Afisarea graficelor unor functii in porti diferite – implementare (3)

```
  // Se adaugă vârfurile care aproximează graficul
  // Se asociază vârfurilor culoarea roşu
  int contor = 4;
  for (x = xmin; x<xmax - pas; x += pas)
  {
    vertices.push_back(VertexFormat(glm::vec3(x, f(x), 0),
                                    glm::vec3(1, 0, 0)));
    indices.push_back(contor); indices.push_back(contor+1);
    contor++;
  }
  // Se adaugă ultimul vârf
  vertices.push_back(VertexFormat(glm::vec3(xmax, f(xmax), 0),
                                    glm::vec3(1, 0, 0)));

  plot->InitFromData(vertices, indices);
  return plot;

  // Sfârșit CreateFunctionPlot
}
```

Afisarea graficelor unor funcții în porți diferite – implementare (5)

```
void TransformareaFereastraPoartaGrafic::Init()
{
  ..............
  // Se crează graficele celor 4 funcții în intervalul xmin=-5, xmax=5
  float y1, y2, x1=-5.0f, x2 = 5.0f;
  // Se crează un mesh cu graficul funcției f1;
  // Se setează spațiul logic pentru graficul funcției f1
  Mesh* mesh_f1 = Object2DGrafic::CreateFunctionPlot("mesh_f1", x1, x2, y1, y2,
                                                     0.01f, f1);
  AddMeshToList(mesh_f1);
  logicSpace1.x = x1; logicSpace1.y = y1;
  logicSpace1.width = x2 - x1;
  logicSpace1.height = y2 - y1;
  // Se crează, în mod analog, mesh_f2, mesh_f3, mesh_f4 și
  // logicSpace2, logicSpace3, logicSpace4
  ..............
}
```

Afisarea graficelor unor functii in porti diferite – implementare (6)

```
void TransformareaFereastraPoartaGrafic::Update(float deltaTimeSeconds)
{
  glm::ivec2 resolution = window->GetResolution();

  // Setează zona de desenare – partea stânga-sus
  viewSpace = ViewportSpace(0, resolution.y/2, resolution.x/2, resolution.y/2);
  SetViewportArea(viewSpace, glm::vec3(1, 0.98, 0.95), true);
  visMatrix = VisualizationTransf2D(logicSpace1, viewSpace);
  // Desenează gaficul funcției f1
  DrawFunctionPlot(visMatrix, 1);

  // Partea dreapta-sus a ferestrei aplicației
  viewSpace = ViewportSpace(resolution.x/2, resolution.y/2, resolution.x/2,
                            resolution.y / 2);
  // Culoarea fondului pentru viewport: bej
  SetViewportArea(viewSpace, glm::vec3(1,0.98,0.95), true);
  visMatrix = VisualizationTransf2D(logicSpace2, viewSpace);
  // Desenează graficul funcției f2
  DrawFunctionPlot(visMatrix, 2);
  // Analog pentru desenarea celorlalte funcții f3 și f4
}
```

Afisarea graficelor unor funcții în porți diferite – implementare (7)

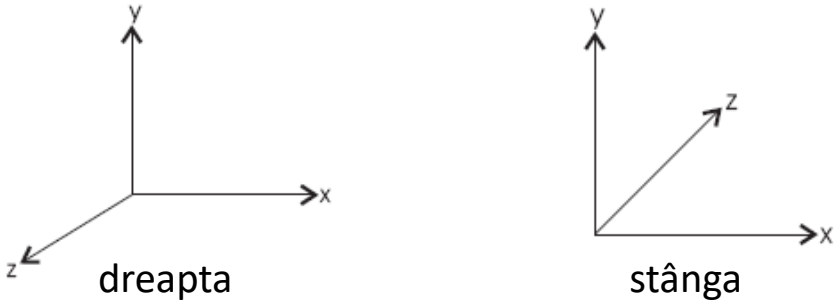
```
void TransformareaFereastraPoartaGrafic::DrawFunctionPlot(glm::mat3
                                                    visMatrix, int nr_functie)
{
      modelMatrix = visMatrix;
      switch (nr_functie)
      {
        case 1:
          RenderMesh2D(meshes["mesh_f1"],shaders["ShaderGrafic"],modelMatrix);
          break;
       case 2:
          RenderMesh2D(meshes["mesh_f2"],shaders["ShaderGrafic"],modelMatrix);
           break;
       case 3:
          RenderMesh2D(meshes["mesh_f3"],shaders["ShaderGrafic"],modelMatrix);
           break;
       case 4:
          RenderMesh2D(meshes["mesh_f4"],shaders["ShaderGrafic"],modelMatrix);
           break;
       default:
           break;
      }
}
```

Transformări geometrice 3D

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

Sisteme de coordonate carteziene 3D



Sisteme de coordonate carteziene 3D

i, j, k: versorii direcțiilor axelor sistemului de coordonate

dreapta: $k = (i \times j)/|i \times j|$

stânga: $k = (j \times i)/|j \times i|$

Sistemul de coordonate în care este descrisă scena într-o aplicație OpenGL: sistem de coordonate carteziene 3D dreapta.

Transformările geometrice 3D elementare

- Translaţia
- Scalarea faţă de origine
- Rotaţiile în jurul axelor principale ale sistemului de coordonate
- Oglindirea faţă de un plan al sistemului de coordonate
- Forfecarea faţă de originea sistemului de coordonate

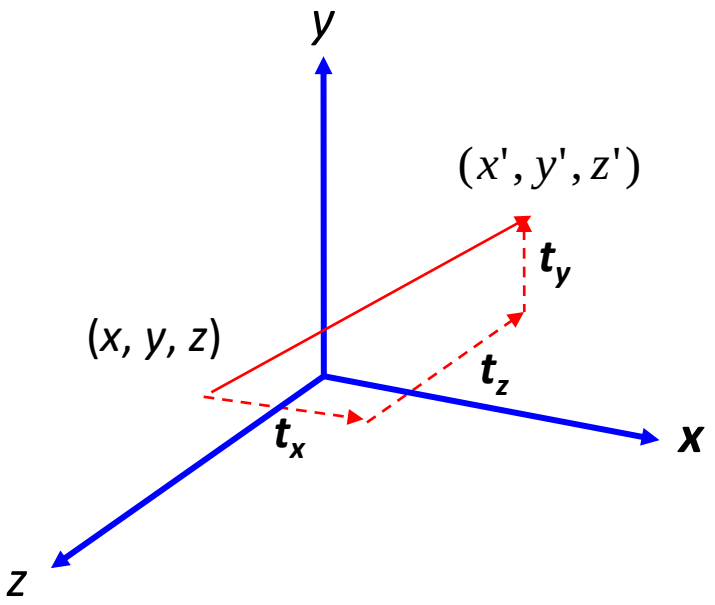
Considerăm punctele din spaţiu reprezentate prin vectori coloană, pentru a respecta convenția din OpenGL:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = M \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

M este matricea de transformare a punctului (x, y, z), în coordonate omogene

Translaţia

Definită printr-un vector T[tx,ty,tz]



$$x' = x + t_x$$

$$y' = y + t_y$$

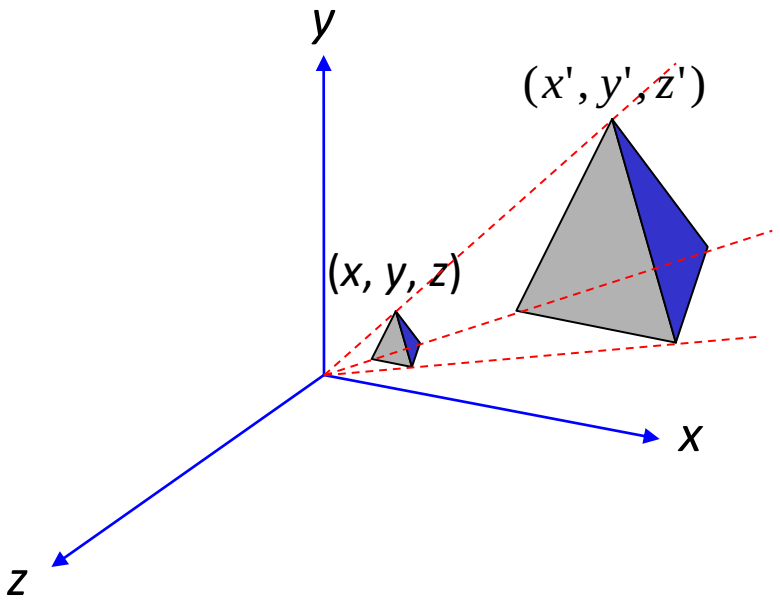
$$z' = z + t_z$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$T(t_x, t_y, t_z)$

Scalarea faţă de origine

Definită prin 3 numere reale: Sx,Sy, Sz – factorii de scalare pe cele 3 axe



$S_x = S_y = S_z \quad \Rightarrow \quad$ scalare uniforma

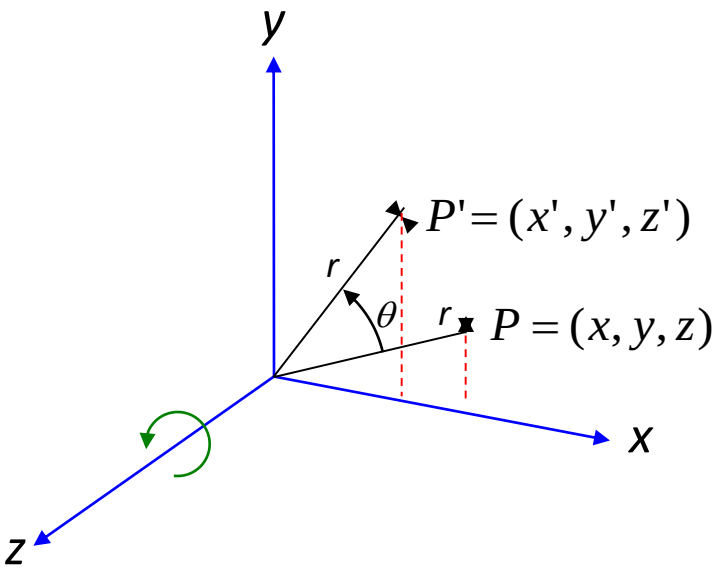
altfel, scalare neuniforma

$$x' = x \cdot s_x$$
$$y' = y \cdot s_y$$
$$z' = z \cdot s_z$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$S(s_x, s_y, s_z)$

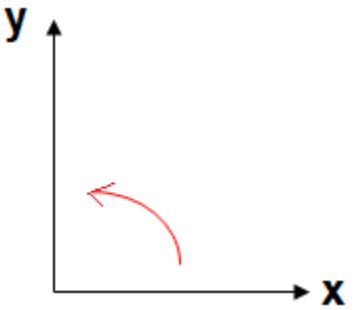
Rotaţia pozitivă (trigonometrică) în jurul axei oz



Este o rotaţie într-un plan de z constant (prin rotaţie nu se modifică coordonata z).

Rotația în planul XOY:

$$x' = x\cos\theta - y\sin\theta$$
$$y' = x\sin\theta + y\cos\theta$$
$$z' = z = 0$$



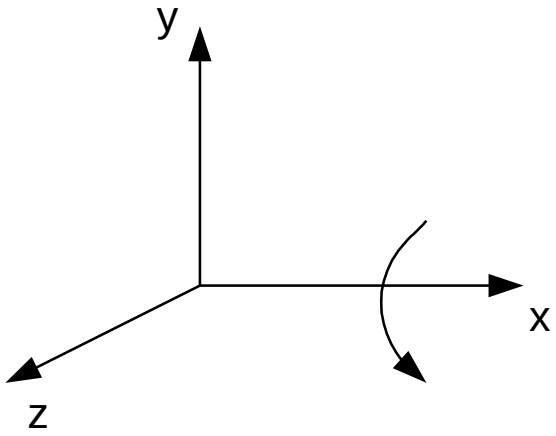
$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$R_z(\theta)$

Rotația punctului P(x,y,z) în jurul axei Oz cu unghiul θ:

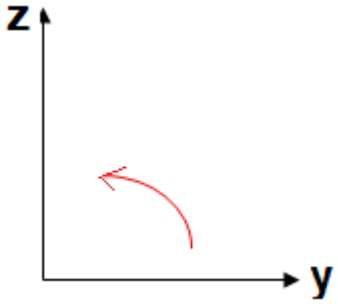
$$x' = x\cos\theta - y\sin\theta$$
$$y' = x\sin\theta + y\cos\theta$$
$$z' = z$$

Rotaţia pozitivă în jurul axei ox



Rotația în planul Y0Z

$$y' = y\cos\theta - z\sin\theta$$
$$z' = y\sin\theta + z\cos\theta$$
$$x' = x = 0$$



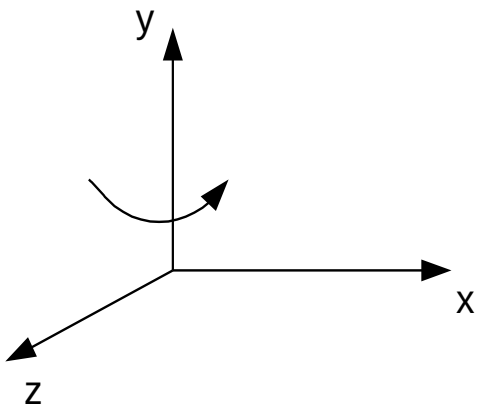
Rotația punctului P(x,y,z) în jurul axei Ox cu unghiul θ:

$$x' = x$$
$$y' = y\cos\theta - z\sin\theta$$
$$z' = y\sin\theta + z\cos\theta$$

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

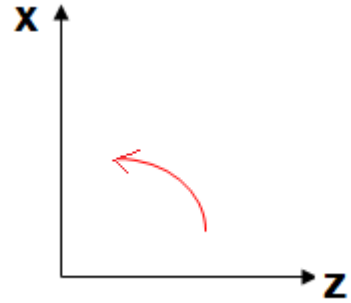
$R_x(\theta)$

Rotaţia pozitivă în jurul axei oy



Rotația în planul ZOX

$$z' = z\cos\theta - x\sin\theta$$
$$x' = x\cos\theta + z\sin\theta$$
$$y' = y = 0$$



Rotația punctului P(x,y,z) în jurul axei Oy cu unghiul θ:

$$x' = x\cos\theta + z\sin\theta$$
$$y' = y$$
$$z' = -x\sin\theta + z\cos\theta$$

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$R_y(\theta)$

Alte transformări geometrice 3D elementare

Oglindirea față de un plan al sistemului de coordonate

$$[O_z] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Forfecarea față de origine

- **De-a lungul axei OZ: modifică x și y proporțional cu z**

  x' = x + Fx*z

  y' = y + Fy*z

  z' = z



- **Analog pentru forfecarea de-a lungul axei OX si a axei OY**

- **Cazul general (forfecarea pe toate cele 3 axe):**

  x' = x + y*d + z*g

  y' = x*b + y + z*i

  z' = x*c + y*f + z

$$[x' \; y' \; z' \; 1] = [x \; y \; z \; 1] * \begin{vmatrix} 1 & b & c & 0 \\ d & 1 & f & 0 \\ g & i & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Transformări geometrice 3D compuse

Exemple:

1. **Scalarea faţă de un punct oarecare, F(xf, yf, zf):**

   T(xf,yf,zf)*S(sx,sy,sz)*T(-xf,-yf,-zf)

2. **Rotația în jurul unei axe paralele cu o axa a sistemului de coordonate**:
   1. Translația obiectului astfel încât axa de rotație să se suprapună peste o axa a sistemului de coordonate.
   2. Rotația obiectului în jurul axei sistemului de coordonate.
   3. Translația inversă celei din pasul 1.

   Rezultă:

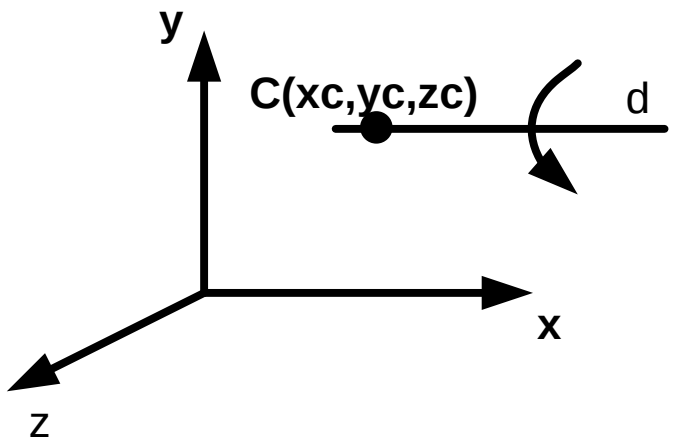
   M = T(xd, yd, zd) * R(u) * T(-xd, -yd, -zd)

   u : unghiul de rotație

   xd, yd, zd : un punct de pe axa de rotație

$$\begin{bmatrix} \mathbf{x'} \\ \mathbf{y'} \\ \mathbf{z'} \\ \mathbf{1} \end{bmatrix} = M * \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ \mathbf{z} \\ \mathbf{1} \end{bmatrix}$$

Rotaţia în jurul unei drepte paralele cu axa OX



$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = T(xc, yc, zc)\, R_X(u)\, T(-xc, -yc, -zc) \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Rotația în jurul unei drepte oarecare (1)

Se consideră dreapta dată printr-un punct (xd, yd, zd) și direcția sa, D[a, b, c].

1. Translație prin care dreapta va trece prin origine: T(-xd, -yd, -zd)
2. Alinierea dreptei cu una dintre axele principale, de ex. cu axa OZ:

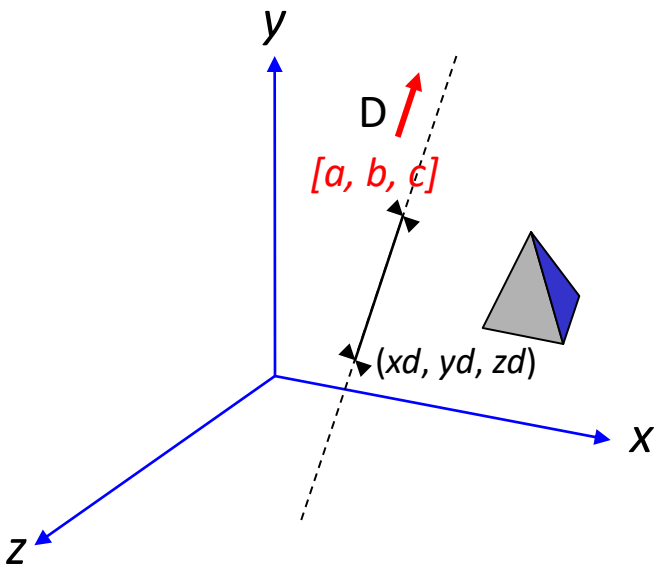
   2.1. Rotație în jurul axei OX, cu un unghi ux, prin care dreapta ajunge în planul XOZ: Rox(ux)

   2.2. Rotație în jurul axei OY, cu un unghi uy, prin care dreapta se suprapune pe axa OZ: Roy(uy)
3. Rotația cu unghiul dat, u, în jurul axei pe care s-a aliniat dreapta: rotație în jurul axei OZ : Roz(u)
4. Transformarea inversă celei din pasul 2:

   4.1. Rotație în jurul axei OY, cu unghiul –uy: Roy(-uy)

   4.2. Rotație în jurul axei OX, cu unghiul –ux: Rox(-ux)
5. Transformarea inversă celei de la pasul 1: T(xd, yd, zd)

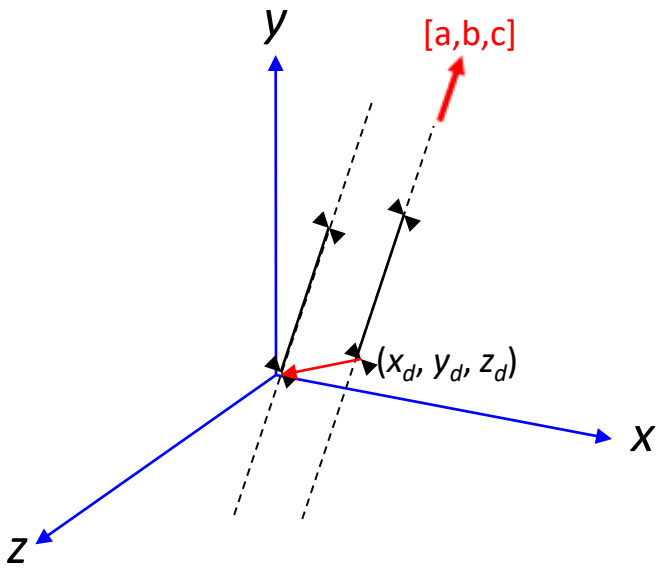
Rotația în jurul unei drepte oarecare (2)



1. Translație astfel încât dreapta să treacă prin origine
2. Aliniere dreaptă cu una din axe
3. Rotația propriu-zisă
4. Transformarea inversă de la punctul 2
5. Transformarea inversă de la punctul 1

Rotația în jurul unei drepte oarecare (3)

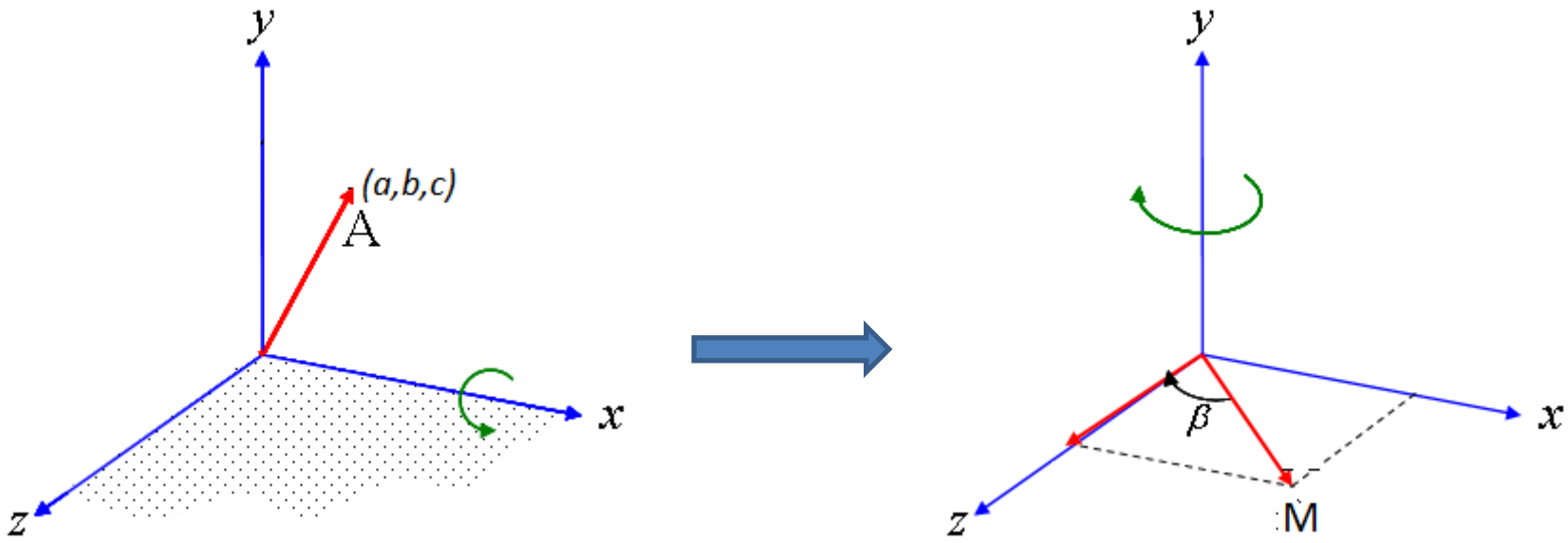
1. Translaţie



$$T = \begin{bmatrix} 1 & 0 & 0 & -xd \\ 0 & 1 & 0 & -yd \\ 0 & 0 & 1 & -zd \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotația în jurul unei drepte oarecare (4)

2. Aliniere *D* cu axa OZ

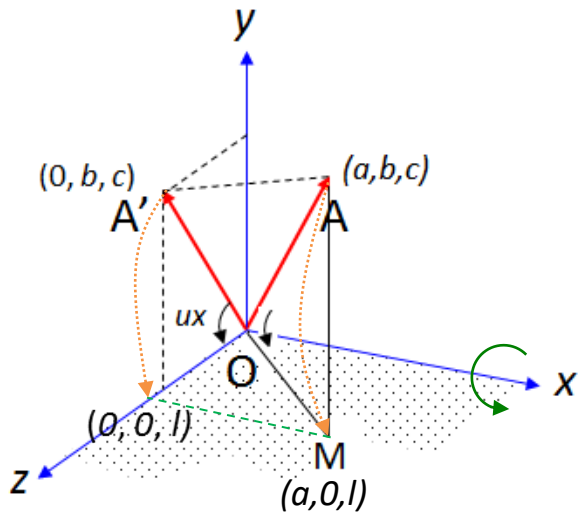


Rotație în jurul axei OX cu un unghi ux, prin care dreapta ajunge în planul XOZ: Rox(ux)

Rotație în jurul axei OY, cu un unghi uy, prin care dreapta se suprapune pe axa OZ: Roy(uy)

Rotația în jurul unei drepte oarecare (5)

2.1. Rotația în jurul axei OX



$R_x(ux)$

A' – proiecția lui A în planul YOZ

Lungime (OA) = L; lungime (OA') = l;

$$L = \sqrt{a^2 + b^2 + c^2}$$

$$l = \sqrt{b^2 + c^2}$$

b = l*sin(ux)
c = l*cos(ux)

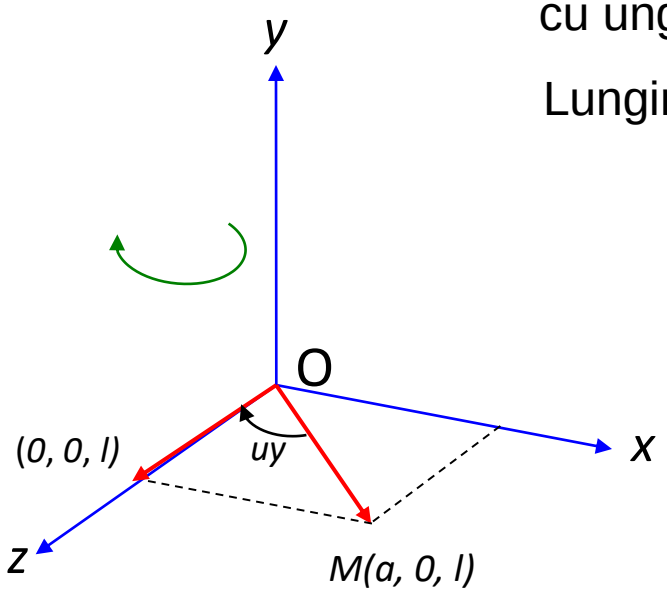
$$\sin(ux) = \frac{b}{l}$$

$$\cos(ux) = \frac{c}{l}$$

Prin rotația lui OA în jurul axei Ox cu unghiul ux, acesta va ajunge în planul XOZ

Rotația în jurul unei drepte oarecare (6)

2.2. Rotația în jurul axei OY



R_y(uy)

M este punctul obținut prin rotația lui A în jurul axei Ox, cu unghiul ux.

Lungime (OM) = Lungime (OA) = L

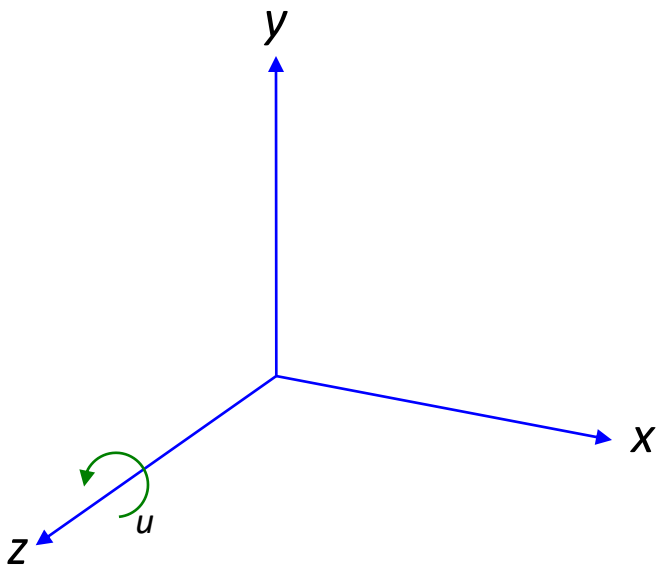
a = L*sin(-uy)

l = L*cos(-uy)

$$\sin(-uy) = a/L -> \sin(uy) = -a/L$$
$$\cos(-uy) = \cos(uy) = l/L$$

Rotația în jurul unei drepte oarecare (7)

3. Rotația în jurul axei OZ cu unghiul dat

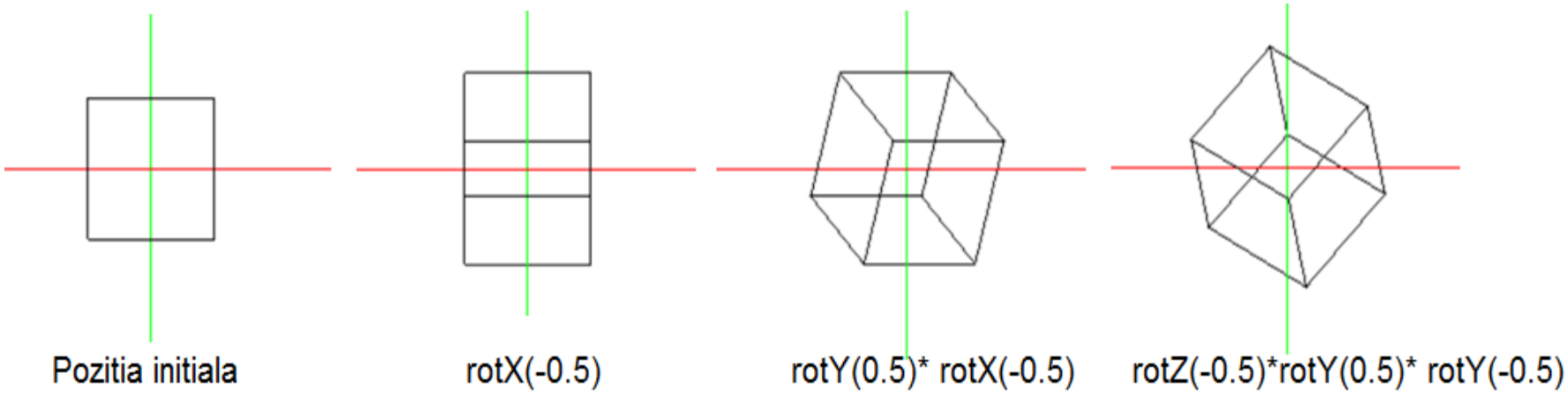


$$R_z(u) = \begin{bmatrix} \cos(u) & -\sin(u) & 0 & 0 \\ \sin(u) & \cos(u) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Exemplu: rotația unui cub (1)

Cub centrat în originea sistemului de coordonate XYZ

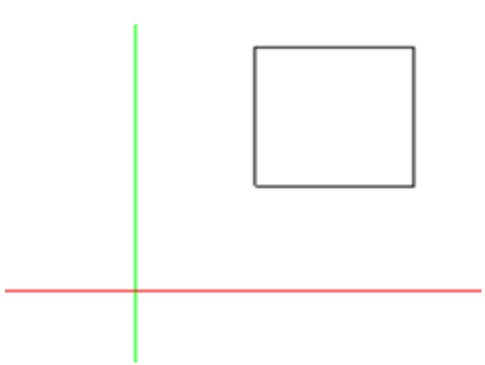
Proiecție ortografică în XOY: (x,y,z) → (x' = x; y' = y)



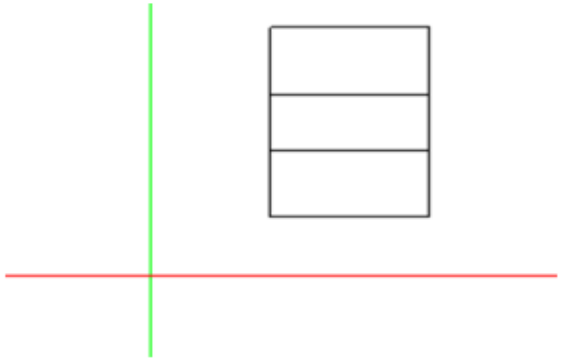
Exemplu: rotația unui cub (2)

Cub cu centrul în (100, 100, 0)

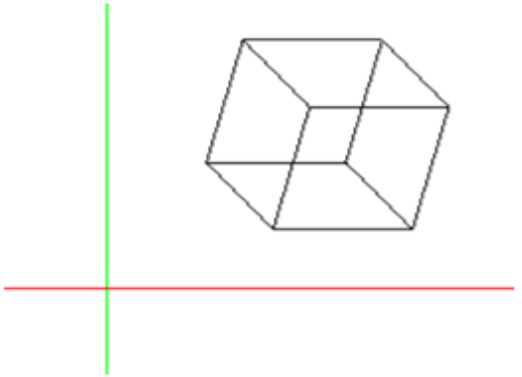
Proiecție ortografică în XOY



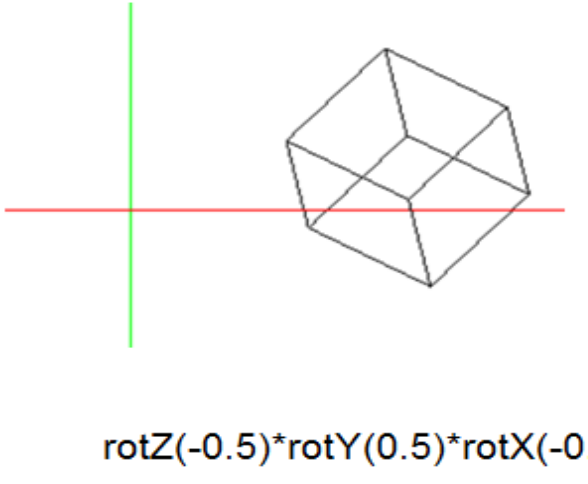
Pozitia initiala



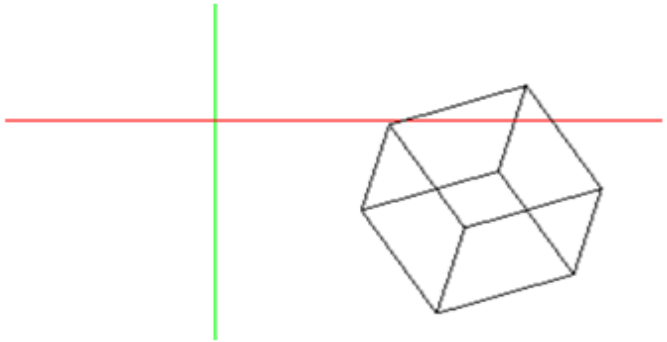
rotX(-0.5)



rotY(0.5)*rotX(-0.5)



rotZ(-0.5)*rotY(0.5)*rotX(-0.5)



rotZ(-0.5)*rotZ(-0.5)*rotY(0.5)*rotX(-0.5)

Proiecții

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

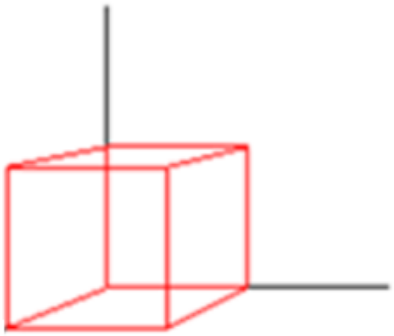
Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

Proiecții

❖ Transformări dintr-un spațiu n-dimensional într-un spațiu k-dimensional, k<n.

Proiecții $R^3 \rightarrow R^2$

- Transformări din spațiul tri-dimensional într-un spațiu bi-dimensional
- Se aplică vârfurilor obiectelor
- Nu modifică legaturile dintre vârfuri (topologia)

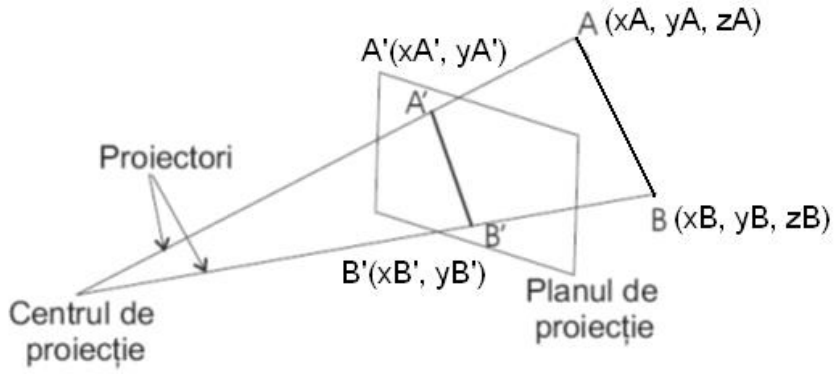


Exemplu:

- Proiecția unui cub în planul XOY
- Legăturile dintre vârfurile 2D sunt aceleași cu legăturile dintre vârfurile 3D proiectate

Proiecții $R^3 \rightarrow R^2$

- Proiecția unui vârf 3D într-un plan 2D este punctul de intersecție dintre plan si proiectorul care pleacă din ***centrul de proiecție*** și trece prin vârf.



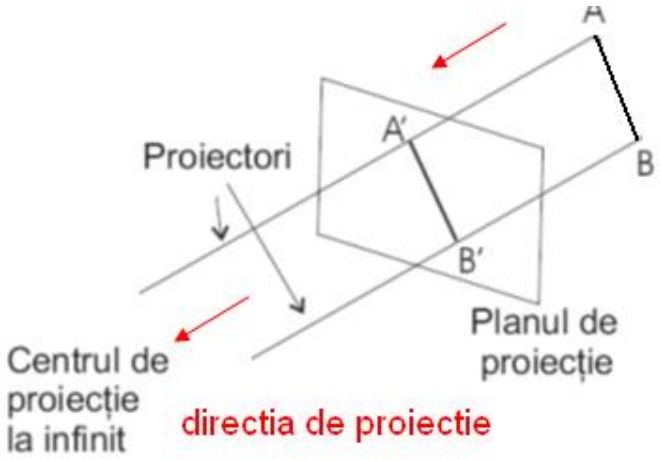
Segmentul A'-B' este proiecția segmentului A-B în planul de proiecţie XOY (z=0).

Există 2 clase de proiecții:

- Proiecții perspectivă: centrul de proiecție este situat la distanță finită față de planul de proiecție (ca în figura de mai sus); proiectorii sunt drepte convergente în CP.

Proiectii $R^3 \rightarrow R^2$

- Proiecții paralele - centrul de proiecție este la infinit; proiectorii sunt linii paralele care trec prin vârfurile obiectului proiectat și au direcția specificată (vectorul ***direcție de proiecție***)



Proiecții perspectivă(1)

Produc imagini asemănatoare celor obținute cu aparatele de fotografiat.

Caracteristici:

1. Efectul de micșorare.

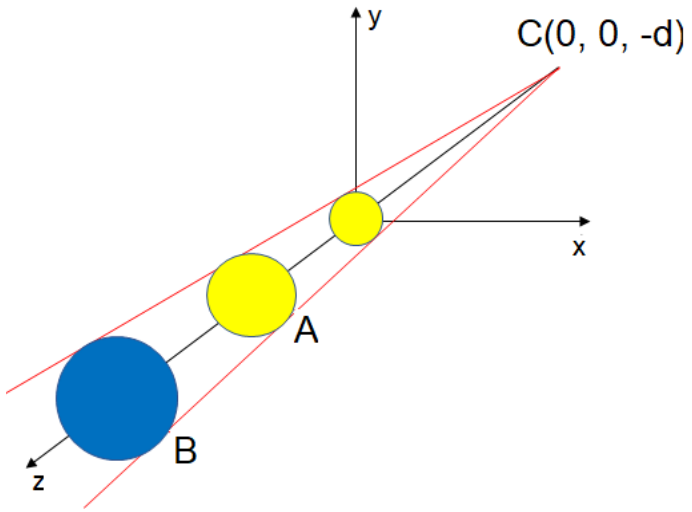
Dimensiunea obiectului în proiecția 2D este invers proporțională cu distanța de la centrul de proiecție la obiect.

Exemplu:

A și B sunt 2 sfere, cu centrele pe axa OZ, care se proiectează în acelaşi disc în planul XOY.

raza sferei B = 2* raza sferei A

Dist(centru_B-C) = 2* Dist(centru_A-C)

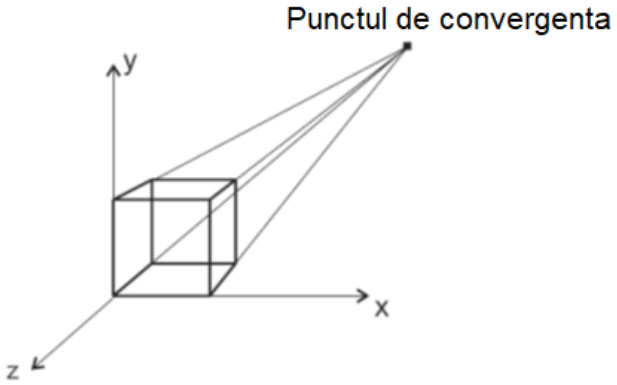


Proiecții perspectivă(2)

2. **Modifică unghiurile dintre dreptele care nu sunt paralele cu planul de proiecție**

Proiectiile liniilor paralele care nu sunt paralele cu planul de proiecție converg către un punct din plan, numit punct de convergență

- Punctul către care converg liniile paralele cu una dintre axele principale: ***punct de convergență principal***



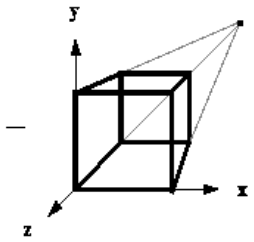
Proiectia perspectivă în planul XOY a unui cub cu feţele paralele cu planele principale

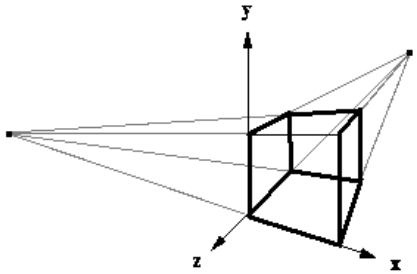


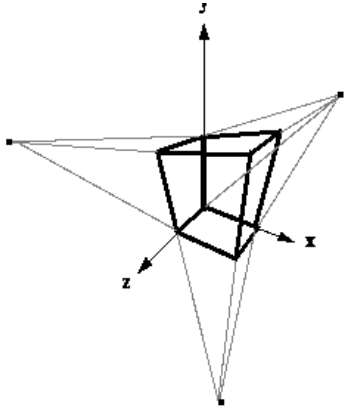
Proiecţii perspectivă(3)

Pot fi efectuate proiecţii cu unul, doua sau trei *puncte de convergenta principale*.

Numărul de puncte de convergenţă principale = numarul de axe principale intersectate de planul de proiecţie.

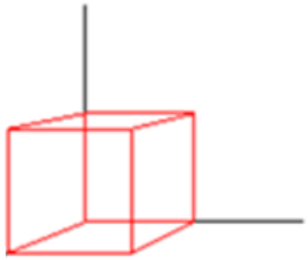




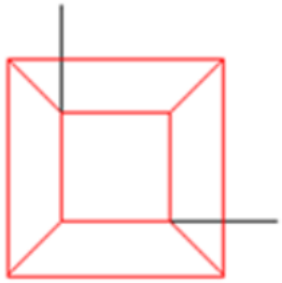


Exemple de proiecții perspectivă

Proiecții ale unui cub cu latura egală cu 1, feţele paralele cu planele principale ale sistemului de coordonate şi coltul de (xmin,ymin,zmin) în originea sistemului de coordonate.

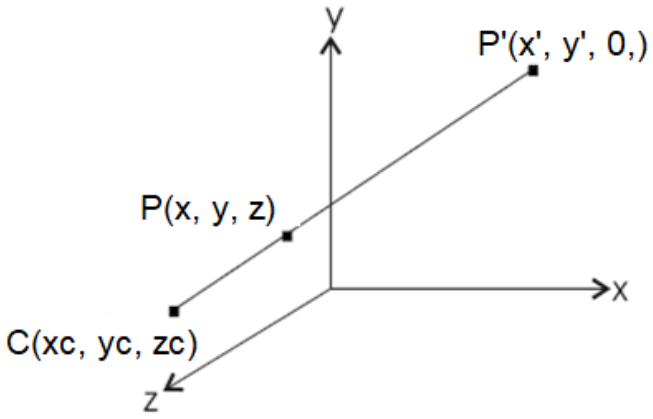


Proiecție perspectivă cu centrul în (5,2,8)



Proiecție perspectivă cu centrul în (0.5,0.5,2)

Proiecția perspectivă în planul XOY (1)



C – centrul de proiecție
P – punctul proiectat
P’ – proiecția lui P în XOY

Ecuațiile parametrice ale proiectorului care trece prin P:

x = xc + (x – xc)*t
y = yc + (y – yc)*t 0<= t <= 1, pentru punctele aflate pe segmentul C-P
z = zc + (z – zc)*t

0 = zc + (z –zc) *t’ → t’ = – zc / (z – zc) → **x’ = (xc*z – zc*x)/(z – zc)**

y’ = (yc*z – zc*y)/(z – zc)

Proiectia perspectivă în planul XOY (2)

Reprezentarea matriceală a transformării de proiecție

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Pperspectiva \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

$$Pperspectiva = \begin{vmatrix} \frac{-zc}{z-zc} & 0 & \frac{xc}{z-zc} & 0 \\ 0 & \frac{-zc}{z-zc} & \frac{yc}{z-zc} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Matricea proiecției depinde de coordonata z a punctului proiectat:

→ nu poate fi folosită pentru toate vârfurile aceluiaşi obiect

$$\begin{vmatrix} xw \\ yw \\ zw \\ w \end{vmatrix} = Pperspectiva \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

$$Pperspectiva = \begin{vmatrix} -zc & 0 & xc & 0 \\ 0 & -zc & yc & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -zc \end{vmatrix}$$

Coordonatele omogene ale punctului P'

xw = -zc*x + xc*z

yw = -zc*y + yc*z

zw = 0

w = z – zc

Coordonatele carteziene ale punctului P'
- împărţirea perspectivă -

x' = xw/w = (-zc*x + xc*z)/(z-zc)

y' = yw/w = (-zc*y + yc*z)/(z-zc)

z' = zw/w = 0

Proiectia perspectivă în planul XOY (3)

Reprezentarea matriceală a transformării de proiecție

$$\begin{vmatrix} xw \\ yw \\ zw \\ w \end{vmatrix} = Pperspectiva \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

$$Pperspectiva = \begin{vmatrix} 1 & 0 & \frac{-xc}{zc} & 0 \\ 0 & 1 & \frac{-yc}{zc} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{-1}{zc} & 1 \end{vmatrix}$$

Coordonatele omogene ale punctului P'

xw = x + (-xc/zc)*z

yw = y + (-yc/zc)*z

zw = 0

w = (-1/zc)*z +1 = -(z – zc)/zc

Coordonatele carteziene ale punctului P'

x' = xw/w = -(x + (-xc/zc)*z)*zc/(z – zc)

= (-zc*x + xc*z)/(z-zc)

y' = yw/w = (-zc*y + yc*z)/(z-zc)

z' = zw/w = 0

Coordonatele omogene ale punctului obtinut prin proiectie perspectiva au w ≠ 1.

Proiectia perspectivă în planul XOY(4)

Pentru conservarea informației de adâncime (necesară pentru eliminarea din imagine a părților nevizibile ale scenei 3D) se folosește matricea :

Coordonatele omogene ale punctului P'

xw = -zc*x + xc*z → x' = (-zc*x + xc*z)/(z-zc)

yw = -zc*y + yc*z → y' = (-zc*y + yc*z)/(z-zc)

zw = z → z' = z/(z – zc)

w = z – zc

$$Pperspectiva = \begin{vmatrix} -zc & 0 & xc & 0 \\ 0 & -zc & yc & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & -zc \end{vmatrix}$$

Proiecția perspectivă standard

Xc = 0, Yc = 0, Zc = - d

xw = x → x' = x*d/(z+d)

yw = y → y'= y*d/(z+d)

zw = z → z' = z/(z+d)

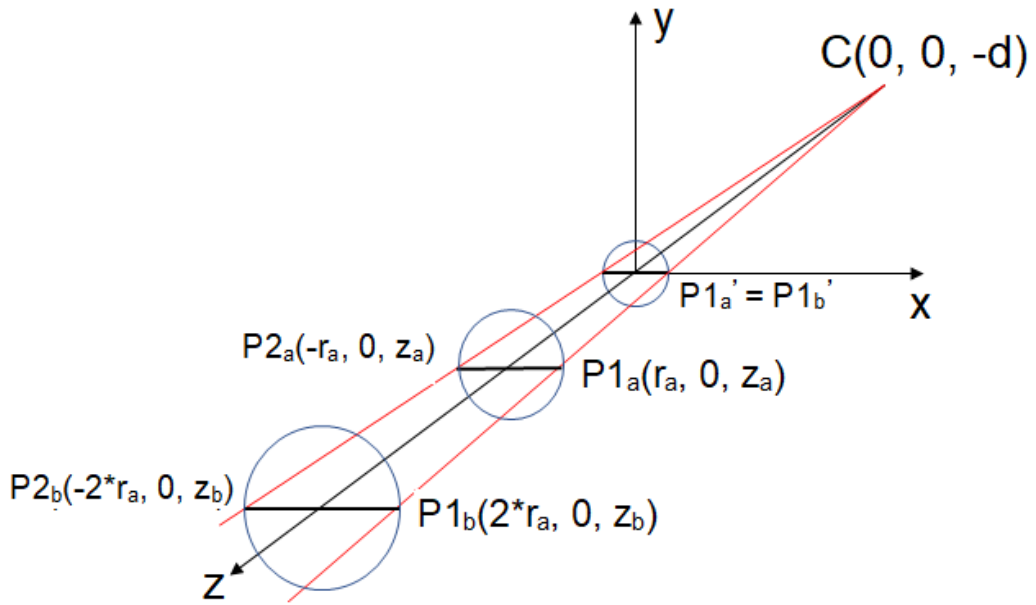
w = z/d + 1 = (z+d)/d

$$Ppers_standard = \begin{vmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \frac{1}{d} & 1 \end{vmatrix}$$

Proiecția perspectivă în planul XOY - exemplu

A şi B sunt 2 sfere cu centrele pe axa OZ, A de rază r_a şi B de rază $2*r_a$.

Distanța de la centrul sferei B la centrul de proiecție C este de 2 ori mai mare decât distanța de la centrul sferei A la C.



Proiecția perspectivă standard

xw = x → x' = x*d/(z+d)

yw = y → y'= y*d/(z+d)

zw = z → z' = z/(z+d)

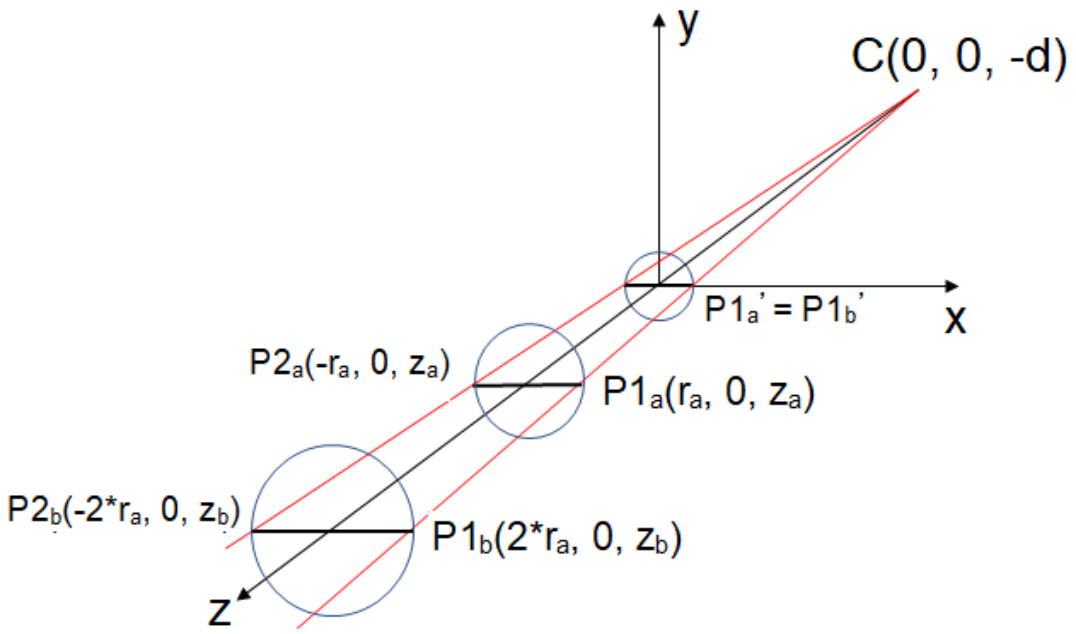
w = z/d + 1 = (z+d)/d

$x_P1_a' = r_a*d/(z_a + d)$; $y_P1_a' = 0 = y_P1_b'$

$x_P1_b' = 2*r_a*d/(z_b + d) = 2*r_a*d/(2*(z_a + d)) = r_a*d/(z_a + d) = x_P1_a'$

Analog se demonstrează că $P2_b' = P2_a'$

Utilizarea coordonatei de adâncime în eliminarea suprafețelor nevizibile din imagini



Proiecția perspectivă standard

zw = z → z' = z/(z+d)

$z_P1_a' = z_a / (z_a + d)$

$z_P1_b' = z_b / (z_b + d)$

Considerăm:

d = 50; z_a =100;

$z_b = 2(z_a + d) - d = 250$

$z_P1_a' = 100/150 = 0.666$;

$z_P1_b' = 250/350 = 0.714$

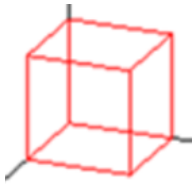
Daca 2 puncte aparținând la 2 suprafețe 3D se proiectează în același punct, se va reda în imagine culoarea punctului mai apropiat de observator (centrul de proiecție):

$z_P1_a' < z_P1_b'$ → discul în care se proiectează cele 2 sfere va fi afișat în culoarea sferei A

Proiecții paralele(1)

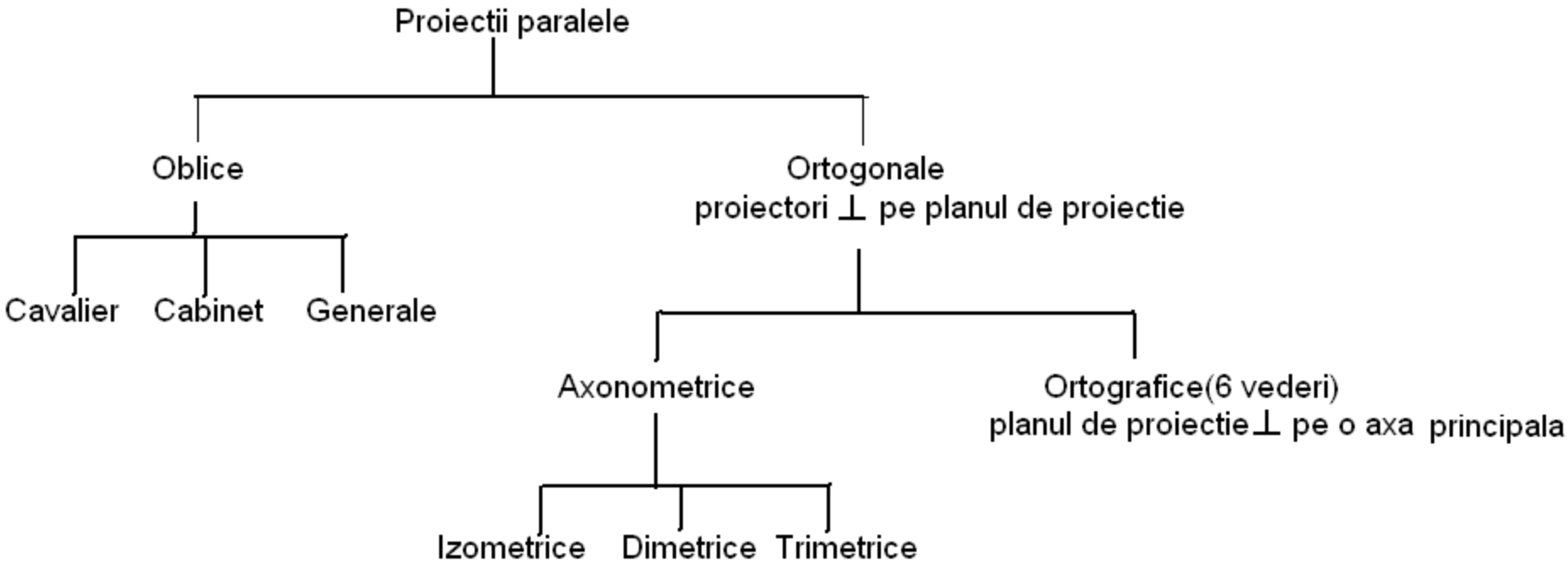
Caracteristici

- **Proiectorii sunt drepte paralele de direcție dată: direcția de proiecție**
- **Sunt transformări afine (conservă paralelismul liniilor)**
- **Unghiurile se conservă doar pentru fețele obiectului paralele cu planul de proiecție.**



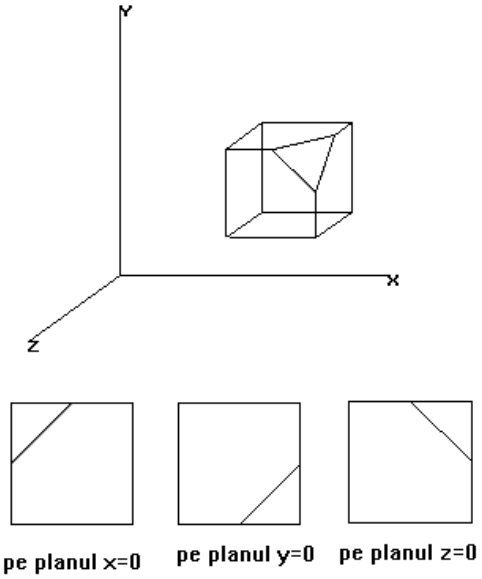
- Clasificare dupa unghiul dintre proiectori și planul de proiecție:
  - *Proiecții **ortogonale**:* proiectorii sunt perpendiculari pe planul de proiecție.
    - Ortografice: planul de proiecție este II cu un plan principal (⊥ pe o axă principală)
    - Axonometrice: planul de proiecție este oarecare
  - *Proiecții **oblice**:* proiectorii nu sunt perpendiculari pe planul de proiecție – cazuri particulare:
    - proiecții Cavalier (unghiul dintre proiectori și plan = 45 grade)
    - proiecții Cabinet (unghiul dintre proiectori și plan = 63.43 grade)

Proiecţii paralele(2) - clasificare



Proiecții paralele(3)

1. Proiecții ortogonale - ortografice



Proiectii ortografice ale unui cub cu un colt taiat

Proiecții în planele sistemului de coordonate (planul de proiecție ⊥ pe o axă principală) → numite şi **vederi:**

- în YOZ: vederea din stânga, vederea din dreapta
- în XOZ : vederea de sus, vederea de jos
- în XOY : vederea din față, vederea din spate.

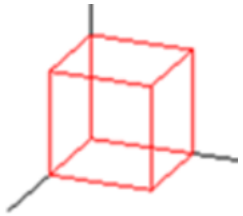
- Conservă lungimile laturilor și unghiurile dintre laturi
- Utile în desenul tehnic

Proiecții paralele(4)

2. Proiecții ortogonale - axonometrice

- Planul de proiecție nu este perpendicular pe nici una din axele sistemului de coordonate
- Redau mai multe fețe ale obiectului proiectat (ca și cele perspectivă): alegând planul de proiecție se poate controla scalarea laturilor.





- În funcție de unghiurile pe care planul de proiectie le face cu axele sistemului de coordonate:

- proiecții *izometrice*, planul face unghiuri egale cu toate cele trei axe → prin proiecție, laturile sunt scalate cu factori egali pe cele 3 axe: sx=sy=sz

- proiecții *dimetrice*, planul face unghiuri egale cu două dintre axe → laturile sunt scalate cu factori de scalare egali pe 2 axe: sx=sy sau sx=sz sau sy=sz

- proiecții *trimetrice*, unghiurile dintre cele trei axe și plan sunt diferite → factorii de scalare ai laturilor pe cele 3 axe sunt diferiti: sx # sy # sz

Proiecții paralele(5)

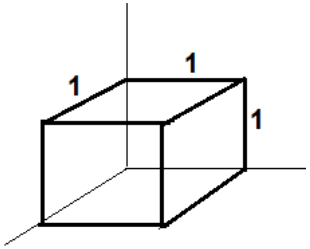
3. Proiecții oblice



- Planul de proiecție este perpendicular pe o axa principală.
- Direcția de proiecție nu este perpendiculară pe planul de proiecție.
- Fețele II cu planul de proiecție se proiectează fără modificarea unghiurilor și a mărimii laturilor
- Alegând direcția de proiecție se obțin cazurile particulare:

❑ **Proiecții Cavalier**

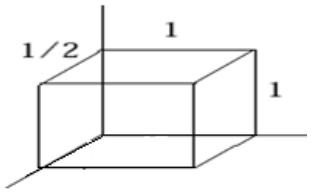
(unghiul proiectorilor cu planul de proiecție = 45 grade)



- Lungimea proiecției unei laturi perpendiculare pe plan este egală cu lungimea laturii 3D (conservă lungimea laturilor perpendiculare pe planul de proiecție)

❑ **Proiecții Cabinet**

(unghiul proiectorilor cu planul de proiectie = 63.43 grade)

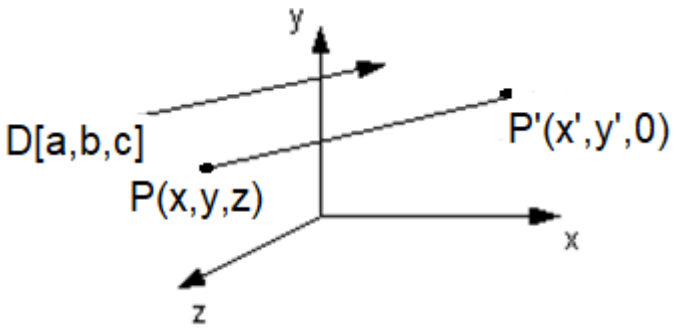


- Lungimea proiecției unei laturi perpendiculare pe plan este egală cu jumătate din lungimea laturii 3D

Proiecţii paralele în planul XOY(6)

Fie punctul P(x,y,z) şi direcţia de proiectie D[a,b,c].

Proiecția lui P în XOY este punctul P’ (x’,y’,0).



Din condiția D || PP’, rezultă:
PP’ = s*D, unde s este un scalar oarecare
x’ – x = s*a
y’ – y = s*b
0 – z = s*c → s = -z/c → x’ = x – (a/c)*z
y’ = y – (b/c)*z

Proiecții paralele în planul XOY(7)

Reprezentarea matriceală a proiecțiilor paralele:

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Pparalela \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

$$Pparalela = \begin{vmatrix} 1 & 0 & \frac{-a}{c} & 0 \\ 0 & 1 & \frac{-b}{c} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Ca și în cazul proiecțiilor perspectivă, pentru conservarea informatiei de adancime se foloseste matricea →

$$Pparalela = \begin{vmatrix} 1 & 0 & \frac{-a}{c} & 0 \\ 0 & 1 & \frac{-b}{c} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Cazuri particulare

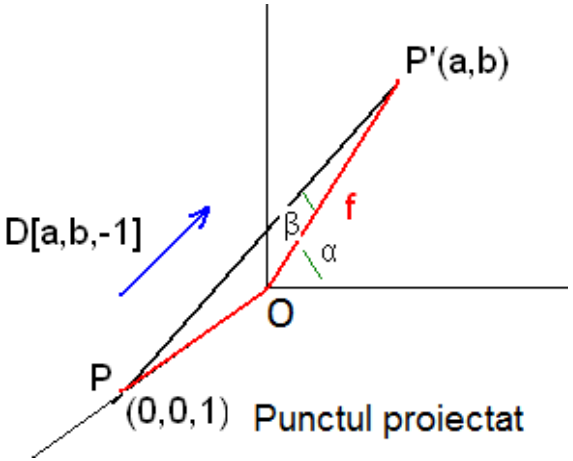
1. Proiecţia ortografică în XOY (proiectorii perpendiculari pe XOY):

D[a, b, c]
a= 0, b = 0 →

$$P_{O\text{-}XOY} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Proiecții paralele în planul XOY(8)

2. Proiecții oblice:



c = -1 → $P_{OBl} = \begin{bmatrix} 1 & 0 & a & 0 \\ 0 & 1 & b & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

$P_{OBl} = \begin{bmatrix} 1 & 0 & f*\cos(\alpha) & 0 \\ 0 & 1 & f*\sin(\alpha) & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

OP de lungime 1 se proiectează în OP' de lungime f: → f este factorul de scalare al laturilor perpendiculare pe XOY (OP este perpendicular pe XOY).

$tg(\beta) = 1/f$

f = 0 → $\beta = 90^0$: proiectie ortografica

f = 1 → $\beta = 45^0$: proiectie Cavalier

f = 0.5 → $\beta = 63.43^0$: proiectie Cabinet

α este un parametru liber

in mod uzual, $\alpha = 30^0$ sau 45^0

Exemple de proiecții oblice





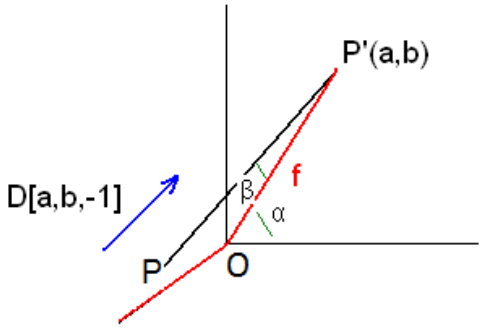
Proiecție Cavalier: f=1, beta=45, alfa=45

Direcția de proiecție: [cos(45),sin(45),-1]

Proiecție Cabinet: f=0.5, beta=63.43, alfa=45

Direcția de proiecție:

[0.5*cos(45),0.5*sin(45),-1]



Proiecții paralele în planul XOY(9)

3. Proiecții axonometrice (1)

2 posibilități de exprimare matematică:

- Proiecție ortogonală într-un plan oarecare

  - obiect fix in spațiu, alegere plan astfel încât să obținem cele 3 cazuri (izometrice, dimetrice, generale):

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Pax(N,R) \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

- Transformare geometrică a obiectului (TA) urmată de proiecție ortografică în XOY (Po-xoy)– cu efect echivalent

  – plan de proiecție fix (XOY), poziționare obiect in spatiu:

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Po\text{-}xoy * TA \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

TA constă din rotații în jurul axelor OX si OY. Particularizând unghiurile de rotație obținem cazurile de proiecții axonometrice: izometrice, dimetrice, generale.

Proiecții paralele în planul XOY(10)

Proiecții axonometrice (2)

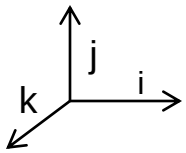
Exemplu:

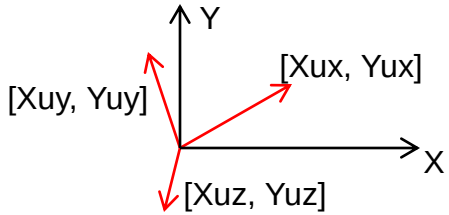
TA = Rx(ux) * Ry(uy) PA = Po-xoy* TA =

$$\begin{bmatrix} \cos(uy) & 0 & \sin(uy) & 0 \\ \sin(ux)^*\sin(uy) & \cos(ux) & -\cos(uy)^*\sin(ux) & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Se aplică transformarea PA versorilor axelor principale:

$$\begin{bmatrix} Xux & Xuy & Xuz \\ Yux & Yuy & Yuz \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} = PA \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$





Factorii de scalare a laturilor pe cele 3 axe sunt lungimile proiecțiilor celor trei vectori unitate:

$$sx = (Xux^2 + Yux^2)^{0.5}$$

$$sy = (Xuy^2 + Yuy^2)^{0.5}$$

$$sz = (Xuz^2 + Yuz^2)^{0.5}$$

Proiecții paralele în planul XOY(11)

Proiecții axonometrice (3)

Se deduc unghiurile de rotație în jurul axelor pentru obținerea celor 2 cazuri particulare de proiecții axonometrice: izometrice și dimetrice.

1. **Proiecţii izometrice:**

- Se impune condiția: sx = sy = sz
- Rezultă: ux = +/- 35.26 grade ; uy = +/- 45 grade

2. **Proiecţii dimetrice:** doi dintre factorii de scalare sunt egali

- Daca se impune condiția: sx = sy, rezultă:

$$ux = \arcsin(+/- sz/\sqrt{2}), \quad uy = \arcsin(+/- sz/\sqrt{(2 - sz^2)})$$

- sz poate fi ales intre 0 si 1;
- pentru fiecare valoare a lui sz exista 4 proiecții dimetrice; de exemplu:
- pentru sz = 0.5 : ux = +/- 20.705 grade, uy = +/- 22.208 grade

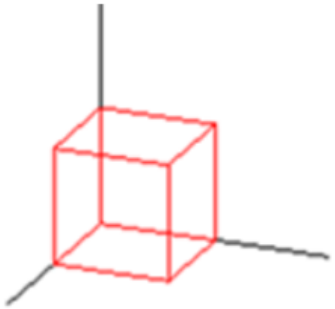
Exemple de proiecții axonometrice



Proiecție izometrică:

P_ort_XOY * R_Ox(35.26)* R_Oy(45)

sx = sy = sz



Proiecție dimetrică:

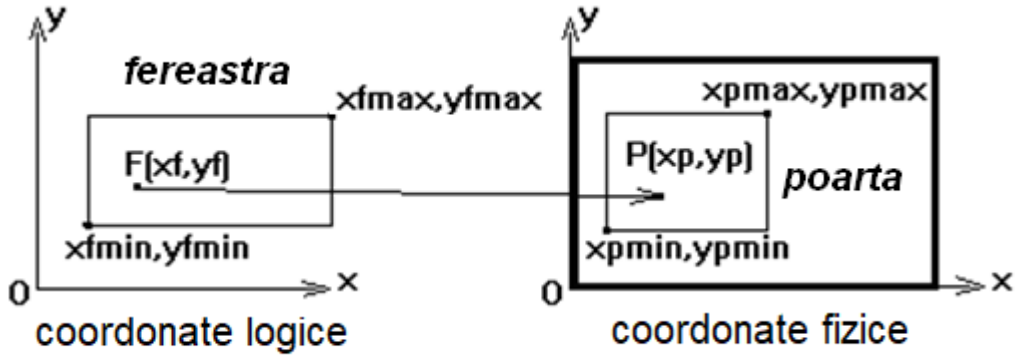
P_ort_XOY * R_Ox(20.705)* R_Oy(22.208)

sx = sy # sz

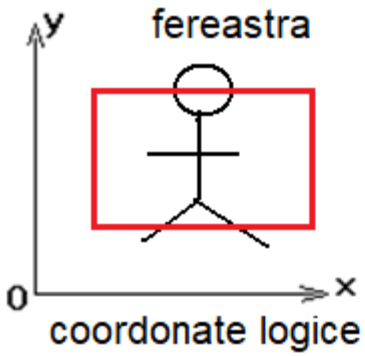
Implementarea proiecțiilor într-o aplicație OpenGL (1)

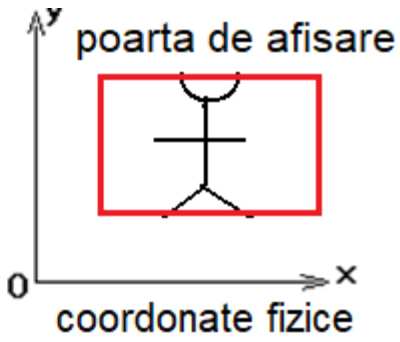
- Proiecția vârfurilor primitivelor 3D este inclusă în **transformarea de vizualizare 3D.**

Analogie între transformarea de vizualizare 2D și transformarea de vizualizare 3D



- Desenul este descris în coordonate logice.
- Coordonatele logice sunt transformate în coordonate fizice folosind matricea transformării de vizualizare 2D

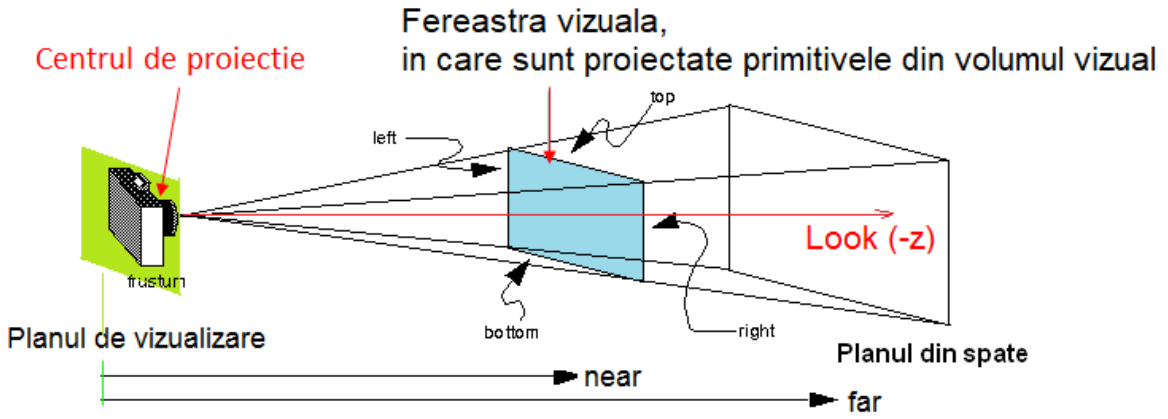




- Părțile din desenul descris în coordonate logice care nu sunt incluse în fereastra sunt decupate (nu sunt afișate în poarta de afișare)

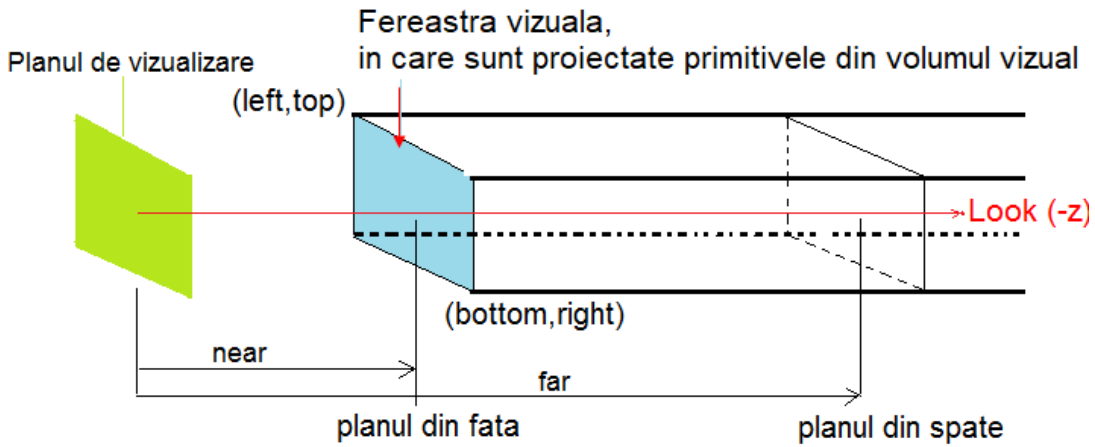
Implementarea proiecțiilor într-o aplicație OpenGL (2)

- ❖ În OpenGL pot fi specificate 2 tipuri de proiecții:
- Proiecția perspectivă
- Proiecția ortografică
- ❖ Obiectele 3D sunt reprezentate în coordonate logice (sistemul de coordonate carteziene 3D).
- ❖ Coordonatele logice sunt transformate în coordonate fizice prin transformarea de vizualizare 3D care include o transformare de proiecție.
- ❖ Părțile din scena 3D, descrisă în coordonate logice, care nu sunt incluse în volumul vizual, sunt decupate la marginile volumului vizual (nu vor fi redate în imagine).



Volumul vizual pentru proiecția perspectivă

Implementarea proiecțiilor într-o aplicație OpenGL (3)



Volumul vizual pentru proiecția ortografică

Cum putem implementa într-un program OpenGL:

- O proiecție axonometrica?

Se foloseste proiecția ortografică asupra varfurilor transformate cu matricea TA

Transformarea de vizualizare din spațiul 3D

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

Transformarea de vizualizare din spațiul 3D

- Are ca scop obținerea unei vederi a modelului unei scene 3D pe o suprafaţă de afişare.
- Poate fi comparată cu transformarea efectuată de o cameră foto asupra lumii reale pentru a se obţine imaginea dintr-o fotografie.
  - Imaginea obţinută depinde de:
    - conținutul din lumea reală
    - poziţia camerei (observatorului)
    - orientarea camerei
    - ceea ce vede ochiul prin vizorul camerei foto (volumul vizual)
- Este o transformare compusă din mai multe transformări, între care şi o proiectie $R^3 \rightarrow R^2$
- Transformările se efectuează asupra vârfurilor obiectelor din scena 3D
- Include şi operaţia de decupare (clipping) a primitivelor grafice la marginea volumului vizual (eliminarea părţilor din scena 3D care sunt în afara volumului vizual).

Transformarea vârfurilor în OpenGL

Spațiul obiect – Object space (Sistemul coordonatelor obiect)

Matricea M — **Transformarea de modelare**

Spațiul scenei 3D – World space (Sistemul coordonatelor globale)

Matricea V — **Transformarea de vizualizare**

Spațiul observator – Eye space (Sistemul coordonatelor observator)

Matricea P — **Transformarea de proiecție (volumul vizual)**

Spatiul de decupare – Clip space (Sistemul coordonatelor de decupare)

Impărţirea perspectivă

Sistemul coordonatelor dispozitiv normalizate

Transformarea în poarta de afişare (viewport)

Sistemul coordonatelor ecran ataşat porţii de afişare

Matricile transformării vârfurilor obiectelor 3D

M – matricea de modelare: diferă de la obiect la obiect

V – matricea de vizualizare: aceeași pentru întreaga scena vizualizată

P – matricea de proiecție: aceeași pentru întreaga scena vizualizată

MV = V · M (matricea “model-view”)

MVP = P · V · M = P · MV (matricea “model-view-projection”)

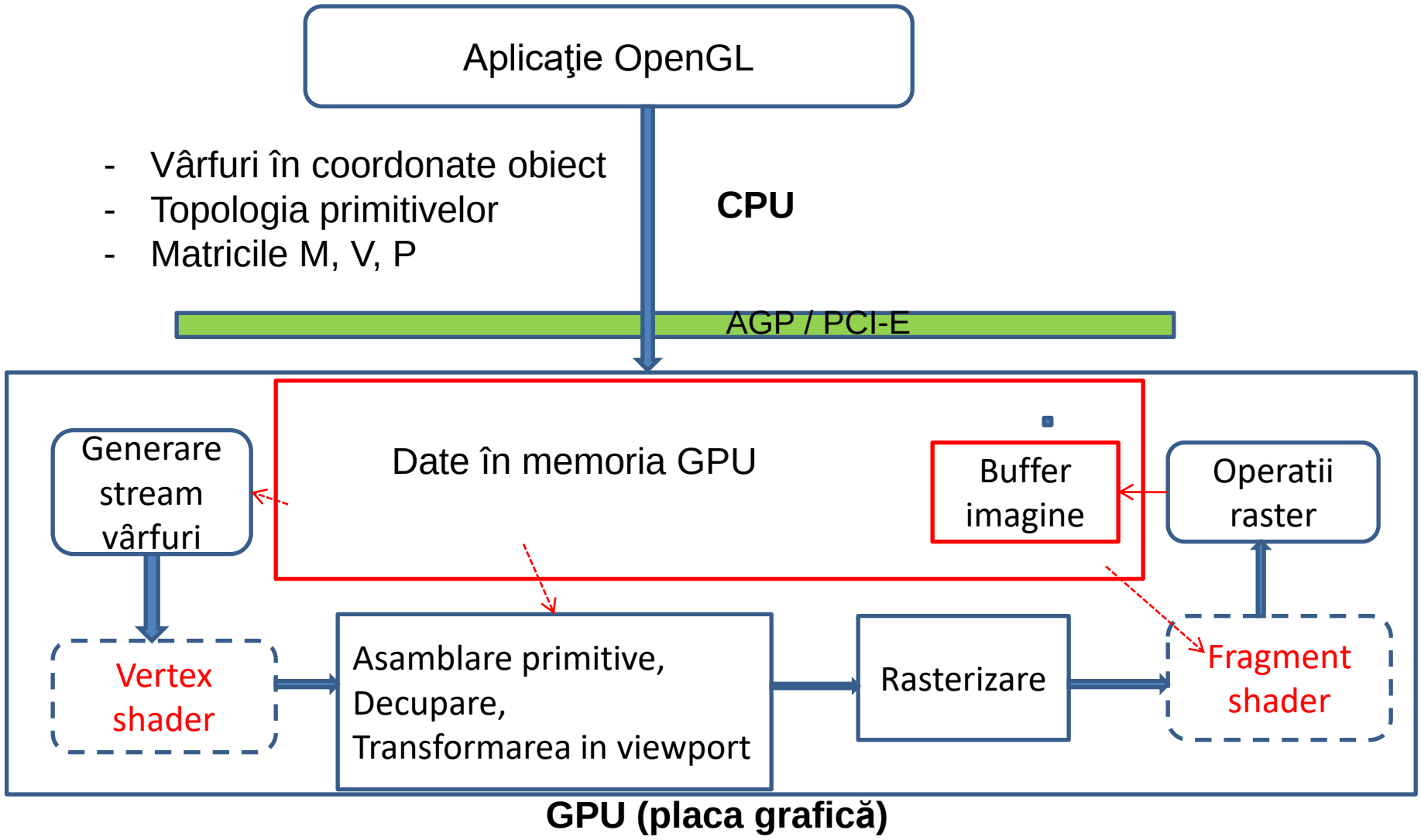
Transformarea vârfurilor din coordonate obiect în coordonate de decupare (clip):

$$[xc, yc, zc]^T = MVP \cdot [xo, yo, zo]^T$$

- Matricile sunt calculate în programul OpenGL folosind functii ale bibliotecii **glm** şi transferate la GPU
- Vârfurile sunt transformate în programul ***Vertex Shader executat pe GPU,*** folosindu-se matricea MVP
- Dupa asamblarea primitivelor, folosindu-se vârfurile transformate şi topologia transmisă la GPU de programul OpenGL, primitivele sunt decupate la frontiera volumului vizual.

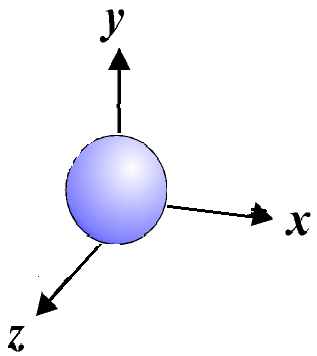
Banda grafică - a V-a generaţie GPU

➢Incepând cu OpenGL 3.0:



Transformarea de modelare (1)

Spațiul obiect (Sistemul coordonatelor obiect)



- În mod uzual, fiecare obiect este definit in propriul sau sistem de coordonate (sistemul coordonatelor locale – coordonate obiect)
- Sistem de coordonate carteziene 3D « dreapta »
- Exemple: cubul, sfera, scheletul unui personaj animat, elemente de mobilier, etc.

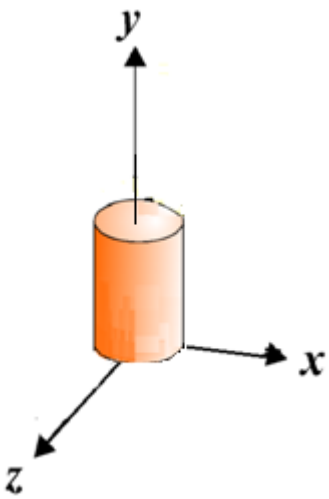
  De regulă, obiectul este definit centrat în originea sistemului de coordonate.
- Avantaje: usurința modelarii şi reutilizarea modelelor obiectelor 3D

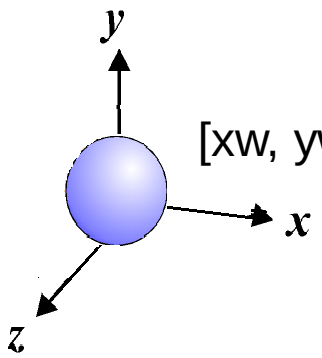
Spațiul scenei 3D -World space (Sistemul coordonatelor globale)

- Spaţiul în care este compusă scena 3D
- Raportat la un sistem de referinţă global – sistem de coordonate carteziene 3D “dreapta”
- Scena redată într-un cadru imagine este reprezentată în sistemul coordonatelor globale

Transformarea de modelare (2)

- Scop: dimensionarea și poziţionarea obiectelor scenei 3D într-un cadru imagine
- Este o transformare geometrică compusă.

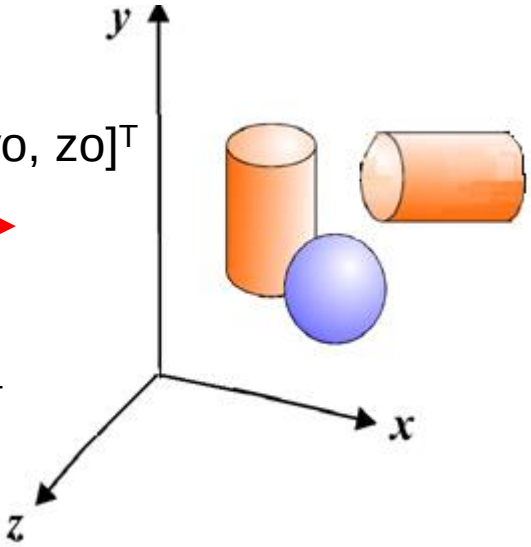




$[xw, yw, zw]^T = M1 \cdot [xo, yo, zo]^T$

$[xw, yw, zw]^T = M2 \cdot [xo, yo, zo]^T$

$[xw, yw, zw]^T = M3 \cdot [xo, yo, zo]^T$



Spațiul scenei 3D

Coordonatele obiect ale vârfurilor obiectelor sunt transformate pentru modelarea scenei 3D

Transformarea de modelare

↓

Rotație, Scalare, Translație

Obiecte definite în sistemul coordonatelor globale

Transformarea de vizualizare

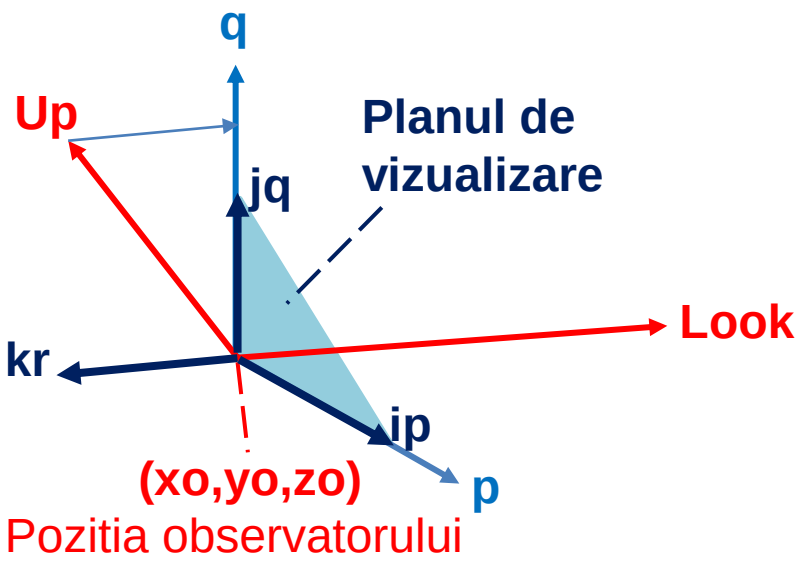
din sistemul de coordonate globale → în sistemul de coordonate observator

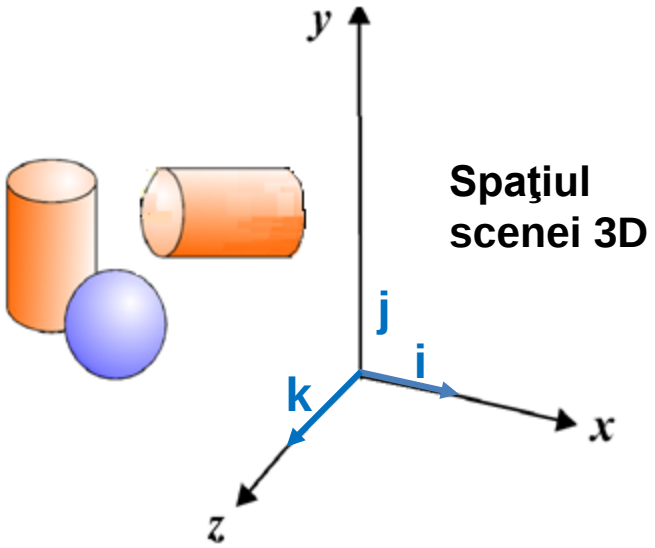
- O imagine este o vedere obținută dintr-un punct al spațiului 3D, similară unei fotografii:
- Depinde de:
  - Poziția camerei (observatorului) în spațiul lumii reale.
  - Orientarea camerei: direcția în care privește observatorul și rotația camerei față de axa sa.

- Spaţiul observator este spaţiul 3D văzut din pozitia observatorului pe direcţia în care priveşte observatorul.
- Obiectele 3D din spațiul observator sunt raportate la un sistem de coordonate cu originea în poziția observatorului (camerei virtuale)
- **Sistemul de coordonate observator este definit prin 3 parametri**:

- Originea sa: **poziția camerei virtuale** în spațiul coordonatelor globale **(xo, yo, zo).**
- **Direcția în care privește observatorul** (spre scena 3D) – **vectorul Look**, reprezintă normala la planul de vizualizare (**N**).
- Direcția “sus” a planului de vizualizare (**vectorul Up**), care determină direcția axei OY a sist. de coordonate observator: reprezinta **rotația camerei în jurul axei privirii (Look)**

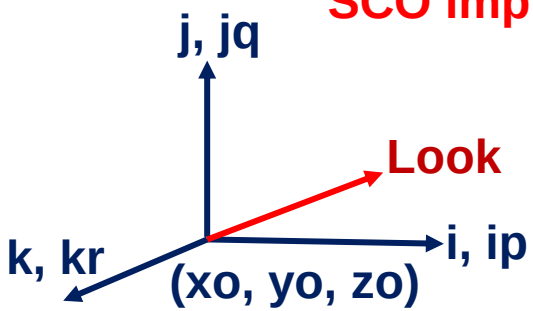
Sistemul de coordonate observator





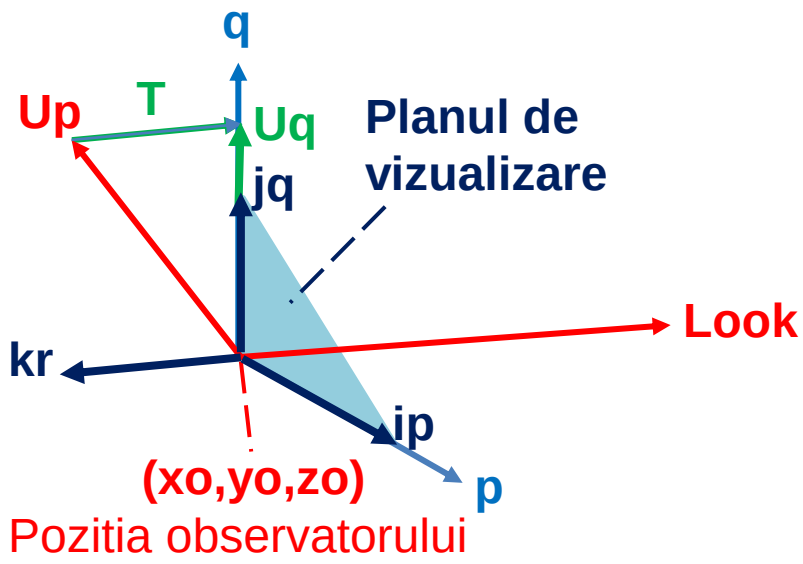
- (xo, yo, zo): originea sistemului de coordonate observator (SCO)
- Look – normala la planul de vizualizare
- axa q: axa verticală a planului de vizualizare – are direcția data de proiecția vectorului UP în planul de vizualizare
- (ip,jq,kr) formează un sistem de coordonate 3D dreapta

SCO implicit



ip, jq, kr: versorii axelor sistemului de coordonate observator coincid cu cei ai sist. coordonatelor globale

Determinarea axelor sistemului de coordonate observator



kr = - Look/|Look|

Uq: proiectia ortogonala a vectorului Up in planul de vizualizare

Uq dă direcţia axei verticale a planului de vizualizare

Uq = Up + T

T || (-kr) → T = s*(-kr) = - s*kr (s un scalar oarecare) → Uq = Up - s*kr

Uq perpendicular pe kr → Uq·kr = 0 → (Up - s*kr)·kr =0 → Up·kr - s*kr·kr = 0 → s = Up·kr

Uq = Up – (Up·kr)·kr

jq = Uq/|Uq| este versorul axei verticale a planului de vizualizare

ip = (jq x kr) / |(jq x kr)| este versorul axei orizontale a planului de vizualizare

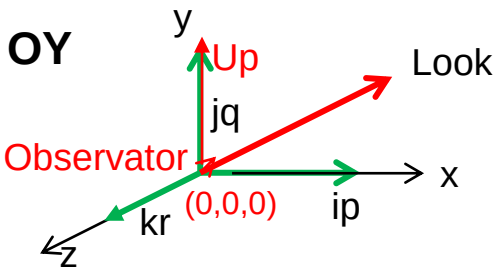
(ip, jq, kr) formează un sistem de coordonate carteziene 3D dreapta

Calculul matricei transformării de vizualizare

Sistemul de coordonate observator implicit în OpenGL

- poziţia observatorului: **originea sistemului de coordonate globale**
- direcţia în care priveşte observatorul: **direcţia negativă a axei OZ**
- direcţia sus a planului de vizualizare: **direcţia pozitivă a axei OY**

Cu aceste valori implicite, transformarea de vizualizare este transformarea identica.



Sistemul de coordonate observator implicit în OpenGL

Pentru calculul matricei transformarii de vizualizare poate fi folosita functia **lookAt:**

glm::mat4 **V** = glm::lookAt (glm::vec3 (xo,yo,zo), glm::vec3 (Px,Py,Pz), glm::vec3 (Upx, Upy, Upz))

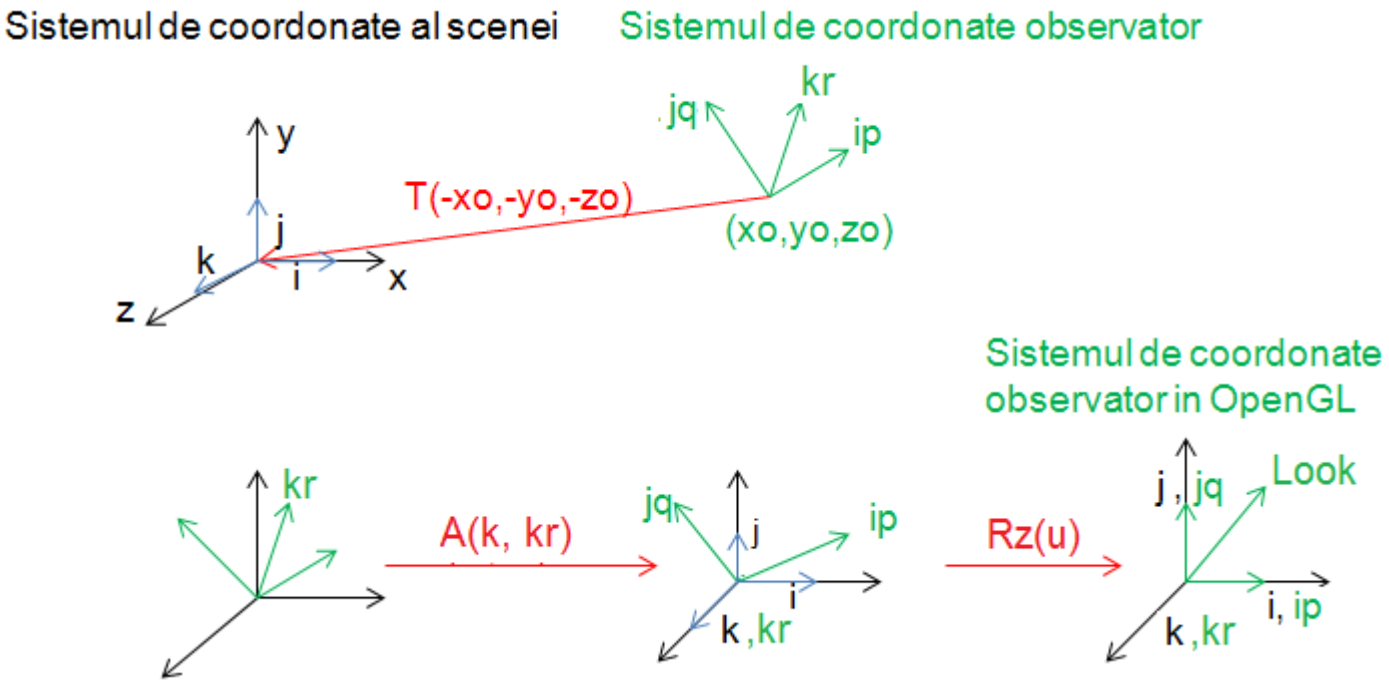
(xo, yo, zo): **poziţia observatorului**

(Px,Py,Pz): punctul din scena 3D către care privește observatorul;

vectorul (Px,Py,Pz) - (xo,yo,zo) reprezintă **vectorul Look**

(Upx, Upy, Upz): **vectorul « sus » al planului de vizualizare (Up)**

Matricea transformării de vizualizare



- A(k,kr) este o transformare prin care se aliniaza axele z ale celor 2 sisteme de coordonate: consta din 2 rotatii – o rotatie in jurul axei X si una in jurul axei Y.
- Prin rotatia in jurul axei z (Rz) se aliniaza celelalte 2 axe.

Matricea transformării de vizualizare: V = Rz(u) * A(k,kr) * T(-xo,-yo,-zo)

Transformarea de modelare și vizualizare

din sistemul de coordonate obiect → în sistemul de coordonate observator

- Transformare compusă (model-view) : MV = V•M

  M – matricea de modelare folosită pentru transformarea unui obiect în spațiul scenei

  V – matricea de vizualizare a scenei

  Transformarea vârfurilor din coordonate obiect în coordonate observator (*eye*):

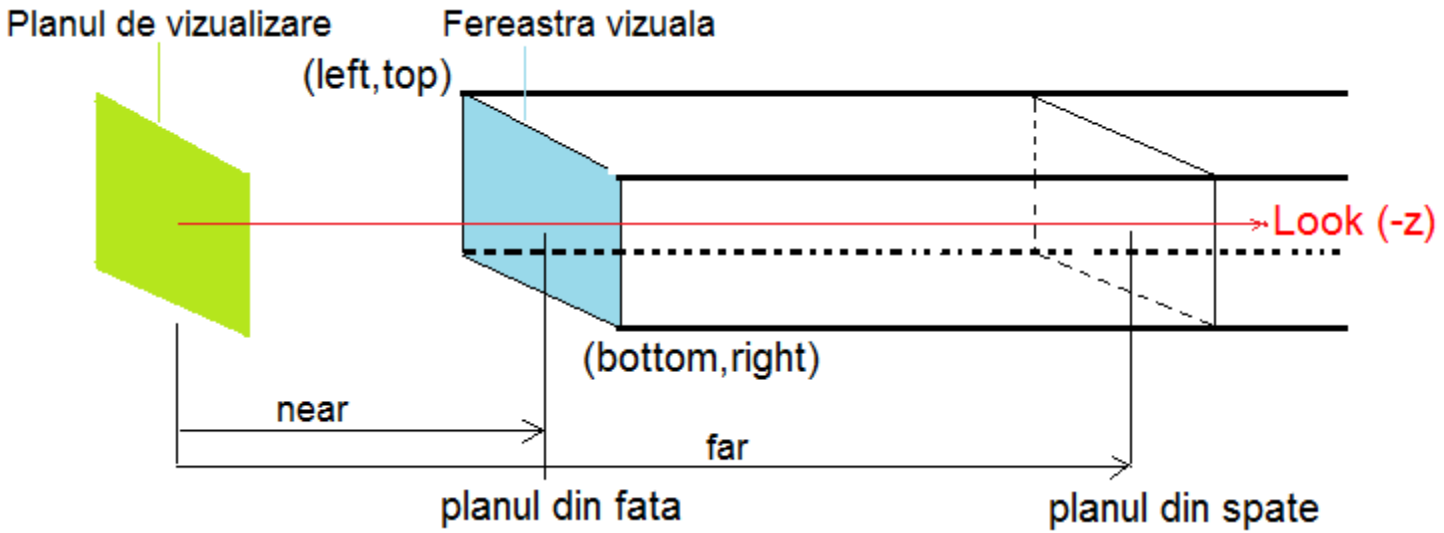
  $[xe\ ye\ \ ze\ we]^T = MV \bullet [xo\ yo\ zo\ wo]^T$

- Pentru calculele de iluminare a obiectelor scenei 3D se folosesc vectori normali atașați vârfurilor obiectelor. Calculele de iluminare se fac, de regulă, în sistemul coordonatelor observator (sau global)

Transformarea de proiecție (1)

Determinată de **volumul vizual** definit de programator

Volumul vizual al proiecției ortografice: paralelipiped dreptunghic cu laturile paralele cu axa z a sistemului de coordonate observator (SCO), delimitat în adancime de planul din față (near) și planul din spate (far) al vederii; near și far sunt distanţe (pozitive) faţă de planul de vizualizare masurate pe axa z negativa a SCO.

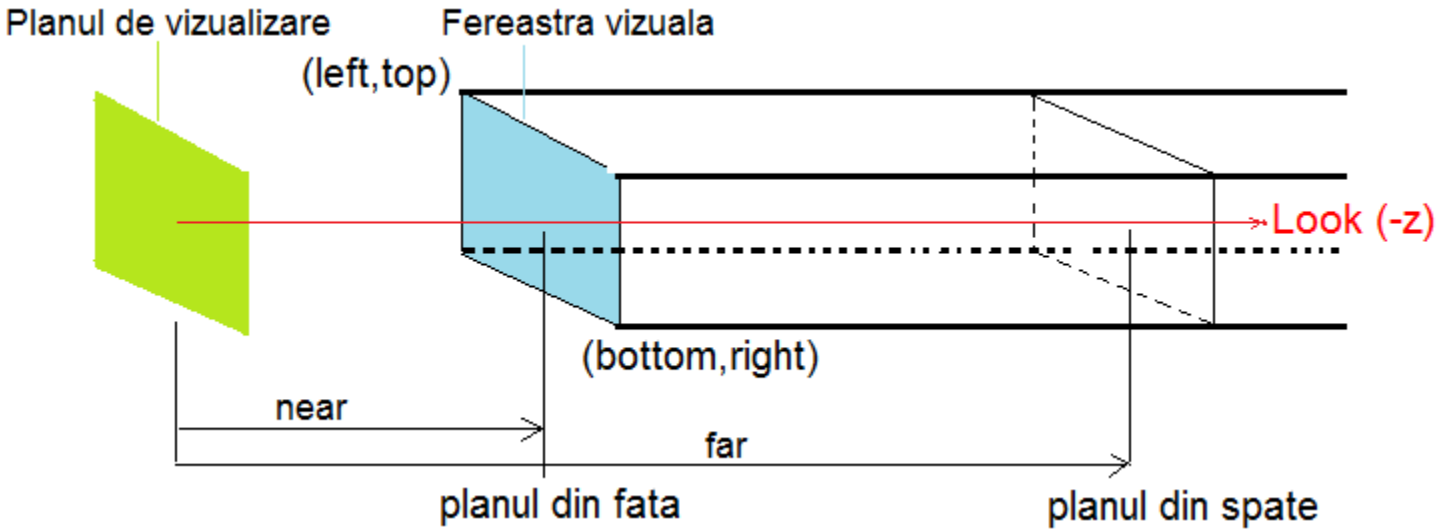


- Coordonatele colțurilor ferestrei vizuale sunt raportate la sistemul de coordonate al planului din faţă, care are axele paralele cu cele ale planului de vizualizare.

Transformarea de proiecție (2)

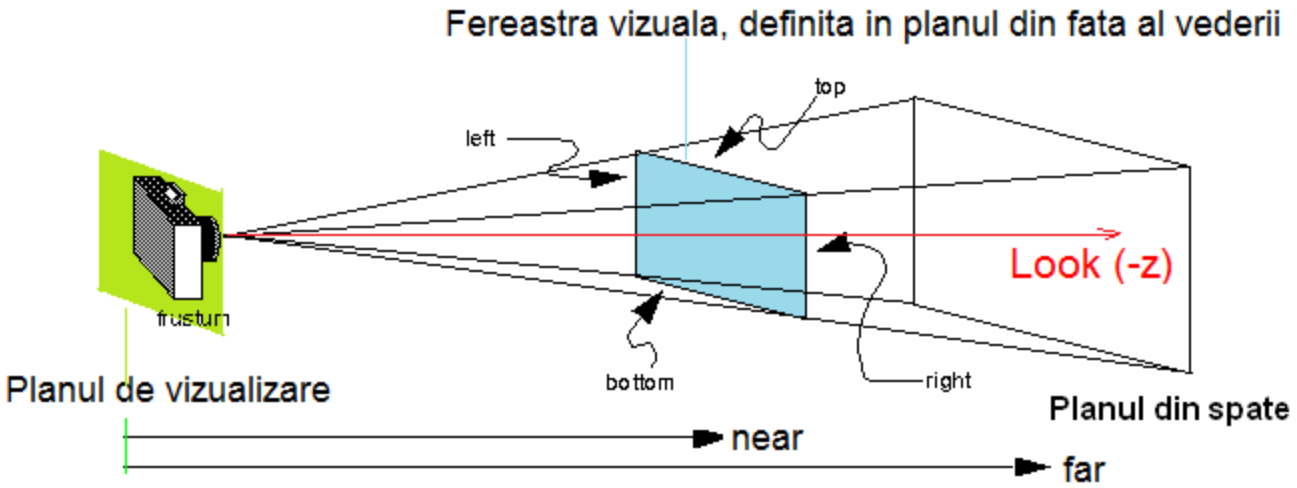
Proiecția ortografică

- Se efectuează proiecția ortografică în fereastra vizuală a obiectelor cuprinse în volumul vizual. Primitivele grafice situate complet in afara volumului vizual sunt eliminate iar cele care intersecteaza volumul sunt decupate la frontiera volumului creându-se noi primitive.



Transformarea de proiecție (3)

Volumul vizual al proiecției perspectivă (View frustum): trunchi de piramida delimitat în adâncime de planul din faţă (near) și planul din spate (far) al vederii, cu vârful în poziția observatorului.

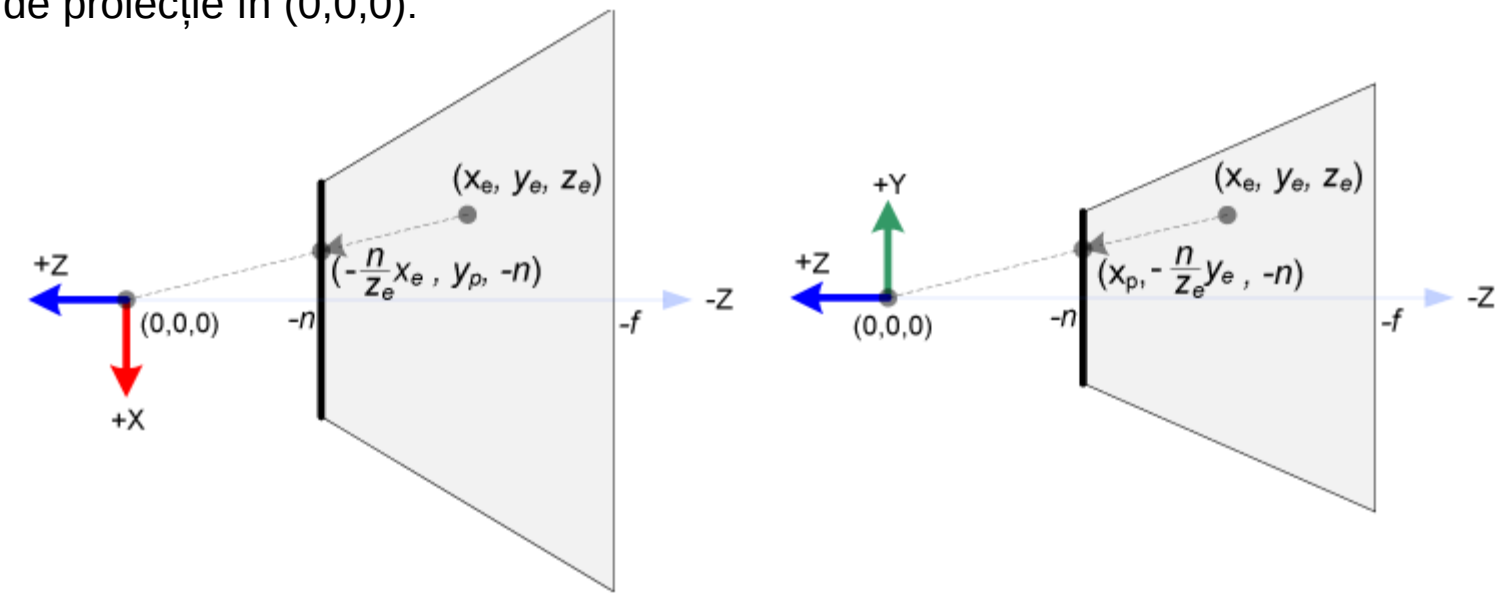


Se efectuează proiecția perspectivă în planul din faţă (în fereastra vizuală), cu centrul de proiecţie în poziţia observatorului, a obiectelor cuprinse în volumul vizual. Obiectele situate în afara volumului vizual sunt eliminate sau decupate la frontiera volumului vizual.

Transformarea de proiecție (4) – proiecția

❖ **Proiecția se efectuează în fereastra vizuală din planul din** faţă: planul z =-near (in SCO).

Proiecția perspectivă a unui punct din SCO, (xe,ye,ze), pe planul din față al vederii, cu centrul de proiecție în (0,0,0):



xp = (-near/ze)*xe, yp =(-near/ze)*ye – se pot obține aplicând formulele proiecției perspectivă cu centrul de proiecție în (0,0,0) și planul de proiectie este z=-near.

Proiectia ortografică a unui punct din sist. coordonatelor observator, (xe,ye,ze), în fereastra vizuală:

xp = xe, yp = ye, zp = -near.

Transformarea de proiecție (5)

- Transformarea de proiecție este însoțită de operația de decupare a primitivelor grafice la marginile volumului vizual.
- Pentru simplificarea calculelor de decupare, volumul vizual definit de programator este transformat în volumul vizual canonic: cub cu latura de 2 unități, centrat în originea sistemului coordonatelor de decupare, cu laturile paralele cu axele.
- Transformarea prin care volumul vizual este transformat în volumul vizual canonic se numește transformarea de normalizare.

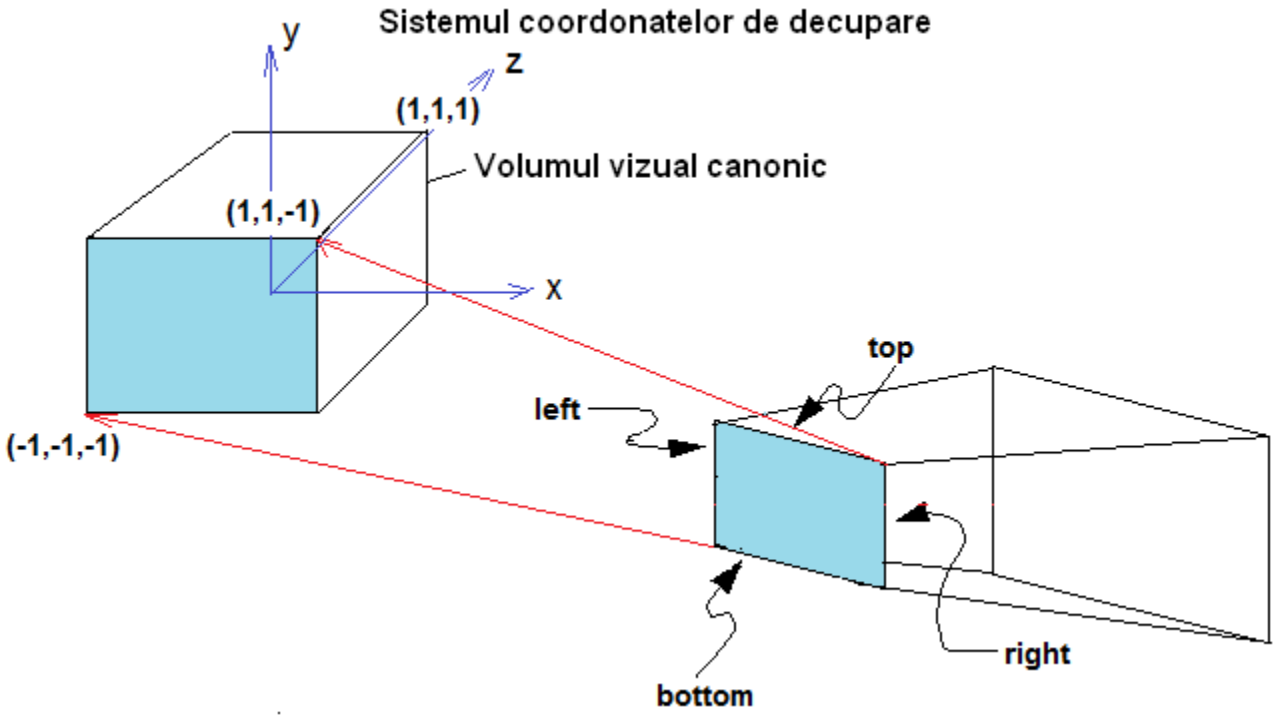
- În OpenGL transformarea de proiecție include: proiecția definită prin volumul de vizualizare compusă cu transformarea de normalizare.

  = transformarea coordonatelor observator în coordonate de decupare.

- După transformarea varfurilor prin transformarea de proiectie, vârfurile sunt unite pe baza topologiei definite de programator, formând primitive grafice 3D. Acestea sunt decupate la marginile volumului vizual canonic.

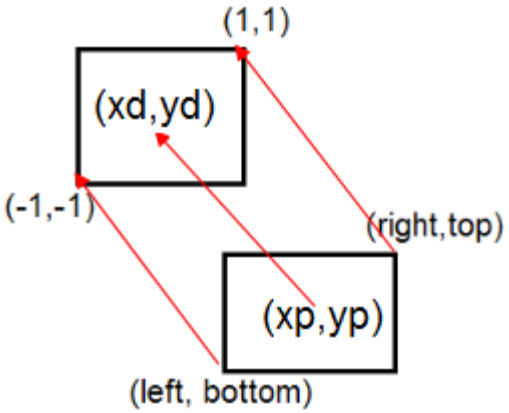
Transformarea de proiecție (6) – normalizarea

- **Sistemul coordonatelor de decupare**: sistem de coordonate carteziene 3D stânga.
- Prin transformarea de normalizare:
  - colţurile ferestrei 2D din planul de proiecţie, (left, bottom,-near) şi (right, top,-near) sunt mapate pe colţurile stânga-jos şi dreapta-sus ale ferestrei 2D din sistemul coordonatelor de decupare, adică (-1,-1,-1) şi (1,1,-1);
  - planul z= -near se mapează pe z= -1, iar z= -far se mapează pe z= 1



Transformarea de proiecție (7) calculul coordonatelor de decupare

- Impunând condiția (left,bottom) → (-1,-1) și (right,top) → (1,1), rezultă transformarea punctelor din planul din faţă, (xp,yp), în sistemul coordonatelor de decupare – este o transformare freastră-poartă:



xd = 2*xp/(right – left) – (right + left)/(right – left)

yd = 2*yp/(top – bottom) – (top + bottom)/(top-bottom)

Înlocuind xp și yp rezultate din proiecția perspectivă în planul -near:

xd = (xe*2*near/(right – left) + ze*(right + left)/(right – left))/(-ze)

yd = (ye*2*near/(top – bottom) + ze*(top + bottom)/(top-bottom))/(-ze)

Pentru transformarea coordonatelor z se impun condițiile: -near → -1, -far → 1; rezultă:

zd = (-ze*(far+near)/(far-near) – 2*far*near/(far-near)) / (-ze)

(xd,yd,zd) sunt coordonate carteziene în sistemul coordonatelor de decupare (clip coordinates); folosind matricea transformării de proiecție rezultă coordonate omogene:

[xc, yc, zc, w] → xd = xc/w; yd = yc/w; zd = zc/w ; w = - ze

Obs: w trebuie sa fie ≠ 0 → punctul proiectat nu poate fi în planul camerei (centrul de proi.)

Transformarea de proiecție (8) matricea transformării

Matricea transformării de proiecție pentru proiecția perspectivă:

$$P = \begin{bmatrix} \frac{2*near}{right-left} & 0 & A & 0 \\ 0 & \frac{2*near}{top-bottom} & B & 0 \\ 0 & 0 & C & D \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

$$A = \frac{right+left}{right-left} \qquad B = \frac{top+bottom}{top-bottom}$$

$$C = -\frac{far+near}{far-near} \qquad D = -\frac{2*far*near}{far-near}$$

Matricea transformării de proiecție pentru proiecția ortografică:

xp=xe si yp=ye → xc = 2*xe/(right – left) – (right + left)/(right – left)

yc = 2*ye/(top – bottom) – (top + bottom)/(top-bottom)

$$P = \begin{bmatrix} \frac{2}{right-left} & 0 & 0 & -A \\ 0 & \frac{2}{top-bottom} & 0 & -B \\ 0 & 0 & \frac{-2}{far-near} & C \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[xc\ yc\ \ zc\ w]^T = P \bullet [xe\ ye\ ze\ we]^T$$

Transformarea de proiecție (9)

Matricile de proiecție pot fi obținute apelând următoarele funcții din biblioteca glm:

glm::ortho(float left, float right, float bottom, float top, float zNear, float zFar);

glm::frustum (float left, float right, float bottom, float top, float zNear, float zFar);

glm::perspective (float fov, float aspect, float znear, float zfar);

Crează o matrice de proiecție pentru un volum perspectivă simetric:

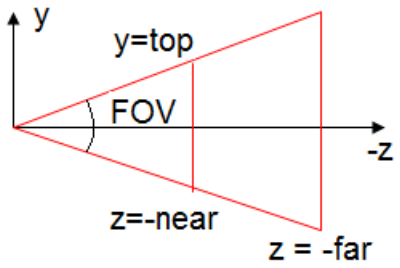
- fov: Field of View

  tan(FOV/2) = top/near

- aspect: aspect ratio

  aspect ratio = (right-left)/(top-bottom)

  = raportul dintre lățimea și înălțimea ferestrei vizuale



Transformarea de proiecție (10)

(xo, yo, zo): **poziția observatorului**

(Px,Py,Pz): punctul din scena 3D către care privește observatorul;

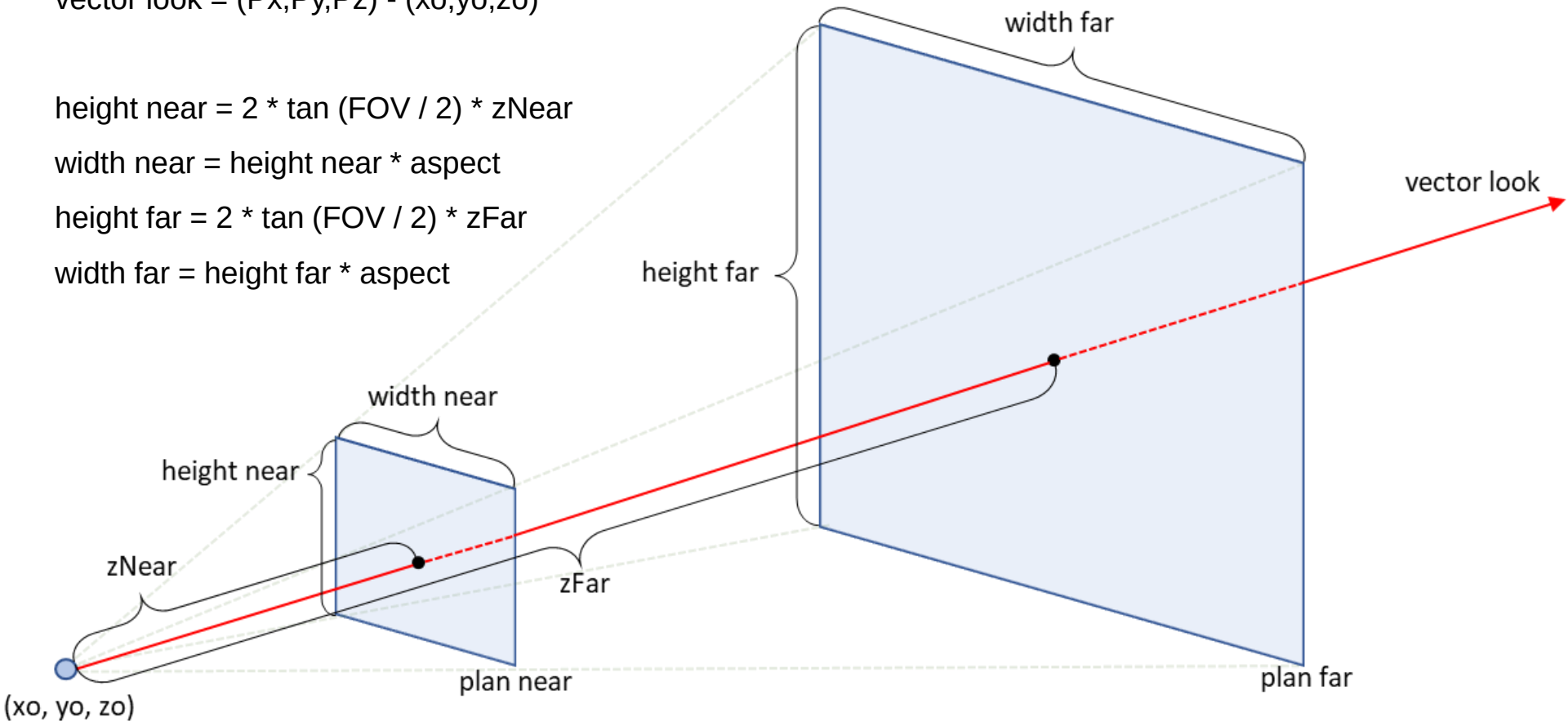
vector look = (Px,Py,Pz) - (xo,yo,zo)

height near = 2 * tan (FOV / 2) * zNear

width near = height near * aspect

height far = 2 * tan (FOV / 2) * zFar

width far = height far * aspect



Transformarea de proiecție (11)

din coordonate observator → în coordonate de decupare

$[xc\ yc\ zc\ w]^T = P \bullet [xe\ ye\ ze\ we]^T$ (clip coordinates) ← eye coordinates)

Împărțirea perspectivă

Coordonate omogene → coordonate carteziene: xd = xc/w , yd = yc/w , zd= zc/w

(xd, yd, zd): coordonate dispozitiv normalizate

- **Vârfurile rezultate prin împărțirea perspectivă sunt asamblate în primitive grafice care sunt decupate la frontiera volumului vizual canonic.**

Transformarea de modelare-vizualizare-proiecție

❖ Prin înmulțirea matricilor transformarilor de modelare, vizualizare și proiecție se obține matricea numită "model-view-projection", a.i. asupra vârfurilor definite în sistemul coordonatelor obiect poate fi efectuată o singură transformare:

MVP = P•V•M (matricea ModelViewProjection)

Transformarea in poarta de afișare

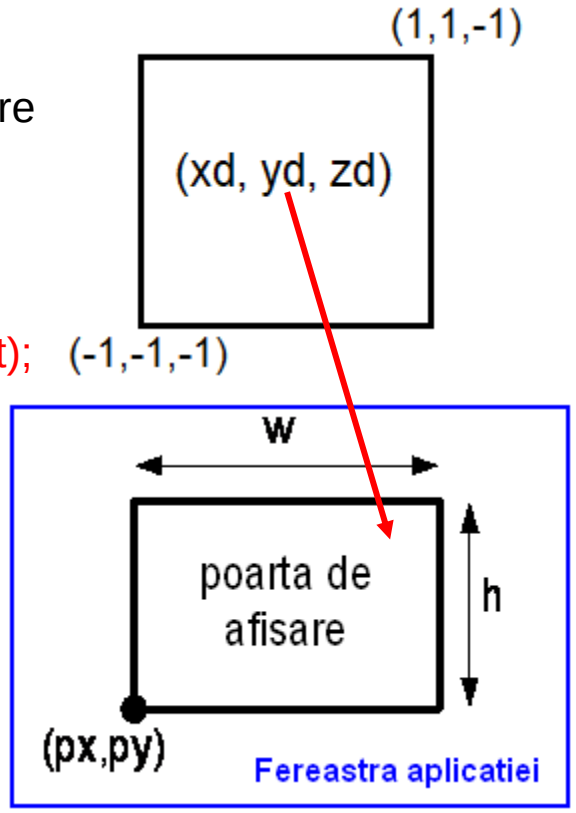
- Este o transformare fereastră-poartă, în care:
- Fereastra este fereastra 2D din sistemul coordonatelor de decupare

  având colţurile în (-1, -1, -1) - (1, 1, -1).
- Poartă este definită de programator folosind funcția:

  void **glViewport**(GLint Px, GLint Py, GLsizei width, GLsizei height);

  (*Px, Py*) reprezintă coordonate relative la colțul stânga-jos al ferestrei aplicației (în pixeli). Valorile implicite : (0,0).

  width, height reprezintă lăţimea / înălţimea porţii de afişare. Valorile implicite sunt: lăţimea şi înălţimea ferestrei aplicației.



- **Transformarea în poarta de afișare este definită astfel:**

  xv = (width/2)*xd + ox , ox = px+ width/2

  yv = (height/2)*yd + oy, oy = py + height/2

  zv = ((f-n)/2)*zd + (n+f)/2 // conservă coordonata z pentru eliminarea părților nevizibile

  n (near) şi f (far) au valorile implicite 0.0 respectiv 1.0, pentru care 0<=zv<=1

  n și f pot fi modificate la valori cuprinse în intervalul [0,1] folosind funcţia ***DepthRange()***

Eliminarea părților nevizibile ale scenelor 3D din imagini

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

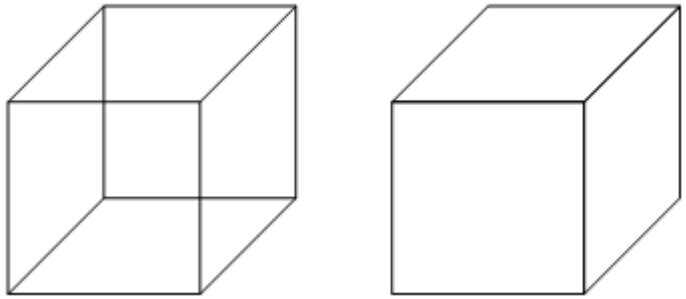
Eliminarea părților nevizibile ale scenelor 3D, din imagini (1)

Vizibilitatea obiectelor dintr-o scenă 3D depinde de:

– Poziția observatorului și direcția în care privește

– Volumul vizual

– Poziționarea obiectelor din scena unul față de celălalt



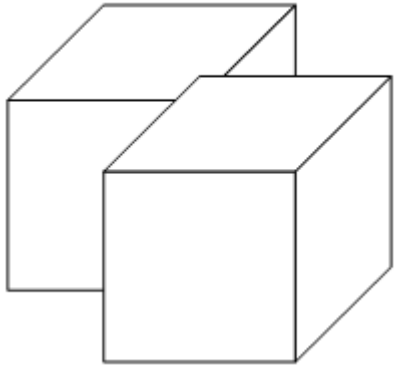
Părțile unei scene 3D nevizibile într-o imagine sunt:

– Părți aflate total sau parțial în afara volumului vizual

– Fețe auto-obturate

– Părți obturate (ascunse) de obiecte ale scenei,

aflate în fața lor în raport cu observatorul



Eliminarea părților nevizibile ale scenelor 3D, din imagini (2)

Efectuată la nivel de:

1. Obiecte sau grupuri de obiecte - **Object culling**:

   - **Frustum Culling**: eliminarea din banda grafică a obiectelor sau grupurilor de obiecte care sunt complet în afara volumului vizual
   - **Occlusion Culling**: – eliminarea din banda grafică a obiectelor sau grupurilor de obiecte care sunt complet obturate de alte obiecte

   - **nu este efectuată de OpenGL/GPU**
   - poate fi efectuată prin algoritmi implementați într-o bibliotecă folosită de aplicația grafică 3D (motorul grafic)

2. Fețe ale obiectelor - ***Back face culling:*** eliminarea din banda grafică a fețelor auto-obturate ale obiectelor (este efectuată pe GPU).

Eliminarea părților nevizibile ale scenelor 3D, din imagini (3)

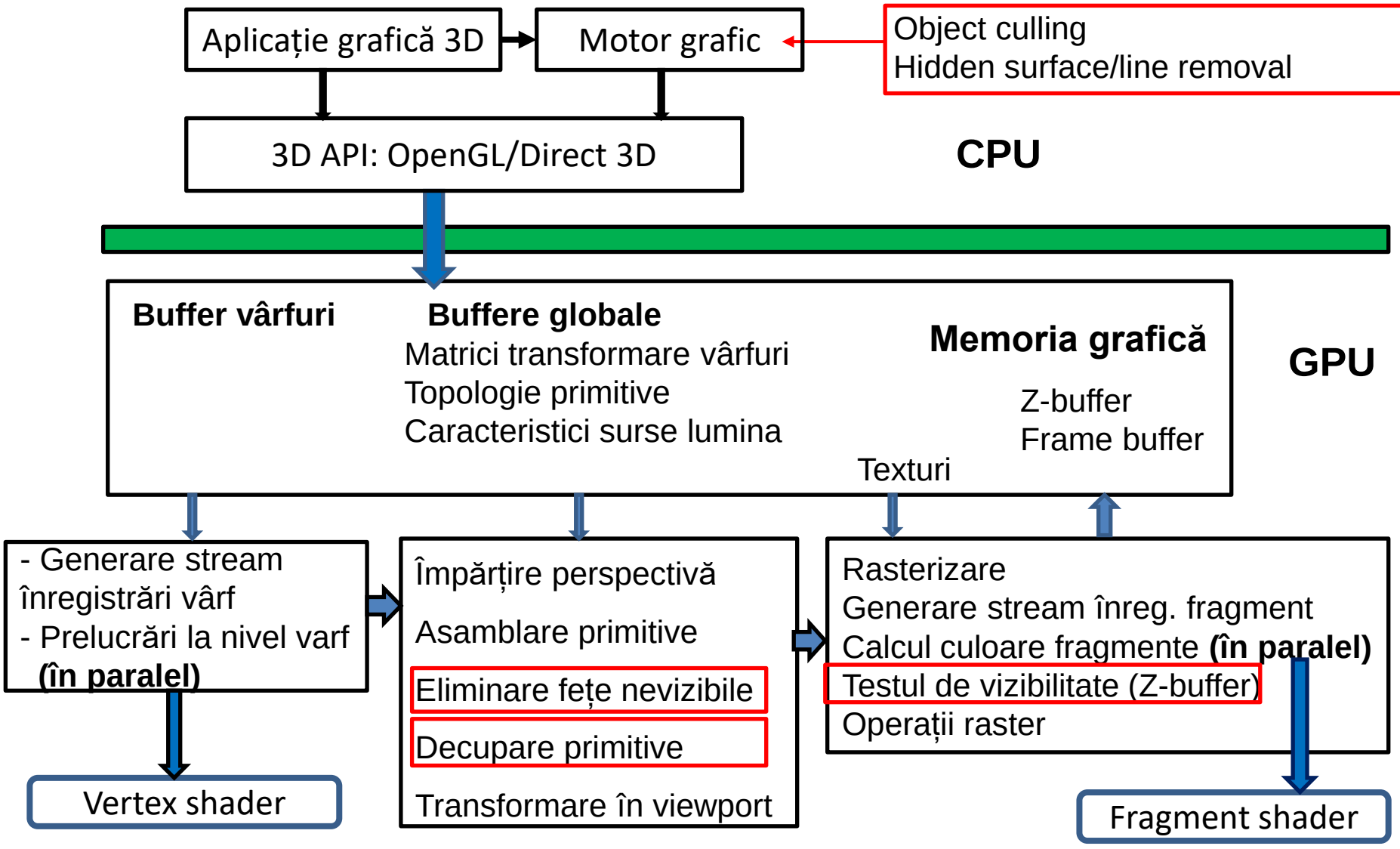
3. Primitive grafice (poligoane, linii):

- Eliminarea primitivelor aflate total sau parțial în afara volumului vizual prin operația de **decupare (clipping)** efectuată de GPU

- Eliminarea părților obturate (ascunse) ale primitivelor grafice: ***hidden surface/hidden line removal***

  - poate fi efectuată prin algoritmi implementați într-o bibliotecă folosită de aplicația grafică 3D (motorul grafic): BSP, Pictorului, ş.a.
  - este efectuată de GPU, ca operație raster la nivel de fragment (algoritmul Z-buffer)

Eliminarea părților nevizibile în banda grafică



Înregistrare vârf: intrare Vertex shader

Înregistrare fragment: intrare Fragment shader

Eliminarea fețelor auto-obturate pentru poliedre convexe (Back face culling) (1)

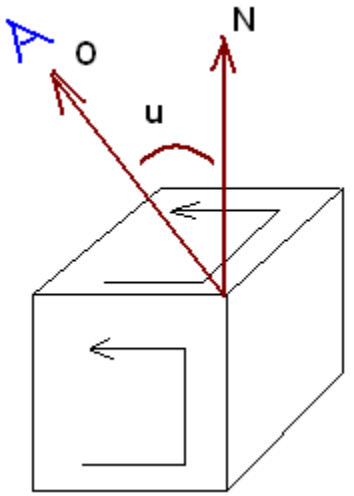
- Poate reduce substanțial numărul de poligoane rasterizate.

Algoritmul:

- Observatorul și obiectul sunt raportați la același sistem de coordonate.

- Conturul fiecărei fețe a obiectului este orientat în sens trigonometric atunci când poliedrul este văzut din exterior.

- Observatorul este situat în afara obiectului.



Fața este vizibila daca : $0 <= u < 90^o$ sau $0 < \cos(u) <= 1$

N: normala la față

O: vectorul orientat către observator

Produsul scalar: $N \bullet O = |N| * |O| * \cos(u) \rightarrow \cos(u) = N \bullet O / (|N| * |O|) = Nu \bullet Ou$

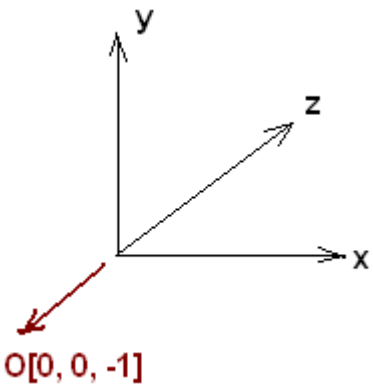
Condiția de vizibilitate: Nu • Ou > 0 (produsul scalar al versorilor)

Determinarea fețelor auto-obturate pentru poliedre convexe (2)

Determinarea feţelor auto-obturate poate fi efectuată:

1. În sistemul coordonatelor globale (în care este definit și observatorul – camera virtuală)
2. În sistemul de coordonate observator
3. În sistemul coordonatelor de decupare (după aplicarea transformării de proiecție)

În cazul 3



Sistemul coordonatelor de decupare

Nu • Ou = [nx, ny, nz] • [0, 0, -1] = - nz

Condiția de vizibilitate devine: nz <0

După transformarea de proiecție poziția observatorului este la infinit, pe axa z negativă.

Vectorul orientat către observator este același pentru orice punct al unui obiect

Determinarea fețelor auto-obturate pentru poliedre convexe (3)

Calculul normalei la o faţă a poliedrului

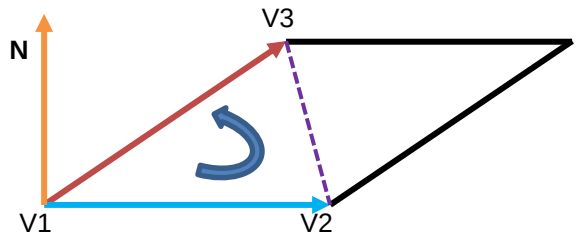
Fie V1(x1, y1, z1), V2(x2, y2, z2), V3(x3, y3, z3), 3 varfuri succesive ale conturului feţei.

Normala la față se poate obține calculând produsul vectorial: (V2-V1)x(V3-V1)

$$\mathbf{N} = (V2-V1)\times(V3-V1) = \begin{vmatrix} i & j & k \\ x2-x1 & y2-y1 & z2-z1 \\ x3-x1 & y3-y1 & z3-z1 \end{vmatrix}$$

$$= [(y2-y1)*(z3-z1) - (z2-z1)*(y3-y1)]*\mathbf{i}$$
$$- [(x2-x1)*(z3-z1) - (z2-z1)*(x3-x1)*\mathbf{j}$$
$$+[(x2-x1)*(y3-y1) - (y2-y1)*(x3-x1)]*\mathbf{k}$$

unde **i**, **j**, **k** sunt versorii direcțiilor axelor sistemului de coordonate carteziene 3D



Normalele:

- Pot fi calculate în sistemul coordonatelor obiect și atașate vârfurilor în aplicația OpenGL → sunt transformate în sistemul coordonatelor de decupare, in vertex shader.
- Nu sunt atașate vârfurilor: sunt calculate pe GPU dupa asamblarea primitivelor, folosind vârfurile transformate în sistemul coordonatelor de decupare.

Determinarea fețelor auto-obturate pentru poliedre convexe (4)

Pentru eliminarea feţelor auto-obturate se apelează următoarele funcţii OpenGL:

glCullFace(GL_FRONT / GL_BACK / GL_FRONT_AND_BACK);

- eliminare fețe: din faţă / din spate / toate (nu vor fi afișate fețe, ci numai alte primitive: puncte, linii)

glFrontFace(GL_CCW/GL_CW);

- specifică orientarea feţelor din faţă: trigonometrică / sensul acelor de ceas

➢ Dacă ciclul de vârfuri al fiecarei fețe este orientat trigonometric atunci când obiectul este privit din exteriorul său:

- atunci când observatorul este în spațiul exterior obiectului se va apela glFrontFace(GL_CCW)
- atunci când observatorul este în spațiul interior obiectului se va apela glFrontFace(GL_CW)

(văzute din interior, feţele obiectului au conturul orientat anti-trigonometric).

Algoritmul z-buffer (1)

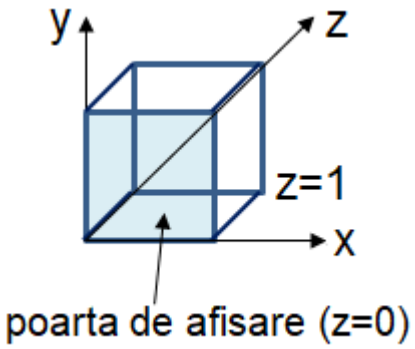
- Algoritm de eliminare a fragmentelor nevizibile ale primitivelor grafice.

- Executat de GPU, fiind integrat în procesul de rasterizare a primitivelor

- Observatorul este la infinit, pe axa z negativă

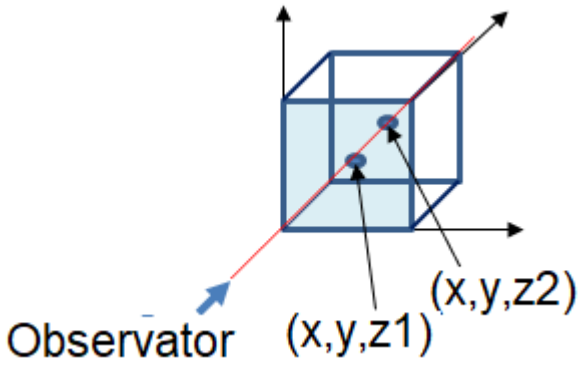
- Asupra primitivelor se efectueaza proiecție ortografică

  în planul XOY (poarta de afișare)



Dacă două fragmente aflate pe același proiector, rezultate din rasterizarea a două primitive diferite, au coordonata z diferită:

- Fragmentul avand coordonata z mai mica va fi afisat în pixelul (x,y).



Algoritmul z-buffer (2)

- Primitivele grafice (triunghiuri, linii) în care a fost descompusă scena 3D sunt rasterizate în ordinea în care au fost transmise din programul de aplicație (nu sunt sortate).

- Buffer-ul imagine (frame buffer) este actualizat pe măsura rasterizării primitivelor, astfel:

daca (fragmentul curent se proiecteaza in pixelul (x,y) și
z-fragment < z-fragment afișat în pixelul (x,y))
atunci
actualizează culoarea pixelului (x,y) în bufer-ul imagine, la culoarea fragmentului curent

Rezultă: este necesar să se memoreze coordonatele z ale fragmentelor afișate în pixelii imaginii



Z-buffer: - tablou bidimensional cu numar de elemente egal cu numărul de pixeli ai suprafeței de afișare
- în memoria GPU

Algoritmul z-buffer (3)

Algoritmul z-buffer (integrat în procesul de rasterizare):

*Inițializează buffer-ul imagine la culoarea de fond
*Inițializează Z-buffer la coordonata z maximă (z=1)

Pentru fiecare fragment f(x,y,z) rezultat din rasterizarea unei primitive

* calculează culoarea fragmentului (în fragment shader)

// testul de vizibilitate (adâncime)

daca z < Z-buffer[y][x] atunci

{

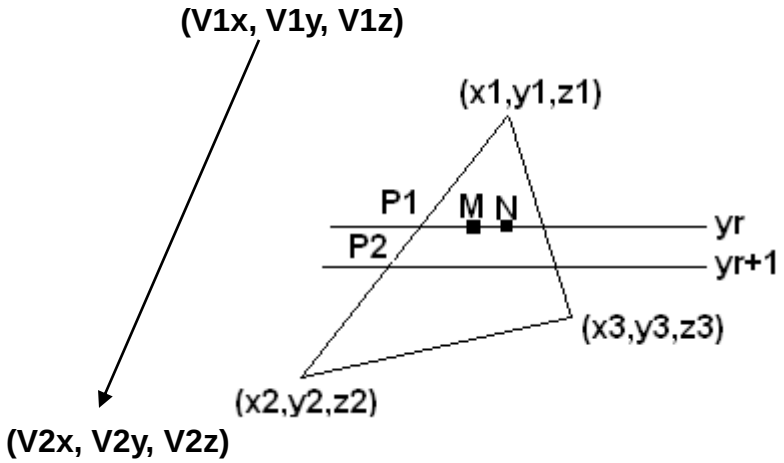
Z-buffer[y][x] = z

*actualizează culoarea pixelului (x,y) in buffer-ul imagine folosind culoarea fragmentului f

}

Algoritmul z-buffer (4)

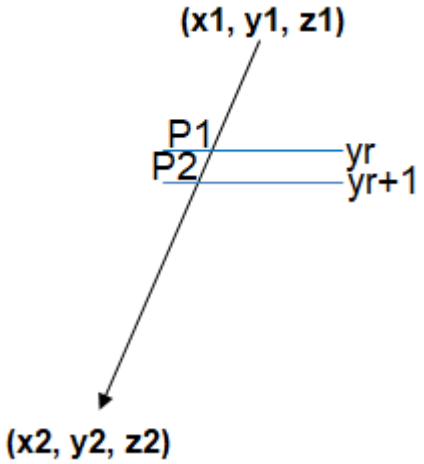
Calculul coordonatei z a unui fragment



Coordonatele z ale fragmentelor rezultate la rasterizarea vectorilor și a suprafețelor triunghiulare se obțin prin calcul incremental, în algoritmul de rasterizare.

Algoritmul z-buffer (5)

Calculul coordonatei z la rasterizarea vectorilor



x = x1 + t(x2-x1)
y = y1 + t(y2-y1) Ecuațiile parametrice ale vectorului
z = z1 + t(z2-z1)

yr, yr+1: ordonatele a 2 linii raster (ecran) care intersectează vectorul

y(P1) = yr = y1 + t'(y2-y1) → t' = (yr-y1)/(y2-y1)

z(P1) = z1 + (yr-y1)(z2-z1)/(y2-y1) = z1 + (yr-y1)*K,
K este o constantă la rasterizarea vectorului

z(P2) = z1 + (yr+1-y1)*K = z1 + (yr-y1)*K + K

Z(P2) = Z(P1) + K → coordonatele z ale fragmentelor de pe vector se obțin prin calcul incremental.

Algoritmul z-buffer (6)

Calculul coordonatei z la rasterizarea poligoanelor

P1(xP1, yr, zP1), P2(xP2, yr+1, zP2)

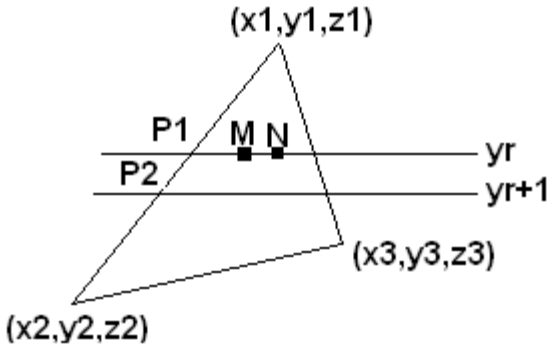
m- panta laturii (x1,y1,z1) – (x2,y2,z2)

xP2 = xP1 + 1/m (vezi curs - rasterizare)

A*x + B*y + C*z + D = 0 ec planului poligonului

C*z = -A*x - B*y - D

zP1 = (-A*xP1 - B*yr –D)/C



zP2 = (-A*xP2 - B*(yr +1) – D)/C = (-A(xP1 + 1/m) - B*yr –B – D)/C = zP1 + (-A/m –B) → **zP2 = zP1 + K1**

M(xM, yr, zM), N(xM + 1, yr, zN)

zM = (-A*xM – B*yr – D)/ C

zN = (-A*(xM + 1) – B*yr – D)/ C = zM + (-A/C) → **zN = zM + K2**

Observație: atunci când un vector se suprapune pe suprafața unui poligon pot aparea defecte datorită modului diferit în care se calculează coordonatele z ale fragmentelor care se afișează în aceiași pixeli!

Algoritmul z-buffer (7)

Aprecieri asupra algoritmului z-Buffer

1) Numărul de comparații de valori z pentru un pixel = numărul de fragmente care se proiectează în acel pixel.

   Timpul de calcul tinde sa devină independent de numărul de poligoane: în medie, numărul de pixeli acoperiți de un poligon este invers proporțional cu numărul de poligoane.

2) “Depth complexity”: numărul de suprascrieri ale unui pixel in buffer-ul imagine, la generarea unui cadru imagine

   - ❖ calcul culoare fragment: model iluminare, utilizare texturi

3) Pentru o precizie bună, este necesară o reprezentare pe 32 biţi a valorilor din z-buffer.

Algoritmul BSP (Binary Space Partitioning) (1)

- Inițial, arborele BSP a fost folosit numai pentru operația de eliminare a obiectelor (grupurilor de primitive) nevizibile din banda grafică (object culling).
- Ulterior a devenit algoritm pentru reprezentarea și afișarea scenelor 3D cu "eliminarea" părților nevizibile.
- Este eficient pentru eliminarea părților nevizibile ale poligoanelor (hidden surface removal) dar poate fi folosit și pentru eliminarea partilor nevizibile ale liniilor dintr-o scenă compusă din poligoane și linii.

Intrarea: lista primitivelor care compun scena 3D
(nu are importanță din care obiect face parte fiecare primitivă)

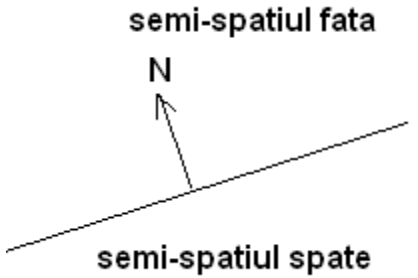
– Scena 3D este reprezentată printr-un arbore binar: arborele BSP

– Arborele BSP al scenei este independent de poziția observatorului (reprezintă scena în sistemul coordonatelor globale).

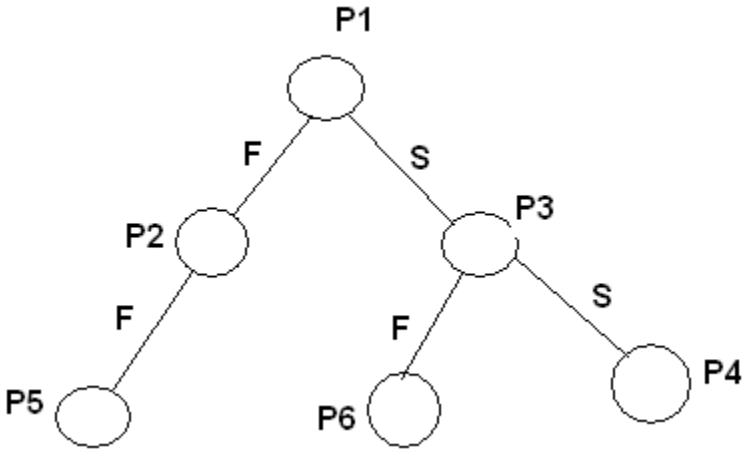
Algoritmul BSP (2)

Construirea arborelui BSP al unei scene

- Fiecare nod al arborelui corespunde unui plan de partiționare al spațiului 3D (de regulă, planul unui poligon al scenei)
- Fiecare plan de partiționare împarte spațiul în 2 semi-spații:
  - cel din fața planului (de aceeași parte cu normala la plan)
  - cel din spatele planului



1. Se începe cu un poligon oarecare din lista (de regulă primul), pentru care se crează nodul rădacină al arborelui
   - Primitivele din semi-spatiul "faţă" formeaza "lista-faţă", care va genera subarborele "faţă" al nodului
   - Primitivele din semi-spațiul "spate" formeaza "lista-spate", care va genera subarborele "spate" al nodului.



2. Se alege un poligon din lista-față și se crează nodul rădacină al subarborelui "față, etc.

Algoritmul BSP (3)

Algoritmul de creare a arborelui BSP:

arbore * creareBSP (Poligon * LP)

{ Poligon P, * ListaFata, * ListaSpate, * ListaNod;

dacă (LP este vidă) return NULL;

ListaFata = ListaSpate = ListaNod = NULL;

* alege un poligon P din LP;

* elimină P din LP; adaugă P la ListaNod;

pentru (fiecare primitiva Q din LP) execută

{ **dacă** (Q este în semispațiul față al planului lui P) **atunci**

* adauga Q în ListaFata

altfel

Algoritmul BSP (4)

daca (Q este în semispațiul spate al planului lui P) **atunci**

* adauga Q în ListaSpate

altfel

daca Q este în același plan cu P **atunci**

* adaugă Q in ListaNod

altfel // Q este intersectată de planul lui P

* divizează Q cu planul lui P

* adaugă fiecare primitivă rezultată în ListaFata sau ListaSpate, în funcție de poziția sa

} // pentru fiecare primitivă

arbore * radacina = combina(creareBSP(ListaFata), creareBSP(ListaSpate));

*ataseaza ListaNod la radacina;

return radacina;

}

Algoritmul BSP (5)

Afişarea arborelui BSP al unei scene

- Ține cont de poziția observatorului

- Afișare "front-to-back" și "back-to-front"

Afișare "back-to-front": se începe cu primitiva cea mai îndepărtată de observator:

- Se pornește din rădacina arborelui și se avansează în arbore până la frunze, în funcție de poziția observatorului față de planul atașat fiecărui nod.

- În fiecare nod: se avansează în subarborele nodului care este opus observatorului față de plan.

```
void afisareBSP(arbore * A, Pozitie Observator)
{
  daca (! A) return;
  daca (Observator este în semispațiul față al planului radacinii arborelui A) atunci
    { afisareBSP(A->spate, observator); *afișare primitive din nodul rădăcină; afisareBSP(A->fata,
            observator);}
  altfel
    { afisareBSP(A->fata, observator); *afișare primitive din nodul rădăcină; afisareBSP(A->spate,
observator);}
}
```

Algoritmul BSP (6)

- **Eliminarea obiectelor aflate în afara volumului vizual (frustum culling):**

➢ Dacă planul de partiționare al unui nod nu intersectează volumul vizual, atunci numai subarborele aflat de aceeaşi parte cu volumul vizual va fi afişat, celalalt subarbore fiind eliminat din banda grafică. La fel şi primitivele ataşate nodului (fiind în afara volumui vizual).

```
void afisareBSPcull (arbore * A, Pozitie Observator)
{ daca (! A) return;
  daca (volumul vizual este în fața planului radacinii arborelui A) atunci
        afisareBSPcull(A→fata, observator); return;
  daca (volumul vizual este in spatele planului radacinii arborelui A) atunci
        afisareBSPcull(A→spate, observator); return;
  // volumul vizual este intersectat de planul rădăcinii
    daca (Observator este în semispațiul față al planului rădăcinii) atunci
     { afisareBSP(A->spate, observator); *afișare primitive din nodul rădăcină; afisareBSP(A->fata, observator);}
   altfel
    { afisareBSP(A->fata, observator); *afisare primitive din nodul rădăcină; afisareBSP(A->spate, observator);}
}
```

Algoritmul BSP (7)

Aprecieri

- Arborele nu trebuie să fie construit pentru fiecare cadru imagine: avantaj pentru scenele statice
- Modificarea poziției observatorului → execuția funcției de afișare (dacă scena nu s-a modificat)
- Permite eliminarea din banda grafică a unui numar mare de poligoane care sunt în afara volumului vizual (Object culling)
- Pentru obiectele dinamice se pot face diverse adaptări ale algoritmului; de ex:

- pentru fiecare cadru imagine se inserează în arborele BSP cu obiectele statice, noduri corespunzând obiectelor dinamice;

- se transmite arborele BSP static nemodificat și obiectele dinamice vizibile, eliminarea părților obturate de acestea fiind realizată în algoritmul z-buffer (la rasterizare).

Algoritmul BSP (8)

Aprecieri (cont):

- Dacă primitivele grafice sunt trimise în banda grafică în ordinea “din spate in faţă”, nu mai este necesar testul de adâncime la rasterizarea primitivelor (fragmentele care se proiectează în același pixel sunt suprascrise).
- Testul de adâncime efectuat asupra fragmentelor la rasterizare poate fi activat/dezactivat:

  glEnable(GL_DEPTH_TEST) / glDisable(GL_DEPTH_TEST)

- Dacă se folosește algoritmul BSP cu afisare «back-to-front» împreună cu testul de adâncime, numărul de suprascrieri ale pixelilor este mai mare decât în cazul unei ordini aleatoare a poligoanelor (algoritmul z-buffer fara BSP).
- Dacă se folosește BSP cu afișare «front-to-back» împreună cu testul de adâncime, afișarea poate fi mai rapidă decât folosind BSP «back-to-front» fără testul de adancime, în funcție de amplasarea poligoanelor în scenă.
- Construirea și afișarea arborelui BSP sunt funcții de management al scenei 3D, implementate de regulă într-un motor grafic 3D.

Algoritmul pictorului (1)

– Algoritm din categoria “Hidden surface removal”.

– Intrare: lista poligoanelor care alcătuiesc scena 3D, transformate în spațiul de afișare

– Ideea: eliminarea părților nevizibile se rezolvă prin afișare “din spate în față”, la fel ca în pictură.

➢ Nu este necesar testul de adâncime la rasterizarea poligoanelor.

Forma generală a algoritmului

1. Se calculează “extensia” fiecarui poligon din listă pe axele OX, OY, OZ (xmin,ymin,zmin – xmax, ymax, zmax): paralelipipedul încadrator al poligonului, cu feţele paralele cu planele principale ale sistemului de coordonate.

2. Se ordonează poligoanele crescător după coordonata zmin a fiecarui poligon: primul în listă va fi cel mai apropiat de observator.

3. Se descompun poligoanele ale căror extensii pe axa OZ se suprapun, astfel încât extensiile lor pe axa OZ sa fie disjuncte.

4. Se transmit în banda grafică poligoanele începând cu ultimul din listă (cel mai îndepărtat de observator).

Algoritmul pictorului (2)

Pasul 3. Se descompun poligoanele ale căror extensii pe axa OZ se suprapun

- Sunt multe aplicații în care acest pas nu este necesar, poligoanele fiind amplasate în plane de Z-constant: cartografie, generarea straturilor circuitelor imprimate, etc.
- Poate fi optimizat, știind că nu întotdeauna atunci când extensiile pe axa OZ se suprapun este necesară descompunerea poligoanelor; descompunerea nu este necesară dacă:

  - extensiile pe axa OX nu se suprapun

  - extensiile pe axa OY nu se suprapun

  - proiecțiile poligoanelor nu se suprapun în imagine
- Testele se efectuează progresiv, în funcție de complexitatea calculelor presupuse.

- **Principalul efort de calcul:** sortarea listei de poligoane și descompunerea, dacă este necesară.

Eliminarea părților nevizibile ale scenelor 3D din imagini - structuri de date de accelerare -

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

Tehnici de eliminare a obiectelor nevizibile - Frustum culling -

❖ Scopul: eliminarea din banda grafică a obiectelor sau grupurilor de obiecte care sunt complet în afara volumului vizual

❖ Se bazeaza pe reprezentarea scenei printr-o structură de date spațială.

Structuri de date spațiale pentru managementul scenei 3D

- Ierarhie de volume încadratoare (BVH)
- Arbore BSP
- Arbore AA-BSP
- Arbore octal

- Managementul scenei este realizat în cadrul unui motor grafic 3D, implementat peste OpenGL / Direct 3D.

- Structurile de date sunt utile și pentru alte operații: mai sunt numite "**structuri de date de accelerare**".

1. Scena - Ierarhie de volume încadratoare (1) (Bounding Volume Hierarchy - BVH)

❖ **Scena este reprezentată printr-un arbore care are în noduri volume încadratoare de obiecte sau grupuri de obiecte**: sfera, paralelipipedul aliniat cu axele (Axis-Aligned minimum Bounding Box: AABB), poliedrul convex minimal încadrator, cilindrul, elipsoidul.

Construirea arborelui:

1. Top-down – rădăcina arborelui este asociată cu întreaga scenă

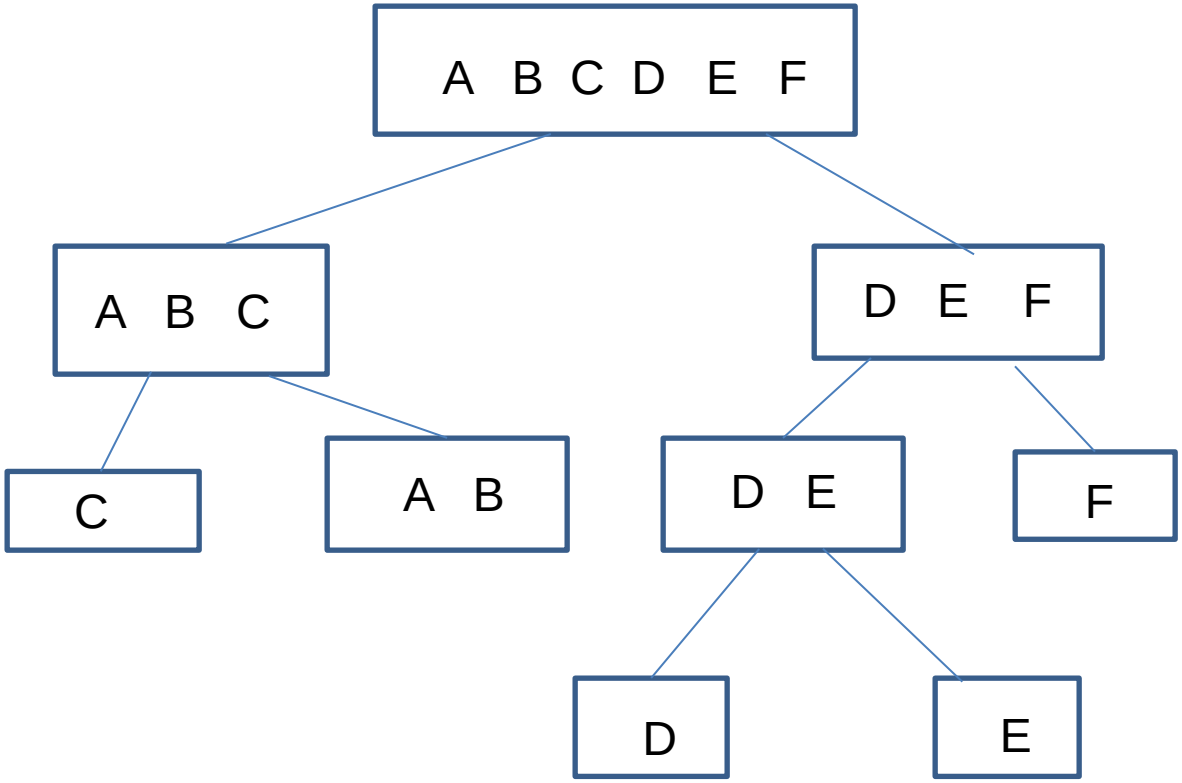
- Se partiționează setul de obiecte în 2 sau mai multe subseturi și se alocă fiecare subset unui volum încadrator – rezultă primul nivel al arborelui
- Se continuă procedura recursiv la nivelul fiecărui grup de pe nivelul anterior până când se ajunge la un singur obiect (sau grup de obiecte) pe nod – nod frunză
- Metoda este simplă dar, în general, nu produce arbori optimi.

2. Bottom-up

- Se pleacă de la setul de obiecte și se definește un volum încadrator pentru fiecare obiect sau grup de obiecte – nodurile frunză.
- Se grupează 2 sau mai multe volume nod frunză, rezultând alte noduri ale arborelui.
- Se continuă gruparea până când totul este grupat într-un singur volum – nodul radacină

❖ Construirea bottom-up produce arbori mai eficienți decât cea top-down.

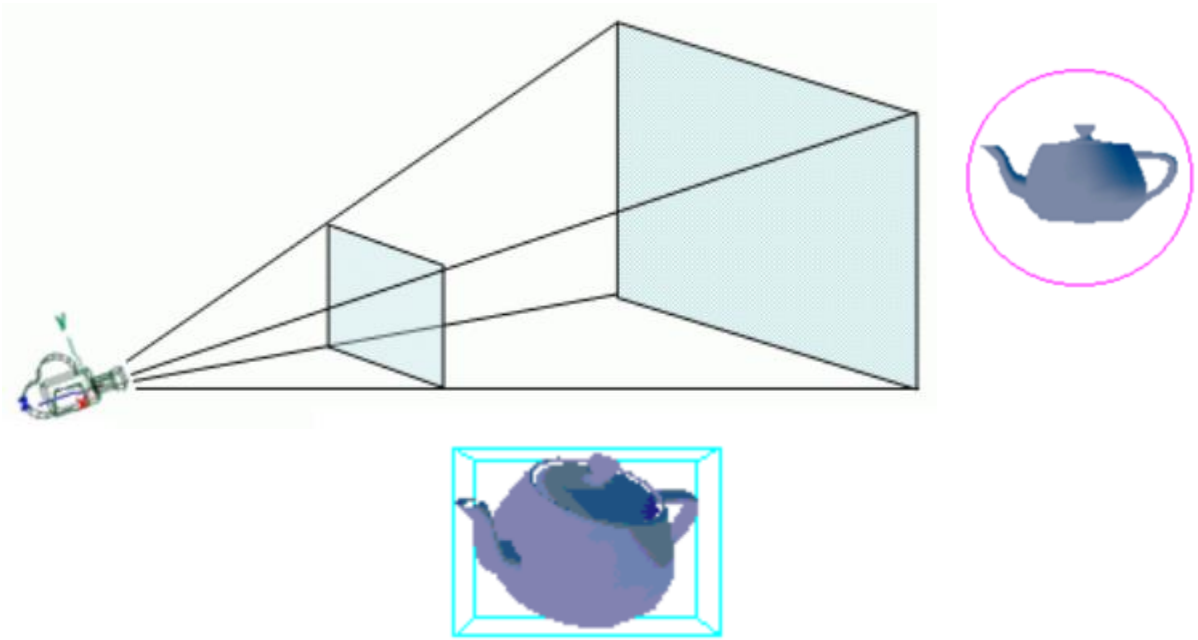
Ierarhie de volume încadratoare (2)



Ierarhie de volume încadratoare (3)

- **Afișarea scenei 3D folosind arborele BVH**

Dacă volumul încadrator al unui nod este în afara volumului vizual atunci obiectele din subarborele său **NU** sunt trimise în banda grafică.



Ierarhie de volume încadratoare (4)

❖ **Afișarea scenei 3D folosind arborele BVH**

- testele de vizibilitate încep cu rădăcina arborelui;

- **dacă** volumul încadrator al nodului curent este complet în afara volumului vizual

atunci se revine pe nivelul anterior al arborelui (întregul grup de obiecte din subarborele nodului curent este exclus din banda grafică)

altfel se coboară în arbore;

- **daca** se ajunge la o frunză al carei volum încadrator nu este în afara volumului vizual,

atunci - grupul de primitive (obiectul) asociat frunzei este trimis în banda grafică;

- se revine pe nivelul anterior al arborelui;

➢ **Pentru simplitate și eficienţă, testele de vizibilitate se pot face în spațiul coordonatelor de decupare, transformând volumele încadratoare prin matricea MVP și testând intersecția cu volumul vizual canonic (cub cu laturile paralele cu axele)**

❖ Structura BVH este frecvent utilizată pentru detecția coliziunilor dintre obiectele unei scene dinamice.

Ierarhie de volume încadratoare (5)

❖ **Eficiența structurii BVH privind eliminarea obiectelor nevizibile din banda grafică**

- depinde mult de felul în care sunt grupate obiectele în volume încadratoare; poate fi eficientă pentru anumite poziții ale observatorului și ineficientă pentru alte poziții.

Alegerea tipului de volum încadrator pentru un nod:

- Să aibă o formă simplă, a.i. testele de intersecție cu volumul vizual să fie simple și rapide
- Să încadreze minimal obiectul/obiectele

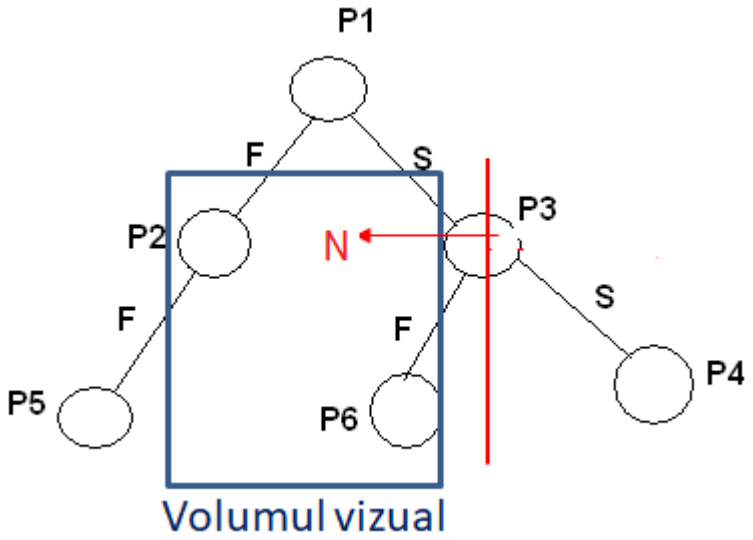
Proprietăți dorite ale ierarhiei de volume încadratoare:

- Volumele nodurilor dintr-un subarbore trebuie sa fie apropiate spațial, cu atât mai apropiate cu cât sunt situate mai jos în arbore
- Volumul fiecărui nod să încadreze minimal obiectul/grupul de obiecte corespunzător
- Volumul încadrator al intregii scene trebuie să fie minimal
- Suprapunerea volumelor nodurilor vecine să fie minimală

2. Scena - Arbore BSP (Polygon Aligned BSP tree) (1)

- **Planele de partiționare sunt planele poligoanelor (Polygon Aligned - PA BSP tree)**
  - Arborele scenei se construiește ca în algoritmul BSP pentru eliminarea suprafețelor nevizibile
- **Eliminarea grupurilor de primitive nevizibile**

Daca planul de divizare atasat unui nod nu intersecteaza volumul vizual, atunci subarborele atașat nodului care este situat de partea opusă a planului față de volumul vizual este complet nevizibil și poate fi eliminat din procesul de redare. La fel şi primitivele ataşate nodului.



Eliminarea se efectuează în spațiul coordonatelor globale, folosind volumul vizual definit de aplicație (trunchi de piramidă sau paralelipiped dreptunghic).

3. Scena - Axis Aligned BSP Tree (1)

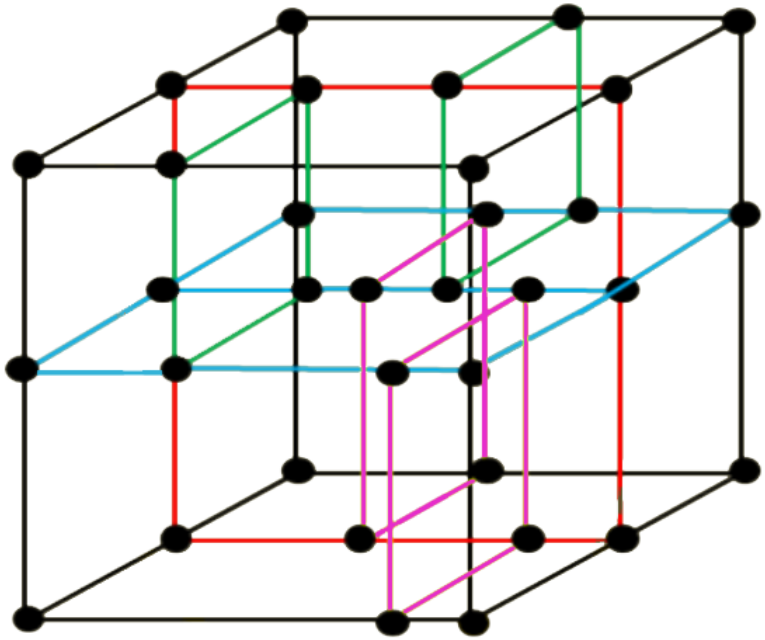
- **Planele de partiționare sunt aliniate cu axele sistemului de coordonate 3D -** perpendiculare pe cele 3 axe: Axis-Aligned BSP Tree (AA-BSP)
- AA-BSP tree este un k-d tree (*k-dimensional tree):* arbore binar în care fiecare nod corespunde unei coordonate într-un spațiu k-dimensional. Pentru k=3, un nod va corespunde uneia dintre coordonatele x, y sau z. De exemplu, dacă planul de partiționare al unui nod este perpendicular pe axa OX, atunci nodul este asociat coordonatei x a planului.
- **Construirea arborelui:**

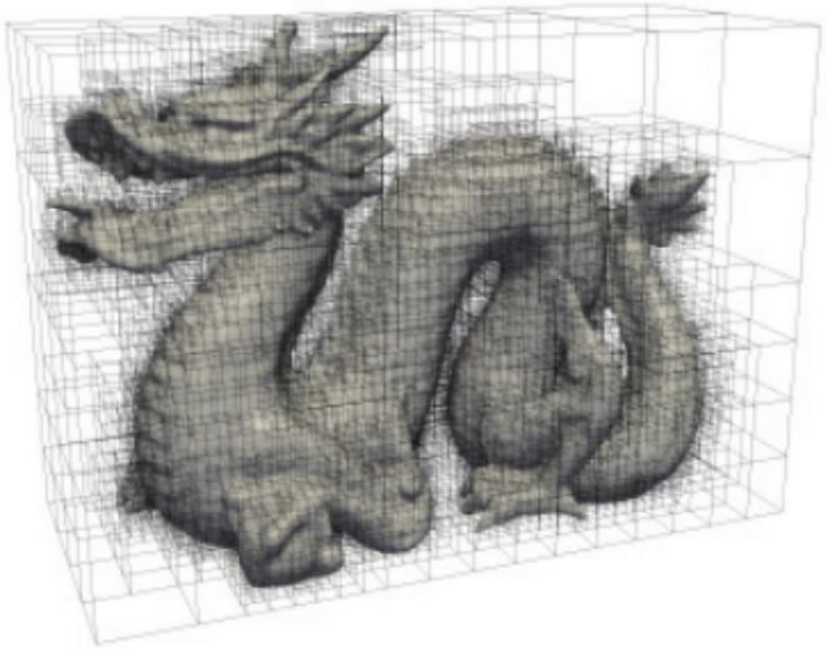
  - Scena este divizată recursiv prin plane paralele cu planele sistemului de coordonate, într-o ordine fixă, ținând cont și de geometria scenei; de exemplu, în pasul 1 se alege un plan perpendicular pe OX, în pasul 2 un plan perpendicular pe OY, în pasul 3 un plan perpendicular pe OZ, în pasul 4 un plan perpendicular pe OX, s.a.m.d.
- **Alegerea planelor:**

  - numărul de obiecte din fiecare semispațiu să fie același sau diferit cu 1, pentru ca arborele să fie echilibrat

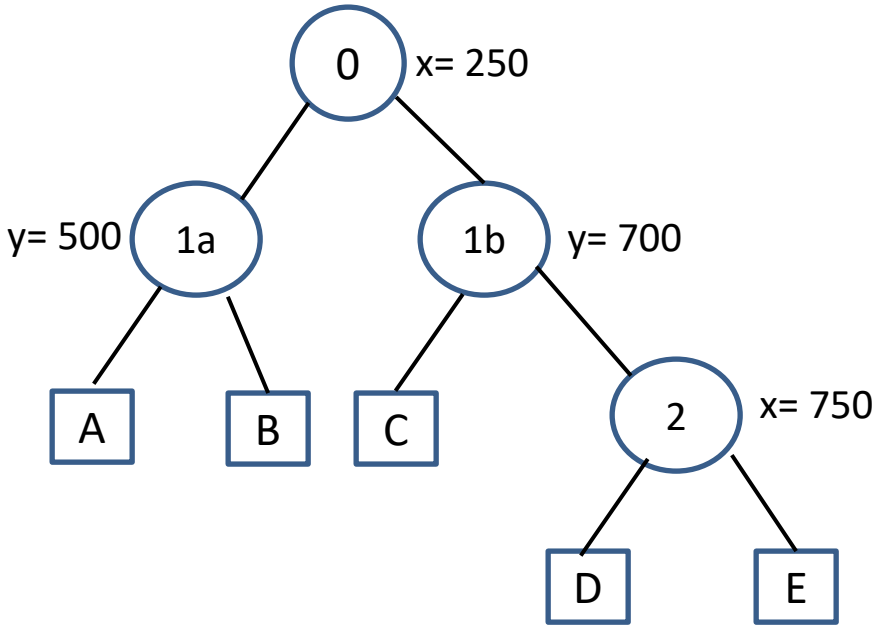
  - numărul de intersecții dintre planele de partiționare și obiectele scenei să fie minimizat.

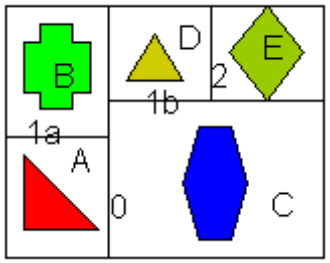
3. Scena - Axis Aligned BSP Tree (2)





3. Scena– Axis Aligned BSP Tree (3)





Exemplu de divizare
cu plane aliniate cu axele, în 2D:

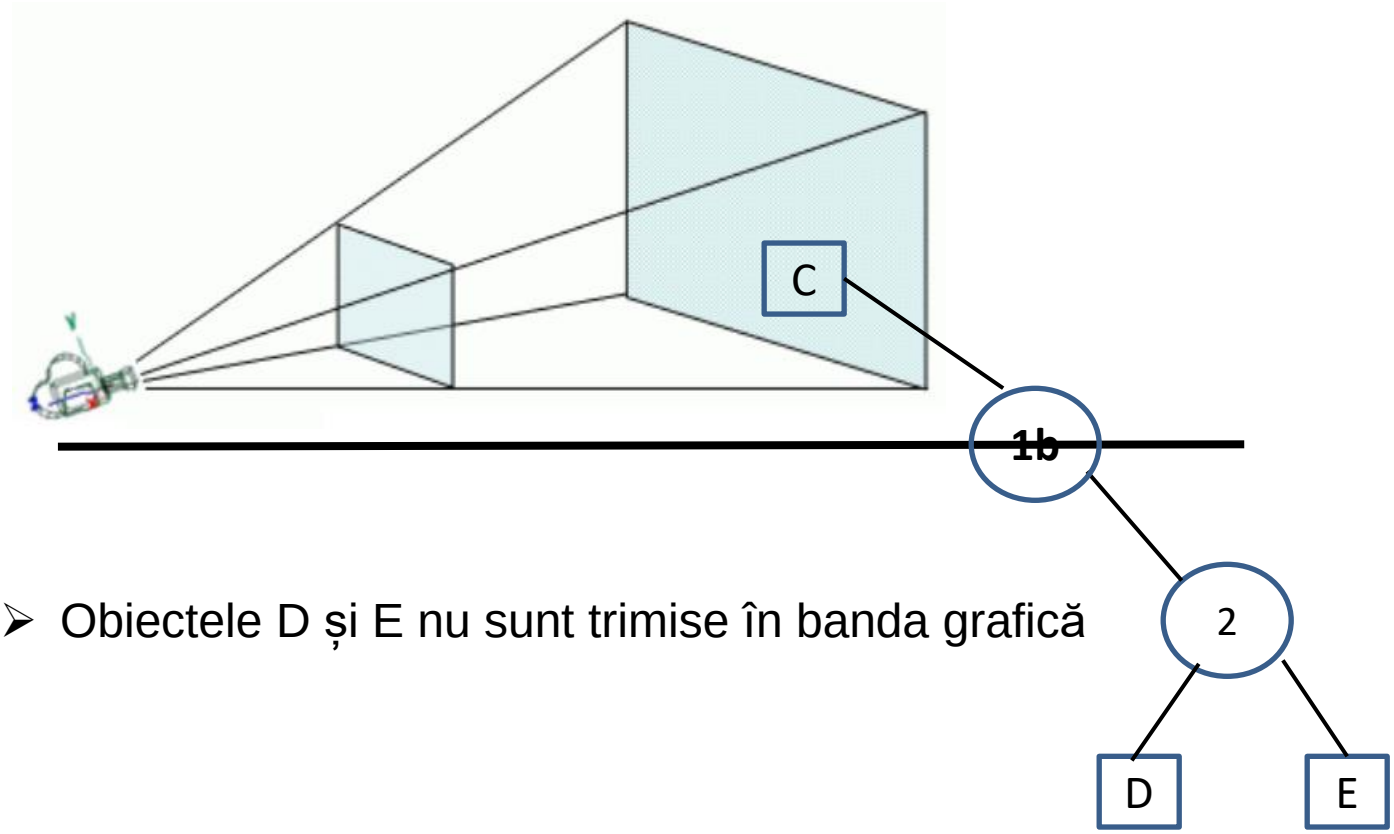
În fiecare pas planul de divizare este ales a.i. să se evite intersecția sa cu obiectele scenei.

❖ **Eliminarea obiectelor nevizibile: la fel ca în cazul arborelui PA- BSP:** dacă planul de partitionare al unui nod nu intersectează volumul vizual, întregul grup de obiecte din subarborele aflat de partea opusă volumului este eliminat din banda grafică.

3. Scena– Axis Aligned BSP Tree (4)

❖ **Afisarea scenei 3D folosind arborele AABSP**

Dacă planul atașat unui nod nu intersectează volumul vizual atunci întregul grup de obiecte din subarborele aflat de partea opusa volumului **NU** este trimis în banda grafică

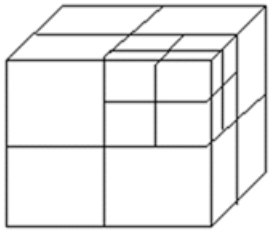


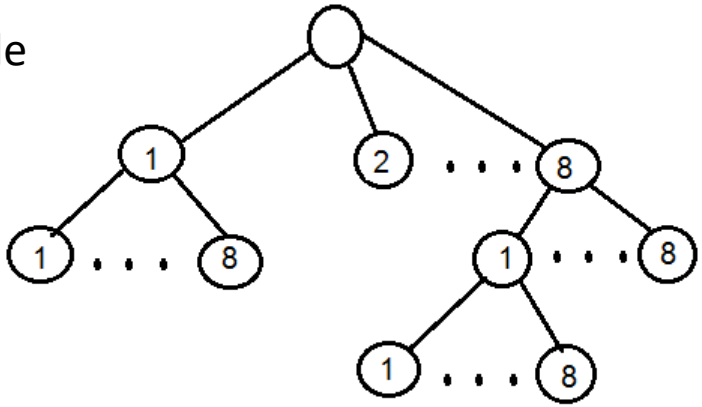
➢ Obiectele D și E nu sunt trimise în banda grafică

4. Scena - Arbore octal (1)

❖ **Construirea arborelui octal al scenei**

- Se pleacă de la volumul încadrator al scenei, aliniat cu axele, care se asociază nodului rădăcină al arborelui.
- Se divizează volumul încadrator în 8 subvolume egale
- Se împart primitivele între cei 8 octanți





- Se continuă cu divizarea recursivă a subvolumelor și asocierea primitivelor cu subvolumele;

fiecare primitivă este asociată cu cel mai mic octant care o încadrează complet.

- Un subvolum nu este divizat dacă s-a ajuns la un nivel de divizare prestabilit sau subvolumul conține suficient de puține primitive sau nici o primitivă.
- Rezultă un arbore in care fiecare nod intern (care nu este frunză) are 8 copii.

Scena - Arbore octal (2)

❖ **Fiecare nod al arborelui octal are ataşat un volum incadrator**

➢ **Eliminarea obiectelor nevizibile folosind arborele octal al scenei : la fel ca în cazul BVH**

- Se efectuează testând intersecțiile subvolumelor arborelui cu volumul vizual (cub, în sistemul coordonatelor de decupare), începând cu nodul rădăcină.

- Daca volumul asociat nodului curent intersectează volumul vizual, se coboară în arbore, testând intersecția dintre fiecare copil al său și volumul vizual.

- Daca subvolumul asociat unui nod este în afara volumului vizual, atunci se revine în arbore pe nivelul anterior (se elimină din banda grafică toate obiectele din subarborele nodului).

Scena - Arbore octal (3)

❖**Eficiența arborelui octal al scenei**

- Subdivizarea strictă a spațiului împreună cu metoda de asociere a fiecarei primitive cu cel mai mic octant care o încadrează complet, poate conduce la ineficienţă;

de ex., daca o primitivă este situată în apropierea centrului volumului încadrator al scenei, primitiva va fi asociata cu acest volum, chiar daca este foarte mică în raport cu volumul!

- O alternativă: o primitivă poate fi asociată cu 2 sau mai multi octanți vecini care împreună o încadrează complet și formează un volum minimal (crește însă complexitatea construirii arborelui).

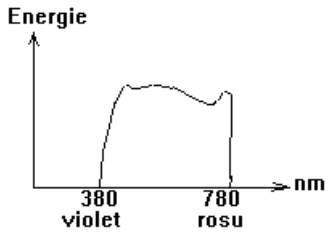
Proprietățile luminii
Modele de culoare

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

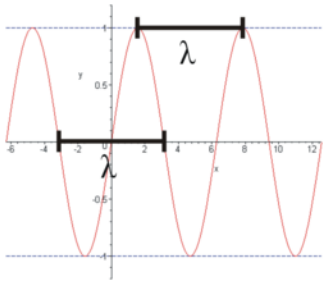
Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

Lumina

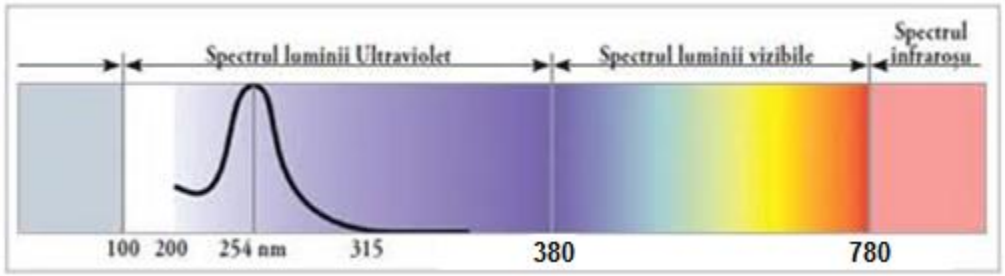
- Lumina este energie electromagnetică: spectrul vizibil
- Spectrul vizibil este alcătuit din undele cu lungimi cuprinse intre 380-780 nanometri.



Spectrul vizibil



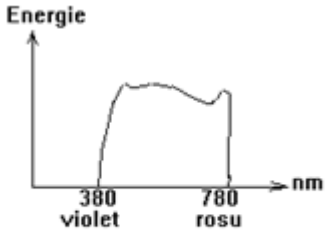
Lungimea de undă: distanța dintre două puncte din spațiu între care unda a parcurs o perioada de repetiție.



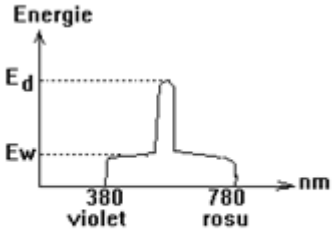
Proprietățile luminii (1)

- Atunci când cade pe suprafaţa unui obiect, lumina poate fi: absorbită, reflectată sau transmisă

- **Sistemul vizual uman percepe: - lumina provenită direct de la o sursă şi**
  - lumina reflectată sau transmisă de obiectele din mediu.



Lumina acromatica



Lumina cromatica

- Lumina care conţine toate lungimile de undă din spectrul vizibil în proporţii aproximativ egale se numeşte **acromatică**.
- Lumina în care predomină anumite lungimi de undă se numeşte **cromatică.**

- Lumina acromatică provenită de la o sursă este percepută ca albă.
- Lumina acromatică provenită de la un obiect este percepută ca albă, neagră sau ca o nuanţă de gri în funcţie de proprietăţile fizice ale suprafeţei obiectului.
- Obiectele care reflectă acromatic mai mult de 80% din lumina incidentă albă apar ca albe. Cele care reflectă acromatic mai puţin de 3% din lumina incidentă apar ca negre.

Proprietățile luminii (2)

Culoarea unui obiect, percepută de ochiul uman, depinde atât de distribuţia lungimilor de undă în lumina care cade pe obiect cât şi de caracteristicile fizice ale suprafeței obiectului: reflectă/absoarbe anumite lungimi de undă.

- Energia electromagnetică nu are culoare!
- **Culoarea este rezultatul unui proces psiho-fiziologic**.

Definiţia psiho-fiziologică a unei culori cuprinde: nuanța, luminozitatea (strălucirea), saturația

–**Nuanţa:** roşu, galben, verde, etc. – determinată de lungimea de undă dominantă a distribuţiei spectrale a luminii.

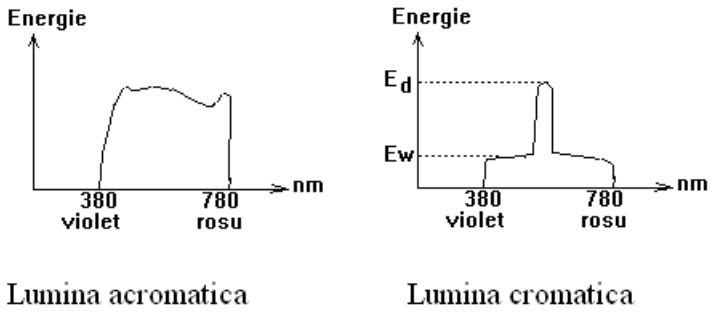
–**Luminozitatea sau strălucirea** - reprezintă intensitatea luminii.

– Luminozitatea: caracteristică a unui obiect emiţător de lumină

– Strălucirea: caracterizează un obiect neemiţător, care reflectă lumina.

Proprietățile luminii (3)

- **Saturaţia sau puritatea -** o măsură a amestecului de alb într-o culoare pură;
  - permite să se facă distincţie între roşu şi roz, între albastru şi bleu, etc;
  - o culoare pură are saturaţia 100%;
  - lumina acromatică are saturaţia 0%.



Lumina acromatica Lumina cromatica

Culoarea este data de lungimea de unda dominantă.

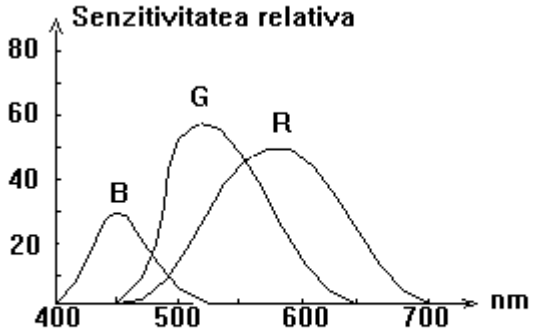
Ew=0 – culoarea are puritatea 100% : monocromatică

(largime de banda de un nanometru)

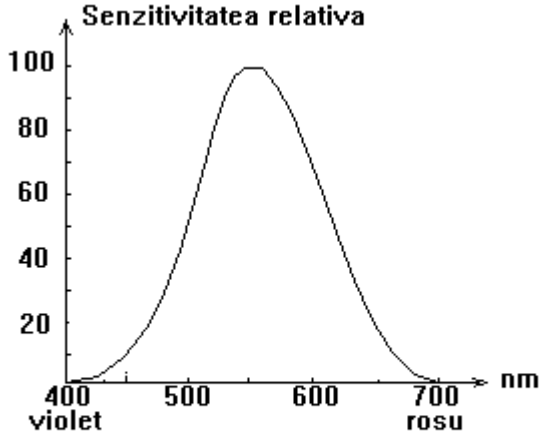
– Cea mai cunoscută dintre teoriile privind formarea culorilor în sistemul ochi-creier este aceea conform căreia în retina ochiului uman există trei tipuri de conuri, fiecare tip fiind sensibil la una dintre culorile roşu, verde şi albastru: teoria tricromaticității (Tristimulus theory)

Teoria tricromaticității (1)

- Culorile roșu, verde și albastru se numesc culori primare.
- Cf. teoriei tricromaticității, orice culoare care poate fi percepută de ochiul uman se obține printr-un amestec aditiv al celor 3 culori primare.
- Prin amestecul lor în proporţii egale se obţine alb.



Raspunsul ochiului la cele 3 culori primare



Curba de luminozitate
Suma celor 3 curbe de senzitivitate

Senzitivitatea maximă corespunde luminii cu lungimea de undă în jur de 550 nm, percepută ca galben-verde.

Teoria tricromaticității (2)

- **Două culori care prin amestec produc lumină albă se numesc complementare.**
- Culorile complementare culorilor primare sunt : cian (Cyan), magenta (Magenta), galben (Yellow).
- Amestecând două culori primare în proporții egale se obţine culoarea complementară celei de a treia:

  R **G** **B**

  C M Y

- De exemplu, albastru+verde=cian, roşu+verde=galben, roşu+albastru= magenta.
- Culorile **roşu, verde şi albastru** se numesc şi **“primitive aditive“** deoarece ele permit formarea de culori prin **adunarea** lor în diferite proporţii.
- Acest mod de definire a culorilor corespunde echipamentelor emiţătoare de lumină (dispozitive de vizualizare cu ecran).

Teoria tricromaticității (3)

- Culorile **cian, magenta, galben** se numesc **"primitive extractive"**.
- Se obtin prin **extragerea** culorilor primare din lumina albă:

cian = alb-roşu, magenta = alb-verde, galben = alb-albastru

R G B
C M Y

– **Se folosesc pentru a defini culorile reflectate de documentele imprimate**: pigmenţii existenţi în cernelurile tipografice absorb din lumina incidentă componentele complementare acelora ale pigmenţilor.

Ex: pigmentul de culoare magenta absoarbe din lumina incidentă lumina verde, iar cel de culoare galben, lumina albastra.

➢ O suprafaţă care conţine pigmenţi magenta şi galben va reflecta (sau transmite) lumină roşie.

– **C, M, Y permit specificarea de nuanţe prin extragerea culorilor primare în diferite proporţii din alb. Scăzându-le în proporţii egale din alb se obţin diferite nuanţe de gri.**

Modele de culoare folosite în sinteza imaginilor (1)

- Orientate către echipamente
  - se bazează pe culorile primare folosite de echipamente pentru redarea culorilor:

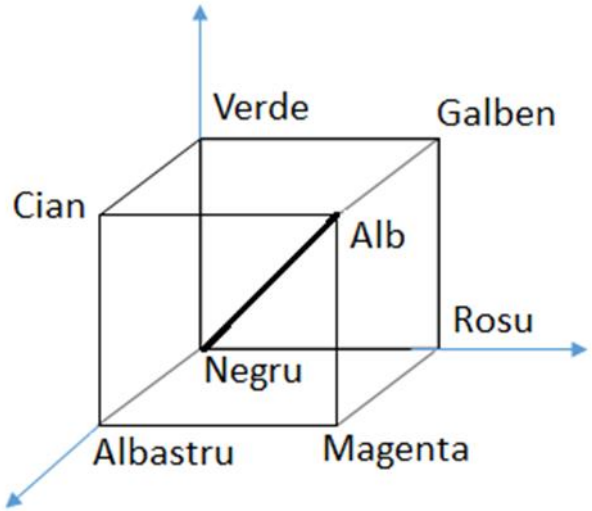
    RGB, CMY
  - YIQ: pentru transmisia imaginilor TV color

- Orientate către utilizator
  - se bazează pe proprietăţile psiho-fiziologice ale culorilor :
    - HSV şi HSL.

Modele de culoare (2)

- Un model de culoare specifică un sistem de coordonate 3D şi un subspaţiu al culorilor în sistemul de coordonate respectiv.
- Fiecare culoare se reprezintă printr-un punct în subspaţiul culorilor.

Modelul RGB (Red, Green, Blue)



Rosu: 1,0,0

Verde: 0,1,0

Albastru: 0,0,1

Alb: 1,1,1; Negru: 0,0,0

Cian: 0, 1, 1 - culoarea complementară culorii Roșu !

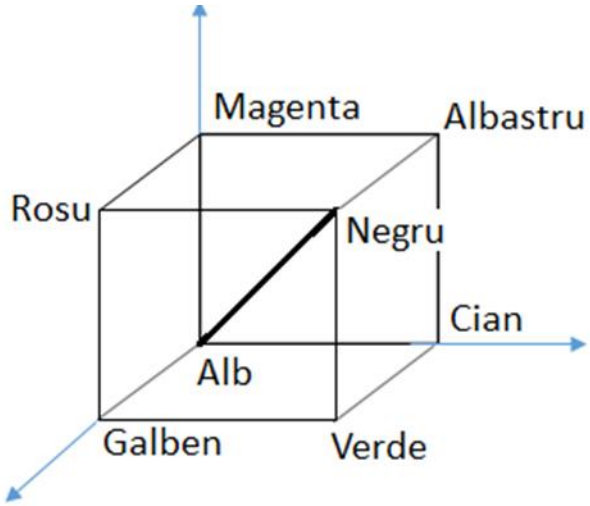
Pe diagonala Negru-Alb: nuanțe de gri – obținute prin amestec în proporții egale între R,G,B

Modele de culoare (3)

Modelul CMY (Cyan, Magenta, Yellow)

CMYK (Cyan, Magenta, Yellow, Key) → key = negru





Cian: 1,0,0

Magenta: 0, 1, 0

Galben: 0,0,1

Alb: 0,0,0

Negru: 1,1,1 - (R,G,B) sunt extrase în proporție maximă din lumina alba

Roșu: 0, 1, 1 - Complementara culorii Cian!

Unele dispozitive de imprimare includ culoarea negru între culorile C,M,Y: CMYK

Conversia RGB ←→ CMY

[C M Y] = [1 1 1] - [R G B]
[R G B] = [1 1 1] - [C M Y]

Culorile reflectate (RGB):
Cian: 1,1,1 – 1,0,0 = 0,1,1
Magenta: 1,1,1 – 0,1,0 = 1,0,1
Negru: 1,1,1 – 1,1,1 = 0,0,0

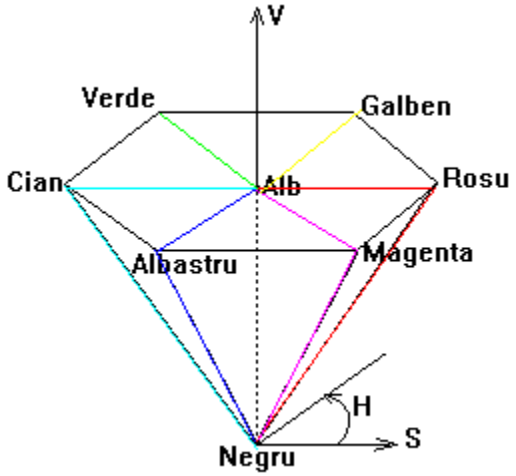
Modele de culoare (4)

Modelul HSV (Hue, Saturation, Value)

- Model de culoare orientat către utilizator

- Artiştii specifică culorile prin tente, nuanţe şi tonuri





0<= V<= 1, 0<= S <= 1, 0<= H <= 360

V=1 - baza hexaconului

S=0 - pe axa hexaconului

S=1 - pe frontiera bazei hexaconului: culorile pure

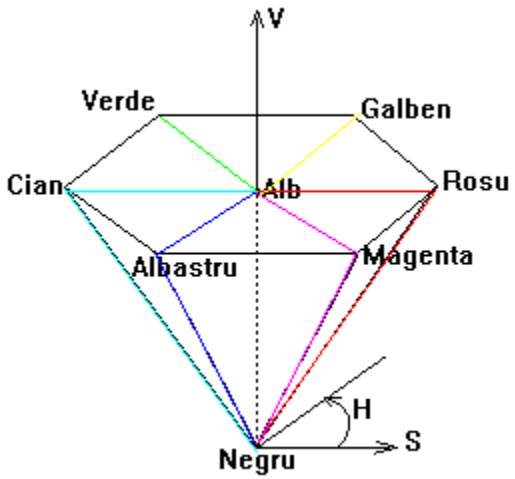
Rosu: (0, 1, 1)

Culorile complementare – la 180 grade pe baza hexaconului

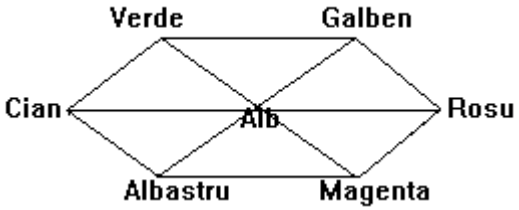
Cian: (180, 1, 1)

- Hexaconul din figură reprezintă subspațiul culorilor în modelul HSV

Modele de culoare (5)



- Saturaţia într-un punct este definită prin distanţa de la axa hexaconului la punctul respectiv.
- Combinaţia (S=0,V=1) corespunde culorii alb iar prin (S=0, 0<=V<=1) se reprezintă nivelurile de gri.
- Pentru S=0 valoarea lui H este nesemnificativă.
- Culorile pure sunt reprezentate prin V=1 şi S=1.

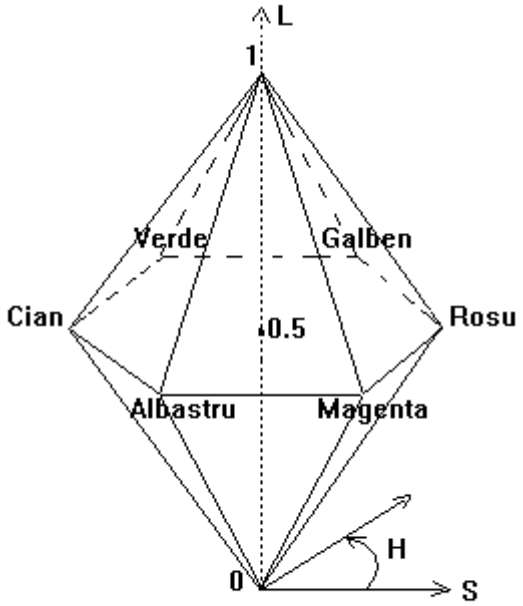


- Baza hexaconului corespunde suprafeţei văzute atunci când se priveşte cubul RGB dinspre vârful (1,1,1) de-a lungul diagonalei principale.

- Fiecare plan de V constant din spaţiul HSV corespunde vederii unui subcub din cubul RGB.
- Diagonala principală a cubului RGB devine axa principală în spaţiul HSV.
- Această interpretare sta la baza algoritmilor de conversie între modelele RGB şi HSV.

Modele de culoare (6)

Modelul HSL (Hue, Saturation, Lightness)



Modelul HLS

Culorile primare cu saturaţie maximă şi complementarele lor sunt reprezentate prin S=1, L=0.5.

Conversii RGB → HSV/ HSL

max = sup(R, G, B) min = inf(R, G, B)

V = max

S= (max-min)/max daca max ≠ 0

0 daca max =0

H = 0 daca (max – min) = 0

(G – B) / (max – min) daca R = max

(B – R) / (max – min) + 2 daca G = max

(R – G) / (max – min) + 4 daca B = max

H = H x 60

daca H<0 H = H + 360

RGB → HSV

L = (max-min)/2

S = (max – min)/(max + min) daca L<=0.5

= (max – min)/ (2 – max – min) daca L > 0.5

H = 0 daca (max – min) = 0

(G – B) / (max – min) daca R = max

(B – R) / (max – min) + 2 daca G = max

(R – G) / (max – min) + 4 daca B = max

H = H x 60

daca H<0 H = H + 360

RGB → HSL

Interpolarea în spațiul culorilor

- Rezultatul interpolării între două culori depinde de modelul de culoare în care sunt specificate.
- Fie două culori specificate în modelul RGB, C1=(1,0,0) şi C2=(0,1,0). Le interpolăm cu ponderi egale în modelele RGB şi HSV:

- Interpolare in modelul RGB:

  C=t*C2 + (1-t)*C1 unde t=0.5, se obţine

  C=(0.5, 0.5, 0), care convertită în HSV ne dă (60, 1, 0.5)

- Interpolare in modelul HSV:

  C1 se reprezintă în HSV prin (0, 1, 1) iar

  C2 prin (120, 1, 1)

  C= 0.5*(0,1,1) + 0.5*(120,1,1) = (60, 1, 1)

Modele de reflexie

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

Modele empirice pentru calculul reflexiei luminii

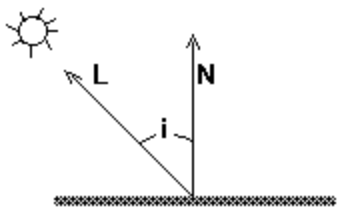
- Caracteristicile luminii reflectate într-un punct al suprafaţei unui obiect depind de :
  - lungimile de undă conţinute în lumina incidentă,
  - proprietăţile materialului din care este construită suprafaţa,
  - direcţia şi geometria sursei luminii incidente,
  - orientarea suprafeţei în punctul respectiv.

- Expresia care modelează intensitatea luminii reflectate într-un punct al unei suprafeţe este definită pentru o suprafaţă necolorată şi o **lumină incidentă monocromatică**, deci caracterizată printr-o anumită lungime de undă, λ.
- În cazul general, **lumina reflectată nu este monocromatică**, de aceea pentru calculul său expresia ar trebui să fie evaluată continuu pe întregul domeniu al spectrului de modelat.
- În practică, expresia se evaluează separat pentru cele trei culori primare, R, G, B.

Modelarea reflexiei difuze a luminii provenite de la o sursă

Aproximarea reflexiei difuze într-un punct al unei suprafețe 3D

- Lumina reflectată difuz de o suprafaţă este dispersată regulat în toate direcţiile.
- **Legea lui Lambert** defineşte reflexia luminii provenite de la o **sursă punctiformă**, de către un difuzor perfect:



Reflexia difuza

$Id = Isursa * kd * \cos(i) \qquad 0 <= i <= \pi/2$

N – normala la suprafață în punctul considerat

Isursa – este intensitatea luminii incidente (provenită de la sursa de lumină)

Kd – este coeficientul de difuzie a luminii incidente, dependent de materialul suprafeței: suprafața reflectă/absoarbe/transmite lumina în anumite proportii: $0 <= kd <= 1$

Id este intensitatea luminii reflectate difuz de suprafață

- Dacă i este mai mare ca π/2, suprafaţa nu primeşte lumină de la sursă (sursa de lumină se află în spatele suprafeţei).

Modelarea reflexiei luminii ambientale

Lumina ambientală

- Modelează lumina provenită de la celelalte obiecte ale scenei 3D: este o sursă de lumină distribuită uniform în spațiu.
- Lumina ambientală este reflectată la fel în orice punct al unei suprafețe. Intensitatea sa depinde de intensitatea luminii ambientale și de proprietățile suprafeței.
- Reflexia luminii ambientale este modelată prin:

  Ia*ka, unde

  Ia – este intensitatea luminii ambientale iar

  0<= ka<= 1 este coeficientul de reflexie a luminii ambientale, dependent de materialul suprafeței.

Atunci:

Id – intensitatea luminii reflectate difuz într-un punct al unei suprafețe

Id = reflexia luminii ambientale + reflexia difuză a luminii de la sursa =

Ia * ka + Isursa * kd* cos(i)

0 <= i <= π/2

Modelarea atenuării luminii de la o sursă

Sursă direcțională: sursa aflată la distanță foarte mare (infinit) de scena 3D

- Vectorul L este același în orice punct al suprafeței
- Pentru 2 suprafețe paralele, cu aceleași proprietăți de material, rezultă aceeași Id → dacă proiecțiile suprafețelor se suprapun în imagine, ele nu se vor distinge.

Intensitatea luminii descreşte proporţional cu inversul pătratului distanţei de la sursa de lumină la suprafață.

Id = Ia* ka + fat * Isursa* kd* cos(i), unde fat = $1/d^2$ este funcţia de atenuare a luminii de la o sursă

d este distanţa de la sursă la punctul de pe suprafaţă considerat.

- Corecţia nu satisface cazurile în care sursa este foarte îndepărtată: atenuare prea mare
- Dacă sursa este la distanţă foarte mică de scenă, atenuările obţinute pentru două suprafeţe aflate la distanțe puțin diferite de sursă, vor fi mult diferite.
- În practică se folosește factorul de atenuare: fat = $\min(1/(c1 + c2*d + c3*d^2), 1)$

  c1, c2 şi c3 sunt trei constante care se asociază sursei de lumină.
- Constanta c1 se alege astfel încât numitorul să nu devină prea mic atunci când sursa este foarte apropiată. Valoarea funcţiei este limitată la 1 pentru a se asigura atenuarea.

Culoarea luminii reflectate difuz (1)

- Lumina incidentă poate conține mai multe lungimi de undă: → reflectate în mod diferit de o suprafață

  Csursa = [IsursaR, IsursaG, IsursaB] – culoarea luminii sursei

  Ca = [IaR, IaG, IaB] – culoarea luminii ambientale

- O suprafață reflectă în mod diferit componentele R, G, B ale luminii incidente.

❖ Ex: o suprafață care absoarbe din lumina incidentă componentele de verde (G) și albastru (B),

- Dacă lumina incidentă este albă, suprafața va fi văzută ca roșie:

  [1,1,1] – [0,1,1] = [1,0,0]

- Dacă lumina incidentă este roșie, suprafața va fi vazută tot ca roșie:

  [1,0,0] – [0,1,1] = [1,0,0] (componentele G și B au valoarea zero – nu pot fi extrase)

- Dacă lumina incidentă este verde, suprafața va fi văzută ca neagră (componenta verde a fost absorbită):

  [0,1,0] – [0,1,1] = [0,0,0]

➢ Rezultă: coeficienții de difuzie a luminii de către o suprafață depind de componentele luminii incidente și de proprietățile suprafeței

Culoarea luminii reflectate difuz (2)

ka = [kaR, kaG, kaB] – coeficienții de difuzie a componentelor R, G, B din lumina ambientală

kd = [kdR, kdG, kdB] – coeficienții de difuzie a componentelor R, G, B din lumina provenita de la sursă

- Expresia luminii reflectate difuz se evaluează separat pentru cele 3 componente ale luminii incidente, R,G,B:

  $Id\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda * kd\lambda * \cos(i)$, unde λ: lungimea de undă

sau:

- Cd = [IdR, IdG, IdB] - culoarea luminii reflectate difuz

  Cd = [IaR *kaR + fat*IsursaR *kdR *cos(i),

  IaG *kaG + fat*IsursaG *kdG *cos(i),

  IaB *kaB + fat*IsursaB *kdB *cos(i)]

- ka (kaR, kaG, kaB) și kd (kdR, kdG, kdB) sunt caracteristici ale materialului suprafeței, numite: culoarea ambientală și culoarea difuză a materialului.

Modelarea reflexiei difuze

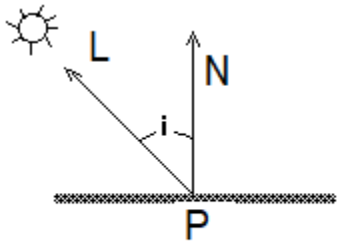
- Știind că

  cos(i)=L·N / (|L|·|N|) = Lu·Nu

  unde: N este normala la suprafață în P

  L este vectorul din P către sursa de lumină

  (L·N) – produsul scalar al vectorilor L, N



Reflexia difuza in punctul P

- Rezultă ecuația care modelează lumina reflectată difuz:

❖ $Id\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda * kd\lambda * (Lu \cdot Nu)$

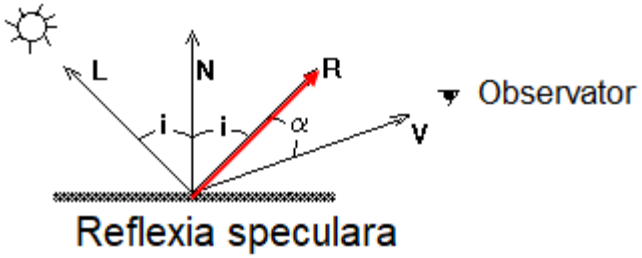
Cd = [IdR, IdG, IdB] – culoarea luminii reflectate difuz

Pentru a include și cazul în care $i > \pi/2$ (lumina de la sursă nu ajunge în punctul considerat):

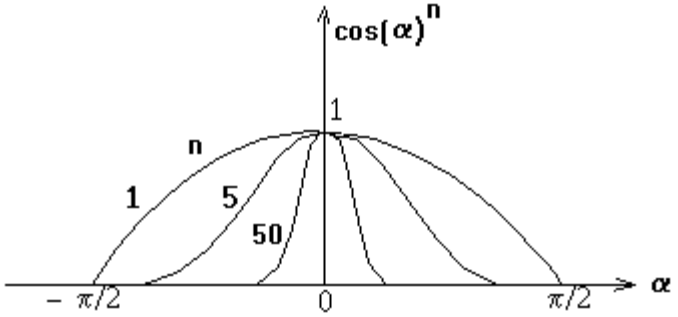
$I\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda * kd\lambda * \max((Lu \cdot Nu), 0)$

Modelarea reflexiei speculare (1)

- Un reflector perfect, de exemplu o oglindă, reflectă lumina numai într-o singură direcţie, R, care este simetrică cu L faţă de normala la suprafaţă → numai un observator situat exact pe direcţia respectivă va percepe raza reflectată:



Reflexia speculara



Functia de distributie spatiala a luminii reflectate specular

• Pentru materialele imperfect reflectante cantitatea de lumină care ajunge la observator depinde de distribuţia spaţială a luminii reflectate specular:

- la suprafeţele netede (ex. metale) distribuţia este dreaptă şi focalizată;
- la suprafeţele cu rugozităţi (ex. hârtia) ea este dispersată.
- se aproximeaza prin $\cos(\alpha)^n$ (modelul Bui-Tuong Phong) unde n este exponentul de strălucire (shininess) al materialului.

Modelarea reflexiei speculare (2)

Modelul Phong pentru aproximarea reflexiei speculare într-un punct al unei suprafețe 3D:

$$Is\lambda = Isursa\lambda * w(i,\lambda) * \cos(\alpha)^n$$

w(i, λ) este funcţia de reflectanţă, i - unghiul de incidenţă iar λ - lungimea de undă a luminii incidente

În practică, w(i, λ) este înlocuit cu o constantă determinată experimental, numită **coeficientul de reflexie speculară al materialului, notat** ksλ.

- Modelul practic al reflexiei speculare:

$$Is\lambda = Isursa\lambda * ks\lambda * \cos(\alpha)^n$$

$\cos(\alpha)=R \cdot V / (|R|\cdot|V|) = Ru \cdot Vu$

Rezultă:

$$Is\lambda = Isursa\lambda * fat * ks\lambda * (Ru \cdot Vu)^n$$

Pentru a include și cazul în care α >= 90:

$$Is\lambda = Isursa\lambda * fat * ks\lambda * \max((Ru \cdot Vu)^n , 0)$$

Modelarea reflexiei speculare (3)

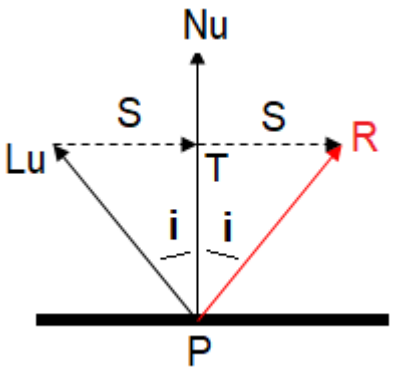
Reflexia speculară nu poate avea loc dacă în punctul considerat nu se primește lumină de la sursă:

$$Is\lambda = lum * Isursa\lambda * fat * ks\lambda * \max((Ru \cdot Vu)^n, 0)$$

$$lum = 1 \text{ daca } (Lu \cdot Nu) > 0 \quad (0 <= i < \pi/2)$$

$$= 0 \text{ altfel}$$

Calculul direcției reflexiei speculare



- Vectorul R este simetricul vectorului L față de N.
- Proiecția scalară a vectorului Lu pe direcția normalei este mărimea segmentului PT: $|Lu|\cos(i) = \cos(i) = Lu \bullet Nu$
- Proiecția vectorială a lui Lu pe direcția normalei este **vectorul PT**, care are lungimea segmentului PT și direcția normalei:

$(Lu \bullet Nu) * Nu / |Nu| = (Lu \bullet Nu) * Nu$

$R = Lu + 2 * S$, $PT = Lu + S \rightarrow S = PT - Lu$

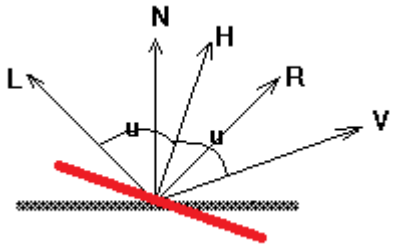
$R = Lu + 2(PT - Lu) = Lu + 2 * (Lu \bullet Nu) * Nu - 2 * Lu = 2 * (Lu \bullet Nu) * Nu - Lu$

$R = 2 * (Lu \bullet Nu) * Nu - Lu$ **sau în glsl: R = *reflect*(-Lu, Nu)**

Modelarea reflexiei speculare (4)

O altă formulare a modelului Phong pentru reflexia specular folosește vectorul median, notat cu H în figură. El face unghiuri egale cu L şi cu V.

- Dacă suprafaţa ar fi orientată astfel încât normala să aibă direcţia lui H, atunci observatorul ar percepe lumina specular maximă (deoarece ar fi pe direcţia razei reflectate specular).



- Termenul care exprimă reflexia specular este în acest caz:

$I_{s\lambda} = I_{sursa\lambda} * f_{at} * k_{s\lambda} * (N_u \bullet H_u)^n$, unde $H_u = (L_u + V_u) / |(L_u + V_u)|$

Atunci când sursa de lumină şi observatorul sunt la infinit, utilizarea termenului $N_u \bullet H_u$ este avantajoasă deoarece Hu este constant (același în toate punctele unei suprafețe).

Modelul de iluminare locală (1)
- lumina reflectată într-un punct al unei suprafețe -

Modelul de iluminare locală (numit şi Modelul de reflexie Phong)

Combină culorile datorate:

reflexiei luminii ambientale – culoarea_ambientala,

reflexiei difuze a luminii de la o sursă / surse – culoarea_difuza

reflexiei speculare a luminii de la o sursă / surse – culoarea_speculara

❖ Reflexia luminii într-un punct al unei suprafeţe determină culoarea sa în acel punct.

$$I\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda \, [kd\lambda * max((Lu \bullet Nu),0) + lum * ks\lambda * max((Hu \bullet Nu)^n, 0)]$$

lum = 1 daca (Lu • Nu) > 0 (0<= i < π/2)

= 0 altfel

Modelul de iluminare locala(2)

- Dacă scena 3D este iluminată de m surse de lumină:

$$I\lambda = Ia\lambda * ka\lambda + \sum_{1<=i<=m} fat_i * Isursa\lambda_i * [kd\lambda * max((Lu_i \bullet Nu),0) + lum_i * ks\lambda * max((Hu_i \bullet Nu)^n,0)]$$

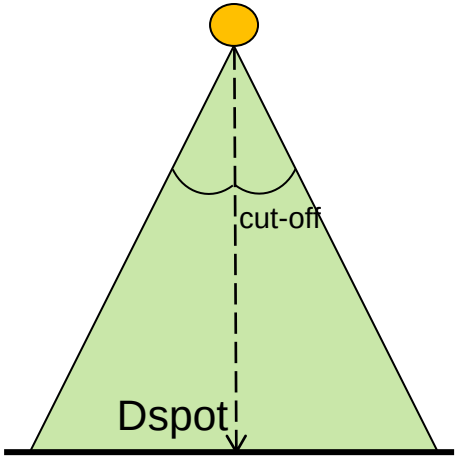
- La cele 3 componente se poate adăuga culoarea_emisa de suprafață. Rezultă:

  Culoarea intr-un punct al unei suprafețe
  = culoarea_emisa + culoarea_ambientala + culoarea_difuza + culoarea_speculara

- **Culoarea emisă** :
  - lumina emisă de o suprafață
  - independentă de sursele de lumină
  - aceeași în orice punct al suprafeței
  - nu luminează obiectele din jur
  - culoarea_emisa = Ke, o constantă

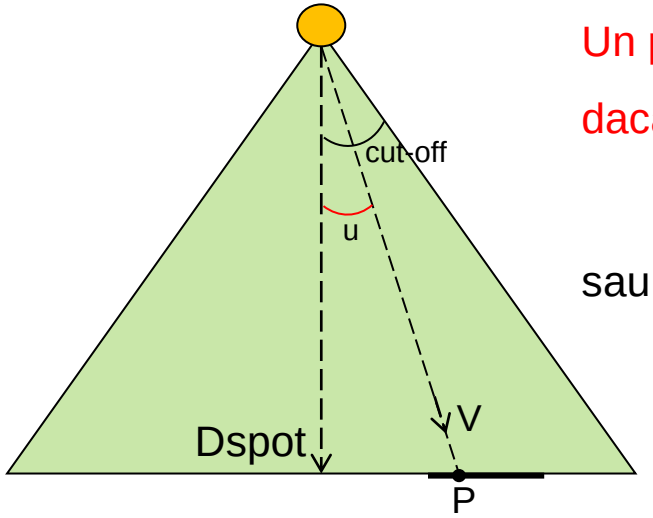
Efectul de spot

- În modelul de iluminare locală, sursa de lumină este considerată punctiformă. Lumina produsă de o sursa punctiformă este omnidirecțională.
- Efectul de spot simulează o sursă de lumină care produce un con de lumină.
- Numai suprafețele aflate în conul de lumină primesc lumina de la sursă.



Conul de împrăștiere a luminii se calculează folosind:

- Poziția spotului (sursei de lumina)
- Direcția spotului
- Împrăștierea luminii spotului (θspot sau cut-off)



Un punct P al unei suprafețe primește lumina de la spot daca u < cut-off:

$$\cos(u) > \cos(\text{cut-off})$$

sau

$$Vu \bullet Du > \cos(\text{cut-off})$$

Transparența
în modelele de iluminare locală

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

MODELAREA TRANSPARENȚEI (1)

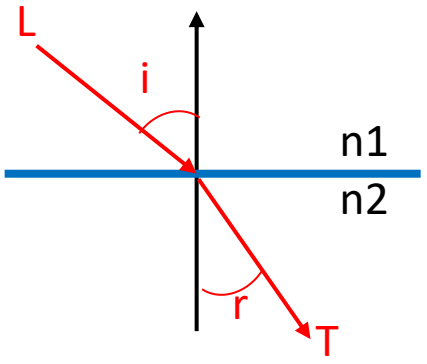
Unele obiecte ale scenei sintetizate pot fi construite din materiale transparente sau translucide.

Transmisia luminii prin obiectele transparente este specular, în timp ce prin cele translucide este difuză.

- Atunci când lumina trece dintr-un mediu într-altul (de exemplu, din aer în apă), direcţia sa se modifică datorită refracţiei.
- Relaţia dintre unghiul razei incidente, i, şi cel al razei refractate, r, este dată de legea lui Snell:

sin(r)/sin(i) = n1/n2

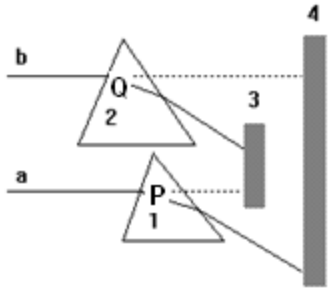
n1 şi n2 sunt indicii de refracţie ai celor două medii (materiale) traversate de lumină.



MODELAREA TRANSPARENȚEI (2)

Indicele de refracţie al unui material este dependent de lungimea de undă a luminii incidente şi chiar de temperatură, dar în modelele de sinteza a imaginilor el este considerat constant.

Culoarea vizibilă într-un punct al unei suprafețe transparente provine de la obiectul aflat pe direcția razei transmise.



3 și 4 sunt poligoane opace

1 si 2 sunt poligoane transparente, aflate în fața poligoanelor 3 și 4

a și b sunt raze de lumină provenind de la surse, incidente în P și Q

Ținând cont de refracție:

- Culoarea în P este dată de poligonul 4
- Culoarea în Q este dată de poligonul 3

Neglijând refracția (T are direcția razei incidente)

- Culoarea în P este dată de poligonul 3
- Culoarea în Q este dată de poligonul 4

MODELAREA TRANSPARENȚEI (3)

- Refracţia produce, de asemenea, o distorsionare a obiectelor, asemănătoare cu aceea produsă de o proiecţie perspectivă: → dacă se doreşte obţinerea de imagini realiste, trebuie să se ţină cont de refracţie.

- Multe metode practice de modelare a transparenţei ignoră refracţia, astfel încât obiectele vizibile printr-o suprafaţă transparentă sunt cele aflate pe direcţia razei incidente.

Motivul ignorării:

- reducerea volumului de calcule;
- obţinerea realismului fotografic în totalitate (fără deformare)

Atunci când suprafaţa vizibilă într-un pixel este transparentă, culoarea în care va afişat pixelul se poate obţine combinând culoarea suprafeţei vizibile cu aceea a suprafeţei aflată imediat în spatele său (sau a fondului), folosind următoarea formulă de interpolare:

MODELAREA TRANSPARENȚEI (4)

$I_\lambda = (1-kt_1)*I_{\lambda 1} + kt_1*I_{\lambda 2}$

$I_{\lambda 1}$ este culoarea suprafeței vizibile în pixel

$I_{\lambda 2}$ este culoarea suprafeței din spatele celei vizibile, care se proiecteaza în același pixel

Coeficientul de transmisie, kt_1, este o măsură a transparenţei suprafeţei vizibile în pixel,

$0 <= kt_1 <= 1$

$kt_1 = 0$ → suprafaţa vizibilă este opacă şi deci pixelul va fi afişat în culoarea sa, $I_{\lambda 1}$

$k_{t1} = 1$ → suprafaţa vizibilă este perfect transparentă şi nu contribuie la culoarea pixelului. Pixelul va fi afişat în culoarea suprafeţei din spate, $I_{\lambda 2}$

Dacă $k_{t1}=1$ şi suprafaţa din spatele celei vizibile este la rândul său transparentă, metoda de calcul se aplică recursiv, până când se întâlneşte o suprafaţă opacă sau fondul.

Aproximarea liniară din model nu dă rezultate bune pentru suprafeţele curbe: în apropierea laturilor siluetei unei suprafeţe curbe (de exemplu, o vază sau o sticlă) grosimea materialului reduce transparenţa.

MODELAREA TRANSPARENȚEI (5)

Soluția propusă de D.S. Kay (SIGGRAPH 79, "Transparency For Computer Synthesized Images"):

kt se calculează în funcție de normala la suprafață în punctul considerat:

$k_t = kt_{min} + (kt_{max} - kt_{min})(1-(1-N_z)^m)$

unde

kt_{min} şi kt_{max} reprezintă transparenţa minimă şi cea maximă a suprafeţei,

N_z este componenta z a normalei normalizate la suprafaţă în punctul pentru care se calculează kt,

m este un exponent ce caracterizează transparenţa. Valorile uzuale pentru m sunt 2 şi 3.

- **Majoritatea algoritmilor de determinare a vizibilităţii suprafețelor la afişarea scenelor 3D pot fi adaptaţi pentru a îngloba transparenţa.**

- In algoritmii care afişează poligoanele scenei 3D în ordinea “din spate în faţă” (back to front), de ex. algoritmul Pictorului şi BSP, $I_{\lambda 1}$ corespunde poligonului care se rasterizează la un moment dat iar $I_{\lambda 2}$ este valoarea existentă în memoria imagine pentru pixelul considerat.

MODELAREA TRANSPARENȚEI (6)

- Adăugarea efectului de transparenţă în algoritmul Z-Buffer este mai dificilă, deoarece poligoanele sunt rasterizate în ordinea în care sunt transmise în banda grafică, neţinându-se cont de distanţa lor faţă de observator.

-Se poate realiza folosind mai multe memorii buffer-imagine și generând imaginea în mai multe etape, imaginea finală obținându-se prin combinarea imaginilor poligoanelor opace cu cele ale poligoanelor transparente

În OpenGL, transparența poate fi simulată prin amestecul dintre culoarea fragmentului curent și cea a pixelului în care se afișează, folosind modelul (R, G, B, A);

A – opacitatea: 0 – transparent; 1 – opac

MODELAREA TRANSPARENȚEI (7)

glEnable(GL_BLEND) - activează amestecul culorilor

glBlendFunc(..) – specifică modul de amestec al culorilor. Exemplu:

glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA):

noua_culoare_pixel = Culoare_fragment * A_fragment + Culoare_pixel * (1 – A_fragment)

- Operația de amestec este o operație raster: efectuată de procesorul de rasterizare dupa testul de adâncime, înainte de actualizarea valorii pixelului în bufferul imagine (frame buffer).

Modele de “shading”

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

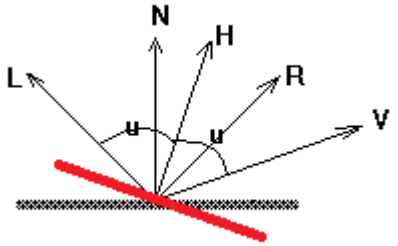
Calculul culorii fragmentelor la redarea suprafețelor 3D (Shading models) (1)

- În vederea vizualizarii, suprafeţele 3D sunt descompuse în faţete poligonale/triunghiulare conectate. Poligoanele procesate în banda grafică pot fi feţele unui corp poliedral (fețe plate) sau pot proveni din descompunerea unei suprafețe oarecare (fețele sunt petice de suprafață curbă).
- La momentul rasterizării nu se cunoaște din ce suprafață face parte un poligon.

Modelul LAMBERT(1)
(Iluminare constantă sau plată)

- Toate fragmentele unui poligon sunt afișate cu aceeași culoare, calculată folosind modelul de iluminare locală (de regulă considerând numai componenta difuză):

$$Id_\lambda = Ia_\lambda * ka_\lambda + fat * Isursa_\lambda * [kd_\lambda * \max((Nu \bullet Lu),0) + lum * \max((Nu \bullet Hu)^n,0)]$$



Modelul LAMBERT (2)

Modelul se bazează pe următoarele presupuneri:

1. Poligonul face parte din suprafaţa de vizualizat şi nu este o aproximare a unui petic de suprafaţă curbă: vectorul Nu este același în orice punct al poligonului.

2. Sursa de lumină este la infinit: produsul scalar (Nu • Lu) este atunci constant pe întreaga suprafaţă a poligonului.

3. Observatorul este la infinit: (Nu • Hu) este constant pe suprafaţa poligonului;

Dacă ultimele două cerinţe nu sunt satisfăcute, se poate adopta o convenţie de calcul al vectorilor L şi V pentru un întreg poligon. De exemplu, cei doi vectori pot fi calculaţi în centrul poligonului.

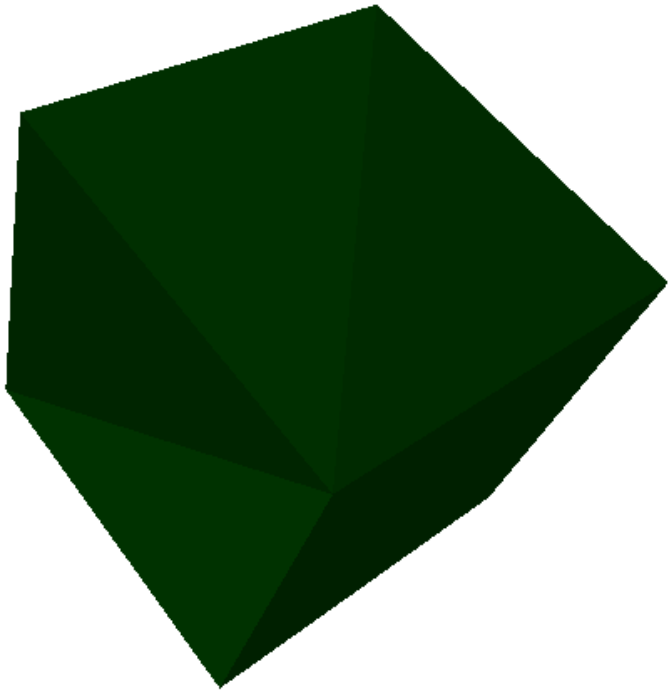
Dacă prima cerinţă nu este îndeplinită, aproximarea unei suprafeţe curbe prin reţeaua de faţete poligonale este vizibilă deoarece intensităţile calculate pentru faţete adiacente cu orientare diferită sunt diferite.

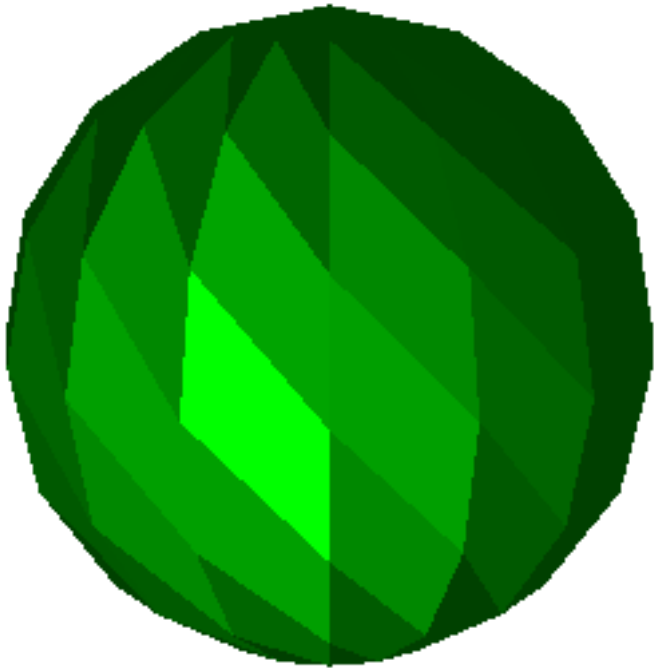
Modelul LAMBERT (3)

Indicații de implementare

```
//vertex shader
layout(location = 0) in vec3 v_position;
layout(location = 1) in vec3 v_normal;
uniform vec3 light_pos;                 // poziția sursei de lumină
uniform vec3 eye_position;              // poziția observatorului
flat out vec3 color;            // culoarea vârfului "provocator" va fi folosită la afișarea
                                // fragmentelor; în mod implicit ultimul vârf al primitivei (ex.:
                                // al 3-lea vârf pt un triunghi)
void main() {
 // calculează culoarea vârfului aplicând modelul de iluminare în poziția vărfului
 color = ambient + diffuse + specular;
 ……
}
//fragment shader
flat in vec3 color; // culoarea vârfului "provocator"
void main() {
gl_FragColor = color;
}
```

Modelul LAMBERT(4)





Efectul de bandă Mach

- Efectul de bandă Mach (descoperit de Mach în 1865) este o iluzie optică datorată filtrarii componentei de luminozitate a imaginii formate pe retină, de catre sistemul vizual uman.
- Efectul măreşte contrastul dintre zone adiacente cu intensitate ușor diferită, la frontiera dintre zone, declanşând detecţia frontierei în sistemul vizual uman.
- Atunci când o suprafaţa curbă (de exemplu o sferă) este descompusa în faţete poligonale, este de dorit ca laturile faţetelor să nu fie vizibile în imaginea sferei.
- În cazul folosirii modelului Lambert pentru calculul culorilor fragmentelor, datorită folosirii unei singure normale pentru toată suprafaţa unei faţete, culorile faţetelor adiacente vor fi ușor diferite, iar efectul de bandă Mach măreşte percepţia schimbării de intensitate pe laturile faţetelor adiacente.

Modelul Gouraud (1)

- La momentul rasterizării unui poligon, fiecărui vârf al poligonului îi este asociată o culoare.
- Culorile fragmentelor poligonului se obțin prin interpolarea liniară a culorilor din vârfuri.

1. Culoarea într-un vârf (ieşire din vertex shader: out vec3 color);

- se calculează aplicând modelul de iluminare locală în vârf (cu N, L, V în cordonate globale)

2. Normala unui vârf, necesară în modelul de iluminare locală, trebuie să fie transmisă la vertex shader din programul de aplicaţie (este intrare pentru vertex shader):

- poate fi calculată exact din ecuația analitică a suprafeței pe care se afla vârful
- poate fi aproximată ca medie aritmetica a normalelor la feţele adiacente în vârf

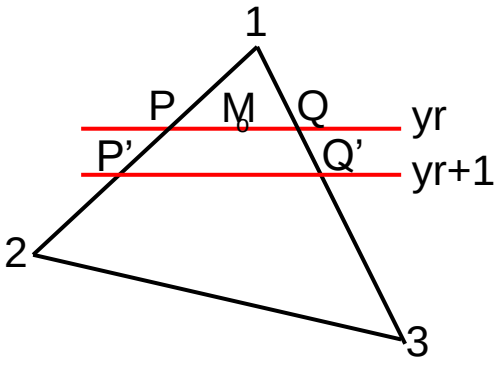
3. La rasterizarea unui poligon, procesorul de rasterizare calculează culoarea fiecarui fragment astfel:

- prin interpolarea liniară a culorilor vărfurilor, pentru fragmentele de pe laturi
- prin interpolare liniară între culorile din capetele segmentului interior pe care se află fragmentul.

Culoarea fragmentului este intrare pentru programul fragment shader (in vec3 color).

Modelul Gouraud (2)

Interpolarea liniară a culorilor



$y = y1 + t\,(y2 - y1) \rightarrow tP = (yr - y1)/(y2 - y1)$

$IP = I1 + tP\,(I2 - I1) = I1 + (I2 - I1)\,(yr-y1) / (y2-y1) = I1 + (yr-y1) * \mathbf{C_{1-2}}$

$C_{1-2} = (I2 - I1) / (y2 - y1)$ - constanta pentru latura 1-2

$IQ = I1 + (I3 - I1)\,(yr-y1) / (y3-y1) = I1 + (yr - y1) * \mathbf{C_{1-3}}$

$C_{1-3} = (I3 - I1) / (y3 - y1)$ - constanta pentru latura 1-3

$x = xP + t(xQ - xP) \rightarrow tM = (xM - xP)/(xQ - xP)$

$IM = IP + tM\,(IQ - IP) = IP + (IQ - IP)\,(xM - xP) / (xQ - xP) = IP + (xM-xP)* \mathbf{C_{P-Q}}$

$C_{P-Q} = (IQ - IP) / (xQ - xP)$ - constanta pentru segmentul P-Q

Calculul incremental al culorilor

$IP' = I1 + (I2 - I1)\,(yr+1 - y1) / (y2 - y1) = I1 + (yr+1 - y1) * \mathbf{C_{1-2}} = IP + \mathbf{C_{1-2}}$

$IP' = IP + C_{1-2}$

analog, **$IQ' = IQ + C_{1-3}$**

M' (xM + 1, yr)

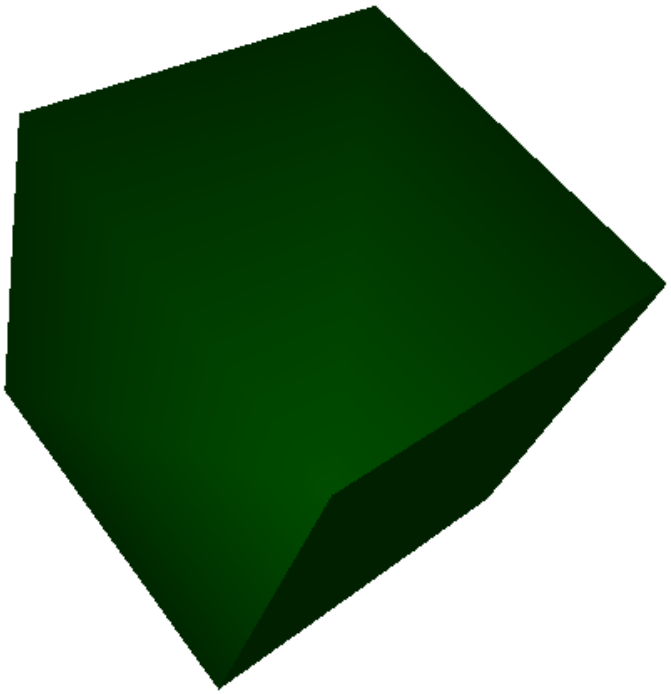
$IM' = IP + (xM +1 - xP) * \mathbf{C_{P-Q}} \rightarrow$ **$IM' = IM + C_{P-Q}$**

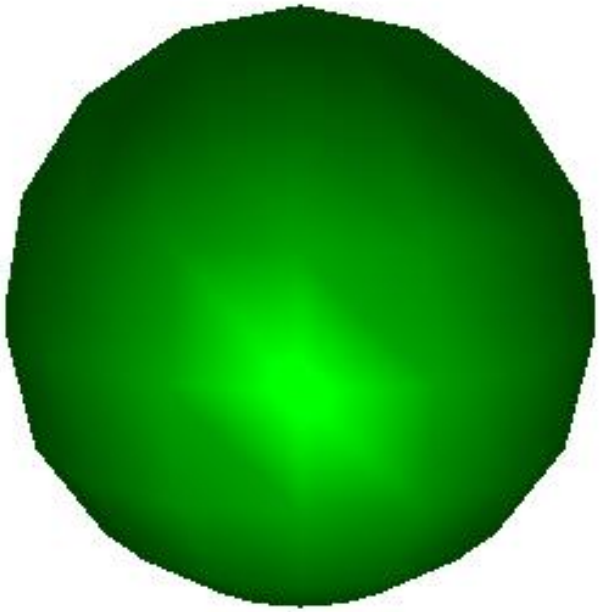
Modelul Gouraud (3)

Aprecieri asupra modelului Gouraud

- Este integrat în algoritmul de rasterizare a poligoanelor
- Calcule simple pentru culoarea unui fragment (calcul incremental) → rapid
- Nu permite calculul luminii reflectate specular pentru fragmentele interioare unui poligon (intensitățile fragmentelor interioare nu pot fi mai mari decât cele din vârfuri)
- Reduce efectul de banda Mach, dar nu îl elimină:
  - Culorile pe latura de adiacență a 2 poligoane sunt aceleași
  - Nu ține cont de tangentele la fețele adiacente pe o latură (culorile de pe laturi se calculează folosind numai culorile din vârfuri)
  - Efectul de banda Mach poate fi observat în vecinătatea siluetei unei suprafeţe 3D și a zonelor de curbura mare.

Modelul Gouraud (4)





Modelul Phong (1)

- La momentul rasterizării unui poligon, fiecărui vârf îi este asociată normala vârfului.
  - Normala vârfului este transmisă din programul de aplicație, ca și pentru modelul Gouraud.
  - Normala și poziția vârfului, în coordonate globale, sunt ieşiri din vertex shader:
  - Procesorul de rasterizare interpolează liniar pozitiile şi normalele vârfurilor, transmițând la fragment shader poziția și normala fragmentului.
- În fragment shader se calculează culoarea fragmentului aplicând modelul de iluminare locală în poziția fragmentului

La rasterizarea unui poligon se calculează o normală pentru fiecare fragment astfel:

- prin interpolarea liniara a normalelor vârfurilor, pentru fragmentele de pe laturi (la fel ca în modelul Gouraud, pentru culori);
- prin interpolare liniară între normalele capetelor segmentului interior pe care se află fragmentul (la fel ca în modelul Gouraud, pentru culori).

Modelul Phong (2)

Dezavantaje față de modelul Gouraud

❑ Componentele Nx, Ny, Nz ale normalei unui fragment se obţin (în algoritmul de rasterizare) printr-un calcul incremental analog cu cel folosit pentru calculul culorilor în modelul Gouraud dar, pentru folosirea lor în calculul culorii, normala trebuie să fie normalizată în fragment shader:

Nu = N/|N|, unde $|N| = (Nx^2 + Ny^2 + Nz^2)^{0.5}$ (se folosește funcția GLSL *normalize()*)

❑ De asemenea, vectorii L (către sursa de lumină) și V (către observator), folosiți în modelul de iluminare locală la nivel de fragment, sunt calculați și normalizați în programul fragment shader pentru a fi folosiți în calculul culorii fragmentului.

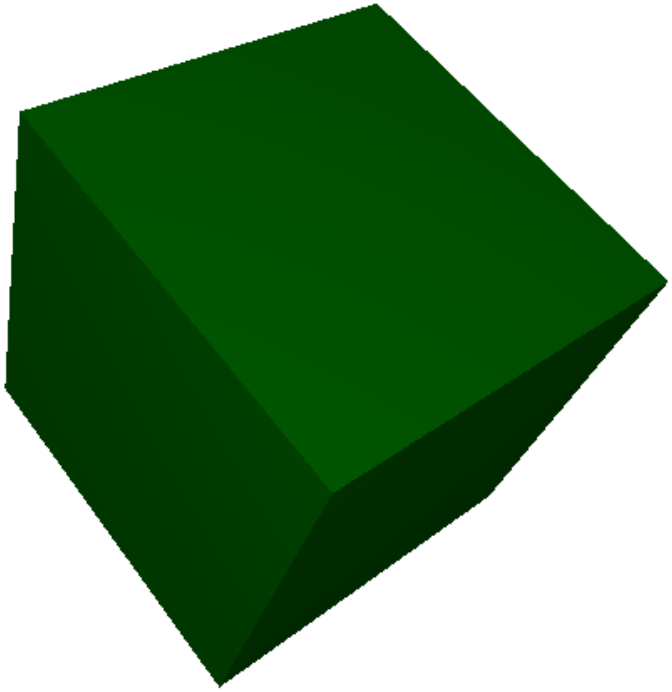
➢ Calcule mai complexe la nivel de fragment decât în modelul Gouraud

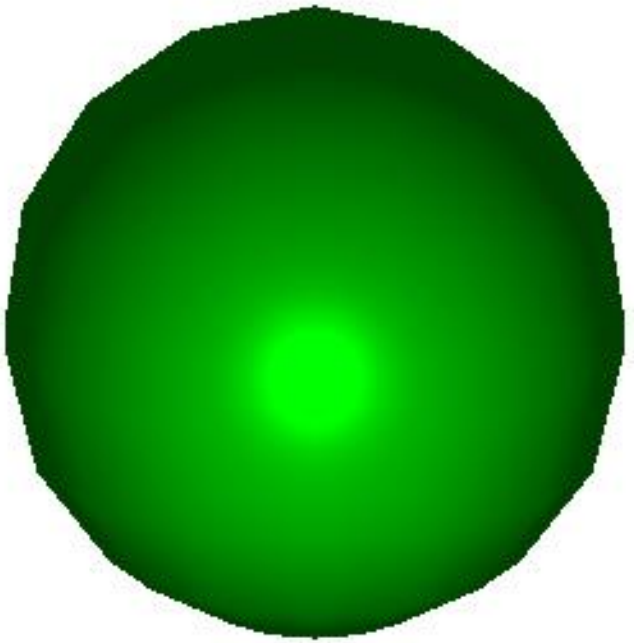
Avantaje

❑ Permite redarea reflexiei speculare in orice fragment al unui poligon

❑ Reduce mult efectul de bandă Mach

Modelul Phong (3)





INTRODUCEREA CEȚII ÎN IMAGINI

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

MODELAREA CEȚII(1)

Ceaţa adaugă realism imaginii şi ajută în eliminarea defectelor produse de decuparea la nivelul planului din spate al volumului vizual:

- Fără ceaţă, pe măsură ce observatorul se îndepărtează de un obiect, obiectul se apropie de planul din spate şi este decupat atunci când planul din spate îl intersectează.
- Cu ceaţă, dacă intensitatea ceţii creşte proporţional cu distanţa de la poziţia observatorului, se crează efectul de distanţă pentru obiectele îndepărtate, iar obiectele decupate la nivelul planului din spate nu mai sunt vizibile.

Cfog – culoarea de ceaţă

Clocal – culoarea calculată într-un punct folosind modelul de iluminare locala

$0 <= \Phi <= 1$ – factorul de ceaţă, proporţional cu distanţa de la observator la punct

($\Phi=1$: opacitate)

Culoarea punctului, afectată de ceață: **$C_{loc_fog} = (1-\Phi)*C_{local} + \Phi*C_{fog}$**

MODELAREA CEȚII(2)

Fie dist distanța de la observator la un punct din scena vizualizată.

Factorul de ceaţă liniar

- Acționează între 2 distanțe definite în sistemul coordonatelor observator, z0 si z1, măsurate pe direcția în care privește observatorul.

  $\Phi = 0$ pentru dist < z0

  $\Phi = (dist - z0) / (z1 - z0)$ pentru z0 <= dist <= z1

  $\Phi = 1$ pentru dist > z1

Factorii de ceaţă exponenţiali

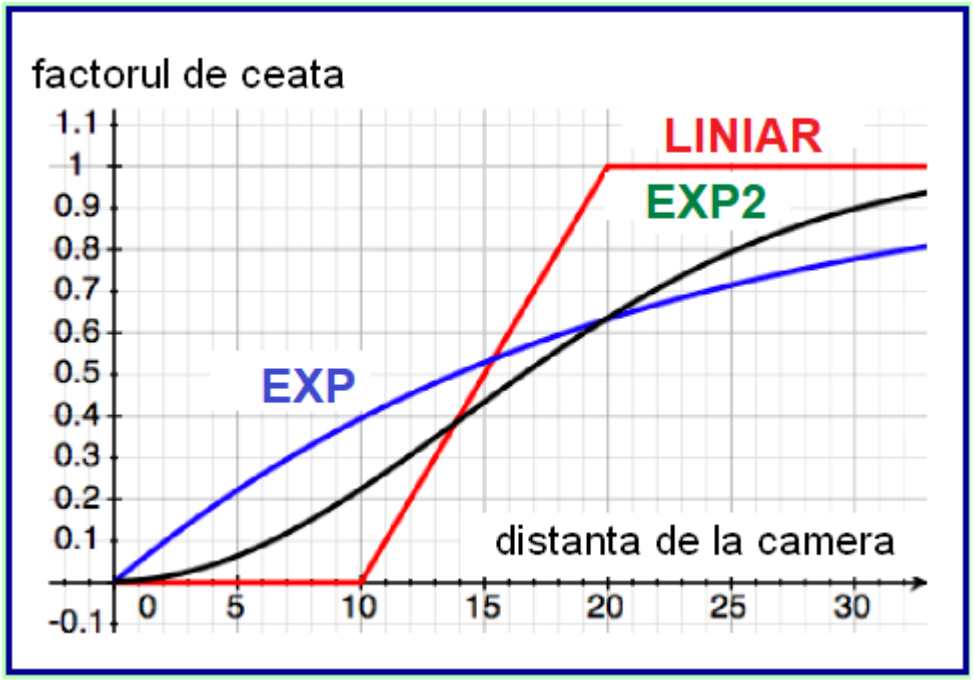
$\Phi = 1.0 - e^f$

EXP: $f = -(fog_density*dist)$

EXP2: $f = -(fog_density*dist)^2$

- fog_density: densitatea ceții (un număr real, de ex. 0.05) - modelează atenuarea luminii datorata ceții

MODELAREA CEȚII(3)



z0 = 10; z1 = 20; fog_density = 0.05

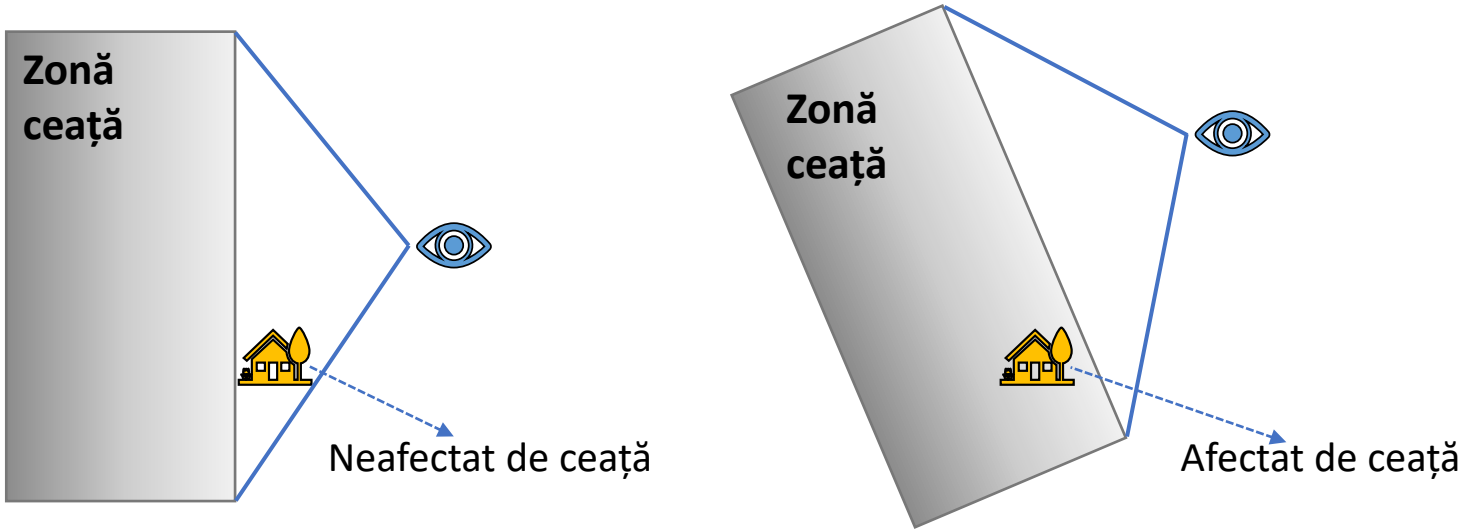
DISTANȚA - dist

Distanța față de observator a punctului în care se calculează culoarea:

- Punctul este în coordonate observator

1. **dist** este coordonata z a punctului, la fel ca distanțele z0 și z1

Dacă se rotește camera, fără a-i modifica poziția, coord. z a punctului se modifică → un obiect aflat la o distanță d de observator poate să intre/iasă din zona de ceață:



2. **dist** este lungimea vectorului de la observator (0,0,0) la punct.

Varianta 1 este mai ieftină computațional și preferată atunci când nu este rotită camera

INTRODUCEREA CEȚII ÎN IMAGINI (1)

Includerea ceții în calculul culorii se poate efectua:

- În vertex shader (modelul Gouraud):

- se calculează **Φ** folosind distanța (dist) de la observator la poziția vârfului transformată în coordonate observator

- se calculează culoarea vârfului afectat de ceață: **Cloc_fog = (1-Φ)*Cvrf + Φ*Cfog**

- **Cloc_fog** este ieșire din vertex shader

- culorile afectate de ceață ale vârfurilor sunt interpolate de GPU la nivel de fragmente

- culoarea de ceață interpolată, **Cloc_fog,** este intrare pentru fragment shader

- În fragment shader (modelul Phong):

Varianta (1): fragment shader primeşte poziția fragmentului în coordonate observator

- se transmite de la vertex shader poziția vârfului în coordonate observator

- pozițiile vârfurilor în coordonate observator sunt interpolate de GPU la nivel de fragmente

- în fragment shader se calculează **Φ** folosind ca distanța (dist) - poziția fragmentului (pe z) în coordonate observator

INTRODUCEREA CEȚII ÎN IMAGINI (2)

```
// Calculul culorii de ceață în fragment shader - Varianta (1)

// vertex shader
layout(location = 0) in vec3 in_position;
layout(location = 1) in vec3 in_normal;
uniform mat4 model_matrix, view_matrix, projection_matrix;

out vec3 world_pos;          // pozitia varfului in coord globale
out vec3 world_normal;       // normala transformata in coord globale
out vec4 view_pos;           // pozitia varfului in coord observator

void main(){
  world_pos = (model_matrix * vec4(in_position,1)).xyz;
  world_normal = normalize(mat3(model_matrix) * in_normal);
  view_pos = view_matrix * model_matrix * vec4(in_position,1);
  gl_Position = projection_matrix * view_pos;
 }
```

```
// fragment shader
layout(location = 0) out vec3 out_color;
uniform vec3 light_position;
uniform vec3 eye_position;
uniform int fogSelector; //0 linear; 1 exponential; 2 exponential square
uniform vec3 Cfog = vec3(0.5, 0.5,0.5);// culoarea ceață
……..
in vec3 world_pos;          // poziția interpolată a fragmentului în coord globale
in vec3 world_normal;       // normala în coord globale interpolată
in vec4 view_pos;           // poziția interpolată a fragmentului în coord observator
void main(){
//Calculează Cfragment aplicând modelul de iluminare locală în poziția fragmentului(world_pos)
   Cfragment = ….
// Calculeaza distanța de la poziția interpolată a fragmentului la observator
dist = abs(view_pos.z); //coordonata Z a fragmentului
if(fogSelector == 0)//linear fog
{  fogFactor = (dist - 4)/(8 - 4); // z0 =8; z1=4
  fogFactor = clamp( fogFactor, 0.0, 1.0 ); //restrânge fogFactor la intervalul 0-1
  out_color = (1-fogFactor)*Cfragment + fogFactor*Cfog
} ……..}
```

INTRODUCEREA CEȚII ÎN IMAGINI (3)

```
// Calculul culorii de ceață în fragment shader- Varianta (2)
// poziția fragmentului în coordonate observator se poate calcula în fragment shader
// sau mai simplu, se poate primi (interpolata) de la VS

// Vertex shader
out vec3 world_pos;                 // poziția vârfului în coord globale
out vec3 world_normal;              // normala transformată în coord globale
out vec4 view_pos;                  //pozitia varfului in coord observatory

void main(){
  world_pos = (model_matrix * vec4(in_position,1)).xyz;
  world_normal = normalize(mat3(model_matrix) * in_normal);
  view_pos = view_matrix * model_matrix * vec4(in_position,1);
  gl_Position = projection_matrix * view_pos;
 }
```

INTRODUCEREA CEȚII ÎN IMAGINI (4)

```
// Fragment shader
layout(location = 0) out vec3 out_color;
uniform vec3 light_position;
uniform vec3 eye_position;
uniform vec3 Cfog = vec3(0.5, 0.5,0.5);  // culoare ceață
in vec3 world_pos;                       // poziția interpolata a fragmentului în coord globale
in vec3 world_normal;                    // normala în coord globale interpolată
in vec4 view_pos;                        // poziția interpolată a fragmentului în coord observator
void main(){
//Calculeaza Cfragment  aplicând modelul de iluminare locală în poziția fragmentului (world_pos)
   Cfragment = ….
// Calculează poziția fragmentului în coordonate observator: frg_pos → frg_eye_pos
// Calculează distanța de la poziția fragmentului, frg_eye_pos (sau view_pos de la VS), la
// observator
   dist = length(view_pos);
// Calculeaza fogFactor folosind dist
……
```

INTRODUCEREA CEȚII ÎN IMAGINI (5)

Calculează poziția fragmentului în coordonate observator: frg_pos → frg_eye_pos

// transformarea în viewport

gl.FragCoord.x = (width/2)*frg_xd + ox , ox = px + width/2

gl.FragCoord.y = (height/2)*frg_yd + oy, oy = py + height/2

gl.FragCoord.z = ((f-n)/2)*frg_zd + (n+f)/2 = frg_zd/2 + 0.5 // implicit: n=0, f=1

Rezultă coord. disp. normalizate →

frg_xd = (gl.FragCoord.x – ox)/(width/2)

frg_yd = (gl.FragCoord.y – oy)/(height/2)

frg_zd = (gl.FragCoord.z – 0.5)*2

// transformarea de normalizare

frg_ xd = frg_ xc/frg_wc; frg_ yd = frg_ yc/frg_wc; frg_ zd = frg_ zc/frg_wc; frg_ wc = -eye.z

// poziția fragmentului în coordonate de decupare

frg_clip_pos = $[frg_xc, frg_yc, frg_zc, fgr_wc]^T$

// transformarea de proiecție

frg_clip_pos = projection_matrix * frg_eye_pos



// poziția fragmentului în coordonate observator

frg_eye_pos = $(projection_matrix)^{-1}$ *frg_clip_pos

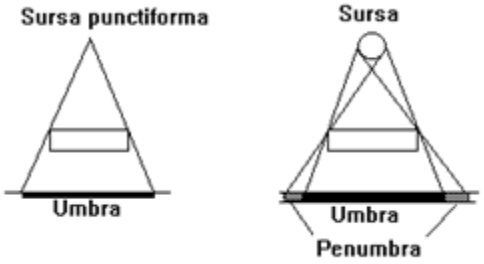
Redarea umbrelor în imagini

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

INTRODUCEREA UMBRELOR ÎN IMAGINI (1)

- Atunci când un observator priveşte o scenă 3D (iluminată de o sursă de lumină) dintr-o poziţie diferită de aceea a sursei de lumină, va vedea umbrele produse de obiectele scenei.
- Umbrele au o contribuţie însemnată la realismul imaginii, îmbunătăţind percepţia profunzimii.



În general, se consideră numai umbra pură: calcule mai simple.

Volumul de calcule depinde şi de poziţia sursei de lumină:

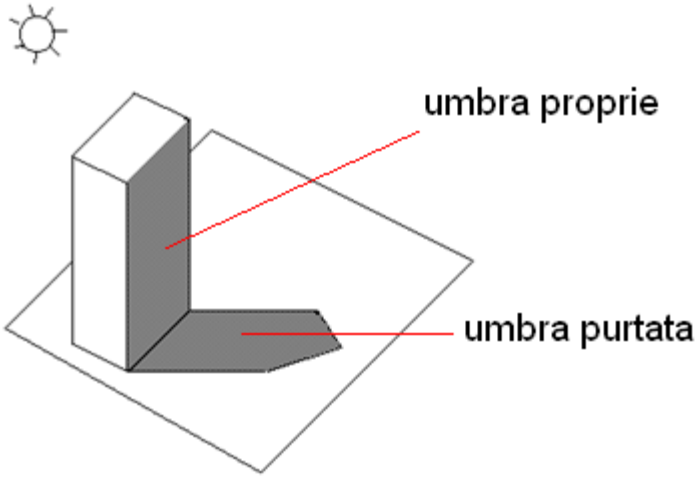
- Dacă sursa este la infinit calculele sunt mai simple.
- În cazul unei surse situate la distanţă finită dar în afara volumului vizual, este necesară o proiecţie perspectivă din poziţia sursei.
- Cazul cel mai dificil este acela în care sursa este situată în volumul vizual.

INTRODUCEREA UMBRELOR ÎN IMAGINI (2)

- Problema determinării umbrelor este similară cu cea de determinare a părților nevizibile ale unei scene 3D: părțile scenei 3D care nu sunt vizibile din poziţia sursei de lumină sunt în umbră.

  → Crearea unei imagini cu umbre presupune rezolvarea de două ori a problemei suprafeţelor nevizibile: o dată privind scena din poziţia sursei de lumină, a doua oară privind-o din poziţia observatorului.

- Sunt două tipuri de umbre: **umbre proprii** și **umbre purtate**.



POLIGOANE DETALIU DE UMBRĂ (1)

– Umbrele proprii sunt generate de obiectul însuşi, care împiedică lumina să ajungă la unele dintre feţele sale: feţele umbrite de umbra proprie sunt feţele auto-obturate atunci când scena este văzută din poziţia sursei de lumină.

– Umbra purtată este umbra pe care un obiect o produce pe alte părţi ale scenei, la care lumina nu ajunge din cauza obiectului.

Poligoane detaliu

❑ Umbrele purtate se pot determina proiectând în scenă, din poziţia sursei, toate poligoanele (feţele) neumbrite de umbra proprie.

- Rezultă un set de **poligoane de umbră**, care se adaugă la reprezentarea scenei.

❑ Atât fețele umbrite de umbra proprie cat şi poligoanele de umbră se folosesc ca poligoane-detaliu la redarea scenei 3D.

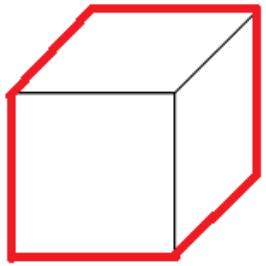
❑ Poligoanele detaliu sunt poligoane coplanare cu poligoanele scenei, care se folosesc în calculul culorii fragmentelor.

POLIGOANE DETALIU DE UMBRĂ (2)

- Poligoanele de umbră au o culoare specifică umbrei pure.
- Poligoanele de umbră se trimit în banda grafică după poligoanele scenei, care sunt vizibile din poziţia observatorului, cu testul de adâncime dezactivat (fragmentele poligoanelor de umbră suprascriu unele dintre fragmentele vizibile) .

- Pot fi generate mai multe vederi ale scenei fără a recalcula umbrele, deoarece umbrele depind numai de poziţia sursei (surselor) de lumină.

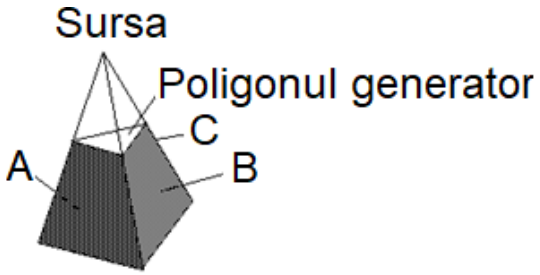
- Numărul de poligoane de umbră este mai mic,
dacă în loc să se proiecteze fiecare faţă luminată de sursă,
se proiectează silueta fiecărui obiect văzută din poziţia sursei de lumină.



- Calculul poligoanelor de umbră este destul de complicat, de aceea în prezent se folosesc alte metode pentru introducerea umbrelor în imagini.

VOLUME DE UMBRĂ (1)

- Sursa de lumină este considerată punctiformă iar scena 3D alcătuită din poligoane.
- Un **volum de umbră** este definit de o sursă de lumină şi un poligon luminat (vizibil din poziţia sursei de lumină), pe care-l vom numi **poligonul generator (“shadower”)**.



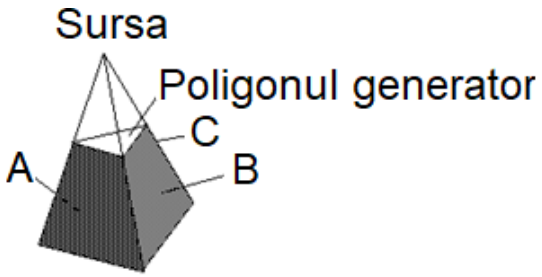
- Volumul de umbră determinat de o sursă şi un poligon generator este delimitat de o faţă care reprezintă poligonul generator scalat. Această faţă este situată la o distanţă faţă de sursă dincolo de care intensitatea luminii sursei este neglijabilă, deci orice punct aflat dincolo de această limită este umbrit.
- Volumul de umbră poate fi decupat la marginile volumului vizual.

VOLUME DE UMBRĂ (2)

❑ Feţele laterale ale volumului sunt numite **poligoane de umbră**. Un poligon de umbră este determinat de o latură a poligonului generator şi cele două drepte care pleacă din sursa de lumină, fiecare trecând printr-un vârf al laturii.

❑ Normalele la feţele laterale punctează înspre exteriorul volumului.

❑ Poligoanele de umbră se folosesc pentru determinarea umbririi produse de poligonul generator în scenă.



Notăm cu :

– PUV poligoanele de umbră care sunt vizibile din poziţia observatorului (A şi B în figura) şi cu

– PUN poligoanele de umbră care nu sunt vizibile din poziţia observatorului (poligonul C).

➢ Clasificarea poligoanelor de umbra în PUV și PUN se poate face pe baza normalelor la poligoane și a poziției observatorului (back face culling: front-face și back-face).

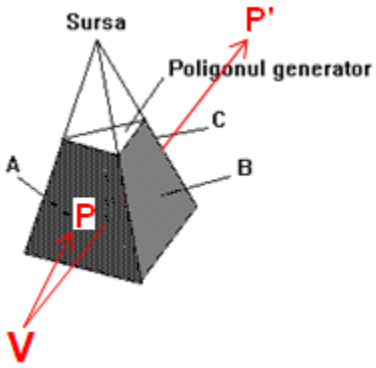
VOLUME DE UMBRĂ (3)

Fie

– un punct P al unei suprafeţe din scena 3D şi

– VP vectorul din poziţia observatorului (V) în punctul P.

Punctul P este umbrit dacă numărul de poligoane de tip PUV intersectate de vectorul VP este mai mare decât numărul de poligoane de tip PUN intersectate de vector.



❑ În general, pentru a determina dacă un punct este în umbră, se poate folosi un contor în care iniţial se memorează numărul de volume de umbră care conţin poziţia observatorului.

– Se asociază poligoanelor de tip PUV valoarea +1 iar celor de tip PUN valoarea -1.

– Atunci când vectorul VP traversează un poligon de umbră, se adună la contor valoarea asociată poligonului.

– Punctul P este umbrit dacă valoarea contorului este pozitivă în P.

VOLUME DE UMBRĂ (4)

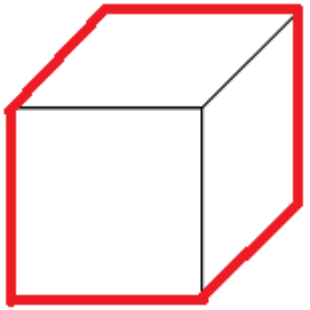
➢ Volumul de calcule presupus de acest algoritm poate fi redus dacă în loc să se calculeze volumul de umbră pentru fiecare poligon vizibil din poziţia sursei, se calculează un singur volum de umbră pentru o suprafaţă poliedrală.

În acest scop, se determină poligoanele de umbră numai pentru laturile care fac parte din silueta suprafeţei, văzută din poziţia sursei de lumina.

❑ Silueta unei suprafeţe, corespunzătoare unui punct de observare, este un set conectat de laturi care aparţin poligoanelor vizibile din punctul de observare.

❑ O latură de siluetă este fie o latură de margine a unei suprafeţe deschise, fie o latură care separă un poligon vizibil de unul nevizibil.

➢ Pentru determinarea laturilor de siluetă, este necesar să se folosească o structură de date care reflectă adiacenţa poligoanelor. Cunoscându-se poligonul adiacent pe fiecare latură a fiecărui poligon vizibil din poziţia sursei, se pot determina rapid laturile de siluetă.



VOLUME DE UMBRĂ (5)

Implementarea metodei folosind stencil-buffer (1)

❑ Buffer-ul de marcaje (stencil-buffer) este prezent pe plăcile grafice actuale (în memoria GPU), alături de buffer-ul imagine (frame-buffer) și buffer-ul de adâncime (z-buffer).

– este o memorie de numere intregi asociată pixelilor imaginii, de regulă un byte/pixel;

– poate fi utilizat pentru a implementa diferite operații grafice, de regulă în combinație cu z-buffer, cum este și cazul implementarii volumelor de umbră:

– valorile din stencil-buffer pot fi incrementate/ decrementate automat pentru fiecare fragment care trece / nu trece testul de adâncime.

❑ Există mai multe metode de a utiliza stencil-buffer pentru implementarea metodei volumelor de umbre.

VOLUME DE UMBRĂ (6)

Implementarea metodei folosind stencil-buffer (2)

❑ În metoda prezentată în continuare, se crează în stencil-buffer o mască a umbrelor din imagine: orice pixel de umbra, (x,y), va fi reprezentat în stencil-buffer printr-o valoare ≠ 0.

❑ **Pixelii de umbră pot fi afișați in culorile fragmentelor vizibile, calculate cu formula.**

Cfragment = Cemisiva + Cambientala = ke + ka*globalAmbient

ke, ka – proprietăți de material

– Se presupune ca s-au calculat poligoanele de umbră.

– Conturul fiecarui poligon de umbră trebuie să fie orientat în sens trigonometric atunci cand este privit din exteriorul volumului de umbră. În acest fel, poligoanele PUV sunt cele “din faţa” (“front-face”) observatorului, iar poligoanele PUN sunt cele “din spate” (“back-face”)

VOLUME DE UMBRĂ (7)

Implementarea metodei folosind stencil-buffer(3)

Considerăm cazul în care observatorul nu este în umbră.

Pasul 1

Inițializează z-buffer și stencil-buffer; // nu trebuie programate, acestea sunt
// inițializate în mod automat pentru fiecare cadru imagine

Activează scrierea în z-buffer și dezactivează scrierea în stencil-buffer ; //implicite

*** Rasterizează scena (folosindu-se algoritmul z-buffer) fără lumini**

Efectul:

– în frame-buffer se obține imaginea scenei în umbră (fără lumini). Culoarea fiecărui pixel este:

Cpixel = culoarea emisiva + culoarea ambientala;

La sfârșitul acestui pas, z-buffer conține coordonatele z ale fragmentelor vizibile în imagine.

VOLUME DE UMBRĂ (8)

Implementarea metodei folosind stencil-buffer(4)

Pasul 2

glDepthMask(GL_FALSE); // Dezactivează scrierea în z-buffer

glColorMask(GL_FALSE, GL_FALSE, GL_FALSE, GL_FALSE); // Dezactivează scrierea în frame-buffer

glFrontFace(GL_CCW);// feţele “din faţă” sunt orientate trigonometric

glCullFace(GL_BACK);//se vor elimina feţele din spate (nu sunt rasterizate poligoanele PUN)

glStencilFunc(GL_ALWAYS, 0x0, 0xff);// se activează scrierea în stencil buffer

glStencilOp(GL_KEEP, GL_KEEP, GL_INCR); //se incrementează stencil buffer pt toate
// fragmentele care trec testul de adâncime

***Rasterizează poligoanele de umbră**

Efectul:

Sunt rasterizate poligoanele de umbră de tip PUV.

Pentru fiecare fragment rezultat, f, se face testul:

daca f.z < z-buffer[f.y][f.x] // fragmentul f în față fragmentului vizibil în pixelul (x,y)

atunci stencil-buffer[f.y][f.x]++;

// se numără poligoanele PUV aflate între observator și obiectele scenei

VOLUME DE UMBRĂ (9)

Implementarea metodei folosind stencil-buffer(5)

Pasul 3

glCullFace(GL_FRONT);//se vor elimina feţele din faţă (nu sunt rasterizate poligoanele PUV)

glStencilOp(GL_KEEP, GL_KEEP, GL_DECR); // se decrementează stencil buffer pt toate
// fragmentele care trec testul de adâncime

***Rasterizează poligoanele de umbră**

Efectul:

Sunt rasterizate poligoanele de umbră de tip PUN.

Pentru fiecare fragment rezultat, f, se face testul:

daca f.z < z-buffer[f.y, f.x] // f este in faţa fragmentului vizibil în pixelul (x,y)

atunci stencil-buffer[f.y, f.x]--; // se numără poligoanele PUN aflate între observator
// și obiectele scenei

În stencil-buffer s-a obținut o masca cu “găuri” în punctele care primesc lumina:

valorile din stencil-buffer sunt diferite de zero pentru pixelii care trebuie afișați în umbră.

VOLUME DE UMBRĂ (10)

Implementarea metodei folosind stencil-buffer(6)

Pasul 4

```
glEnable(GL_STENCIL_TEST);// Activează testul stencil
glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP); // Dezactivează scrierea în stencil-buffer
glStencilFunc(GL_EQUAL, 0x0, 0xff); // Specifică testul stencil
glDepthMask(GL_TRUE); // Activează scrierea în z-buffer
glColorMask(GL_TRUE, GL_TRUE, GL_TRUE, GL_TRUE); // Activează scrierea în frame
                                                 // buffer
```

***Rasterizează scena (folosindu-se algoritmul zbuffer) cu lumini**

Efectul:

Se rasterizează scena modificându-se în frame-buffer numai culorile pixelilor (x,y) pentru care stencil-buffer [y][x] =0 (fragmentul nu este în umbră)

VOLUME DE UMBRĂ (11)

Implementarea metodei folosind stencil-buffer (7)

Cazul general:

Se inițializează stencil-buffer cu o valoare reprezentând numărul de volume de umbră care conțin poziția observatorului.

Aprecieri asupra metodei volumelor de umbră

Pozitive:

- Este generală şi poate fi aplicată pentru orice scenă, pentru oricâte surse de lumină.
- Produce forme de umbre exacte.

Negative:

- Complexă din punct de vedere al timpului de calcul:

  se rasterizează scena de 2 ori și poligoanele de umbră o singură dată dar în 2 pași!
- Sursele de lumină sunt considerate punctiforme, de aceea nu pot fi obținute penumbre.

Introducerea umbrelor în algoritmul z-buffer - shadow mapping -

Williams [WiLL78] a propus o metodă de generare a umbrelor bazată pe rasterizarea de 2 ori a scenei, cu execuţia algoritmului Z-Buffer:

- În prima etapă se construieşte buffer-ul Z al scenei văzută din poziţia sursei de lumină (în transformarea de vizualizare a scenei – funcția lookAt - se consideră poziţia sursei).

  Vom nota acest buffer cu ZS. El conține coordonatele z ale fragmentelor care primesc lumina de la sursă. Mai este numit și « **shadow map** », de unde și denumirea sub care este cunoscută metoda în prezent.

- În etapa a doua se construieşte imaginea văzută din poziţia observatorului, utilizând buffer-ul ZS pentru determinarea punctelor aflate în umbră.

În cazul mai multor surse de lumină, se utilizează câte un buffer ZS pentru fiecare sursă.

Implementarea moderna a metodei este numită “shadow mapping”.

Introducerea umbrelor în algoritmul z-buffer

Etapa I.

Se calculează matricea de transformare a vârfurilor, MVPS, cu observatorul în poziția sursei de lumină

Se transmite MVPS la vertex shader (ca variabilă uniformă).

Se cere folosirea unui buffer ZS ca buffer de adâncime, în locul buffer-ului Z implicit

Se cere rasterizarea scenei:

1. În vertex shader se transformă vârfurile primitivelor folosind MVPS
2. Pentru fiecare primitivă a scenei

    pentru fiecare fragment f(x,y,z) rezultat din rasterizarea primitivei

        dacă z < ZS[y][x] atunci

            ZS[y][x] = z // fragmentul este vizibil în pixelul (x,y) din poziţia sursei

La sfârșitul acestei etape, în ZS sunt memorate coordonatele z ale fragmentelor care primesc lumina de la sursă (cele mai apropiate fragmente de sursa de lumină).

Introducerea umbrelor în algoritmul z-buffer

Etapa II.

Se calculează matricea de transformare a varfurilor MVPO, cu observatorul în poziția sa.

Se transmit la vertex shader (ca variabile uniforme) MVPO și MVPS.

Se cere rasterizarea scenei:

1. În vertex shader se calculeaza varful transformat cu MVPO (gl_Position) și MVPS (poz_sursa)

poz_sursa = pozitia varfului transformata cu MVPS este variabilă de ieșire din vertex shader

2. pentru fiecare primitiva a scenei

pentru fiecare fragment f(x,y,z) rezultat din rasterizarea primitivei

dacă z < Z-Buffer[y][x] atunci // fragment vizibil în pixelul (x,y), din poziţia observatorului

{Z-Buffer[y][x] = z; frame-Buffer[y][x] = Culoare fragment (întoarsa de fragment shader);}

În fragment shader

poz_sursa este variabilă de intrare pt fragment shader (poziție fragment transformată cu MVPS)

Culoare_fragment = culoare calculată folosind modelul de iluminare locală;

dacă ZS[poz_sursa.y][poz_sursa.x] < poz_sursa.z atunci //fragmentul f(x,y,z) este în umbră

Culoare_fragment = factor_umbra* Culoare_fragment (0< factor_umbra<1)

Detalii de implementare umbre

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

STENCIL BUFFER (1)

Stencil buffer-ul pune la dispoziție multe opțiuni pentru a restricționa desenarea pe ecran și se poate folosi în multe locuri acolo unde bufferul de adâncime nu poate fi utilizat.

Privit în modul cel mai simplist stencil buffer-ul este folosit pentru a bloca la desenare anumite zone din ecran.

Activare: glEnable(GL_STENCIL_TEST)

Exista 4 funcții stencil în OpenGL :

- void glClearStencil(GLint s)
- void glStencilFunc(GLenum func, GLint ref, GLuint mask)
- void glStencilMask(GLuint mask)
- void glStencilOp(GLenum fail, GLenum zfail, GLenum zpass)

glClearStencil() funcționeaza în mod similar cu glClearColor și glClearDepth

deasemenea inițializează stencil buffer-ul cu valoarea inițială dată ca parametru când este apelat glClear(GL_STENCIL_BIT). În mod default această valoare este 0.

STENCIL BUFFER (2)

În mod default doar prin activarea folosirii stencil bufferului nu se întampla nimic toate desenările pe ecran fiind permise fara nici o restricție. Pentru a lucra cu stencil bufferul trebuie să "marcăm" valori în acesta folosind glStencilFunc() și glStencilOp()

Functia glStencilFunc(GLenum func, GLint ref, GLuint mask) definește :

- o funcție de comparație
- o valoare de referință cu care se face comparația
- o mască (se face ȘI logic cu valoarea de referință && valoarea din stencil buffer)

Funcțiile de comparare valide pentru stencil test sunt :

- GL_NEVER – eșuează întotdeauna (nu se va desena nimic pe ecran)
- GL_ALWAYS – reușește întotdeauna (desenarea are loc mereu - aceasta este comportarea default)
- GL_EQUAL – reușeste dacă valoarea de referință este egală cu valoarea din stencil buffer
- GL_NOTEQUAL – reușește dacă valoarea de referință nu este egală cu valoarea din stencil buffer
- GL_LESS – reușește dacă valoarea de referință este mai mică decât valoarea din stencil buffer
- GL_LEQUAL – reușește dacă valoarea de referință este mai mică sau egală cu valoarea din stencil buffer
- GL_GREATER – reușește dacă valoarea de referință este mai mare decât valoarea din stencil buffer
- GL_GEQUAL – reușește dacă valoarea de referință este mai mare sau egală cu valoarea din stencil buffer

STENCIL BUFFER (3)

Legate de funcțiile stencil sunt operațiile stencil definite cu

glStencilOp(GLenum fail, GLenum zfail, GLenum zpass) care setează comportarea valorii stencil bufferului pentru cei trei parametri ai funcției :

- fail : testul stencil a eșuat
- zfail : testul stencil a reușit dar testul de adâncime a eșuat
- zpass : testul stencil a reușit și testul de adâncime a reușit

Operațiile valide asupra valorilor din stencil buffer sunt :

- GL_ZERO - setează valoarea din stencil pentru pixelul/fragmentul curent la 0
- GL_KEEP - păstrează valoarea din stencil pentru pixelul/fragmentul curent
- GL_REPLACE – înlocuiește valoarea din stencil pentru pixelul/fragmentul curent cu valoarea de referință
- GL_INCR – mărește valoarea din stencil pentru pixelul/fragmentul curent
- GL_DECR - scade valoarea din stencil pentru pixelul/fragmentul curent

Exemple de folosire :

- glStencilFunc(GL_ALWAYS, 1, 1); glStencilOp(GL_KEEP, GL_KEEP, GL_REPLACE);

  Se va desena tot timpul pe ecran iar în locul unde a fost desenat ceva stencil bufferul va avea valoarea 1
- glStencilFunc(GL_EQUAL, 0, 1); glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);

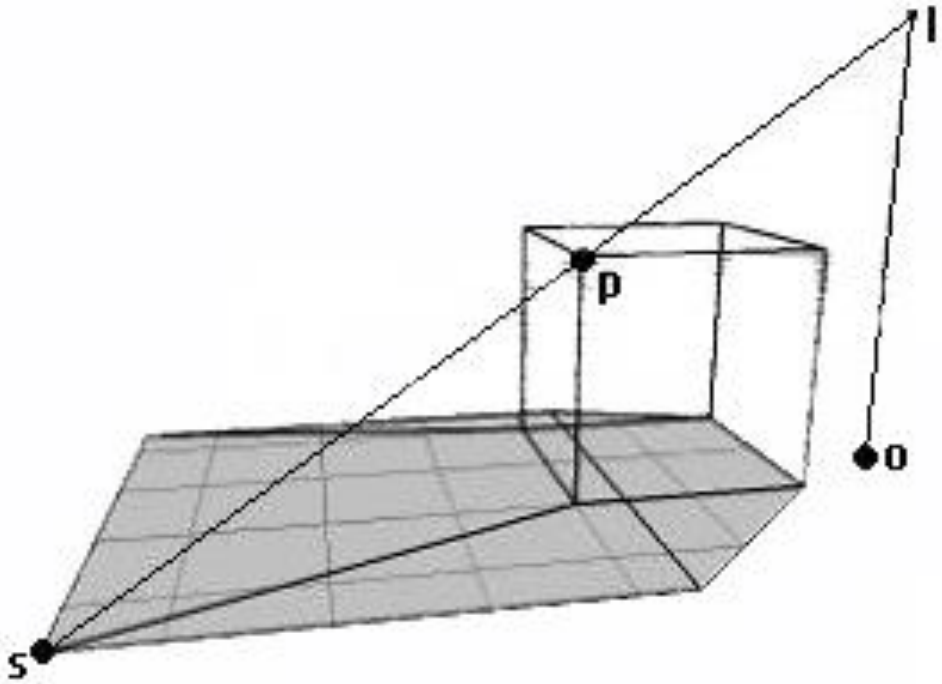
  Se va desena pe ecran numai unde stencil bufferul este 0 și valorile din stencil buffer vor fi păstrate

UMBRE PLANARE (1)

O metodă simplă care permite afișarea de umbre pe suprafețe plane a fost dezvoltată în 1988 de James Blinn.

Ideea care stă în spatele metodei propuse de Blinn este aceea de a găsi o transformare care proiectează geometria fiecărui obiect generator de umbre (occluder) pe un plan stabilit rezultând astfel umbra obiectului respectiv.

Proiecția poate fi descrisă de o matrice 4x4 de transformare în coordonate omogene.



L – sursa de lumină punctiformă

O – punct de pe plan

P – punct al obiectului generator de umbră

S – proiecția lui P pe plan

UMBRE PLANARE (2)

Pentru a simplifica lucrurile să considerăm pentru început situația din figură unde avem o sursă de lumină la pozitia L = (Lx,Ly,Lz) și planul care va primi umbra este y = 0

Un punct P = (Px,Py,Pz) de pe obiect văzut din sursa de lumina L va proiecta o umbră pe plan în punctul S = (Sx,0,Sz).

Punctul S poate fi exprimat ca:

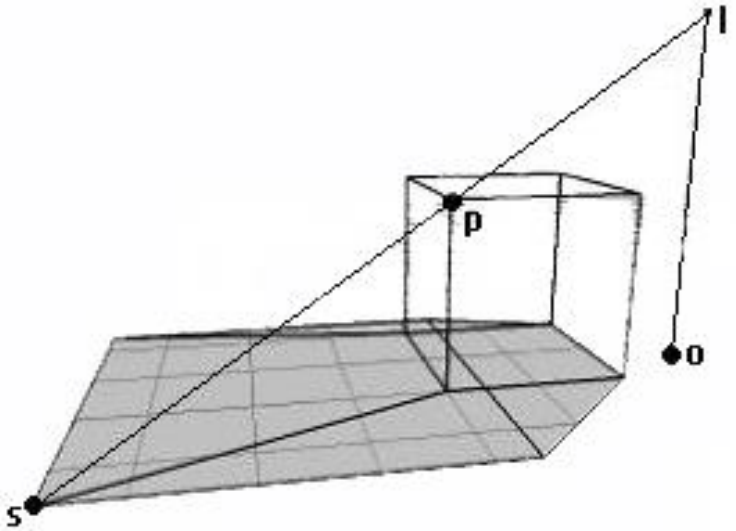
$$s = L + t(P - L) \qquad (1)$$

Dar cum Sy = 0 rezultă:

$$t = \frac{L_y}{L_y - P_y}$$

Introducând în formula (1) obținem coordonatele lui S:

$$\begin{bmatrix} S_x \\ 0 \\ S_z \\ w \end{bmatrix} = \begin{bmatrix} L_y & -L_x & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & -L_z & L_y & 0 \\ 0 & -1 & 0 & L_y \end{bmatrix} \begin{bmatrix} P_x \\ P_y \\ P_z \\ 1 \end{bmatrix}$$



Unde w = $L_y - P_y$

UMBRE PLANARE (3)

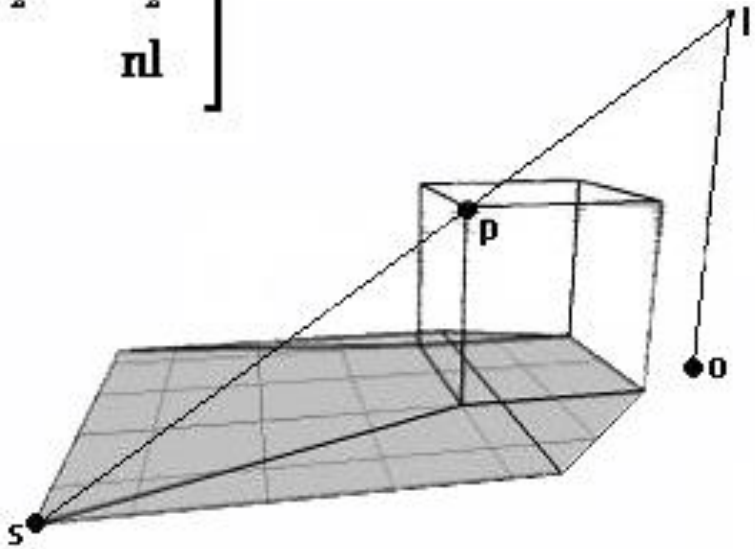
În cazul general poligonul pe care se vor proiecta umbrele nu se află în planul y = 0.

Pentru un plan oarecare **NX** + d = 0, unde **N** este vectorul normalei planului și **X** este un punct oarecare de pe plan se poate construi matricea transformării M într-un mod similar. Combinând formula (1) cu ecuația planului și introducand pe **X = S** rezultă :

$$t = \frac{NL+d}{N(L-P)}$$

Similar rezulta matricea de transformare M care satisface relatia S = MP

$$\mathbf{M} = \begin{bmatrix} \mathbf{nl} + d - l_x n_x & -l_x n_y & -l_x n_z & -l_x d \\ -l_y n_x & \mathbf{nl} + d - l_y n_y & -l_y n_z & -l_y d \\ -l_z n_x & -l_z n_y & \mathbf{nl} + d - l_z n_z & -l_z d \\ -n_x & -n_y & -n_z & \mathbf{nl} \end{bmatrix}$$



UMBRE PLANARE (4)

Faptul că metoda propusă de Blinn este foarte simplă duce la o serie de probleme/limitări :

• limitarea la suprafețe plane

• datorită problemelor de precizie este posibil ca poligoanele proiectate sa fie afișate sub cele ale planului pe care se proiectează umbrele

• umbrele sunt proiectate pe planul ce conține poligonul primitor și nu direct pe acesta; astfel umbra va fi desenată și în afara poligonului -> folosire stencil buffer pentru rezolvare

• pot fi afișate umbre chiar și pentru obiectele care nu pot în mod normal să aibă umbre; dacă obiectul este situat în spatele sursei de lumină umbrele generate de acesta se numesc anti-umbre (anti-shadows) iar dacă obiectul se află în spatele planului umbrele generate se numesc umbre false (fake shadows); o soluție pentru eliminarea acestor probleme ar fi folosirea de clipping-planes

Avantaje

- realizează proiectarea umbrelor pe suprafete plane cu rezultate f. bune

- metodă rapidă pentru scene mici cu o singură sursă de lumină și un plan pe care se proiectează umbrele

Dezvantaje

- nu permite obiectelor concave sa se auto-umbrească

- crează hard shadows

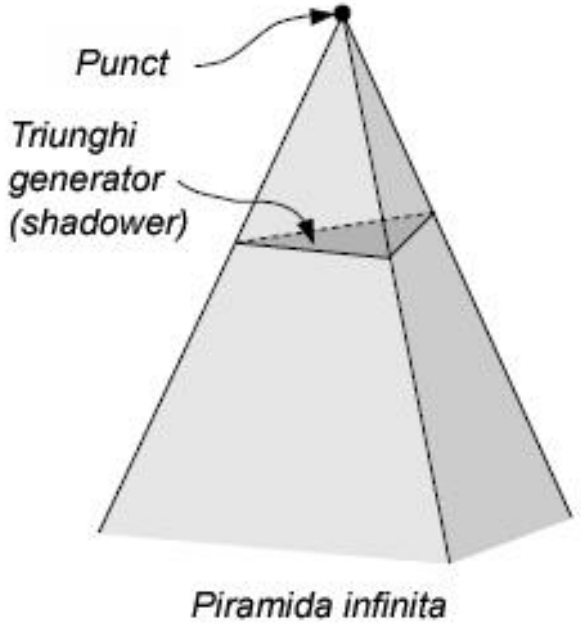
- dacă se dorește realizarea de umbre pentru surse de lumină multiple și mai multe plane de proiectare, costul computațional crește foarte repede

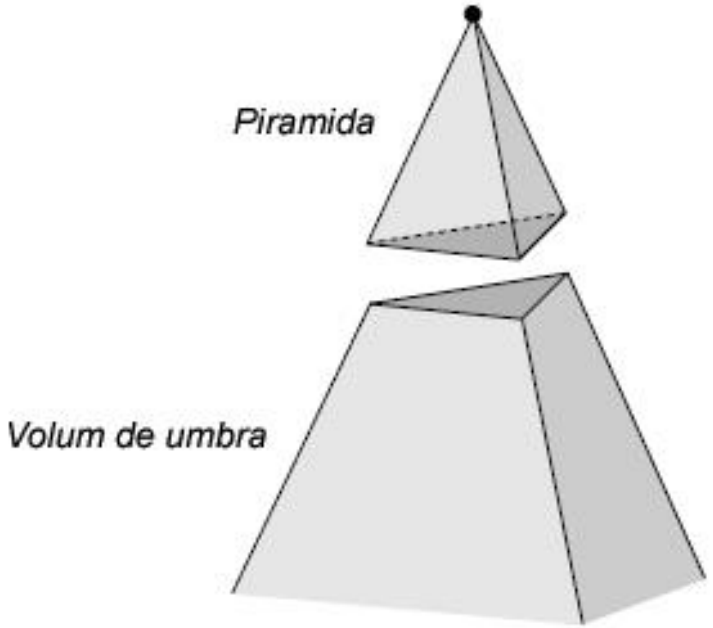
SHADOW VOLUMES (1)

Folosind această metoda se pot proiecta umbre pe obiecte arbitrare și se poate realiza și auto-umbrirea obiectelor

Pentru început sa ne imaginam o sursă de lumină punctiformă și un poligon generator (shadower). Luând ca poligon generator un triunghi extinzând virtual liniile din sursa de lumină și care trec prin vârfurile triunghiului obținem o piramidă infinită.

Putem considera partea de deasupra ca fiind o piramidă iar partea de sub poligon ca fiind un trunchi de piramidă. Orice parte a unui obiect care se află în interiorul volumului trunchiului de piramidă va fi în umbră. Denumirea acestui volum este "volum de umbră" (shadow volume).





SHADOW VOLUMES (2)

Să ne imaginăm că vizualizăm scena dintr-un anumit punct din spațiu iar pentru fiecare pixel afișat ducem o raza până când aceasta va lovi obiectul afișat pe ecran. Pe măsura ce raza este proiectată spre obiect incrementăm un contor de fiecare dată când aceasta intră în volumul de umbră (traversează o față front-facing a acestuia) și decrementăm același contor când raza iese din volumul de umbră (traversează o față back-facing a acestuia).

Dacă în momentul în care raza a lovit pixelul obiectului contorul este mai mare decât 0 atunci acel pixel se află în volumul de umbră (este umbrit) iar în caz contrar se află în afara acestuia.

Dacă această metoda ar fi aplicata geometric pentru fiecare pixel al obiectului ar necesita un număr foarte mare de calcule. De aceea în practică se folosește stencil-bufferul care va face contorizarea.

Folosind și stencil-bufferul și z-bufferul se desenează pe rând întâi fețele front-facing ale volumului de umbră având grija să incrementăm contorul din stencil-buffer iar apoi sunt desenate fețele back-facing ale volumului de umbră având grijă să decrementăm contorul.

În final este afișată întreaga scenă, o valoare de 0 în stencil-buffer însemnând că acel pixel se află în afara volumului de umbră (este iluminat) iar o valoare de 1 înseamnă că acesta este umbrit.

SHADOW VOLUMES (3)

Sinteză implementare algoritm:

1. Se dezactivează bufferele de culoare și adâncime referitor la scriere (acestea nu mai primesc update-uri) dar testul de adâncime rămâne activat
2. Se activează back-face cullingul (vor fi desenate doar fețele front-facing)
3. Se setează operația stencil să incrementeze (GL_INCR) bufferul de marcaj dacă testul de adâncime reușește
4. Se desenează poligoanele volumului de umbră
5. Se activează front-face cullingul (vor fi desenate doar fețele back-facing)
6. Se setează operația stencil să decrementeze (GL_DECR) bufferul de marcaj dacă testul de adâncime reușește
7. Se desenează poligoanele volumului de umbră

Metoda denumită și depth-pass deoarece stencil buffer-ul este modificat când testul de adâncime reușește.

Problemă când observatorul se află în interiorul unuia sau mai multe volume de umbră:

- Se inițializează stencil buffer-ul cu numărul de volume de umbră în care se află observatorul; Calculul dacă observatorul se află în cadrul mai multor volume de umbră poate deveni foarte costisitor -> folosire metoda depth-fail

SHADOW VOLUMES (4)

Metoda depth-fail (stencil buffer-ul este modificat când testul de adâncime eșuează) :

1. Se dezactivează bufferele de culoare și adâncime referitor la scriere (acestea nu mai primesc update-uri) dar testul de adâncime rămâne activat
2. Se activează front-face cullingul (vor fi desenate doar fețele back-facing)
3. Se setează operația stencil să incrementeze (GL_INCR) bufferul de marcaj dacă testul de adâncime eșuează
4. Se desenează poligoanele volumului de umbră
5. Se activează back-face cullingul (vor fi desenate doar fețele front-facing)
6. Se setează operația stencil să decrementeze (GL_DECR) bufferul de marcaj dacă testul de adâncime eșuează
7. Se desenează poligoanele volumului de umbră

Practic se numără poligoanele de umbra din spatele suprafețelor considerate !

Observație: părțile din față și spate a volumului de umbră (bazele trunchiului piramidei) trebuie să fie construite (acoperite)

Cunoscută și sub numele de *Carmack's reverse* (descoperită de John Carmack în dezvoltarea jocului Doom 3).

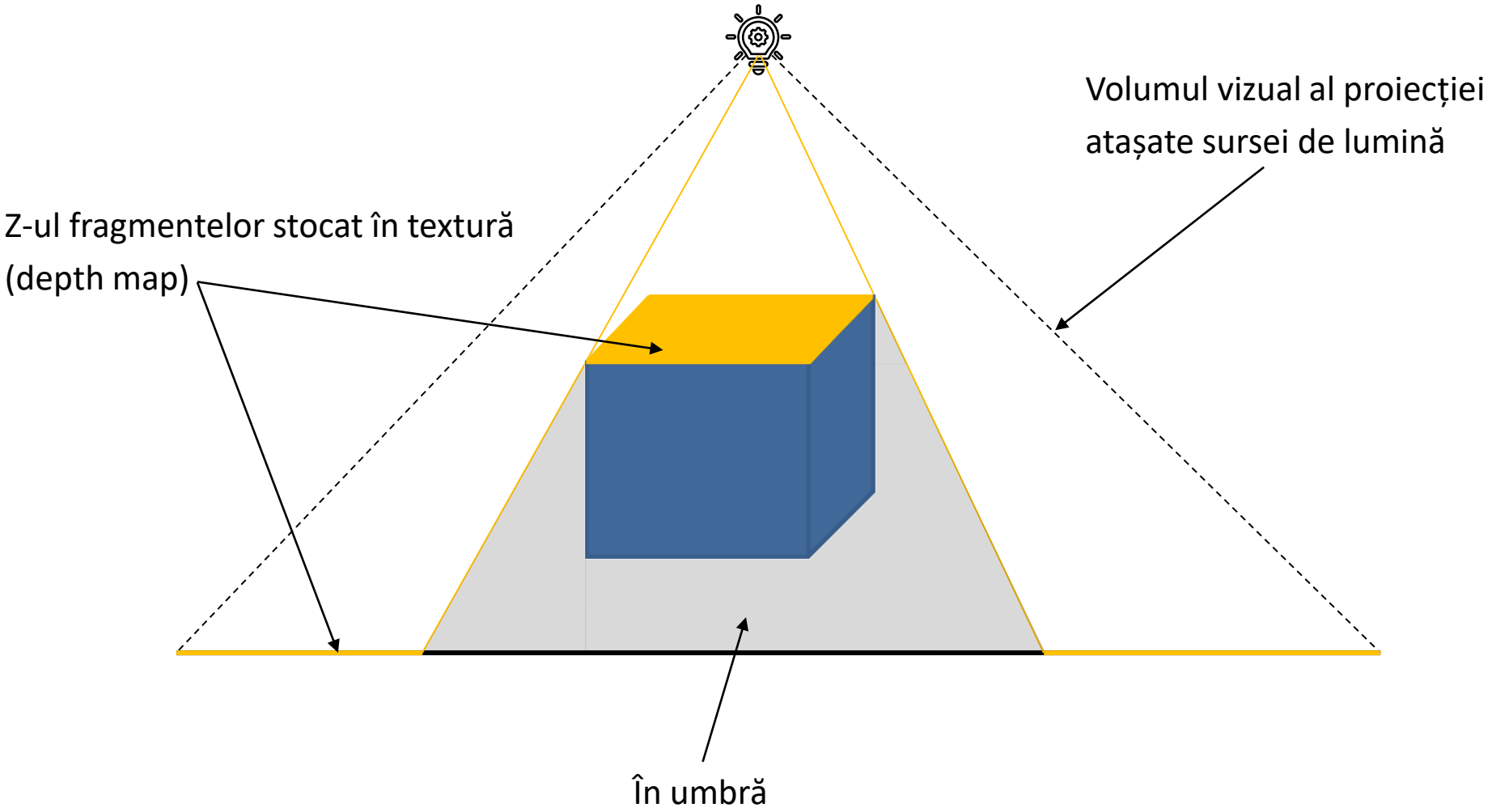
SHADOW MAPPING (1)

Doi pași de redare (render pass)

1. Scena redată din poziția sursei de lumină cu redare direct într-o textură

- Matricea de VIEW calculată cu glm::lookat folosind ca poziție a observatorului poziția sursei de lumină
- Se setează ca buffer de imagine un buffer diferit de cel al redării normale; acest buffer va avea atașată o textură imagine de dimensiune dată -> denumită **shadow map / depth map**
- Se redă scena folosind matricea VIEW ce are ca poziție sursa de lumină (practic scena este văzută din perspectiva sursei de lumină) folosind ca buffer de imagine shadow map-ul
- Se vor memora în shadow map doar coordonatele z ale fragmentelor vizibile din poziția sursei de lumină:
  - Fragment shader-ul nu va scrie nimic în culoarea de output, rasterizatorul atașând automat coordonatele z interpolate în gl_FragDepth

SHADOW MAPPING (2)

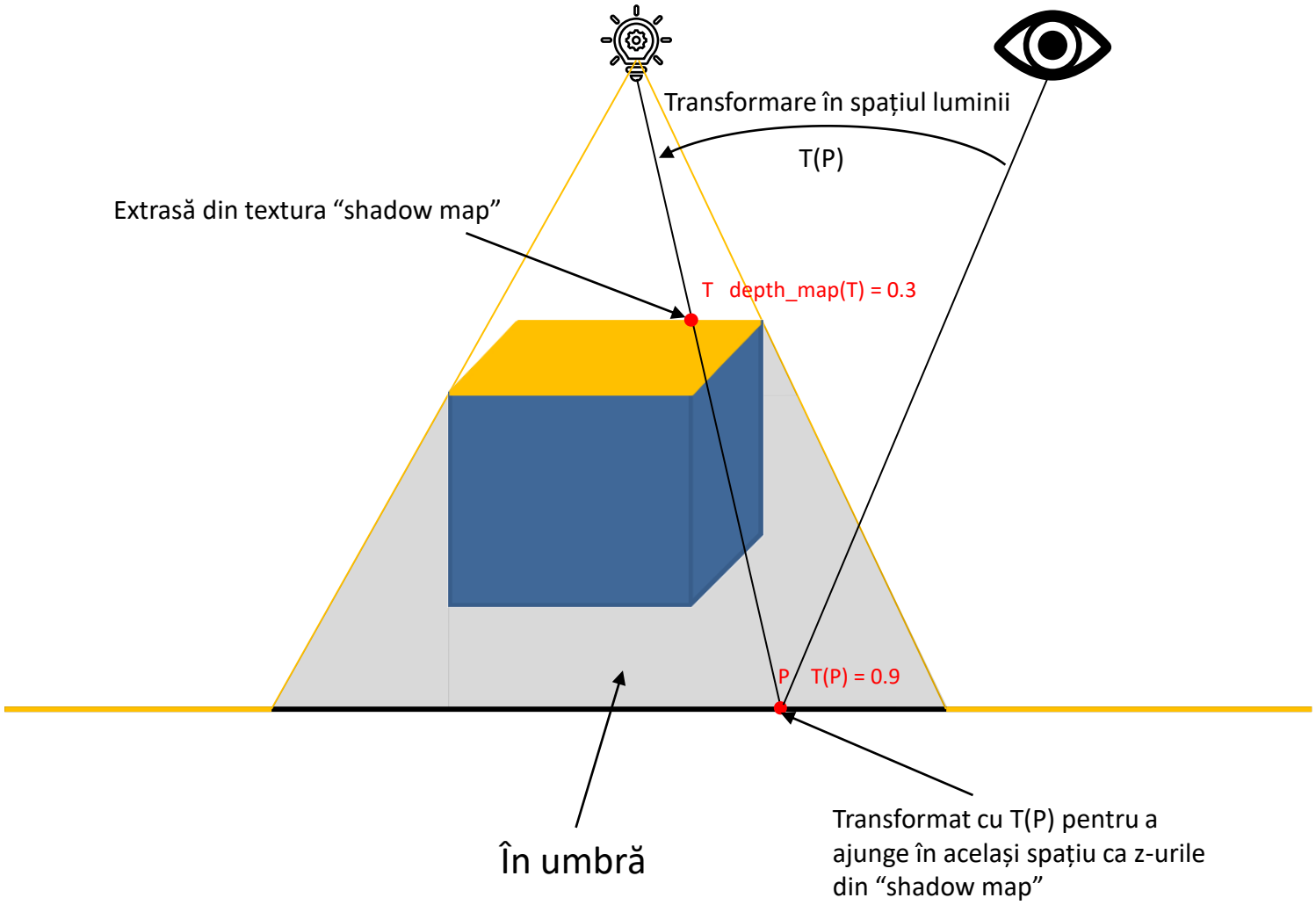


SHADOW MAPPING (3)

2. Scena redată normal

- În vertex shader se transformă varfurile cu MVP
- Suplimentar se transmite matricea VP pentru sursa de lumină ce va fi folosită pentru a transforma vârfurile în spațiul sursei de lumină (MVPS)
- În fragment shader se extrage din textura de shadow map coordonata z corespondentă fragmentului considerat ce a fost transformat în spațiul sursei de lumină
- Se compară aceasta valoare (extrasă din shadow map) cu cea a z-ului fragmentului considerat (transformat în spațiul sursei de lumină) și dacă valoarea este mai mică înseamnă ca atunci fragmentul considerat se află în umbră (sursa de lumină “vede” alt fragment care este mai aproape de ea decât fragmentul considerat)

SHADOW MAPPING (4)

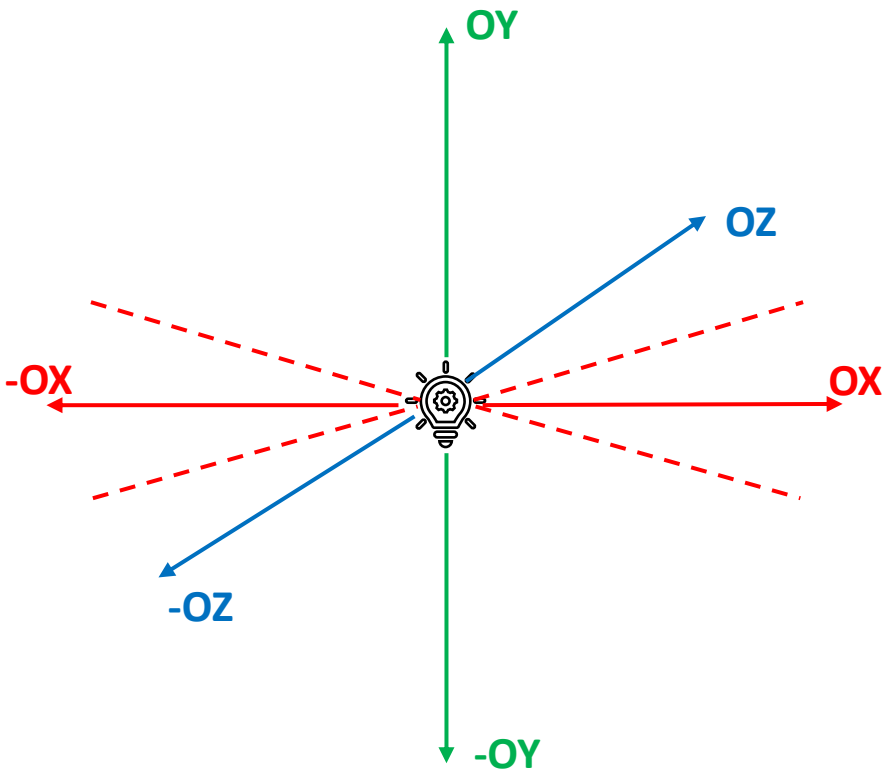


SHADOW MAPPING (5)

Omni-directional shadow mapping:

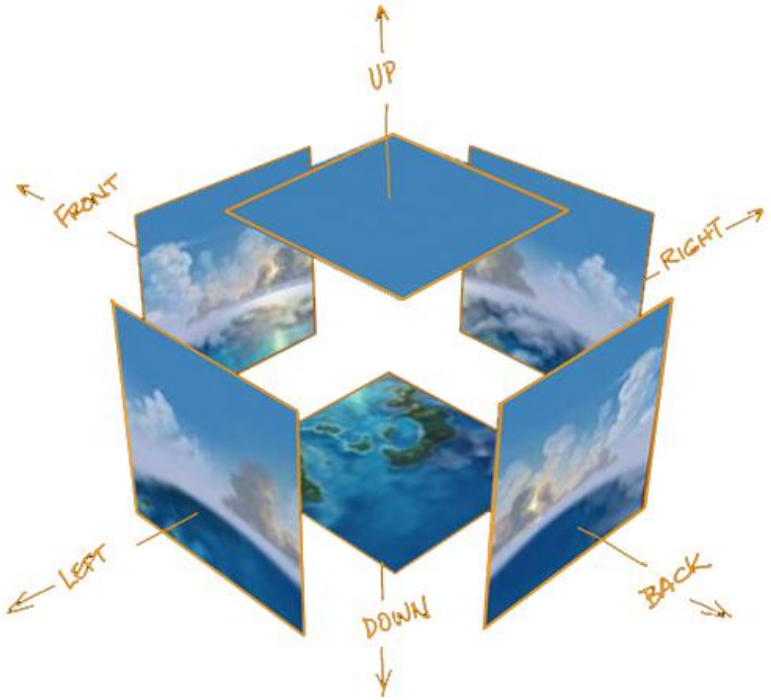
- Sursa de lumină omni-direcționala (punctiformă)
- 6 volume de vizualizare din poziția sursei de lumină ce acoperă întreaga scenă: OX, -OX, OY, -OY, OZ, -OZ
- Se folosește un cubemap unde fiecare față reprezintă textura de shadow map pentru fiecare din cele 6 vizualizări
- Pentru a nu reda explicit de 6 ori se folosește un geometry shader care generează fiecare față a cubemap-ului, fiecare din aceste fețe cu matricea de VIEW corespunzătoare
- În rest algoritm similar cu cel de la shadow map clasic cu singura diferență că extragerea coordonatei Z din shadow map se face prin eșantionarea din cubemap pe direcția de la sursa de lumină la fragmentul considerat

SHADOW MAPPING (6)



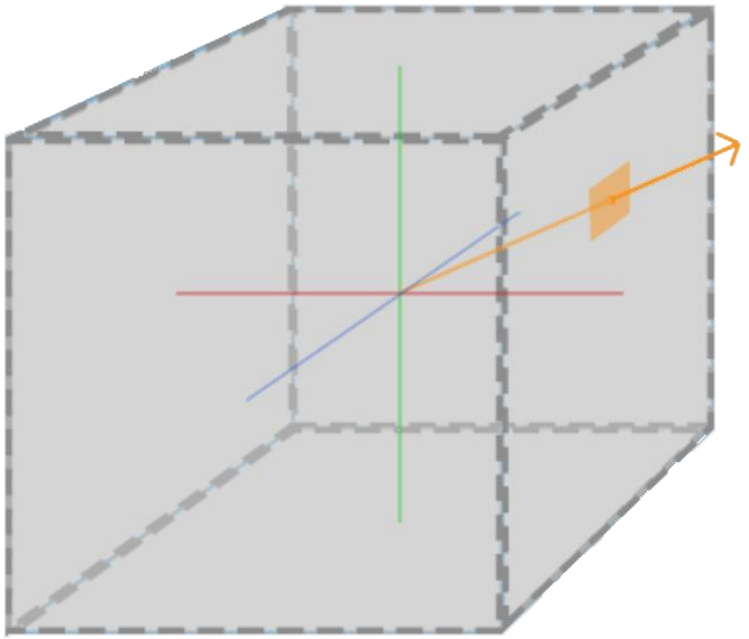
Câte o vedere din poziția sursei de lumină care acoperă fiecare din cele 6 direcții

SHADOW MAPPING (7)



Cubemap

Eșantionare în cubemap folosind un vector direcție



Pentru omnidirectional shadow maps:

- Eșantionare pe directia R unde

  R = world_position - light_position

Redarea suprafețelor 3D folosind texturi

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

Texturi (1)

Se folosesc pentru a reda detaliile suprafeţelor.

Două tipuri de texturi:

1. Texturi reprezentate prin imagini care se aplică pe o suprafaţă netedă, suprafaţa ramânând netedă și după aplicarea texturii

- Imaginea se aplică (« mapează ») pe suprafaţă în timpul generării imaginii suprafeței



- Imaginea mediului înconjurător aplicată pe suprafaţa unui obiect prin simularea reflexiei şi a refracţiei mediului înconjurător de către suprafaţa obiectului.



Texturi (2)

2. Texturi prin care se redau detalii ale geometriei suprafeței.

Se folosesc diferite metode:

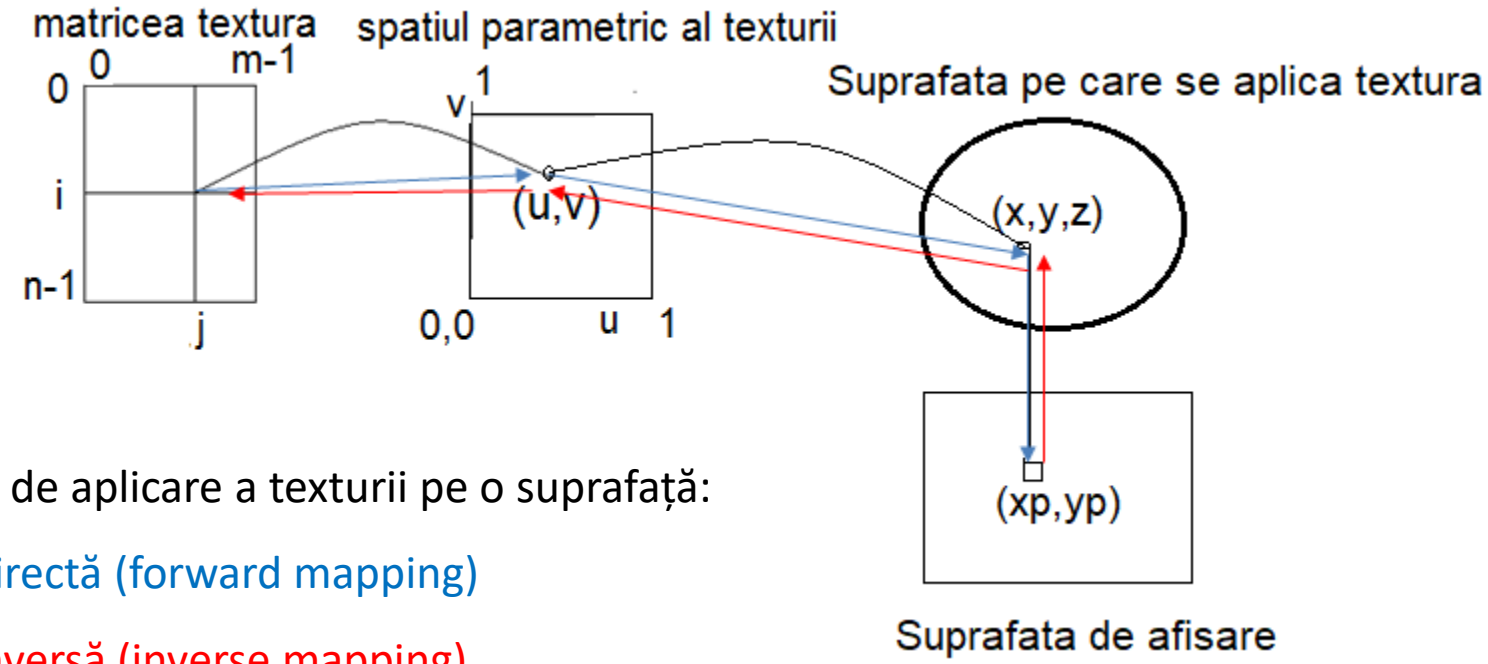
- perturbarea normalelor în punctele suprafeței
- metode fractale
- metode spectrale (funcția textură este definită pentru componentele ei pe diferite frecvențe)
- și altele



Ex: redarea detaliilor geometrice prin perturbarea normalei la suprafață (bump-mapping).

Maparea unei imagini textură pe o suprafață 3D (1)

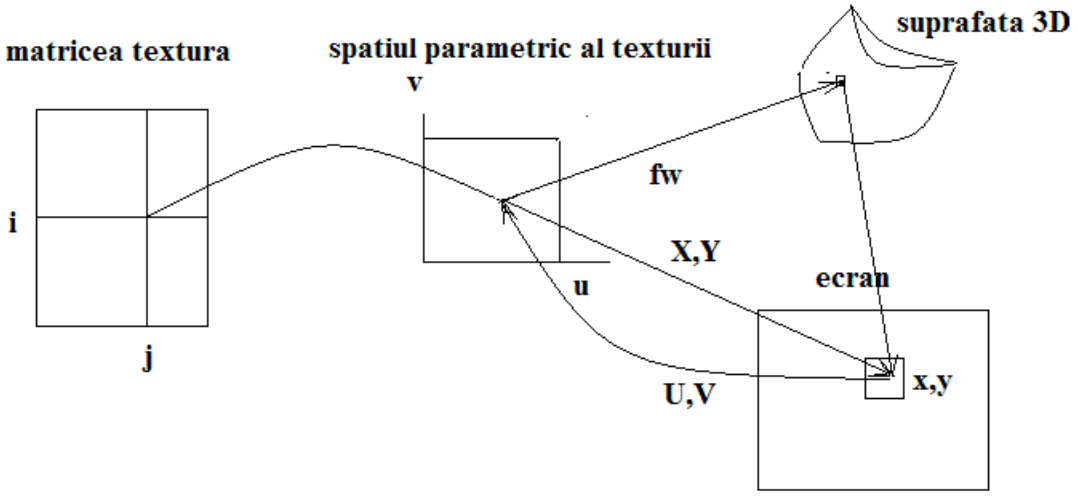
- Imaginea textură este memorată într-o matrice ale cărei elemente sunt culori și sunt adresate prin intermediul unui spațiu parametric (u,v), 0 <= u,v =< 1, care este pus în corespondența suprafeței 3D pe care se aplică textura.
- Elementele matricei textură sunt numite « texeli ».



Sunt 2 metode de aplicare a texturii pe o suprafață:

1) maparea directă (forward mapping)
2) maparea inversă (inverse mapping)

Maparea unei imagini textură pe o suprafață 3D (2)



1) maparea directă (forward mapping): se calculează pentru fiecare texel pixelul în care se afișează.

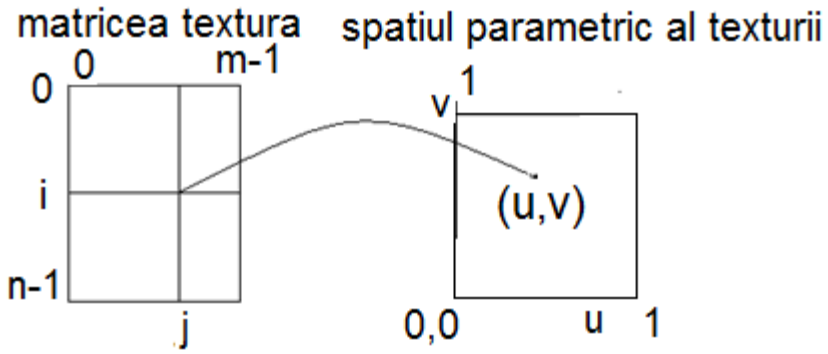
- adresa texel: (i,j) → adresa (u,v) → punct de pe suprafaţa 3D (x,y,z) → adresa pixel, (xp, yp)
- afisare pixel (xp, yp) in culoarea texelului textura[i][j]
- metoda este rar utilizată

2) maparea inversă (inverse mapping): se calculează pentru fiecare fragment (pixel) texelul/texelii din care se obține culoarea textură.

- adresa pixel: (xp,yp) → (x,y,z) → adresa (u,v) → adresa texel, (ir,jr)
- afișează fragment folosind culoarea obținută prin filtrarea texturii.

Spațiul parametric al texturii

Corespondența dintre matricea textură și spațiul parametric al texturii



Maparea directă

Adresa unui texel in spatiul (u,v)

(i, j) ⟹ u = j / (m-1)

v = (1 -(i / (n-1))

Maparea inversă

Accesarea matricei textura din spatiul (u,v)

(u,v) ⟹ jr = u * (m-1)

ir = (1-v) * (n-1)

(ir, jr) - numere reale

- accesarea matricei textura - prin numere intregi

⇩ Filtrarea texturii

2 posibilitati:

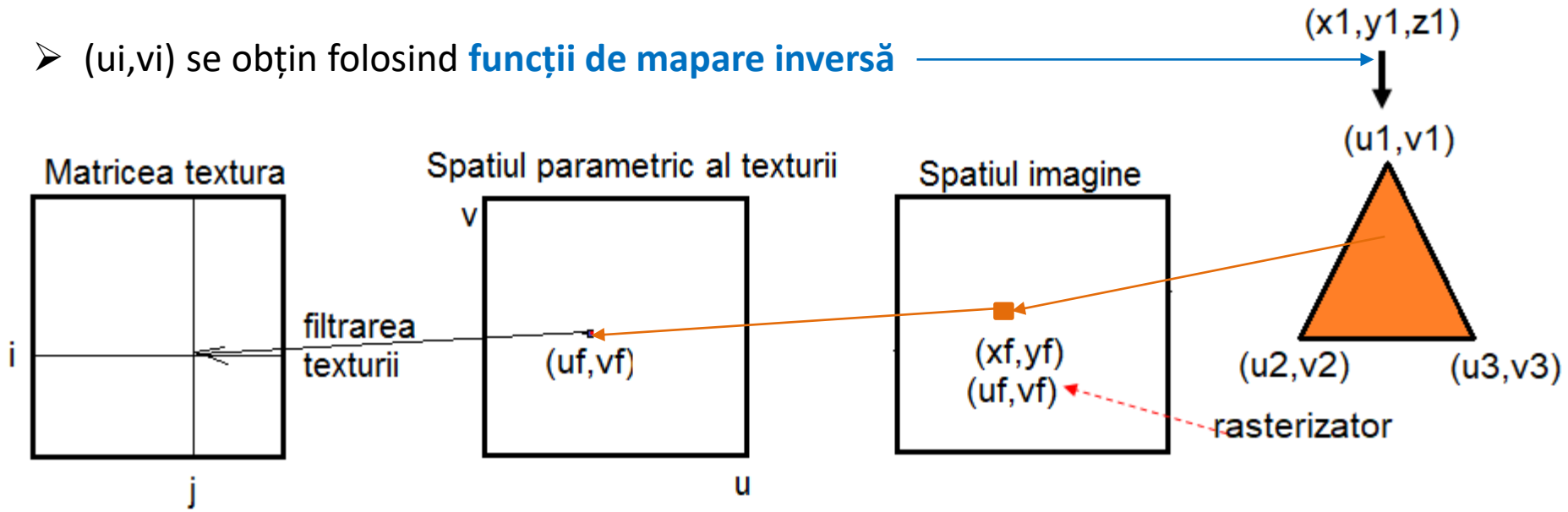
1) culoare = textura[i][j] j= round(jr), i = round(ir)

2) culoare = interpolare_biliniara (textura, ir, jr)

Maparea inversă (1)

Este metoda uzuală de mapare a texturilor pe suprafețele 3D

- Pentru fiecare fragment, (xf,yf), rezultat din rasterizarea unei primitive care are atașată o textură, rasterizatorul calculează adresa (uf,vf), pe care o trimite la fragment shader.
- În fragment shader se obține o culoare din textură folosindu-se (uf,vf)
- (uf,vf) se obține prin interpolarea coordonatelor (ui,vi) ale vârfurilor primitivei
- (ui,vi) se obțin folosind **funcții de mapare inversă**



Maparea inversă (2)

- Corespondența între punctele suprafeței 3D pe care se aplică textura și punctele spațiului (u,v) este stabilită prin funcții de forma:

  $\mathbf{u=f_u(x,y,z)\ ;\ v=f_v(x,y,z)}$

  $f_w^{-1} = (f_u(x,y,z),\ f_v(x,y,z))$ este numită **funcție de mapare inversă**

- Corespondența (x,y,z) ←→ (u,v) se stabilește în programul de aplicație pentru vârfurile primitivelor grafice în care este descompusă suprafața 3D.
- În timpul rasterizării primitivelor, coordonatele (u,v) ale vârfurilor sunt interpolate rezultând coordonatele (uf,vf) corespunzătoare fiecărui fragment
- Coordonatele (uf,vf) sunt intrări pentru fragment shader și se folosesc pentru accesarea texturii.

- Pot fi stabilite funcții de mapare inversă numai pentru suprafeţe 3D care pot fi definite într-un spațiu parametric (t1,t2).
- Corespondența se stabilește între spațiul (t1,t2) și (u,v).

Funcții de mapare standard (1)

Funcțiile de mapare inversă uzuale se numesc **funcţii de mapare standard.**

1. Maparea plană (maparea pe un plan)

Este definită de următoarele ecuații:

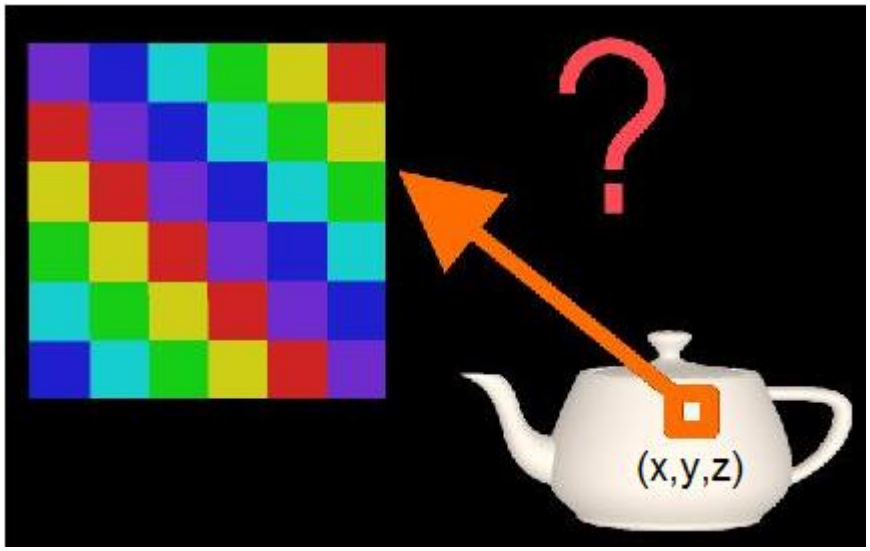
$u = su*x - ou \qquad x \in X,\ y \in Y$

$v = sv*y - ov \qquad 0 <= u, v <= 1$

unde : su, sv sunt factorii de scalare pentru transformarea spațiului (X, Y) în (u,v) iar

(ou,ov) este originea texturii: determină modul în care se așează textura pe suprafaţă.

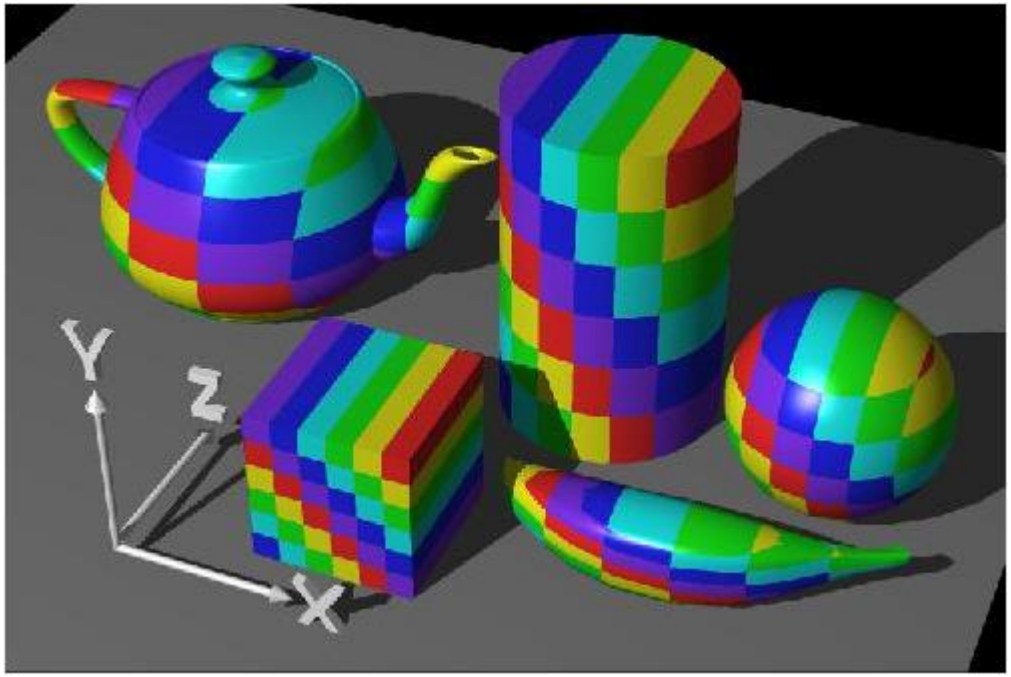
(u, v) ← (x, y) ← (x, y, z) - se renunță la coordonata z: suprafața 3D devine o suprafață plană





http://www.siggraph.org/education/materials/HyperGraph/mapping/r_wolfe/r_wolfe_mapping_2.htm

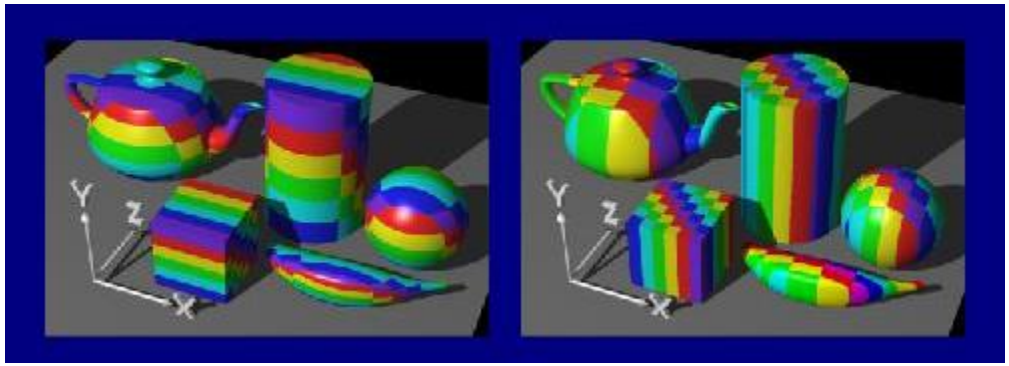
Funcții de mapare standard (2)



Exemple de mapări plane

Maparea plană prin renunțarea la coordonata z: toate punctele suprafeței cu aceleași coordonate (x,y) au aceeași culoare.

http://www.siggraph.org/education/materials/HyperGraph/mapping/r_wolfe/r_wolfe_mapping_2.htm

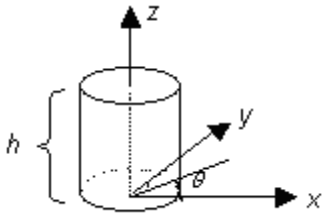


Maparea plană prin renunțarea la coordonata x, respectiv y.

Funcții de mapare standard (3)

2. Maparea cilindrică

- Stabilește corespondența dintre coordonatele (x,y,z) ale unui punct de pe o suprafață cilindrică și spațiul parametric al texturii, (u, v)



Ecuațiile parametrice ale unei suprafețe cilindrice:

x=r*cos(θ)
y=r*sin(θ) 0< θ< 2π 0< w <1
z=w*h

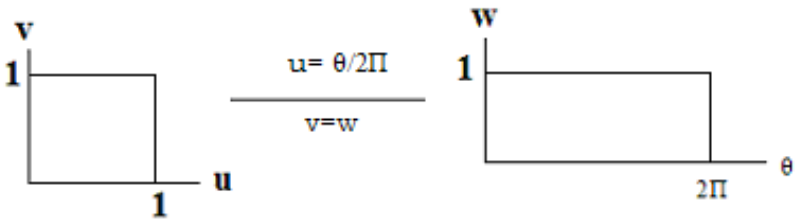
- Se calculează coordonatele cilindrice ale punctului (x, y, z)
- Se pune în corespondenţă spațiul parametric al texturii, (u,v), spațiului parametric al suprafeței cilindrice:

x = r*cos(θ)
y = r*sin(θ)
z = h*w

$\Rightarrow$

$\theta = \text{arctg}(\frac{y}{x})$

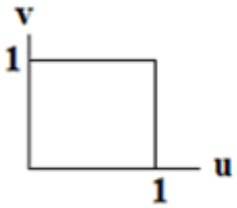
$w = \frac{z}{h}$



Funcții de mapare standard (4)

$\theta = \text{arctg}(\frac{y}{x})$

$w = \frac{z}{h}$



u = θ/2Π ; v=w



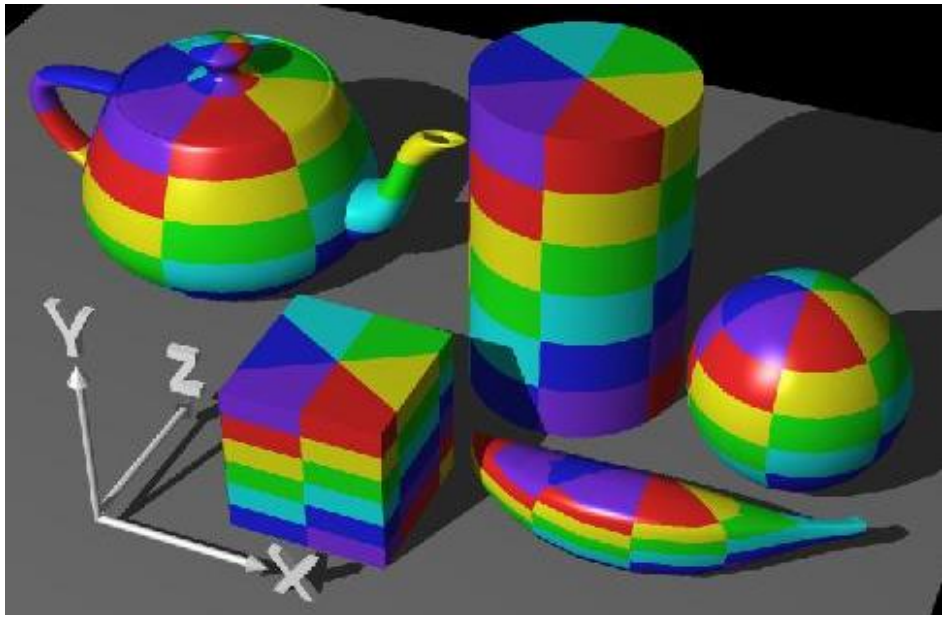
Funcția de mapare cilindrică

u = (1/2π)arctg(y/x)
v = z/h

Textura este “înfaşurată” în jurul obiectului



Transformarea pătratelor texturii la y= ymin și y= ymax



Funcții de mapare standard (5)

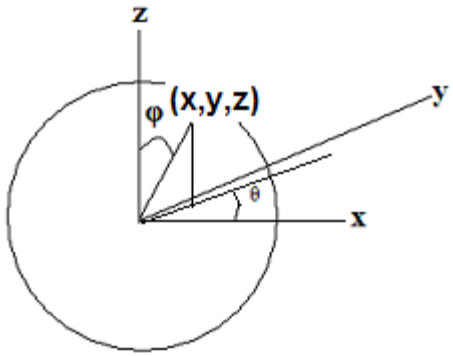
3. Maparea sferică

- Stabilește corespondența dintre coordonatele (x,y,z) ale unui punct de pe o suprafață sferică și spațiul parametric al texturii, (u, v)

Ecuațiile parametrice ale sferei

$$\begin{cases} x = r\cos\theta\sin\varphi \\ y = r\sin\theta\sin\varphi, cu \\ z = r\cos\varphi \end{cases} \begin{cases} 0 < \theta < 2\pi \\ 0 < \varphi < \pi \end{cases} \Rightarrow \begin{cases} \theta = arctg\left(\frac{y}{x}\right) \\ \varphi = \arccos\left(\frac{z}{r}\right) \end{cases}$$

Coord. sferice ale punctului (x,y,z)



Rezultă

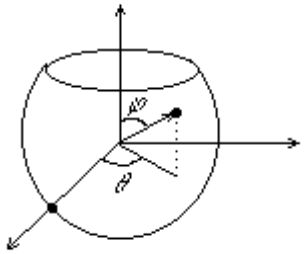
$u = \frac{\theta}{2\pi}$ sau $\begin{cases} u = \frac{1}{2\pi} arctg\left(\frac{y}{x}\right) \\ v = \frac{1}{\pi}\arccos\left(\frac{z}{r}\right) = \frac{1}{\pi}\arccos\frac{z}{\sqrt{x^2+y^2+z^2}} \end{cases}$

$v = \frac{\varphi}{\pi}$

Funcția de mapare sferică

Funcții de mapare standard (6)

Maparea pe un sector de sferă



$0 <= \theta <= \pi/2$

$\pi /4 <= \varphi <= \pi /2$

Functia de mapare

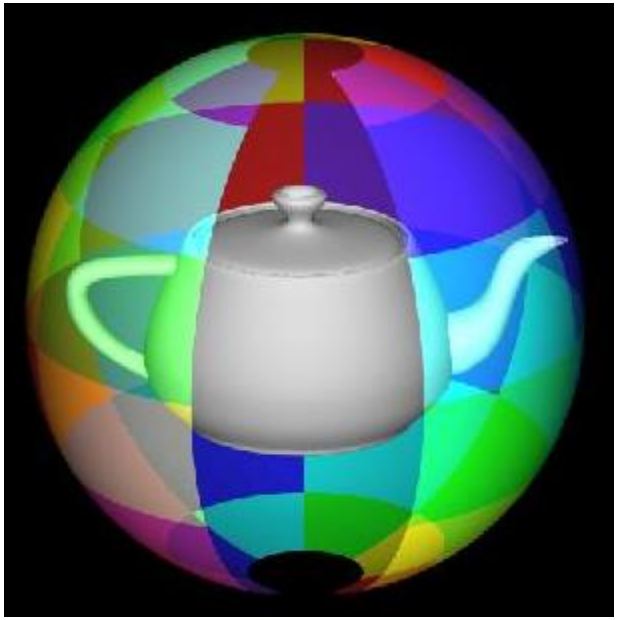
$u = \theta/(\pi/2)$

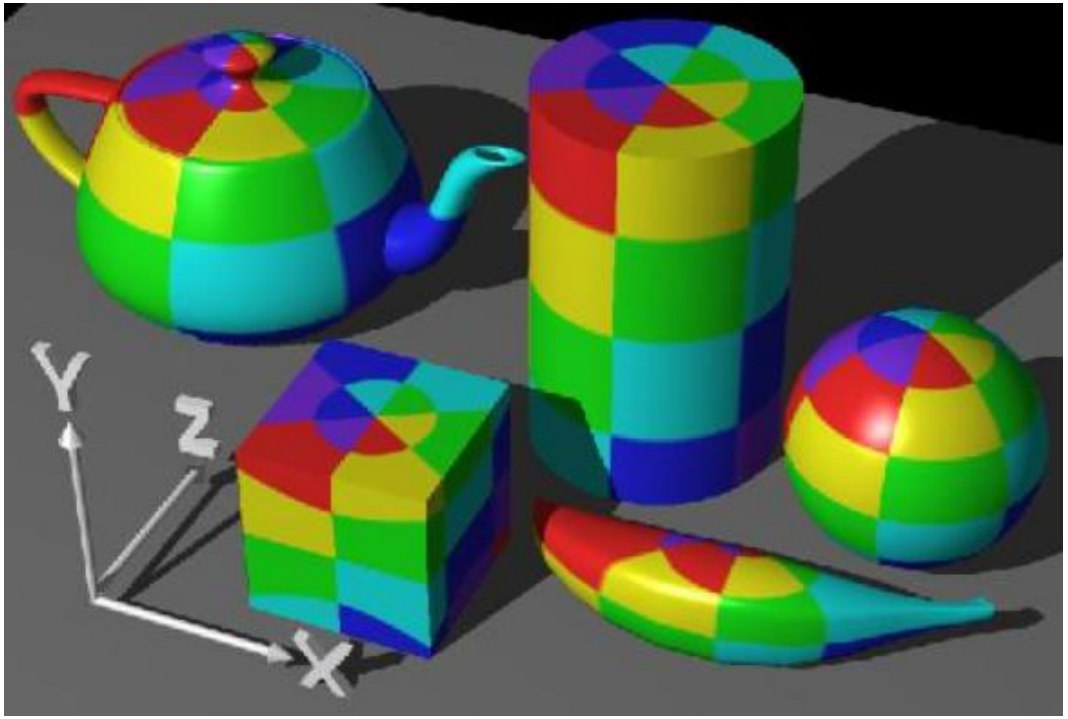
$v = ((\pi /2) – \varphi) / (\pi /4)$

Funcții de mapare standard (7)

Exemple de mapări sferice

Maparea sferică “întinde” pătratele texturii la nivelul ecuatorului și le “comprimă” proporțional cu aproprierea de poli.





http://www.siggraph.org/education/materials/HyperGraph/mapping/r_wolfe/r_wolfe_mapping_2.htm

Maparea unei texturi pe o suprafață oarecare

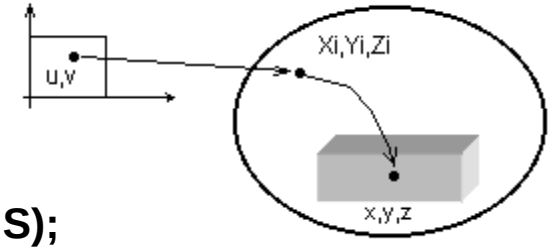
- Nu orice suprafață poate fi modelată analitic prin ecuații parametrice a.i. să se poată stabili corespondența între vârfurile primitivelor în care a fost descompusă suprafața și spațiul parametric al texturii (u,v).
- Exemplu: o suprafață oarecare reprezentată printr-o rețea de triunghiuri (mesh)
- Dacă mesh-ul aproximează o sfera, atunci se pot calcula coordonatele (u,v) atașate vârfurilor folosind funcția de mapare sferică
- Cum atașăm coordonatele (u,v) vârfurilor mesh-ului, dacă nu putem folosi una dintre funcțiile de mapare (mesh-ul reprezintă/aproximează o suprafață oarecare) ?

  → **folosim maparea în 2 pași**

  → tehnică de mapare independentă de forma obiectului pe care se aplică textura. Metoda a fost definita de Bier și Sloan (1986).

Maparea în 2 pași (1)

- Maparea texturii pe suprafața de texturat se realizează folosind o suprafață intermediară, pentru care exista o funcție de mapare.
- Suprafața intermediară încadrează suprafața de texturat.
- Aplicarea texturii este descompusă în două operații,



al căror efect este :

- **mularea texturii pe suprafața intermediara (mapare S);**
- **mularea suprafeței intermediare pe suprafața de texturat (mapare O).**

a) **maparea S este o mapare inversă:** fu și fv sunt funcții de mapare inversă:
u = fu(xi, yi, zi)
v = fv(xi, yi, zi)
unde (xi,yi,zi) este un punct de pe suprafața intermediară

b) (x, y, z) → **mapare O** → (xi, yi, zi)

unde (x,y,z) este un punct de pe suprafața de texturat

Maparea în 2 pași (2)

Maparea S

Se folosesc 4 tipuri de suprafețe intermediare :

1. un plan cu o orientare oarecare;
2. sfera;
3. cilindrul;
4. fețele unui cub.

Alegerea tipului suprafeței intermediare este dependentă de forma obiectului pe care trebuie să fie aplicată textura și de efectul dorit.

Maparea O

Se poate folosi pentru a calcula coordonatele (u,v) ale vârfurilor unui mesh.

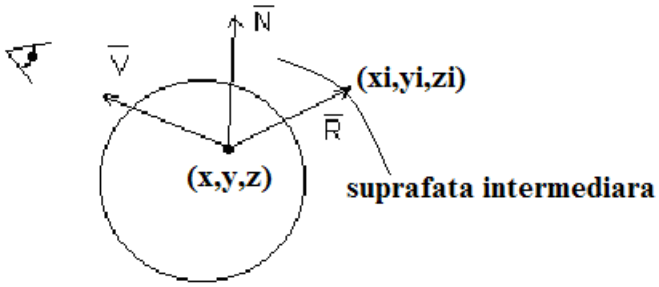
Autorii metodei au definit 4 tipuri de mapări O:

Maparea în 2 pași (3)

Mapări O

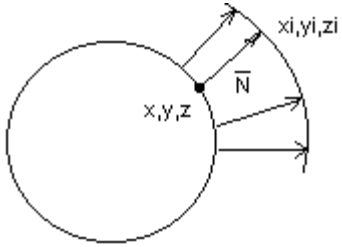
(x, y, z) – un punct al suprafeței, pentru care se dorește calculul coordonatelor (u,v)

(xi, yi, zi) – punctul corespunzător lui (x,y,z) pe suprafața intermediară



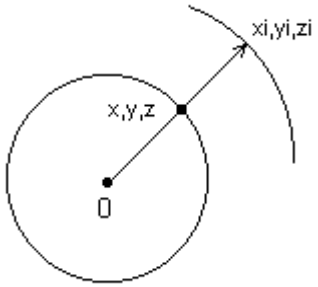
1. (xi, yi, zi), se obține intersectând suprafața intermediară cu vectorul reflectat al vederii în punctul (x,y,z). R este simetric cu V față de N.

Metoda este dependentă de poziția observatorului (se folosește la «maparea mediului înconjurător»)

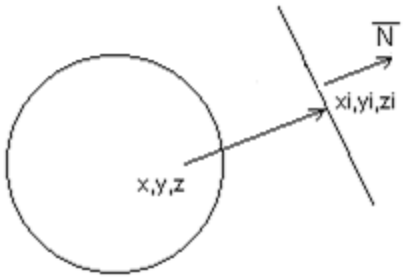


2. (xi,yi,zi) este intersecția suprafeței intermediare cu normala la obiect în (x,y,z).

Maparea în 2 pași (4)



3. (xi,yi,zi) este intersecția suprafeței intermediare cu dreapta care trece prin (x,y,z) și centroidul obiectului.



4. (xi,yi,zi) este intersecția suprafeței intermediare cu dreapta care trece prin (x,y,z) și are direcția normalei la suprafața intermediara.

Acest tip de mapare O este corelat cu o mapare S pe un plan sau pe fețele unui cub.

Combinația « mapare S pe cilindru » - « mapare O de tip 4 » este numită « shrinkwrap ».

Filtrarea texturii (1)

- O textură este definită pe un spațiu discret, fiecare texel fiind adresat prin 2 numere întregi, în timp ce coordonatele textură sunt valori reale. În general, o pereche (uf,vf), care trebuie folosită la colorarea unui fragment, nu corespunde unei adrese de texel în matricea textură.
- **Metoda de calcul a culorii dintr-o textură, folosind coordonatele (uf,vf), se numește "filtrarea texturii".**

Cazul 0<= uf, vf <= 1

Fie (if, jf) adresa coresp. perechii (uf,vf) în spațiul texturii: if = (1-vf)*(n-1), jf = uf*(m-1)

Sunt două moduri standard de filtrare:

1. Se consideră culoarea texelului cu adresa cea mai apropiată (GL_NEAREST) de adresa (if,jf) în spațiul texturii:
   - rapid dar poate produce defecte în imagine
2. Se efectuează o interpolare biliniară (GL_LINEAR) între culorile celor mai apropiați texeli în spațiul texturii:
   - scade viteza de calcul a culorii fragmentului
   - poate produce un efect de încețoșare

Filtrarea texturii (2)

Interpolarea biliniară între culorile din matricea textură

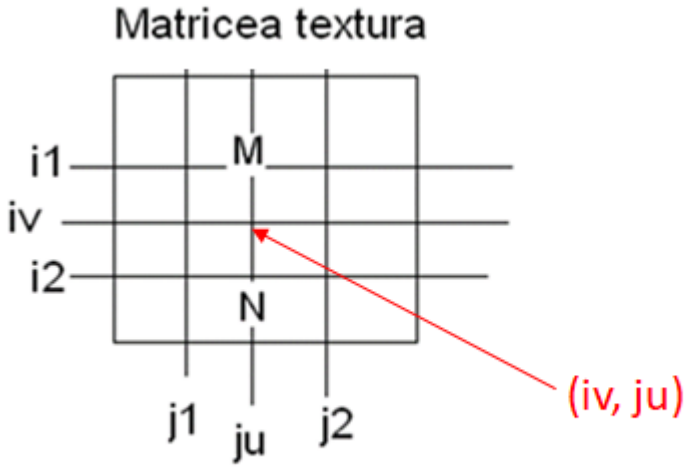
- Dacă (ju, iv) sunt adresele reale corespunzătoare valorilor (uf,vf) în spațiul textură,

  iv = (1-vf)*(n-1); ju = uf*(m-1)

  n - numărul de linii ale matricei textură; m – numărul de coloane

  i1< iv < i2, i2 = i1 + 1 j1< ju < j2, j2 = j1 + 1

- Culoarea extrasă din matricea textură se calculează astfel:



M(i1,ju); N(i2,ju);

C(i,j) – culoarea texelului de indici (i,j)

Cul(M) = (j2-ju)*C(i1,j1) + (ju-j1)*C(i1,j2)

Cul(N) = (j2-ju)*C(i2,j1) + (ju-j1)*C(i2,j2)

Culoare_textura = (i2-iv)*Cul(M) + (iv-i1)*Cul(N)

i1, i2, j1, j2 sunt indici de linie/coloana in matricea textura

Filtrarea texturii (3)

Cazul uf, vf în afara domeniului [0,1]

- Coordonatele (ui, vi) asociate vârfurilor unei primitive pot fi în afara domeniului [0,1].

Rezultă:

coordonatele (uf, vf) ale unui fragment pot fi în afara domeniului [0,1].

- Coordonatele (uf, vf) rezultate in procesul de rasterizare sunt transformate în coordonate

$0 <= u, v <= 1$

- Cele două moduri standard de transformare a coordonatelor textură în domeniul [0,1] sunt denumite “clamping” și “repeating”: ele permit utilizarea eficientă a texturilor și obținerea de efecte interesante.

Clamping (Comprimare)

Coordonatele (uf,vf) sunt transformate în (u, v) astfel:

$(u, v) = (\min(\max(0,uf),1),\min(\max(0,vf),1))$

Deci: daca uf sau vf <0, atunci u , respectiv v =0; daca uf sau vf >1, atunci u, respectiv v=1

daca $0 <= uf, vf <= 1$, atunci u, v = uf, vf.

Filtrarea texturii (4)

Clamping (Comprimare)



Aplicarea acestei transformări are ca efect propagarea culorii din imaginea textură de-a lungul direcției în care coordonata textură depășește limita de 0 sau de 1.

Filtrarea texturii (5)

Repeating

- Pentru accesarea texturii se calculează (u,v) cu formula:

  (u,v) = (uf- [uf]), vf-[vf]),

  unde [w] reprezintă cel mai mare întreg mai mic sau egal cu w.

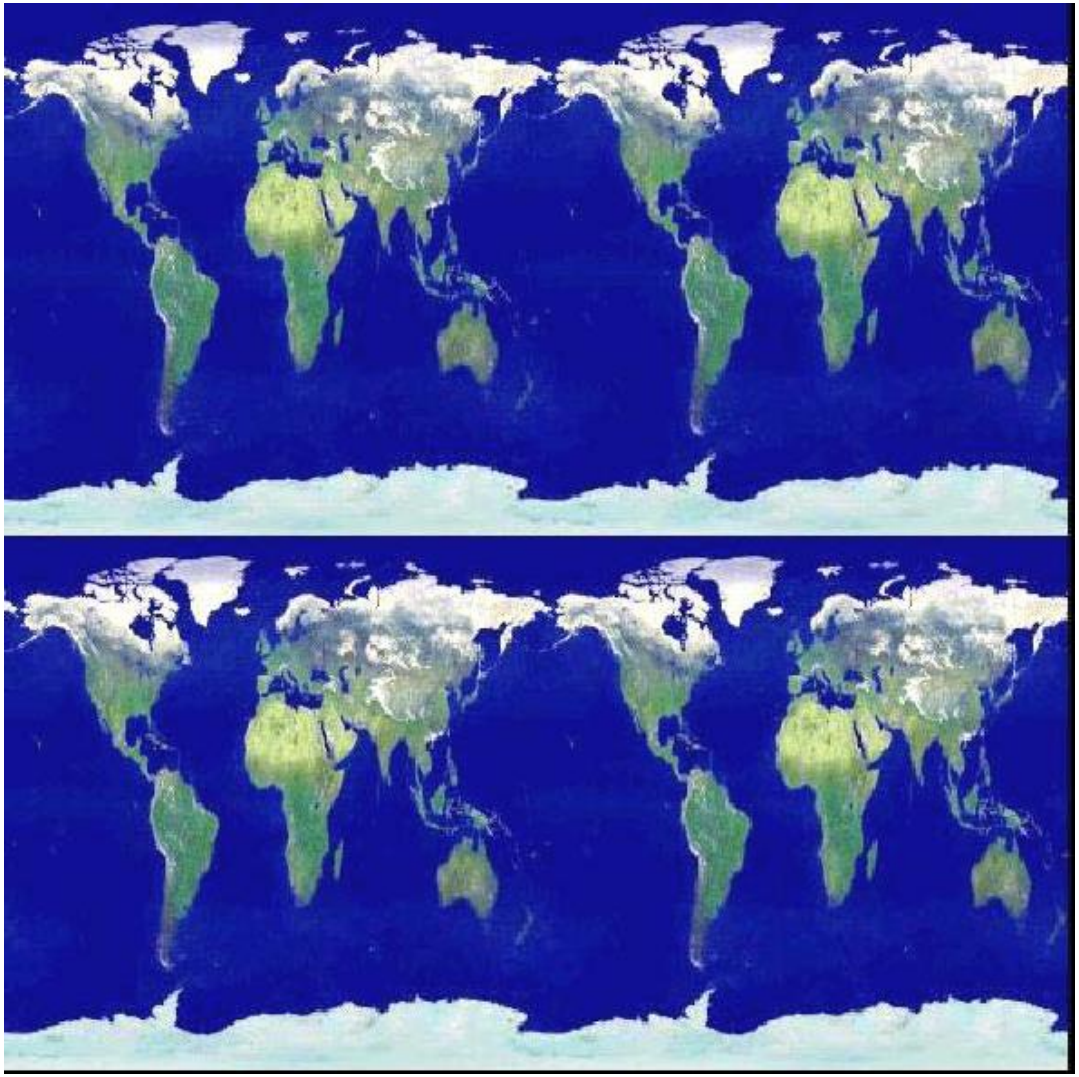
  De exemplu, (-0.1, 1.5) se transformă în (-0.1 – (-1), 1.5 – (1)) = (0.9, 0.5).

- **Acest mod de transformare permite repetarea unei texturi, producând un efect periodic.**
- La utilizarea acestui mod este necesar ca laturile stânga/dreapta, respectiv sus/jos ale imaginii textură să se potrivească, astfel încât să nu se observe marginile texturii.

- Dacă pentru una dintre coordonate se utilizează transformarea “comprimare” iar pentru cealalta “repetare”, textura se numește “cilindrică”.
- Dacă pentru ambele coordonate se utilizează modul “repetare”, textura se numește “ toroidală”.

Filtrarea texturii (6)

Repetare



Transparența texturii

- Imaginea textură poate avea o componentă adițională, numită “canalul alfa”, prin care se controlează transparența sau opacitatea texturii aplicate.
- Fiecare texel este în acest caz reprezentat prin 4 componente: (R,G,B,A), unde A reprezintă opacitatea.
  - daca A = 1, atunci texelul este complet opac și fragmentul va fi afișat în culoarea sa;
  - daca A = 0, texelul este complet transparent; nu se folosește în calculul culorii fragmentului.
- Pentru 0<= A <=1, culoarea texelului modulează culoarea fragmentului, determinată pe baza geometriei (modelul Gouraud sau modelul Phong).
- Interfețele de programare OpenGL și Direct3D permit specificarea modului de combinare între culoarea unui fragment determinată pe baza geometriei primitivei și culoarea rezultată din filtrarea texturii (texturilor), folosind operatori care se aplică culorilor (RGB) și operatori care se aplică opacităților A.
- Combinarea dintre culoarea determinată pe baza geometriei și culoarea texturii poate fi programată în “fragment shader”.

Iluminarea globală
-Ray Tracing-

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

Iluminarea globală în Ray tracing (1)

Îmbină:

- Eliminarea părților nevizibile ale scenelor 3D
- Calculul reflexiei luminii
- Calculul refracției luminii (transparență)
- Calculul umbrelor
- **Interacțiunea globală a reflexiei și refracției luminii la nivelul întregii scene 3D**

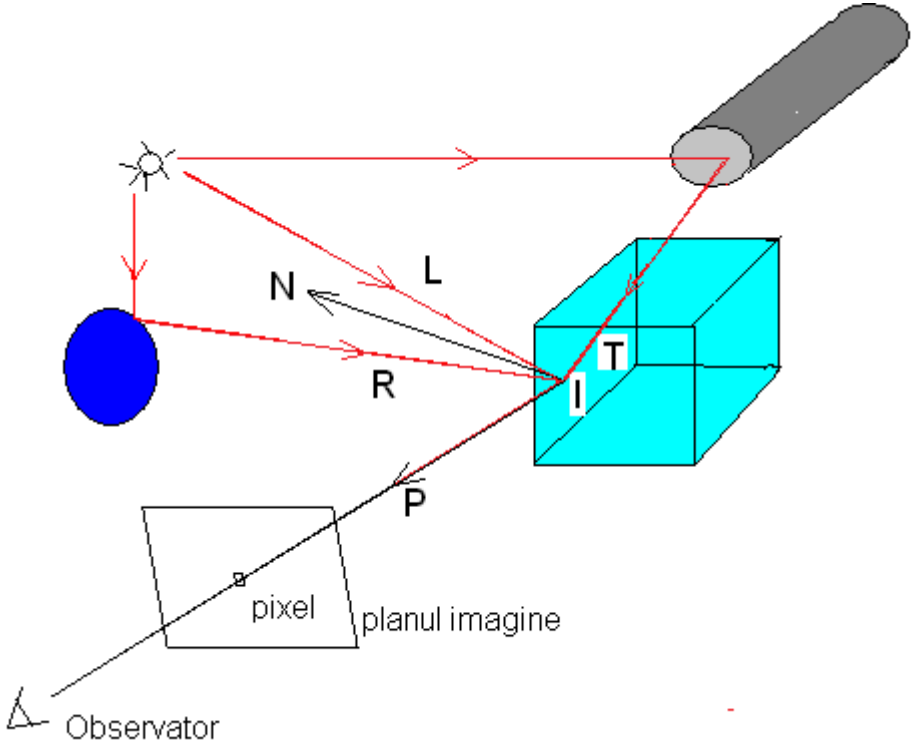
➢ În algoritmul Ray Tracing:

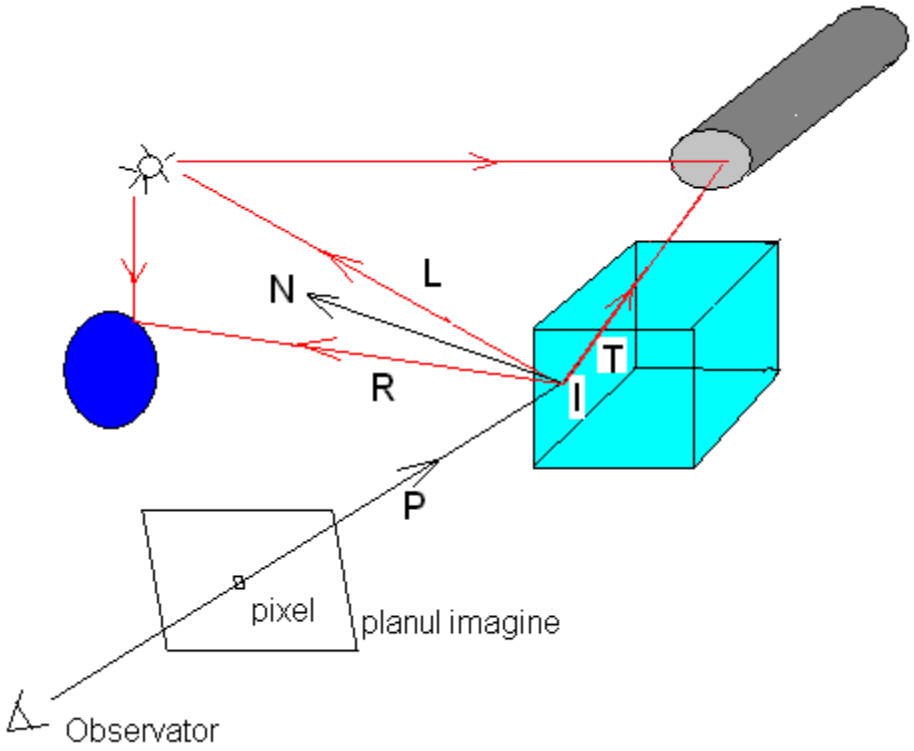
❖ Imaginea unei scene 3D NU se obține prin rasterizarea primitivelor grafice în care a fost descompusă scena

❖ Imaginea este calculată intersectând scena 3D cu raze care pleacă din poziția observatorului, fiecare trecînd printr-un pixel din planul imaginii

➢ Culoarea unui pixel este calculată în punctul de intersecție al razei cu scena, care este cel mai apropiat de observator.

Iluminarea globală în Ray tracing (2)





Directia razelor de lumina în realitate:

- Pixelul este colorat în culoarea punctului I.
- P este direcția pe care lumina din punctul I ajunge la observator.

În calculul culorii pixelului se consideră raze cu direcția inversă celor din realitate

Algoritmul Ray tracing

Imaginea se calculează pixel cu pixel, pornind de la spațiul imagine.

Considerând o singură sursă de lumină:

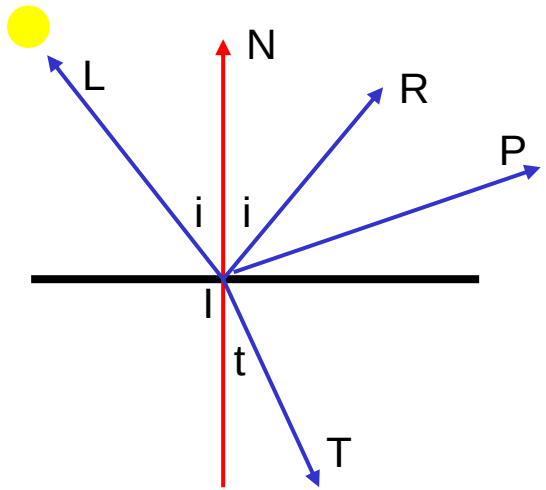
Pentru fiecare pixel al imaginii:

- Se calculează raza primară, P, care pornește din poziția observatorului și trece prin centrul pixelului, în planul imaginii
- Se determină punctul de intersecție, I, al razei primare cu cel mai apropiat obiect de observator
- **Dacă** raza nu intersectează nici un obiect al scenei,
  - Se afișeaza pixelul în culoarea fondului

  Altfel
  - Se calculează culoarea obiectului în punctul I, ținând cont de:
    - Reflexia luminii provenită direct de la sursa de lumină, raza L
    - Lumina provenită în I de la alte obiecte pe direcția reflexiei speculare, raza R
    - Lumina provenită în I de la alte obiecte pe direcția razei transmise, raza T
  - Afișează pixelul în culoarea obținută prin combinarea contribuției celor trei raze

Ray tracing – calculul razelor

Calculul razelor care contribuie la culoarea luminii în punctul de intersecție cu raza primară



P: raza primară – din I către observator

L: raza din I către sursa de lumină

R: raza reflectată specular, simetrică, față de N, cu L

$R = 2(N \cdot L)N - L$

T: raza transmisă, calculată pe baza legii lui Snell:

$n1/n2 = \sin(t)/\sin(i)$, n1,n2: indicii de refracție

$T = L*(n1/n2) – (\cos(t) + (n1/n2)*(L \cdot N))*N$

✓*Razele sunt considerate infinit subțiri*

✓*Reflexia speculară și refracția au loc fără împrăștiere (sunt perfect focalizate)*

➢**Efect**: obiectele din imaginea produsă sunt de regulă strălucitoare, producând reflexii multiple focalizate.

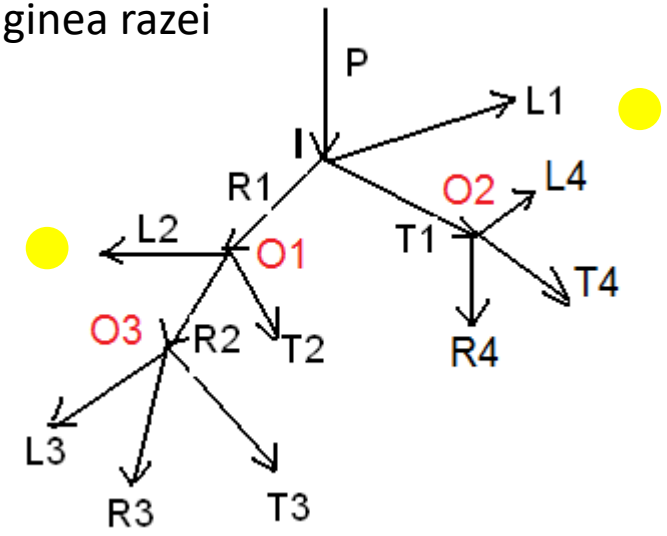
Ray tracing – arborele de raze

Algoritmul Ray tracing este recursiv:

Lumina provenita in punctul I prin reflexie specular de la un obiect O1 (de-a lungul razei R1) sau prin transmisie de la un obiect O2 (de-a lungul razei T1), poate fi compusă din:

- Reflexia luminii provenită direct de la o sursă, de către O1/O2 – raza L2/L4
- Reflexia specular a altor obiecte de către O1/O2 – raza R2/R4
- Transmisia luminii pe O1/O2 provenită de la alte obiecte ale scenei – raza T2/T4

➢ Pentru fiecare rază se consideră punctul de intersecție cu scena, care este cel mai apropiat de originea razei



Arborele de raze

➢*Pentru obținerea culorii pixelului, se evaluează arborele de raze de la frunze către rădăcină*

Ray tracing- calculul reflexiei luminii (1)

Calculul culorii luminii în punctul I:

$$I_\lambda(I) = I_{local\lambda}(I) + Ks*R_\lambda(I) + Kt*T_\lambda(I)$$

unde:

λ – reprezintă lungimea de undă: expresia se evaluează pentru R,G,B

$I_{local\lambda}(I)$ – reprezintă reflexia luminii provenite direct de la sursele de lumină din scena 3D (calculată folosind modelul de iluminare locală)

Ks – este coeficientul de reflexie speculară al materialului obiectului

$R_\lambda(I)$ – reprezintă reflexia speculară a luminii provenite de la alte obiecte ale scenei în punctul I: se obține prin evaluarea arborelui de raze

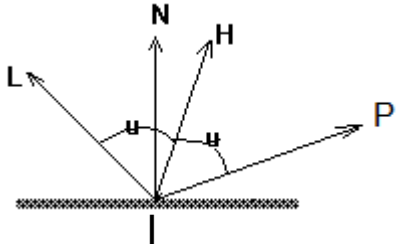
Kt – este coeficientul de transmisie, specific materialului obiectului

$T_\lambda(I)$ - reprezintă lumina provenită prin transmisie (refracție) în punctul I: se obține prin evaluarea arborelui de raze

0<= ks, kt <= 1

Ray tracing- calculul reflexiei luminii (2)

Modelul de iluminare locală (reflexia luminii în punctul I) :



$I_{local\lambda}(I) = Ia_\lambda * Ka + Isursa_\lambda * fat * s * lum * [kd * (N \cdot L) + Ks * (N \cdot H)^n]$

Iaλ – reprezintă intensitatea luminii ambientale

Ka – este coeficientul de difuzie a luminii ambientale, specific materialului obiectului, 0<= ka <= 1

$Isursa_\lambda$ – reprezinta intensitatea luminii provenite de la sursă

L – este versorul direcției din punctul I către poziția sursei de lumină

N – este normala în punctul I (versor)

H – este versorul direcției bisectoare a unghiului dintre L și vectorul din I către observator (raza P)

fat – este factorul de atenuare a luminii de la sursă, proporțional cu distanța de la sursa la punctul I

0<= s <= 1,

s = 0, dacă lumina de la sursă nu ajunge in punctul I: vectorul L intersectează un obiect opac al scenei

s = 1, dacă vectorul L nu intersectează un alt obiect al scenei

0 < s <1, dacă vectorul L intersectează un obiect transparent (sau mai multe)

Punctul I primește lumina de la sursă dacă:

- produsul scalar (N·L)>0 (lum =1, altfel =0)
- raza L nu intersectează un obiect opac al scenei

➢ Daca I nu primește lumina de la sursă, iluminarea locală se reduce la componenta ambientală.

Ray tracing- calculul reflexiei luminii (3)

- Dacă în scena 3D există mai multe surse de lumină, fiecare poate contribui în mod diferit la $I_{local\lambda}(I)$:

$$I_{local\lambda}(I) = Ia_{\lambda} * Ka + \sum_{i=1,n} Isursa_{i,\lambda} * fat_i * s_i * lum_i * [kd * (N \cdot L_i) + Ks * (N \cdot H_i)^n]$$

- Pentru evaluarea componentelor $R_{\lambda}(I)$ si $T_{\lambda}(I)$ din calculul culorii în I,

$$I_{\lambda}(I) = I_{local\lambda}(I) + Ks * R_{\lambda}(I) + Kt * T_{\lambda}(I)$$

se coboară în arborele de raze până la un număr pre-specificat de nivele (energia luminoasă scade destul de repede!).

Algoritmul Ray tracing

Algoritmul Ray tracing

Pentru fiecare pixel al imaginii

```
{    *calculeaza raza primară, P;// din poziția observatorului prin centrul suprafeței pixelului
     *culoare_pixel = TraseuRaza(P, 1); // culoarea în punctul de intersecție al razei P cu scena
     *afiseaza pixelul în culoare_pixel;
}
```

Culoare TraseuRaza(Raza R, int n) // apelată recursiv

```
{    // n este nivelul în arborele de raze
     // întoarce culoarea în punctul de intersectie al razei R cu scena
     *calculeaza intersecțiile razei R cu obiectele scenei;
     dacă (nu există intersecții), atunci
               return  (culoare_fond);
     altfel
               *fie I punctul de intersecție cel mai apropiat de observator și O obiectul intersectat;
               *calculează normala, N, în punctul I;
               return  CuloarePunct(O, R, I, N, n); // culoarea în I
}
```

Algoritmul Ray tracing

```
Culoare CuloarePunct(Obiect O, Raza R, Punct I, Normala N, adancime_arbore n)
{   Culoare  culoare;
    culoare = culoare_ambientală;
    pentru fiecare sursă de lumină S execută
        *calculează vectorul L, din I către S
        dacă ((Nu·Lu) > 0 și vectorul L nu intersectează un obiect opac al scenei) atunci
            *calculează contribuția sursei S la culoare, CS, folosind modelul de iluminare locală
            culoare = culoare + CS
    dacă (n<=nivel_max) atunci // n>nivel_max: ieșirea din recursivitate
        dacă (obiectul O produce reflexii speculare) atunci
            *calculează raza reflectată în punctul I, RS;
            culoare = culoare + Ks* TraseuRaza(RS, n+1); // se coboară în arbore
        dacă (obiectul O este transparent) atunci
            *calculează raza transmisă (refractată) în punctul I, RT;
            culoare = culoare + Kt* TraseuRaza(RT, n+1); // se coboară în arborele de raze
    return (culoare)
}
```

Algoritmul Ray tracing

Deficienţele algoritmului

1. **Efecte de aliasing**: razele sunt convergente și infinit subtiri, eșantionarea spațiului 3D este neuniformă
   - Se produc defecte la marginile suprafețelor (aliasing: marginile au aspect zimţat)
   - Obiectele mici și subțiri pot să apară și să dispară din imagine, la schimbarea poziției observatorului

   Îmbunătățire:

   – mărirea rezoluției eșantionării spațiului 3D; ex. 4 raze primare/pixel → crește complexitatea

2. **Complexitatea computațională**
   - Bună din punct de vedere teoretic:

     - pentru o rază/pixel este de O(p*n), unde p este nr. de pixeli - constant (independent de scenă) = O(n), unde n este nr. de obiecte din scenă
   - Crește liniar cu numărul de raze/pixel
   - Complexitatea calculelor de intersecție depinde de geometria obiectelor intersectate

   Îmbunătățire:

   – Reducerea calculelor de intersecție, prin folosirea de volume încadratoare ale obiectelor și gruparea obiectelor din scena 3D

   – Complexitatea se poate reduce la O(log (n))

Ray Tracing - optimizări

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

Îmbunătățirea calității imaginii (1)

Mărirea numărului de raze primare

1) Uniformă

4 raze primare/pixel, care trec prin colțurile suprafeței pixelului

- Culoarea pixelului se determină ca medie a culorilor obținute cu cele 4 raze primare
- Razele primare utilizate pentru un pixel contribuie și la culoarea pixelilor adiacenți

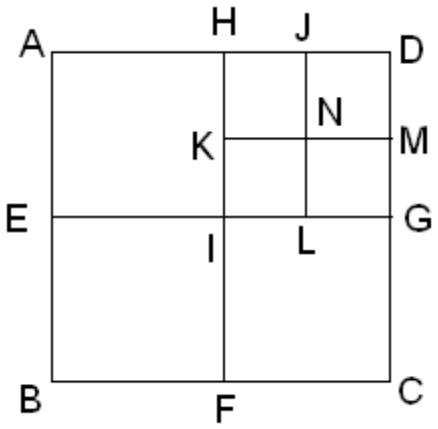
2) **Adaptivă** (Whitted) – subdivizarea adaptivă a suprafeței pixelului

- Se crește rezoluția eșantionarii spațiale numai în zonele în care este necesară operația de anti-aliasing:
  - Dacă diferența dintre culorile celor 4 raze primare pentru un pixel este mare, se subdivizează suprafața pixelului în 4 subzone
  - Se aplică același criteriu de comparație între culorile celor 4 raze primare corespunzătoare unei subzone
  - Subdivizarea se continuă recursiv până la un nivel maxim prestabilit sau până când diferența dintre cele 4 culori scade sub un prag dat
  - Culoarea pixelului se obține ca o medie ponderată a culorilor subzonelor în care a fost divizată suprafața pixelului

Îmbunătățirea calității imaginii (2)

Îmbunătățirea calității imaginii(2)

Exemplu de subdivizare adaptivă



CA, CB, CC, CD, CE, CF, CG, CH, CI, CJ, CK, CL, CM, CN

- sunt culorile obținute cu razele primare care trec prin punctele respective

Culoare pixel (A-B-C-D) =

¼((CA+CE+CI+CH)/4 + (CE+CB+CF+CI)/4 + (CI+CF+CC+CG)/4 +

¼((CH+CK+CN+CJ)/4 + (CK+CI+CL+CN)/4 + (CJ+CN+CM+CD)/4 +

(CN+CL+CG+CM)/4))

Reducerea complexității computaționale

Reducerea calculelor de intersecție

1. **Utilizarea de volume încadratoare la nivel de obiect**

   Se testează intersecția rază-volum încadrator în loc de rază-obiect:

   - Dacă raza nu intersectează volumul încadrator → nu se va calcula intersecția cu obiectul

2. **Divizarea scenei în volume încadratoare**

   Se divizează scena în volume egale până la o anumită rezoluție spațială sau se divizează scena adaptiv.

   Se testează intersecția razei cu volumele în care a fost divizată scena.

3. **Reprezentarea scenei printr-o ierarhie de volume încadratoare (BVH)**

   Scena este reprezentată printr-un arbore în care frunzele sunt obiectele scenei iar nodurile interne sunt grupări logice de obiecte ale scenei.

   Fiecare nod are un volum încadrator.

Utilizarea de volume încadratoare (1)

Utilizarea de volume încadratoare la nivel de obiect/grup de obiecte

- Permite evitarea calculelor de intersecție cu obiecte ale scenei care pot avea geometrie complexă
- Permite eliminarea din calculele de intersecție a unui întreg grup de obiecte care nu este intersectat de rază

Volume încadratoare:

- Sfera, paralelipipedul cu fețele paralele cu planele principale, elipsoidul, cilindrul.
- Calculul intersecției raza-volum încadrator trebuie sa fie mai simplu decât calculul intersecției cu obiectul (de ex. o rețea poligonală)
- Volumul încadrator al unui obiect se alege in funcție de forma obiectului

Utilizarea de volume încadratoare (2)

Calcule de intersecție rază-volum încadrator

Ecuatia razei: r(t) = P0 + t*D

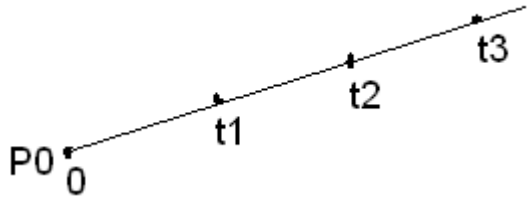
P0 – este originea razei

D – este direcția razei

$x = x0 + t*dx$

$y = y0 + t*dy$

$z = z0 + t*dz$



Punctul de intersecție cel mai apropiat de originea razei este r(tmin)

P0 poate fi:

- poziția observatorului (raza primară)
- un punct de intersecție cu scena (raza secundară)

1) Intersecția cu sfera

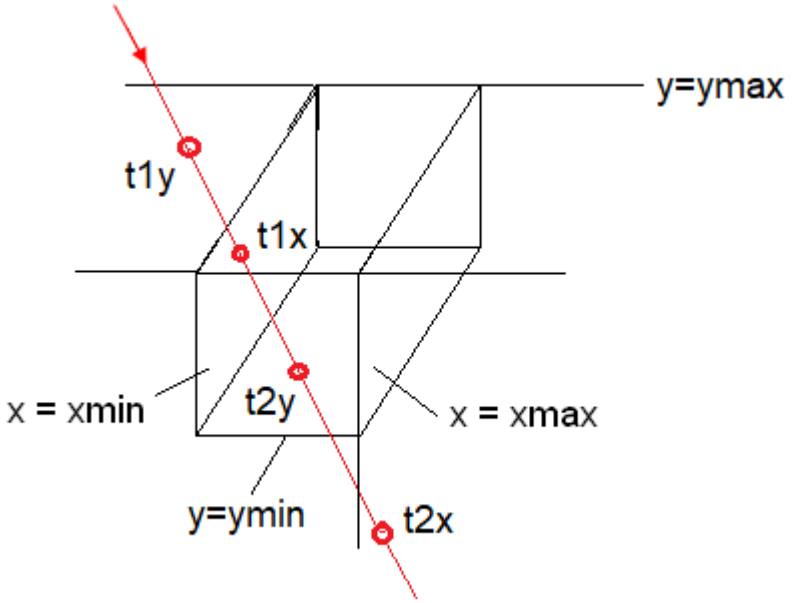
$(x-a)^2 + (y-b)^2 + (z-c)^2 = r^2$

- Se înlocuiesc în ecuația sferei x, y, z, cu cele din ecuația razei
- Rezultă o ecuație de grad 2 în t
- Se calculează discriminantul D, al ecuației:
  - D<0 – raza nu intersectează sfera
  - Dacă sfera este obiect al scenei (interesează punctul de intersecție):
    - D=0 – raza este tangentă la sfera
    - D>0 – se calculează rădăcinile, t1, t2 → punctul de intersecție cel mai apropiat de originea razei este r(tmin(t1,t2))

Utilizarea de volume încadratoare (3)

2) Intersecția cu un paralelipiped cu fețele paralele cu planele principale (1)

Raza intră în volum prin fața x = xmin şi iese prin fața y = ymin.



Planele care delimitează volumul:

x= xmin, x = xmax, y=ymin, y = ymax, z= zmin, z= zmax

Intersecțiile cu x=xmin si x=xmax

xmin = x0 + t*dx → t1x = (xmin – x0)/dx

xmax = x0 + t*dx → t2x = (xmax – x0)/dx

tix = min(t1x, t2x), tex = max(t1x, t2x)

Analog se calculează intersecțiile cu planele y=ymin și y=ymax:

tiy = min(t1y, t2y), tey = max(t1y, t2y)

tix=min(t1x,t2x) = t1x
tex=max(t1x,t2x) = t2x
tiy=min(t1y,t2y) = t1y
tey=max(t1y,t2y) = t2y
max(tix,tiy) = max(t1x,t1y) = t1x
min(tex,tey) = min(t2x,t2y) = t2y
t1x<t2y → se calculează intersecțiile cu z=zmin și z=zmax

daca max(tix,tiy) > min(tex,tey) raza nu intersectează volumul

altfel, se calculează intersecțiile cu z=zmin și z= zmax

dacă max(tix, tiy, tiz) < min(tex, tey, tez)

raza intersectează volumul

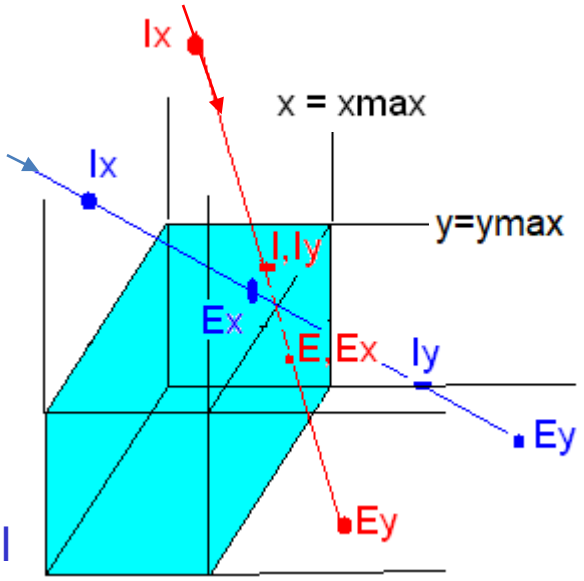
Utilizarea de volume încadratoare (4)

2) Intersecția cu un paralelipiped cu fețele paralele cu planele principale (2)

Punctele de intrare și ieșire din volum sunt:

I = r(max(tix, tiy, tiz))

E = r(min(tex,tey,tez)



Raza albastră intersectează prelungirile fețelor x=xmin, x=xmax

max(tix, tiy) = tiy , min(tex,tey)=tex

max (tix, tiy) > min(tex,tey) → raza nu intersectează volumul

Raza roșie intră în volum prin fața y=ymax și iese prin fața x=xmax

max(tix, tiy) = tiy, min(tex,tey) = tex

max(tix,tiy) < min(tex,tey) → se calculeaza (tiz, tez)

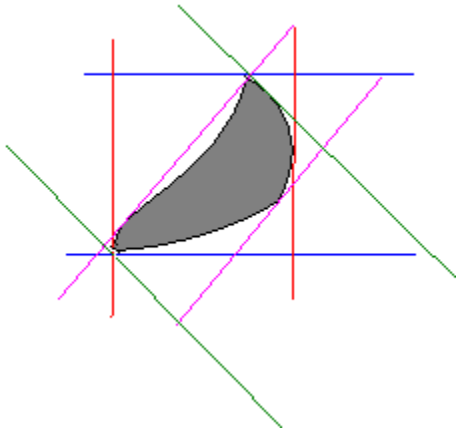
max (tix,tiy,tiz) = tiy, min (tex,tey,tez) = tex

Punctele de intersecție cu volumul: I = r(tiy); E = r(tex)

Utilizarea de volume încadratoare (5)

3) Intersecția cu un volum încadrator poliedru convex

Volumul încadrator (Kay, Kajiya[1986]) este un poliedru convex format din intersecțiile a 4 perechi de plane paralele, înclinate la 0, 45, 90, 135 grade față de orizontală.



Ec. unui plan: A*x + B*y + C*z + D =0

- Fie t11 și t12 - valorile lui t pentru intersecțiile cu o pereche de plane paralele

t1min = min(t11,t12) corespunde punctului de intersecție mai apropiat de originea razei

- Fie t21, t22 - valorile lui t pentru intersecțiile cu următoarea pereche de plane paralele

Daca max (t1min, t2min) > min (t1max, t2max) raza nu intersectează volumul

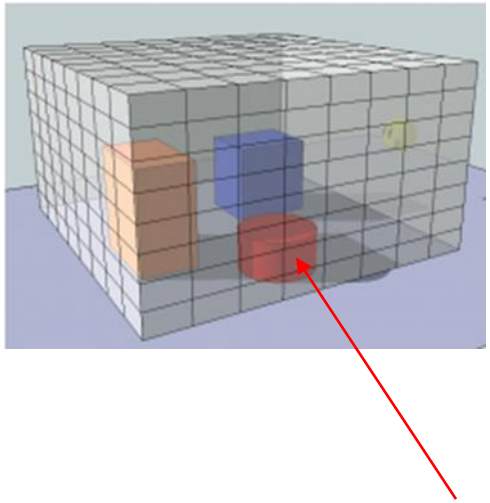
altfel, se continuă cu intersecția următoarei perechi de plane

➢**Punctele de intersecție cu volumul: I = r(max($t_i$min)), E = r(min($t_i$max))**

Divizarea scenei în volume încadratoare (1)

Divizarea regulată a scenei

- Se pornește de la paralelipipedul încadrator al scenei, care se divizează recursiv în opt subvolume egale, până la o anumită rezoluție.
- Subvolumele finale sunt numite **voxeli**.
- La divizare nu se ține cont de structura scenei
- Fiecărui voxel îi este asociată lista de obiecte pe care le conține.
- Un obiect se poate afla în mai mulți voxeli (liste)



Intersecția unei raze cu scena:

- Raza este intersectată numai cu obiectele din voxelii traversați de rază
- Traseul razei prin volumul de voxeli poate fi calculat eficient printr-un algoritm DDA 3D (extensie pentru 3D a algoritmului DDA pentru rasterizarea vectorilor).

Divizarea scenei în volume încadratoare (2)

Voxelul de start: prin care raza primară intră în volum/din care pleacă o rază secundară

Voxel_curent = voxel de start; gata = false; in_volum = true;

cât timp (!gata && in_volum)

{ **daca** voxelul curent nu este gol (lista de obiecte nu este vidă) **atunci**

{ *se intersectează raza cu obiectele conținute în voxelul curent;

dacă există intersecții **atunci**

{ *se retine punctul de intersecție cel mai apropiat de originea razei;

gata = true;

continue;

}

}

// voxelul curent este gol sau nu există intersecții cu obiectele voxelului curent

*determină următorul voxel de pe traseul razei;

dacă raza iese din volum **atunci**

in_volum = false;

altfel voxel_curent = următorul voxel de pe traseul razei;

}

Divizarea scenei în volume încadratoare (3)

Divizarea adaptivă a scenei

- Se pornește de la paralelipipedul încadrator al scenei, care se divide in 8 subvolume egale, în mod recursiv, divizarea unui subvolum terminându-se dacă el nu conține nici un obiect al scenei sau conține un singur obiect.
- Scena se reprezintă printr-un arbore octal, fiecare nod fiind asociat unui subvolum.
- Pentru intersecția razei cu scena se fac teste de intersecție pornind din nodul radacină spre frunze.
- **Daca subvolumul unui nod nu este intersectat de rază,**

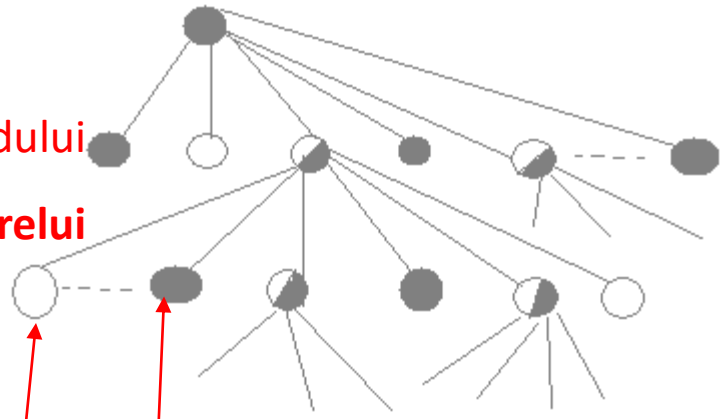
  atunci (niciunul dintre subvolumele din subarborele nodului nu va fi intersectat) **se revine pe nivelul anterior al arborelui**

  altfel daca nodul curent este frunză atunci

  calculează intersecția razei cu obiectul din nod

  altfel se coboară în arbore

Arborele octal al scenei



- subvolum gol
- subvolumul contine un singur obiect

Scena – ierarhie de volume încadratoare

Bounding Volume Hierarchy – BVH

- Scena 3D este reprezentată printr-un arbore (arborele scenei) în care fiecare nod are atașat un volum încadrator pentru un grup de obiecte: vezi cursul 7 (BVH).
- Rădăcina are atașat volumul încadrator al întregii scene
- Fiecare frunză are atașat volumul încadrator al unui obiect (sau mai multor obiecte), celelalte noduri au atașate volume ce încadrează grupuri de obiecte.
- Pentru intersecția razei cu scena este parcurs arborele de la rădăcină spre frunze
- **Dacă raza nu intersectează volumul încadrator al nodului curent atunci**

(- nici unul dintre obiectele din volumul nodului nu este intersectat

- nici unul dintre obiectele din subarborele nodului nu este intersectat)

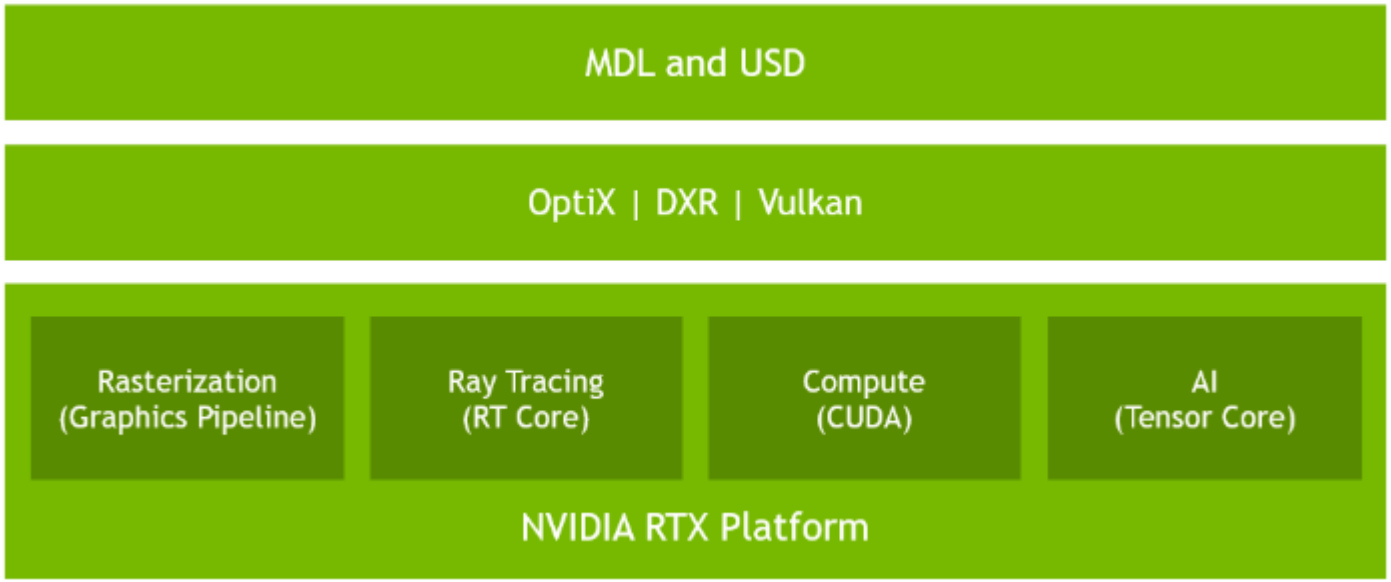
se revine pe nivelul anterior al arborelui

altfel dacă nodul este frunză atunci

calculează intersecția razei cu obiectul/obiectele din nod

altfel se coboară în arbore

Platforma NVIDIA RTX permite implementarea algoritmului Ray-tracing folosind API-uri și kituri software de dezvoltare dedicate: OptiX, Microsoft DXR și Vulkan



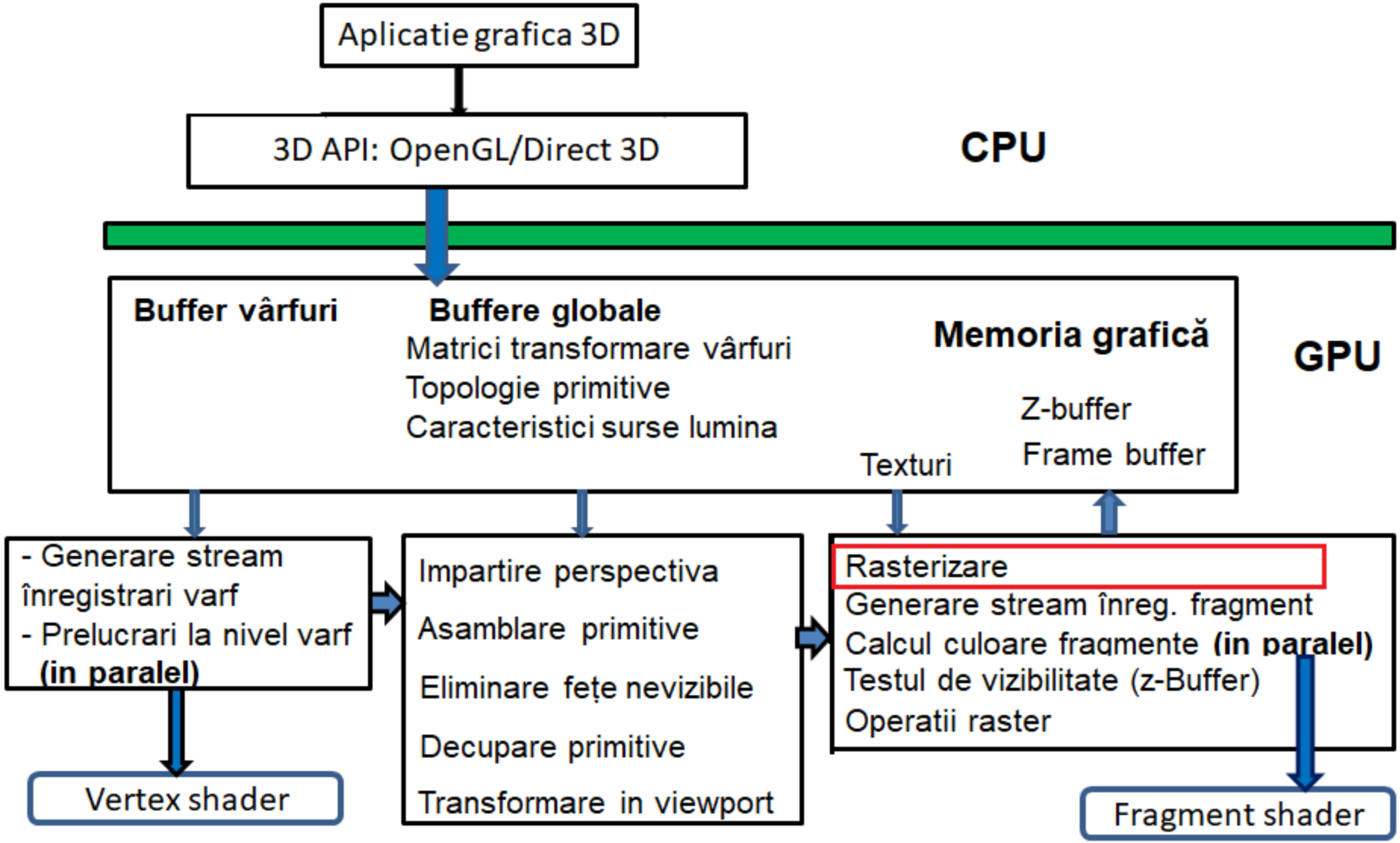
https://developer.nvidia.com/rtx

Rasterizarea primitivelor grafice

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

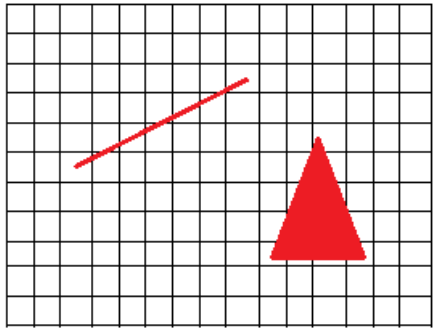
Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

Etapa de rasterizare în banda grafică



Rasterizarea (1)

- Singurele primitive grafice acceptate de plăcile grafice care suportă versiunile noi ale bibliotecii OpenGL (3.x→) sunt:
  - **punctul**, definit prin coordonatele sale (x,y,z);
  - **linia**, definită prin coordonatele extremităților sale, (x1,y1,z1) si (x2,y2,z2); specificate în aceasta ordine, **linia este un vector cu originea în (x1,y1,z1) și vârful în (x2,y2,z2);**
  - **suprafaţa triunghiulară**, definită prin coordonatele 3D ale vârfurilor sale.
- Suprafața de afișare este un spațiu discret, alcătuit din celule de afișare (pixeli) adresate prin coordonate intregi, (0,0) <= (x,y) <= (xmax,ymax)
- Pentru afișare, este necesară descompunerea primitivelor în fragmente care se afișează în celulelele suprafeței de afișare.



- **Rasterizarea:**
  - operația prin care o primitivă grafică este descompusă în fragmente care se afișează în pixelii suprafeței de afișare;
  - are ca rezultat o aproximare în spațiul discret a primitivelor definite analitic.

Rasterizarea (2)

Algoritmii de rasterizare integrează, la nivel de fragment:

1. Calculul coordonatelor (x,y) ale pixelului în care se afișează fragmentul
2. Calculul coordonatei z a fragmentului
3. Calculul culorii fragmentului (în fragment shader)
4. Testul de vizibilitate (adâncime) și actualizarea culorii pixelului (x,y)

```
daca z< Z-buffer[y][x] atunci
{ Z-buffer[y][x] = z
*actualizeaza culoarea pixelului (x,y) în buffer-ul imagine folosind culoarea fragmentului
}
```

Deoarece în algoritmii de rasterizare este important modul de calcul al coordonatelor (x,y), vom include pașii 2,3 si 4 în funcția

```
putpixel (int x, int y);
```

- Pasul 4 este același pentru orice fragment, indiferent de tipul primitivei din care provine.

Rasterizarea vectorilor (1)

La rasterizarea unui vector:

1. Pixelii (x,y) în care se afișează fragmentele rezultate din rasterizare sunt determinați astfel încât să fie cei mai apropiați de adresele punctelor de pe vectorul analitic.

2. Coordonata z a fiecărui fragment se obține prin interpolarea liniară a coordonatelor z ale vârfurilor vectorului (prin calcul incremental – vezi “algoritmul z-buffer”).

3. Culoarea fragmentului se obține prin interpolarea liniară a culorilor varfurilor vectorului (prin calcul incremental); nu este uzual să se calculeze culoarea unui fragment de vector în “fragment shader” (fragment shader întoarce ca rezultat culoarea primită de la rasterizator)

- Algoritmii de rasterizare a vectorilor se deosebesc prin pasul 1.

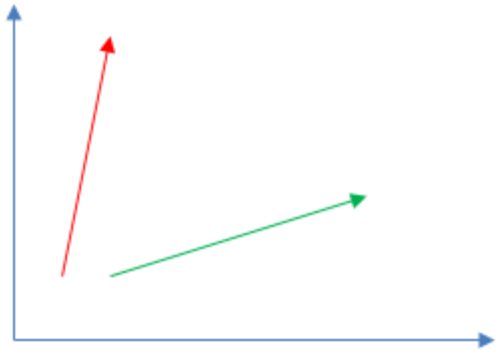
Rasterizarea vectorilor (2)

Ecuația unui vector: y = m*x + b

m = (y2-y1)/(x2-x1) şi b = y1 - m*x1

Considerăm următorul algoritm de rasterizare a unui vector:

```
putpixel(x1, y1);
for (x = x1 + 1; x < x2; x++)
{
   y = m * x + b ;
   putpixel(x, (int)(y + 0.5));
}
putpixel(x2, y2);
```



Dezavantaje algoritm:

- Nu se ține cont de panta dreptei: vectorii cu pantă mare sunt aproximați prin câțiva pixeli!
- Calculul fiecărei adrese de pixel de pe traseul vectorului conține operații cu numere reale

Algoritmul Digital Differential Analyser (DDA)

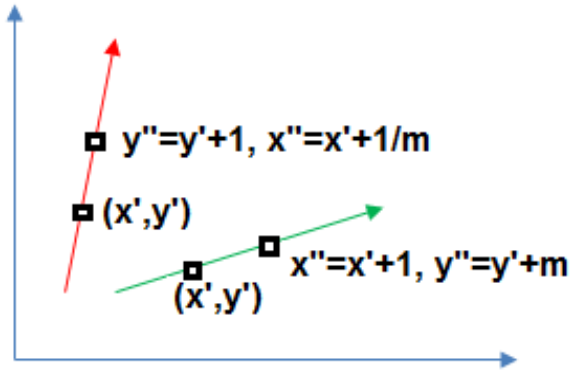
- Ține cont de panta vectorului
- Coordonatele pixelilor de pe traseul vectorului se obțin printr-un calcul incremental (eficient)

Fie (x', y') si (x'',y'') - 2 puncte succesive de pe vector

- (y'' – y') / (x'' – x') = (y2 – y1) / (x2 – x1) = m

|m <1| : se incrementează x : x'' = x' +1 → y'' = y' + m

| m >1| : se incrementează y : y'' = y' + 1 → x'' = x' +1/m




```
void DDA(int x1, int y1, int x2, int y2)
{
   double m, absm, rx, ry; int x, y;
   //vectorul se generează de la (x1,y1) la (x2,y2)
   if (x1 == x2) //vector vertical
   {
      if (y1 > y2) { y = y1; y1 = y2; y2 = y; }
      for (y = y1; y <= y2; y++)  putpixel(x1, y);
      return;
   }
```

Algoritmul DDA (2)

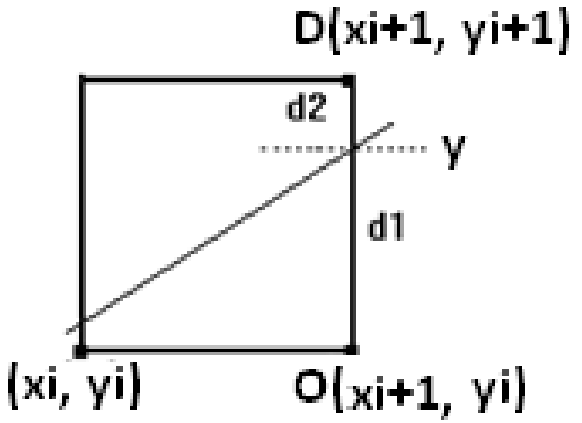
```
    if (y1 == y2) //vector orizontal
    {
       if (x1 > x2)
          { x = x1; x1 = x2; x2 = x; }
       for (x = x1; x <= x2; x++)  putpixel(x, y1);
       return;
    }
    m = (double)(y2 - y1) / (x2 - x1); absm = abs(m);
    if (absm <= 1 && x1 > x2 || absm > 1 && y1 > y2)
    {
       x = x1; x1 = x2; x2 = x; y = y1; y1 = y2; y2 = y;
    }
    putpixel(x1, y1);
    if (absm <= 1)
       for (x = x1 + 1, ry = y1; x < x2; x++)
       {
          ry += m;  putpixel(x, (int)(ry + 0.5));
       }  // y = y + m
    else
    {
       m = 1 / m;
       for (y = y1 + 1, rx = x1; y < y2; y++)
       {
          rx += m;  putpixel((int)(rx + 0.5), y);
       }  // x = x +1/m
    }
    putpixel(x2, y2);
}
```

Dezavantaj: calcule cu numere reale

Algoritmul Bresenham (1)

- Conține numai operații cu numere întregi. **Este definit pentru vectori din primul octant**
- Un vector face parte din ”primul octant” daca panta sa, m, satisface condiția 0<m<1.
- Coordonatele pixelilor de pe traseul vectorului se obțin prin calcul incremental.
- Pentru fiecare valoare a lui x, de la xmin_vector la xmax_vector, se alege acel punct al spaţiului discret care este mai apropiat de punctul de pe vectorul teoretic.

- Fie m=(y2-y1)/(x2-x1) panta vectorului
- (x_i, y_i) ultimul punct al spaţiului discret ales la execuția algoritmului
- d1 distanţa de la punctul de pe vectorul teoretic, (x_i+1, y), la punctul $O(x_i+1, y_i)$
- d2 distanţa de la punctul de pe vectorul teoretic, (x_i+1, y), la punctul $D(x_i+1, y_i+1)$



- O si D sunt adrese de pixeli (puncte ale spațiului discret)

➢ Următorul punct al spațiului discret ales pentru aproximarea vectorului va fi:

- O dacă d1<d2, D în caz contrar.
- Dacă d1=d2 se poate alege oricare dintre cele două puncte.

Algoritmul Bresenham (2)

- Exprimăm diferenţa d1-d2:

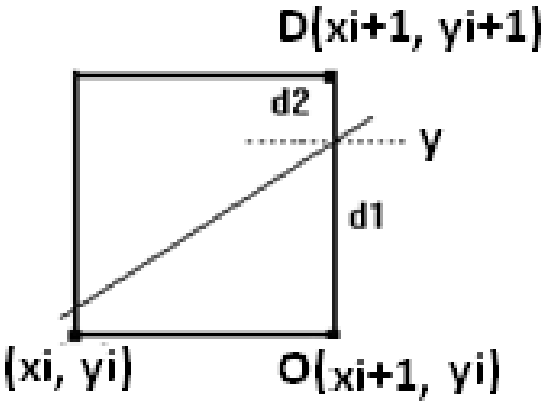
  $y=m*(x_i+1)+b$ este ordonata punctului de pe vectorul teoretic

  $d1= y - y_i= m*(x_i+1)+b-y_i$

  $d2= y_i + 1-y= y_i+1-m*(x_i+1)-b$

  $d1-d2= 2*m*(x_i+1)-2*y_i+2*b -1$

  $dy = y2-y1$
  $dx = x2-x1$



- Se înlocuieşte m cu dy/dx apoi se înmulţeşte în ambele părţi cu dx. Rezultă:

  $t_i= (d1-d2)*dx= 2*dy*(x_i+1)-2*dx*y_i+2*b*dx-dx = 2*dy*x_i - 2*dx*y_i + 2*b*dx - dx + 2*dy$

- t_i reprezintă eroarea de aproximare în pasul i: pe baza ei se alege următorul punct al spațiului discret

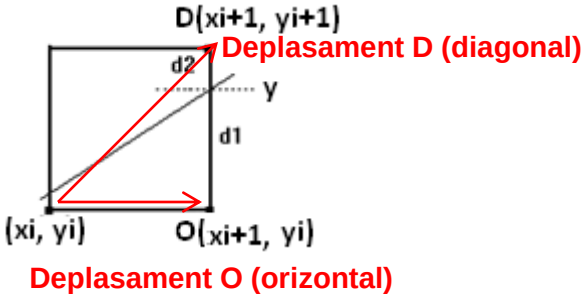
  Notăm cu (x_{i+1},y_{i+1}) punctul care se va alege în pasul i+1.

- Expresia erorii de aproximare în pasul i+1 este:

  $t_{i+1} = 2*dy*x_{i+1} - 2*dx*y_{i+1} + 2*b*dx - dx +2*dy$

Algoritmul Bresenham (3)

$t_i=(d1-d2)*dx = 2*dy*x_i - 2*dx*y_i + 2*b*dx-dx + 2*dy$



(1) Dacă $t_i <= 0$, se alege punctul O, deci $x_{i+1} = x_i+1$ şi $y_{i+1} = y_i$

Rezultă:

$t_{i+1} = 2*dy*(x_i+1) - 2*dx*y_i + 2*b*dx - dx +2*dy$

sau **$t_{i+1} = t_i +2*dy$**

(2) Dacă $t_i > 0$, se alege punctul D, deci $x_{i+1} = x_i + 1$ şi $y_{i+1} = y_i +1$

Rezultă:

$t_{i+1} = 2*dy*(x_i+1) - 2*dx*(y_i+1) + 2*b*dx - dx +2*dy$

sau **$t_{i+1} = t_i + 2*dy - 2*dx$**

- Valoarea variabilei de test se obține prin calcul incremental: adunarea unei constante întregi

Eroarea de aproximare pentru primul pas: $x_i = x1$ şi $y_i = y1$:

$t_1 = 2*dy*x1 - 2*dx*y1 + 2*dy – dx + 2*dx(y1 - (dy/dx)*x1)$

sau **$t_1 = 2*dy-dx$**

Algoritmul Bresenham (4)

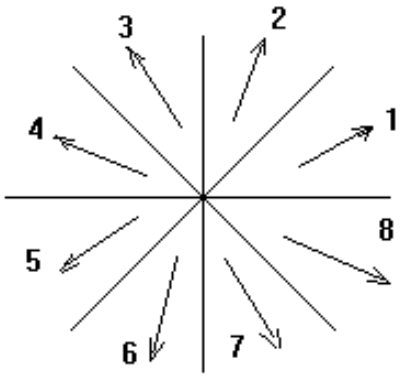
Implementare în C:

```
void Bres_vect(int x1, int y1, int x2, int y2)
{
    //pentru vectori cu panta cuprinsă între 0 și 1
    int dx, c1, c2, x, y, t;
    dx = x2 - x1;
    c1 = (y2 - y1) << 1; // 2*dy
    c2 = c1 - (dx << 1); // 2*dy – 2*dx
    t = c1 - dx;         // 2*dy - dx
    putpixel(x1, y1);
    for (x = x1 + 1, y = y1; x < x2; x++)
    {
        if (t < 0) t += c1;// deplasament O
        else
        { t += c2; y++; } // deplasament D
        putpixel(x, y);
    }
    putpixel(x2, y2);
}
```

Algoritmul Bresenham - generalizare (1)

Generalizarea algoritmului Bresenham pentru vectori de orice pantă

- Vectorii definiţi în planul XOY pot fi clasificaţi, pe baza pantei, în opt clase geometrice, numite **“octanţi“**
- Un vector care aparține unui octant O are 7 vectori simetrici în ceilalți 7 octanți



dx = x2 – x1 și dy = y2 – y1

octantul 1: dx>0 şi dy>0 și dx>=dy;

octantul 2: dx>0 şi dy>0 și dx<dy;

octantul 3: dx<0 şi dy>0 și abs(dx)<dy;

octantul 4: dx<0 şi dy>0 și abs(dx)>=dy;

octantul 5: dx<0 şi dy<0 și abs(dx)>=abs(dy);

octantul 6: dx<0 şi dy<0 și abs(dx)<abs(dy);

octantul 7: dx>0 şi dy<0 și dx<abs(dy);

octantul 8: dx>0 şi dy<0 și dx>=abs(dy);

Algoritmul Bresenham – generalizare (2)

Notăm cu:

+h, deplasamentul orizontal spre dreapta (în sensul crescător al axei x),

-h, deplasamentul orizontal spre stânga (în sensul descrescător al axei x),

+v, deplasamentul vertical în sus (în sensul crescător al axei y),

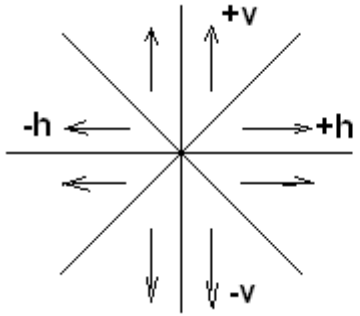
-v, deplasamentul vertical în jos (în sensul descrescător al axei y),

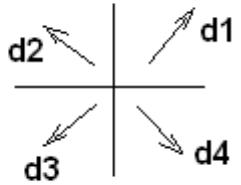
d1, deplasamentul diagonal dreapta-sus,

d2, deplasamentul diagonal stânga-sus,

d3, deplasamentul diagonal stânga-jos,

d4, deplasamentul diagonal dreapta-jos.





Octant	1	2	3	4	5	6	7	8
Deplas O	+h	+v	+v	-h	-h	-v	-v	+h
Deplas D	d1	d1	d2	d2	d3	d3	d4	d4

Corespondenţa între alegerea curentă într-un pas al algoritmului Bresenham (punctul O sau punctul D) şi deplasamentul echivalent în fiecare octant:

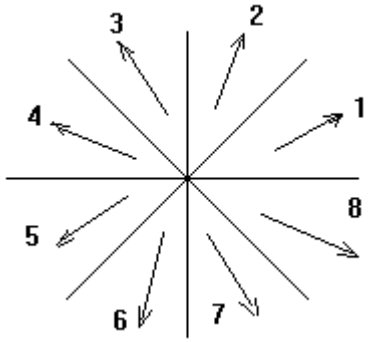
Algoritmul Bresenham – generalizare (3)

Algoritmul Bresenham generalizat - implementare în C

```
void Bres_general(int x1, int y1, int x2, int y2)
{
    int x, y, i, oct, dx, dy, absdx, absdy, c1, c2, t;
    if (x1 == x2) //vertical
    {
        if (y1 > y2) { y = y1; y1 = y2; y2 = y; }
        for (y = y1; y <= y2; y++)
           putpixel(x1, y);
        return;
    }
    if (y1 == y2) //orizontal
    {
        if (x1 > x2) { x = x1; x1 = x2; x2 = x; }
        for (x = x1; x <= x2; x++)
           putpixel(x, y1);
        return;
    }
```

Algoritmul Bresenham – generalizare (4)

```
dx = x2 - x1; dy = y2 - y1; absdx = abs(dx); absdy = abs(dy);
if (dx > 0)    // oct = 1,2,7,8
{
   if (dy > 0) // oct = 1,2
     if (dx >= dy) oct = 1; else oct = 2;
   else        // oct = 7,8
     if (dx >= absdy) oct = 8; else oct = 7;
}
else           // oct = 3,4,5,6
{
   if (dy > 0) // oct = 3,4
     if (absdx >= dy) oct = 4; else oct = 3;
   else        // oct = 5,6
     if (absdx >= absdy) oct = 5; else oct = 6;
}
// Dacă abs(dy) > abs(dx), se inversează rolul variabilelor dx și dy în calculul
// constantelor c1 și c2
if (absdy > absdx) // Adresele de pe traseul vectorului se obtin prin incr. lui y
{ x = absdx; absdx = absdy; absdy = x; }
c1 = absdy << 1;  c2 = c1 - (absdx << 1); t = c1 - absdx;
```



Algoritmul Bresenham – generalizare (5)

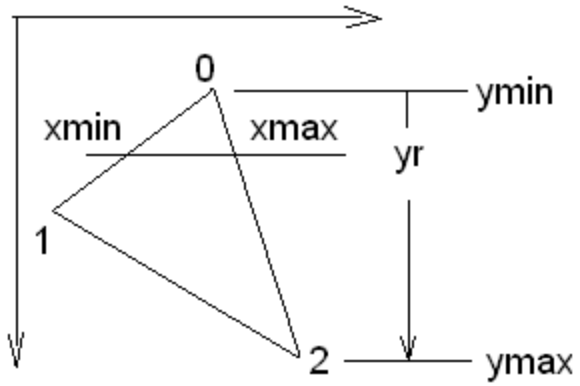
```
    putpixel(x1, y1);
    for (i = 1, x = x1, y = y1; i < absdx; i++)
    {
        if (t < 0) // deplasament O
        {
            t += c1;
            switch (oct)
            {
                case 1:  case 8: x++; break; // +h
                case 4:  case 5: x--; break; // -h
                case 2:  case 3: y++; break; // +v
                case 6:  case 7: y--; break; // -v
            }
        }
        else
        {
            t += c2 // deplasament D
            switch (oct)
            {
                case 1: case 2: x++; y++; break; // d1
                case 3: case 4: x--; y++; break; // d2
                case 5: case 6: x--; y--; break; // d3
                case 7: case 8: x++; y--; break; // d4
            }
        }
        putpixel(x, y);
    } putpixel(x2, y2);
}
```

Rasterizarea suprafețelor triunghiulare

La rasterizarea unei suprafețe triunghiulare

1. Pixelii (x,y) în care se afișează fragmentele rezultate din rasterizare sunt determinați intersectând suprafața triunghiului cu liniile raster de la ymin_triunghi la ymax_triunghi;
2. Fiecare segment dintr-o linie raster, care este interior suprafeței triunghiulare, este descompus in fragmente.
3. Coordonatele z ale fragmentelor se obțin prin calcul incremental (vezi curs “algoritmul z-buffer”).
4. Pentru calculul culorii fiecărui fragment este apelat programul “fragment shader”.
5. Sunt afișate fragmentele care trec testul de vizibilitate.

Rasterizarea suprafețelor triunghiulare - algoritmul -



Bool SupTri (int x0, int y0, int x1, int y1, int x2, int y2)

{ 1. Se sortează vârfurile triunghiului crescător după coordonata y, a.i. y0 = ymin;

2. Dacă triunghiul este degenerat (vârfurile sunt coliniare), return false;

3. Se orientează conturul în sens trigonometric (0-1-2: sens trigonometric);

4. Pentru toate liniile raster de la ymin-triunghi la ymax-triunghi

se calculează extremitățile segmentelor interioare suprafeței triunghiului, și se memorează în 2 tablouri, XMIN, XMAX;

5. Se afișează pixelii segmentelor interioare (de la ymin_triunghi la ymax_triunghi);

return true;

}

Pașii 1 si 2

1. Se sortează vârfurile triunghiului crescător dupa coordonata y, a.i. y0 = ymin

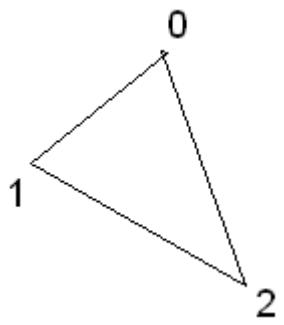
```
if( y1 < y0) { t= y1; y1 = y0; y0 =t; t= x1; x1 = x0; x0 =t;}

if( y2 < y0) { t= y2; y2 = y0; y0 =t; t= x2; x2 = x0; x0 =t;}
```

2. Daca triunghiul este degenerat (vârfurile sunt coliniare), return false

$$\det = \begin{vmatrix} x1 - x0 & y1 - y0 \\ x2 - x0 & y2 - y0 \end{vmatrix}$$

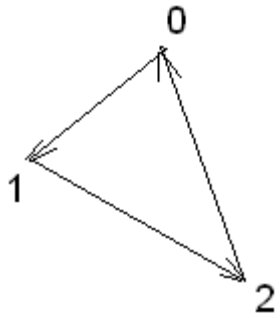
$$= (x1 - x0)(y2 - y0) - (x2 - x0)(y1 - y0)$$




```
if(det==0) return false
```

Pasul 3: orientarea conturului

3. Se orientează conturul în sens trigonometric (0-1-2: sens trigonometric)



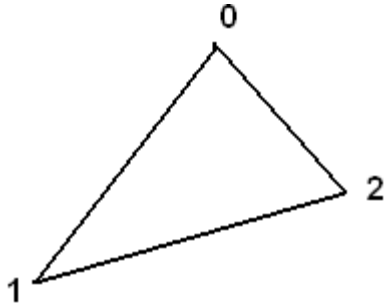
dx0 = x1 – x0; dx1 = x2 – x0; dy0 = y1 – y0; dy1 = y2 – y0;

x1 < x0 , x2 > x0

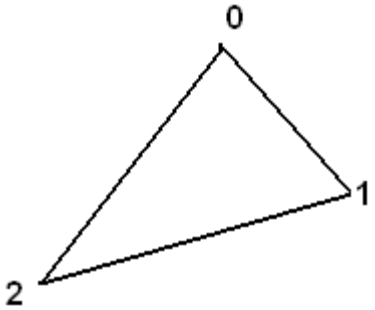
dx0 < 0, dx1 > 0, dy0 > 0, dy1 > 0

det = dx0*dy1- dx1*dy0 **< 0**

sens trigonometric,
Cazul: ymax = y2



sens trigonometric,
Cazul: ymax = y1



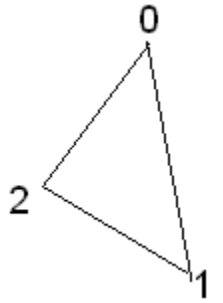
x1> x0, x2 < x0

dx0>0, dx1<0, dy0>0, dy1>0

det = dx0*dy1- dx1*dy0 **> 0**

Se inverseaza varful 1 cu 2

sens anti-trigonometric,
Cazul: ymax = y2



sens anti-trigonometric
Cazul: ymax = y1

Pasul 4: calculul extremităților segmentelor interioare (1)

4. Se calculează extremitățile segmentelor interioare suprafeței triunghiului

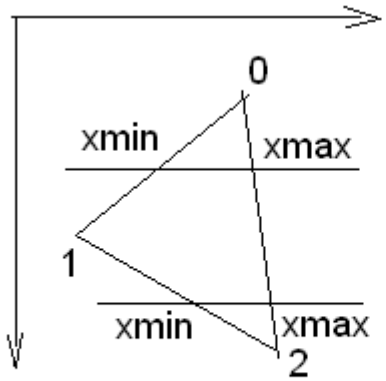
- Coordonatele xmin, xmax ale extremităților segmentelor interioare suprafeței triunghiului se memorează în 2 tablouri:

  int XMIN[H], XMAX[H]; // H este numărul de linii imagine (linii raster)

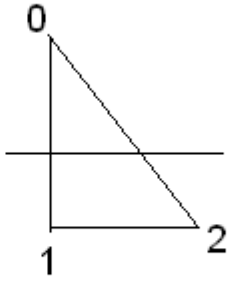
- Calculul extremităților este efectuat în funcțiile **CalculXmin()** și **CalculXmax()**;

Cazuri:

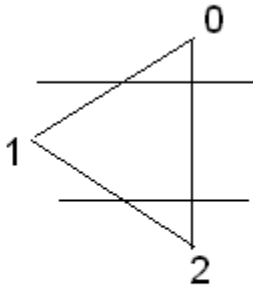
cazul I.



y0 < y1 < y2



x0 = x1

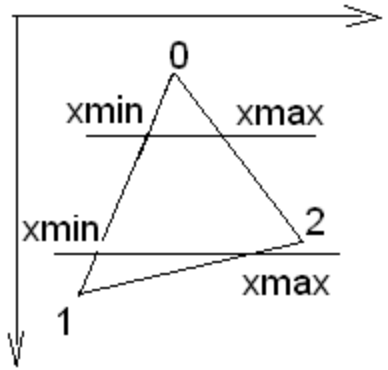


x0 = x2

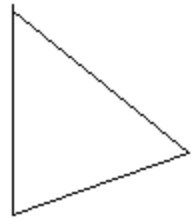
```
CalculXmin(x0, y0, x1, y1);
CalculXmin(x1, y1, x2, y2);
CalculXmax(x0, y0, x2, y2);
```

Pasul 4: calculul extremităților segmentelor interioare (2)

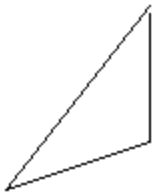
cazul II.



y0 < y2 < y1



x0 = x1



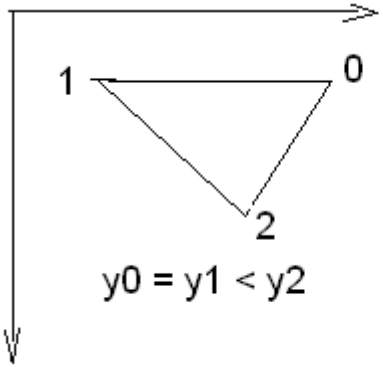
x0 = x2

CalculXmin(x0, y0, x1, y1);

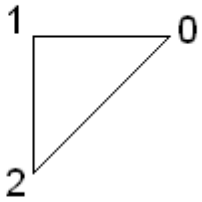
CalculXmax(x0, y0, x2, y2);

CalculXmax(x2, y2, x1, y1);

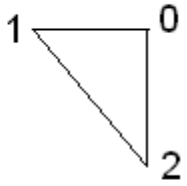
cazul III.



y0 = y1 < y2



x1 = x2

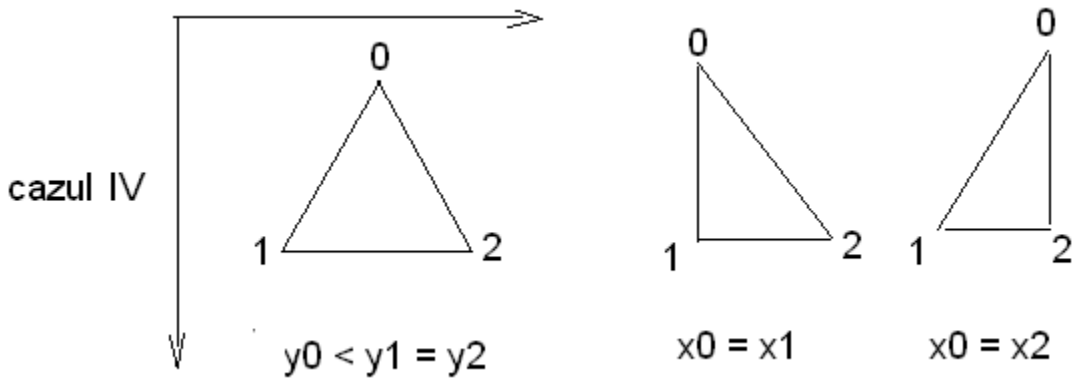


x0 = x2

CalculXmin(x1, y1, x2, y2);

CalculXmax(x0, y0, x2, y2);

Pasul 4: calculul extremităților segmentelor interioare (3)



CalculXmin(x0, y0, x1, y1);

CalculXmax(x0, y0, x2, y2);

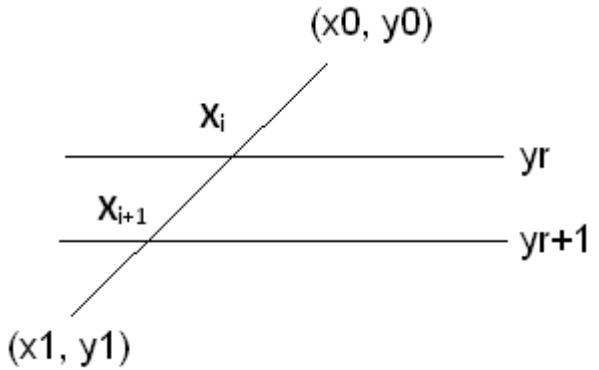
Calculul coordonatelor xmin, xmax

m = (y1 - y0)/ (x1 – x0) - panta laturii

$(yr+1 - yr)/ (x_{i+1} - x_i) = m \rightarrow x_{i+1} = x_i + (1/m)$

dx = 1.0/m - constanta a laturii

$x_{i+1} = x_i + dx \rightarrow$ calcul incremental



Pasul 4: calculul extremităților segmentelor interioare (4)

```
int  XMIN[H], XMAX[H];
void CalculXmin(int x0, int y0, int x1, int y1)
{  // y0 < y1
   int y;
   if (x0 == x1) // latura verticala
   {
      for (y = y0; y <= y1; y++)
      XMIN[y] = x0;
      return;
   }
   float dx = (float)(x1 – x0) / (y1 – y0);
   XMIN[y0] = x0; XMIN[y1] = x1;
   for (y = y0 + 1; y < y1; y++)
      XMIN[y] = XMIN[y - 1] + dx;
}
void CalculXmax(int x0, int y0, int x1, int y1)
{
  - similară cu funcția CalculXmin
  - memorează punctele   succesive  de pe latură în tabloul XMAX
}
```

Pasul 5: afișarea pixelilor suprafeței triunghiulare

5. Se afișează pixelii interiori triunghiului, între xmin și xmax pe fiecare linie raster

```
ymax = y1; if (y2 > y1) ymax = y2;
// pentru fiecare linie raster care intersecteaza suprafața triunghiului
for (int y = y0; y <= ymax; y++)
   for (int x = XMIN[y]; x <= XMAX[y]; x++)
      putpixel(x, y); // afișare fragment interior suprafeței triunghiului
```

Generarea cercurilor în spațiul discret

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

Aproximarea cercurilor în spațiul discret (1)

- Cercurile și elipsele nu sunt primitive grafice în OpenGL, deci nu pot fi transmise în banda grafică.
- Cercurile și elipsele sunt primitive 2D definite în planul XOY.
- Pentru afișare ele sunt descompuse în primitive de tip vector sau primitive de tip punct:
  - le putem asocia coordonata z și transmite în banda grafică sub forma unei secvențe de vectori sau puncte 3D
  - mai eficient: aproximarea cercurilor prin vectori sau puncte se poate face în "Geometry shader"
- Fie (xc,yc,0) coordonatele centrului unui cerc/elipsa. Pentru afișarea sa vom folosi funcțiile:

  point (int x, int y, RGB culoare);

  - funcția generează o primitivă "point" având coordonatele (x,y,0) și culoarea specificată.

  line (int x1, int y1, int x2, int y2, RGB culoare)

  - funcția generează o primitivă "line" cu vârfurile (x1,y1, 0) - (x2,y2,0) și culoarea specificată asociată ambelor vârfuri.

Aproximarea cercurilor în spațiul discret (2)

1) Folosind ecuația cercului în coordonate carteziene

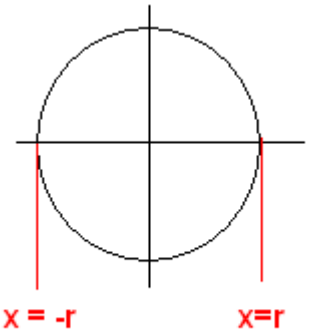
$(x – xc)^2 + (y – yc)^2 = r^2$

Generarea cercului cu centrul în (xc,yc) se reduce la generarea cercului centrat în origine, plus o translație cu (xc, yc) a coordonatelor.

$x^2 + y^2 = r^2$

Se incrementeaza x de la –r la r și se calculează y din ecuația cercului:

```
point (xc - r, yc, culoare);
point (xc + r, yc, culoare);
for (x = -r +1; x< r; x++)
{   y = sqrt(r^2 – x^2);
    point (xc + x, yc + y, culoare);
    point (xc + x, yc - y, culoare);
}
```

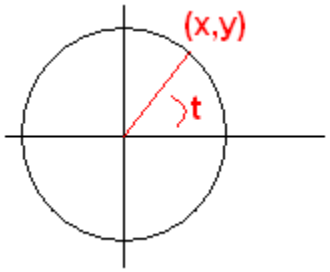


Dezavantaje:

- Spațierea inegală a punctelor de pe cerc afişate
- Calcule complexe cu numere reale

Aproximarea cercurilor în spațiul discret (3)

2) Folosind ecuația cercului în coordonate polare



x = xc + r* cos(t)

y = yc + r* sint(t) 0 <= t <= 2π

Se generează cercul centrat în origine și se aplică punctelor rezultate o translație cu (xc, yc):

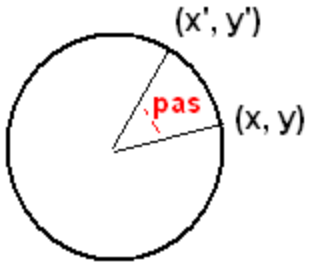
x = r* cos(t)

y = r* sint(t) 0 <= t <= 2π

Cercul poate fi aproximat printr-o polilinie:

- daca se calculează N puncte de pe cerc, egal distanțate, t crește cu pasul 2π/N

Calculul punctelor succesive de pe cerc, pe baza unei relații de recurență:



- (x, y): punctul curent

- (x', y'): punctul urmator

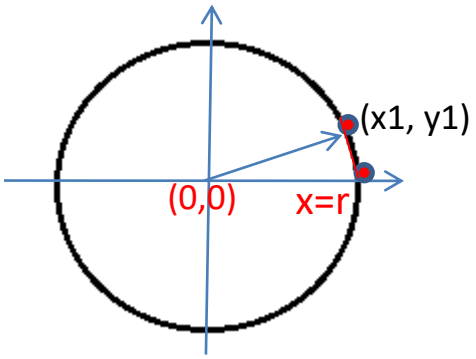
x' = r* cos(t +pas) = r(cos(t) * cos(pas) – sin(t)* sin(pas)) = x*cos(pas) – y*sin(pas)

y' = r* sin(t + pas) = r(cos(t) * sin(pas) + sin(t)* cos(pas)) = x*sin(pas) + y*cos(pas)

Aproximarea cercurilor în spațiul discret (4)

Avantajul calculului prin recurență: nu este necesar să se calculeze sin și cos în fiecare iterație!

```
// Aproximarea unui cerc printr-o linie poligonala
void Cerc(int xc, int yc, int r, int N, RGB culoare)
{
   double pas = 2 * M_PI / N; //pasul unghiular
   double u, x, y, x1, y1, s, c;
   // sin și cos se calculează o singură dată
   c = cos(pas); s = sin(pas);
   for (int i = 1, x = r, y = 0; i < N; i++)
   {
      x1 = x * c - y * s;  // x'
      y1 = x * s + y * c;  // y'
      line(xc + (int)(x + 0.5), yc + (int)(y + 0.5), xc + (int)(x1 + 0.5), yc
        + (int)(y1 + 0.5), culoare);
       x = x1 ; y = y1 ;
   }
   line(xc + (int)(x + 0.5), yc + (int)(y + 0.5), xc + r, yc, culoare);
}
```

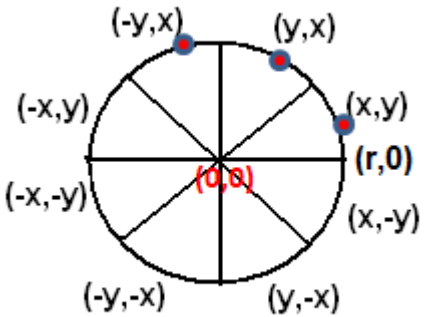


Aproximarea cercurilor în spațiul discret (5)

Simetria punctelor de pe cerc

- **Fiecare punct de pe cerc are 7 puncte simetrice pe cerc:**
  - Este suficient să se calculeze adresele punctelor dintr-un singur octant al cercului, celelalte puncte de pe cerc find calculate prin simetrie

```
// Afișează punctele simetrice de pe cerc
void punct_simetric(int xc, int yc, int x, int y, RGB
{
   // (x,y) – punct de pe cercul centrat în origine
   point(xc + x, yc + y, culoare);
   point(xc + x, yc - y, culoare);
   point(xc - x, yc - y, culoare);
   point(xc - x, yc + y, culoare);
   point(xc + y, yc + x, culoare);
   point(xc + y, yc - x, culoare);
   point(xc - y, yc - x, culoare);
   point(xc - y, yc + x, culoare);
}
```

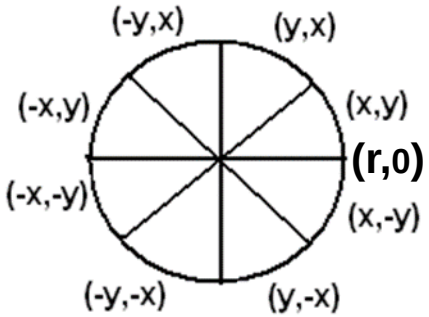


Obs:
Simetricele punctului (r,0): (r, 0), (-r, 0), (-r,0), (0,r), (0,-r), (0,-r), (0,r)
→ pixelii (xc+r,yc), (xc-r,yc), (xc,yc+r), (xc,yc-r) vor fi suprascriși
→ nu va fi apelată funcția pentru punctul (r,0)

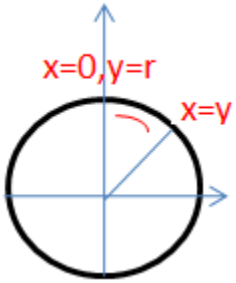
Aproximarea cercurilor în spațiul discret (5)

- Pentru afișarea punctelor succesive de pe circumferință, un pasul unghiular adecvat trebuie să fie aprox. (1/raza).

```
// Calculeaza numai punctele din primul octant si afiseaza toate punctele simetrice
void Cerc_sim(int xc, int yc, int r, RGB culoare)
{
  double pas, u, x, y, xx, s, c;
  // pas unghiular pentru obținerea de puncte succesive de pe cerc
  pas = 1.0 / r;
  c = cos(pas); s = sin(pas);
  point(xc + r, yc, culoare);
  point(xc - r, yc, culoare);
  point(xc, yc + r, culoare);
  point(xc, yc - r, culoare);
  for (u = pas, x = r, y = 0; u < M_PI_4; u += pas)
  {
    xx = x;
    x = xx * c - y * s;
    y = xx * s + y * c;
    punct_simetric(xc, yc, (int)(x + 0.5), (int)(y + 0.5), culoare);
  }
}
```

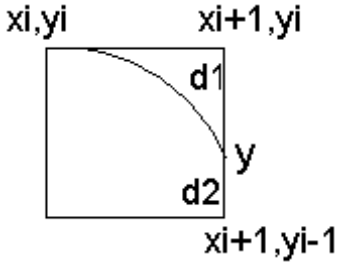


Algoritmul Bresenham pentru aproximarea cercurilor în spațiul discret (1)



- Considerăm cercul cu centrul în originea sistemului de coordonate și raza r.
- În cadrul algoritmului **se calculează numai punctele din octantul al 2-lea,** celelalte obținându-se prin simetrie.
- Se calculează punctele de pe cerc începând cu (x=0,y=r) până la (x=y), prin incrementarea lui x.
- Fie (x_i,y_i) ultimul punct al spațiului discret, ales pentru aproximarea cercului.

  Următorul punct va fi unul dintre (x_i+1,y_i) și (x_i+1,y_i-1):



Fie y ordonata punctului de pe cercul teoretic, cu abscisa x_i+1.

$$y^2 = r^2-(x_i+1)^2$$

Fie d1 și d2 distanțele de la cele 2 puncte ale spațiului discret la punctul de pe cercul teoretic.

Interesează semnul diferenței (d1-d2):

- daca $(y_i - y) > (y- (y_i-1))$ atunci

$$(y_i^2 - y^2) > (y^2 - (y_i-1)^2)$$

$$d1' = y_i^2 - y^2 = y_i^2 - r^2 + (x_i+1)^2$$

$$d2' = y^2 - (y_i-1)^2 = r^2 - (x_i+1)^2 - (y_i-1)^2$$

Algoritmul Bresenham (2)

- Notăm cu t_i eroarea de aproximare în pasul curent, pe baza căreia se va alege următorul punct:

  $t_i = d1' - d2' = y_i^2 + 2*(x_i+1)^2 + (y_i-1)^2 - 2*r^2$

- Se obține o relație de recurență pentru calculul erorii de aproximare în pasul următor:

  $t_{i+1} = y_{i+1}^2 + 2*(x_{i+1}+1)^2 + (y_{i+1}-1)^2 - 2*r^2$

(1) Dacă **$t_i<0$** ($d1' < d2'$) atunci $x_{i+1}=x_i+1$ si $y_{i+1}=y_i$. Deci,

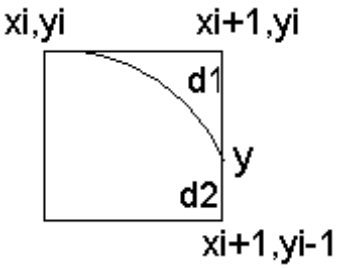
$t_{i+1} = y_i^2 + 2*((x_i+1)+1)^2 + (y_i-1)^2 - 2*r^2$

sau **$t_{i+1} = t_i + 4*x_i + 6$**

(2) Dacă **$t_i >= 0$** atunci $x_{i+1} = x_i + 1$ si $y_{i+1} = y_i - 1$. Deci,

$t_{i+1} = (y_i-1)^2 + 2*((x_i+1)+1)^2 + ((y_i-1)-1)^2 - 2*r^2$

sau **$t_{i+1} = t_i + 4*(x_i - y_i) + 10$**

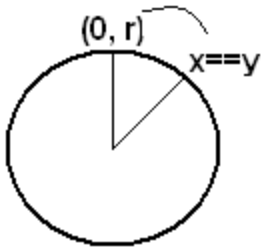


Algoritmul Bresenham (3)

- Valoarea erorii de aproximare în primul pas:

$t_i = y_i^2 + 2*(x_i+1)^2 + (y_i-1)^2 - 2*r^2$: $x_1=0$, $y_1 = r$. Rezultă, **t1 = 3-2*r**

```
void Bres_cerc(int xc, int yc, int r, RGB culoare)
{
  int x, y, t;
  t = 3 - (r << 1);
  point(xc, yc + r, culoare); point(xc, yc - r, culoare);
  point(xc + r, yc, culoare); point(xc - r, yc, culoare);
  for (x = 1, y = r; x < y; x++)
  {
    if (t < 0)
      t += 6 + (x << 2);
    else
    {
      t += 10 + ((x - y) << 2); y--;
    }
    punct_simetric(xc, yc, x, y, culoare);
  }
  // pentru x==y se afișează 4 puncte simetrice
  point(xc + x, yc + y, culoare); point(xc + x, yc - y, culoare);
  point(xc - x, yc + y, culoare); point(xc - x, yc - y, culoare);
}
```

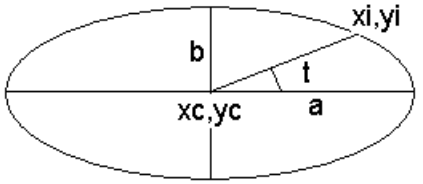


Generarea elipselor în spațiul discret

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

Aproximarea elipselor în spațiul discret (1)



- Ecuaţiile parametrice ale elipsei sunt:

  $x = xc + a*\cos(t)$

  $y = yc + b*\sin(t)$ $\quad 0 <= t <= 2*\pi$

  unde (xc,yc) este centrul elipsei, iar a și b sunt mărimile semiaxelor elipsei.

- Se poate obţine o secvenţă de puncte care aproximează circumferinţa elipsei dându-i lui t valori de la 0 la 2π cu un pas constant.

- Fie (x_i,y_i) ultimul punct calculat de pe circumferinţa elipsei:

  $x_i = xc + a*\cos(t) = xc + a*dx$ $\quad dx = \cos(t)$

  $y_i = yc + b*\sin(t) = yc + b*dy$ $\quad dy = \sin(t)$

- Următorul punct, (x_{i+1},y_{i+1}), este distanţat de cel curent cu pasul unghiular:

  $x_{i+1} = xc + a*\cos(t+pas) = xc + a*dx'$

  $y_{i+1} = yc + b*\sin(t+pas) = yc + b*dy'$

  $dx'=\cos(t+pas)=\cos(t)*\cos(pas) - \sin(t)*\sin(pas) = dx*c - dy*s$

  $dy'=\sin(t+pas)=\cos(t)*\sin(pas) + \sin(t)*\cos(pas) = dx*s + dy*c$

  unde c=cos(pas), s=sin(pas) sunt constante

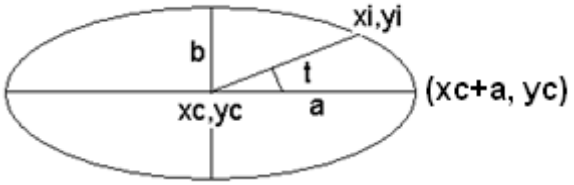
Aproximarea elipselor în spațiul discret (2)

Rezultă:

$x_{i+1} = xc+a*dx' = xc + a*(dx*c - dy*s)$

$y_{i+1} = yc+b*dy' = yc + b*(dx*s + dy*c)$

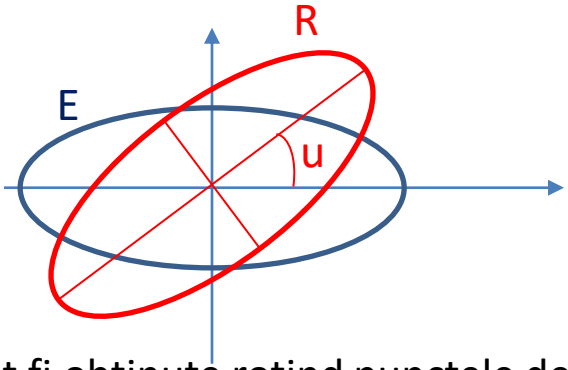
```
// Generarea unei elipse cu axele paralele cu axele sistemului de coordonate
   carteziene 2D
 void Elipsa(int xc, int yc, int a, int b, int N, RGB culoare)
 {
   // N: numarul de puncte calculate pe circumferinta elipsei
   double c, s, t, dx, dy, u, pas = 2 * M_PI / N;
   int x, y, x1, y1 ;
   c = cos(pas); s = sin(pas);
   x = xc + a; y = yc; // dx = cos(0) = 1; dy = sin(0) = 0
   for (i = 1, dx = 1, dy = 0; i < N; i++)
   {
     t = dx;
     dx = dx * c - dy * s; // dx'
     dy = t  * s + dy * c;  // dy'
     x1 = xc + (int)(a*dx + 0.5); y1 = yc + (int)(b*dy + 0.5);
     line(x, y, x1, y1, culoare);
     x = x1; y = y1;
   } line(x, y, xc + a, yc, culoare);
 }
```



Aproximarea elipselor în spațiul discret (3)

Generarea unei elipse rotite

- Fie R, o elipsă cu centrul în originea sistemului de coordonate şi semiaxele a, b rotite cu unghiul u.
- Fie E, elipsa cu centrul în origine şi semiaxele a şi b suprapuse peste axele sistemului de coordonate.



- Punctele de pe elipsa R pot fi obţinute rotind punctele de pe elipsa E în jurul originii cu unghiul u.

- Punctul curent de pe elipsa E:

$x_i = a*\cos(t) = a*dx \qquad dx = \cos(t)$

$y_i = b*\sin(t) = b*dy \qquad dy = \sin(t)$

- Următorul punct, (x_{i+1},y_{i+1}), de pe elipsa E: dx1=a*dx' , dy1=b*dy‘

$x_{i+1} = a*\cos(t+pas) = a*dx' = dx1$

$y_{i+1} = b*\sin(t+pas) = b*dy' = dy1$

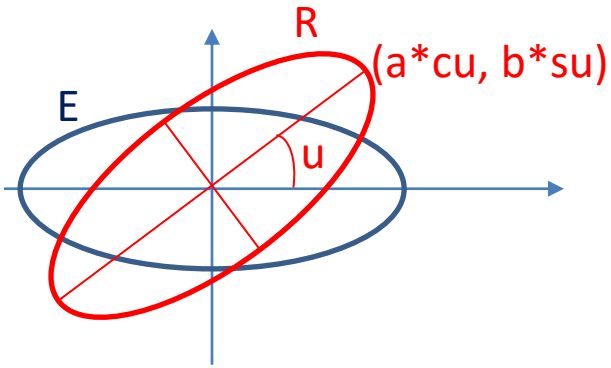
- Următorul punct, (x_{i+1},y_{i+1}), de pe elipsa R se obține prin rotația vectorului (dx1,dy1) în jurul originii :

$x_{i+1} = dxr = dx1*\cos(u) - dy1*\sin(u)$

$y_{i+1} = dyr = dx1*\sin(u) + dy1*\cos(u)$

Aproximarea elipselor în spațiul discret (4)

```
// Generarea unei elipse cu axele rotite față de axele sistemului de coordonate carteziene 2D
void Elipsa_rot(int xc, int yc, int a, int b, int N, double u, RGB culoare)
{
  double c, s, t, dx, dy, dx1, dy1, cu, su, pas = 2 * M_PI / N;
  c = cos(pas); s = sin(pas);
  cu = cos(u); su = sin(u);
  // punctul inițial de pe elipsa rotită
  x = xc + (int)(a*cu + 0.5);
  y = yc + (int)(b*su + 0.5);
  for (i = 1, dx = 1, dy = 0; i < N; i++)
  {
    t = dx;
    dx = dx * c - dy * s; // dx'
    dy = t * s + dy * c;  // dy'
    // punctul de pe elipsa nerotită cu centrul în origine
    dx1 = a * dx; dy1 = b * dy;
    x1 = xc + (int)(dx1*cu - dy1 * su + 0.5);
    y1 = yc + (int)(dx1*su + dy1 * cu + 0.5);
    line(x, y, x1, y1, culoare);
    x = x1; y = y1;
  }
  line(x, y, xc + (int)(a*cu + 0.5), yc + (int)(a*su + 0.5), culoare);
}
```



Decuparea vectorilor

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

Decuparea primitivelor grafice

Decuparea: necesară pentru obiectele parțial vizibile în imagine

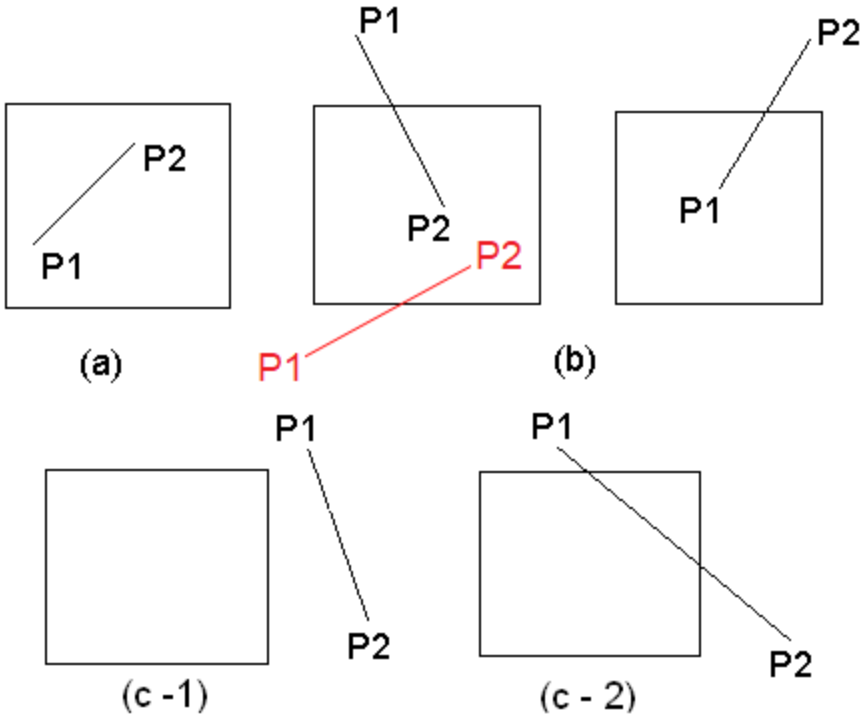
Într-un sistem grafic 2D:

- Decupare la frontiera unui dreptunghi din spatiul de afișare, având laturile paralele cu cele ale sistemului de coordonate carteziene 2D (viewport)
- Decupare la frontiera unui poligon oarecare din spațiul de afișare
- Decuparea se poate efectua:
  - Analitic, prin intersecția primitivelor grafice cu frontiera zonei de decupare (dreptunghi/poligon oarecare), înainte de rasterizare
  - La momentul rasterizarii primitivelor, prin testul de apartenență a fiecărui fragment la dreptunghiul/poligonul de decupare – de ex. pentru cercuri și elipse
- Pentru linii și poligoane, decuparea se efectuează analitic (mai eficient).

Într-un sistem grafic 3D bazat pe OpenGL:

- Decuparea se efectuează la frontiera volumului vizual canonic, în coordonate de decupare (după transformarea de proiecție, înainte de transformarea în poarta de afișare).

Decuparea vectorilor 2D (1)



Poziția unui vector față de dreptunghiul de decupare

Exemplu: intersecție P1-P2 cu dreptunghiul neţinând cont de poziţiile punctelor P1, P2 faţă de dreptunghiul de decupare:

1. Calcul intersecție cu latura din stânga
2. Calcul intersecție cu latura de sus
3. Calcul intersecție cu latura din dreapta
4. Calcul intersecție cu latura de jos

1, 2, 3 – calcule inutile

Optimizări: evitarea calculelor de intersecție inutile

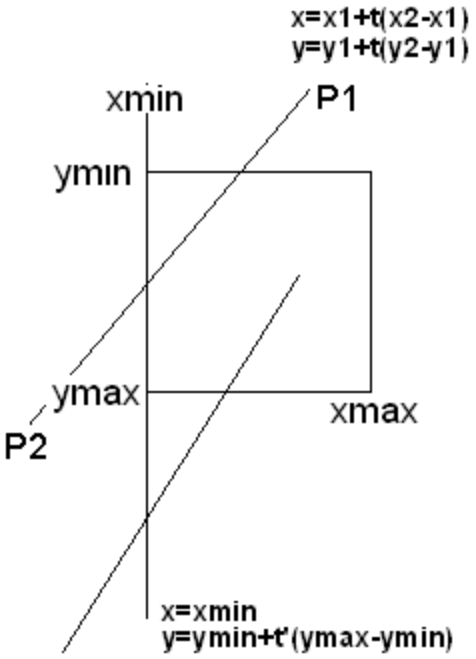
- Evitarea calculelor de intersecție in cazurile de tip (c-1) :

  - cum se diferențiază cazurile (c-1) de cazurile (c-2) fără a efectua calcule de intersecție?

- Determinarea laturii/laturilor intersectate în cazurile de tipul (b) și (c-2), fără calcule de intersecție.

Decuparea vectorilor 2D (2)

Calculul intersecțiilor cu laturile dreptunghiului de decupare



Intersecția segmentului P1-P2 cu latura x = xmin a dreptunghiului

Ec. parametrică a vectorului P1-P2:

P(t) = P1 + (P2-P1)*t 0<= t <=1 între P1-P2

Punctul de intersecție cu dreapta x=xmin are abscisa xmin:

1) xmin=x1+t(x2-x1) → t=(xmin-x1)/(x2-x1)

2) este 0<=t<=1 ???

nu: punctul de intersecție se afla pe prelungirea segmentului

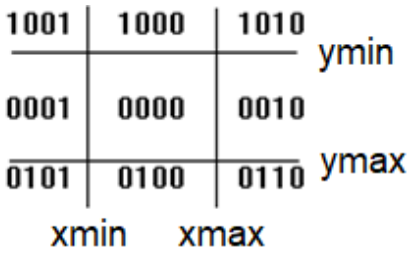
da: punctul de intersecție se află pe segmentul P1-P2

- se calculează yi=y1+(xmin-x1)*(y2-y1)/(x2-x1)
- este ymin<= yi <= ymax ??

da: există intersecție între vectorul P1-P2 și latura x=xmin

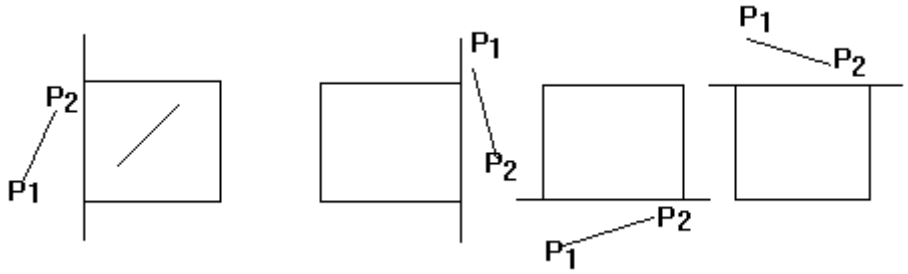
nu: punct de intersecție cu prelungirea laturii – calcul inutil

Algoritmul Cohen-Sutherland (1)



- Punctele din plan sunt codificate prin 4 biți, b3b2b1b0, în funcție de poziția lor față de dreptunghiul de decupare. De exemplu, se face convenția:
- b0=1: pentru punctele din stânga dreptunghiului,
- b1=1: dreapta, b2=1: sub dreptunghi, b3=1: deasupra dreptunghiului

Fie **cod1** codul binar al punctului P1 și **cod2** codul binar al punctului P2.



daca (cod1 == 0 && cod2 == 0) segment este acceptat trivial;

altfel

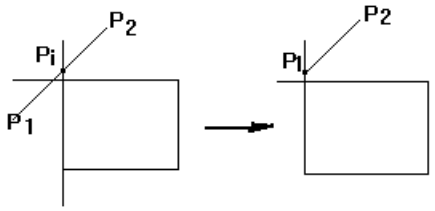
daca ((cod1 & cod2) != 0) segmentul este rejectat trivial
(cele 2 coduri au același bit =1)
P1 si P2 de aceeași parte a dreptunghiului
→ segmentul nu intersectează dreptunghiul

altfel: se intersectează segmentul P1-P2 cu dreptunghiul de decupare

Algoritmul Cohen-Sutherland (2)

- Se alege latura de intersecție cu segmentul P1-P2 pe baza codului vărfului P1, care trebuie să fie situat în afara dreptunghiului de decupare.
- Daca cod1 == 0 → se interschimbă P1 cu P2

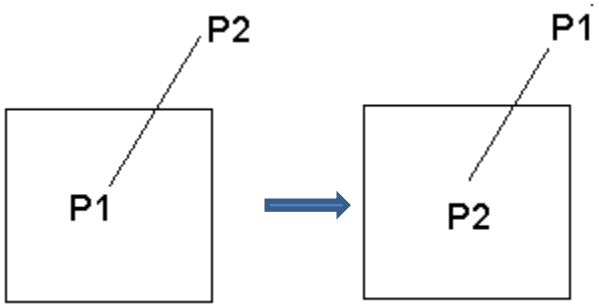
cod1 & 1 != 0
P1 în stânga dreptunghiului



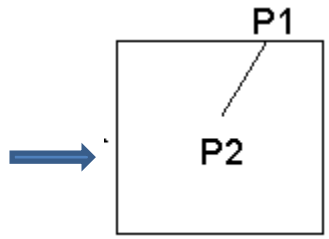
- Se calculează intersectia cu x=xmin
- Se muta P1 în punctul de intersecție
- În iterația urmatoare cod1&cod2 != 0

  → segmentul este rejectat

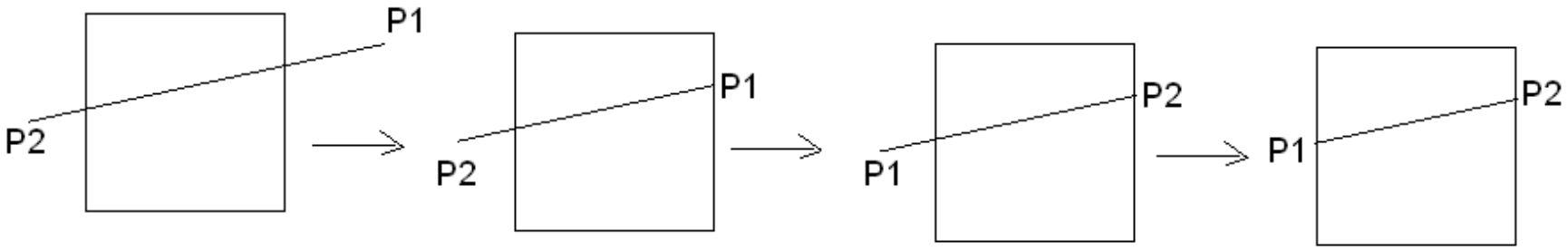
cod1 = 0



cod1 & 8 != 0
P1 deasupra dreptunghiului



Algoritmul Cohen-Sutherland (3)



Calculul punctelor de intersecție

$P(t) = P1 + (P2-P1)*t \rightarrow x(t) = x1 + t(x2-x1);\ y(t) = y1 + t(y2-y1)$

Intersecția cu latura situată pe dreapta y = ymax

ymax=y1+t(y2-y1) de unde, t=(ymax-y1)/(y2-y1)

x=x1+(ymax-y1)*(x2-x1)/(y2-y1) → **xi=x1+(ymax-y1)/m** m: panta segmentului P1-P2

Intersecția cu latura situată pe dreapta x = xmin

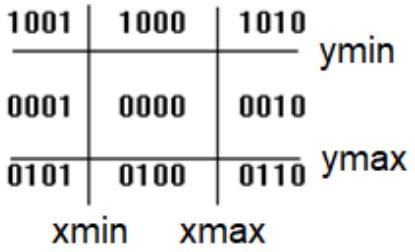
xmin=x1+t(x2-x1), deci t=(xmin-x1)/(x2-x1)

y=y1+(xmin-x1)*(y2-y1)/(x2-x1) → **yi=y1+(xmin-x1)*m**

Algoritmul Cohen-Sutherland (4)

```
// colțurile dreptunghiului de decupare
int xmin, ymin, xmax, ymax;

// functia codif calculează codul binar al unui punct din plan,
// pe baza poziției sale față de dreptunghiul de decupare
int codif(int x, int y)
{
   int cod = 0;
   if (x < xmin)  cod = 1;
   if (x > xmax)  cod = 2;
   if (y > ymax)  cod |= 4;  // b2=1
   if (y < ymin)  cod |= 8;  // b3=1
   return  cod;
}
```



Algoritmul Cohen-Sutherland (5)

```
int Cohen_Suth(int x1i, int y1i, int x2i, int y2i, int* x1d, int* y1d, int*
x2d, int* y2d)
{
  // primește coord. capetelor vectorului de decupat și întoarce coord.
  // părții din vector inclusă în dreptunghi, dacă există
  int acceptat, rejectat, vertical;
  float m, x1 = x1i, y1 = y1i, x2 = x2i, y2 = y2i;
  int cod1, cod2, t;
  acceptat = rejectat = vertical = 0;
  if (x1i != x2i)  // vectorul nu este vertical
    m = (y2 - y1) / (x2 - x1);
  else
    vertical = 1;
  do
  {
    cod1 = codif(x1, y1);  cod2 = codif(x2, y2);
    if (cod1 == 0 && cod2 == 0) { acceptat = 1; break; }
    if ((cod1 & cod2) != 0) { rejectat = 1; break; }
```

Algoritmul Cohen-Sutherland (6)

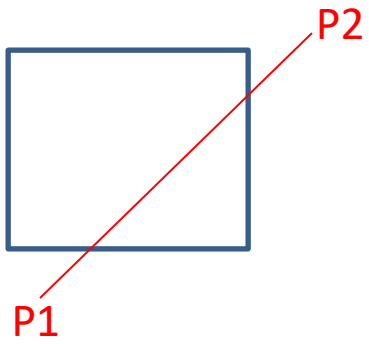
```
  if (cod1 == 0) // P1 este în interiorul dreptunghiului de decupare
  {
    // interschimbare capete
    t = x1; x1 = x2; x2 = t; t = y1; y1 = y2; y2 = t; cod1 = cod2;
  }
  if (cod1 & 1)  // P1 în stânga dreptunghiului
  {
    // punctul de intersecție devine P1 pentru iterația următoare
    y1 += (xmin - x1)*m;  x1 = xmin;
  }
  else
  if (cod1 & 2) // P1 în dreapta dreptunghiului
  {
    y1 += (xmax - x1)*m;  x1 = xmax;
  }
  else
  if (cod1 & 4) // P1 sub dreptunghi
  {
    if (!vertical) x1 += (ymax - y1) / m;
    y1 = ymax;
  }
  else // P1 deasupra dreptunghiului
  {
    if (!vertical) x1 += (ymin - y1) / m;
    y1 = ymin;
  }
}   while (acceptat == rejectat); // cât timp vectorul nu este acceptat sau rejectat
```

Algoritmul Cohen-Sutherland (7)

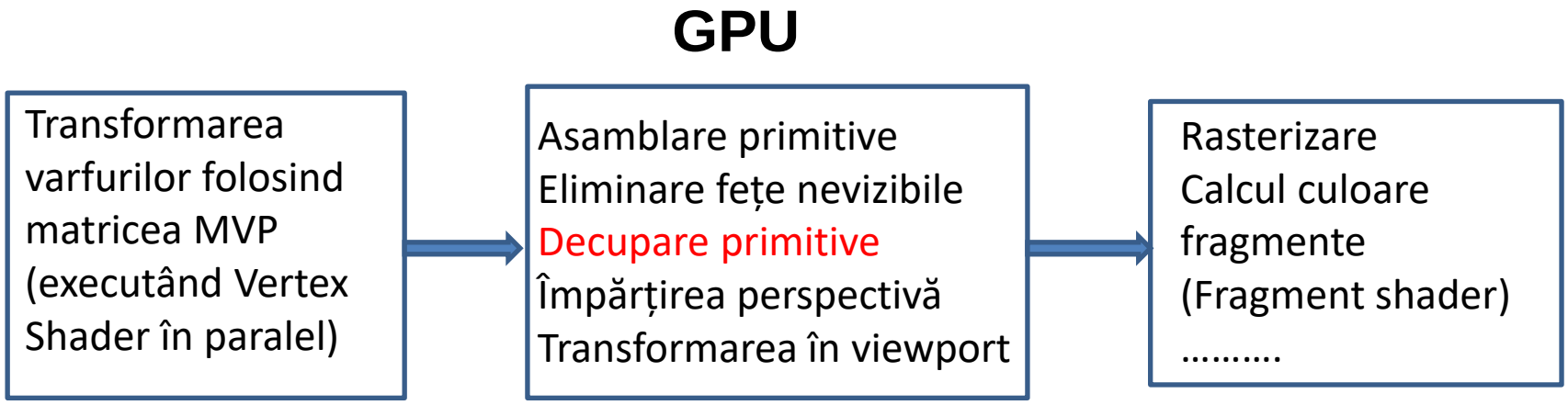
```
  if (acceptat)
  {
    *x1d = x1; *y1d = y1; *x2d = x2; *y2d = y2;
    return 1;
  }
  return 0;
}
```

Exercițiu:

Precizați care sunt pașii în decuparea vectorului din figură, executând algoritmul Cohen-Sutherland.



Decuparea în banda grafică (1)



- Decuparea se efectuează asupra primitivelor 3D, la frontiera volumului vizual canonic
- Din decupare pot rezulta noi vârfuri
- Vărfurile primitivelor pot avea asociate atribute: culoare, normala, coordonate textură
- Pentru vârfurile rezultate din decupare se calculează atribute prin interpolarea atributelor asociate vârfurilor, folosind valoarea variabilei parametrice (t) în punctul de decupare.

Decuparea în banda grafică (2)

Transformarea de proiecție:

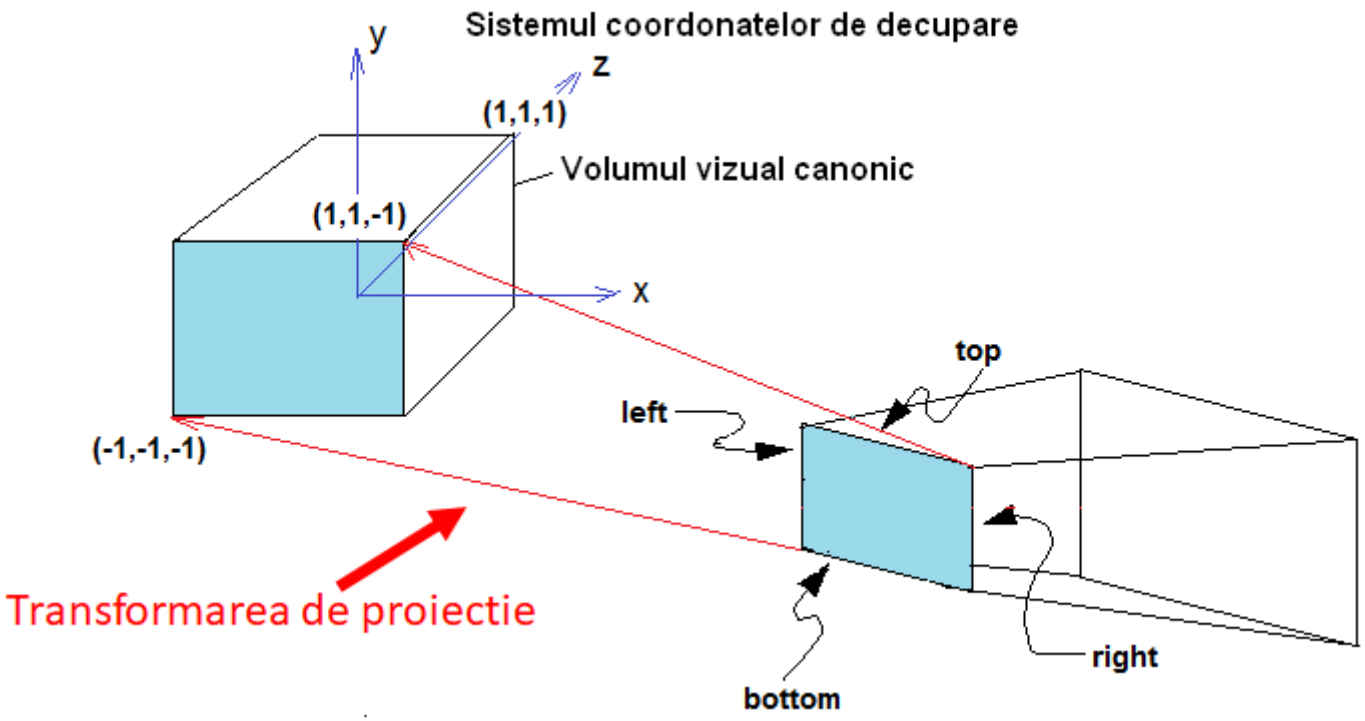
$[xc\ yc\ zc\ w]^T = P \bullet [xe\ ye\ ze\ we]^T$

(clip coordinates) ← (eye coordinates)

Coordonate omogene → împărțirea perspectivă → coordonate carteziene:

xd = xc/w , yd = yc/w , zd= zc/w, w = -ze

(xd, yd, zd): coordonate dispozitiv normalizate (**N**ormalized **D**evice **C**oordinates)



Decuparea în banda grafică (3)

Decuparea: înainte sau dupa împarţirea perspectivă?

1. Înainte: decupare în coordonate omogene
2. După: decupare în coordonate carteziene (NDC)

- Problema: împărțirea perspectivă

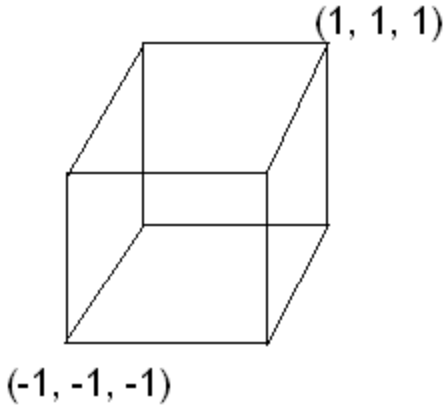
  xd = xc/w , yd = yc/w , zd= zc/w

  w=- ze = 0 pentru proiecția perspectivă daca punctul transformat se află în același plan cu camera (centrul de proiecție): ze = 0

Problema nu apare în cazul 1, deoarece vârfurile / primitivele din planul camerei sunt eliminate la decupare, ele nefiind incluse în volumul vizual (near>0).

În cazul 2, la conversia în NDC trebuie tratat cazul special w=0.

Generalizarea algoritmului Cohen-Sutherland pentru decuparea vectorilor 3D (1) (decuparea în coordonate carteziene)



Față de volumul vizual canonic, un punct din spațiu se poate afla: în interior, în stânga, în dreapta, sub volum, deasupra volumului, în față, în spate:

→pentru codificarea poziției unui punct din spațiu față de volumul vizual canonic sunt necesari 6 biți.

De ex. se poate face convenția:

b0 = 1 pentru puncte (x,y,z) cu x < -1

b1 = 1 pentru puncte (x,y,z) cu x > 1

b2 = 1 pentru puncte (x,y,z) cu y < -1

b3 = 1 pentru puncte (x,y,z) cu y > 1

b4 = 1 pentru puncte (x,y,z) cu z < -1

b5 = 1 pentru puncte (x,y,z) cu z > 1

dacă: cod(P1) ==0 && cod(P2)==0
    segment acceptat trivial

dacă (cod(P1) & cod(P2)) != 0
    segment rejectat trivial
altfel
    se intersectează segmentul cu volumul

Generalizarea algoritmului Cohen-Sutherland pentru decuparea vectorilor 3D (2)

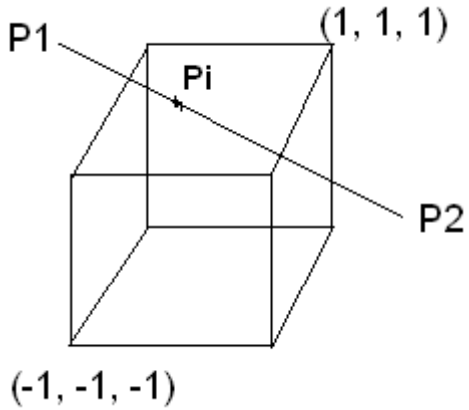
Intersecțiile segmentului cu volumul vizual

Ec. parametrice ale segmentului

$x = x1 + t(x2 – x1)$

$y = y1 + t(y2 –y1)$

$z = z1 + t(z2 – z1)$



dacă (cod(P1) & 8 !=0) // P1 deasupra volumului

Se efectuează intersecția cu planul y = 1:

$1 = y1 + t(y2 – y1) \rightarrow tinters = (1-y1)/(y2-y1)$

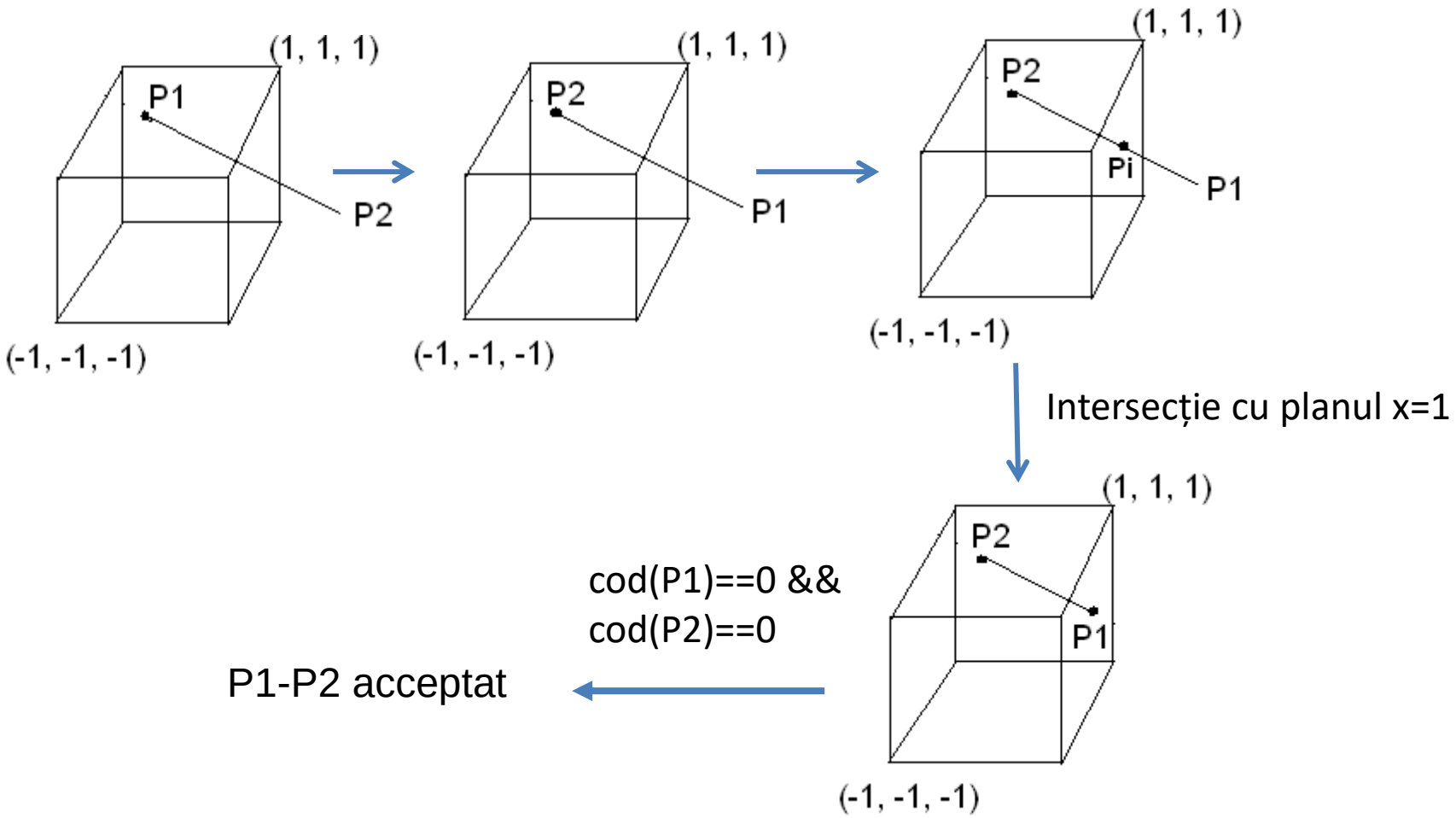
Calculează punctul de intersectie și mută P1 în punctul de intersecție:

$x1 = x1 + tinters(x2 – x1)$

$y1 = 1;$

$z1 = z1 + tinters(z2 – z1)$

Generalizarea algoritmului Cohen-Sutherland pentru decuparea vectorilor 3D (3)

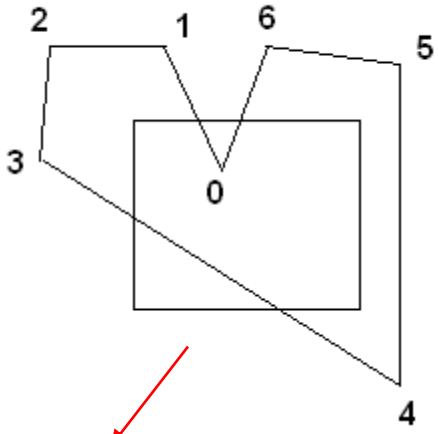


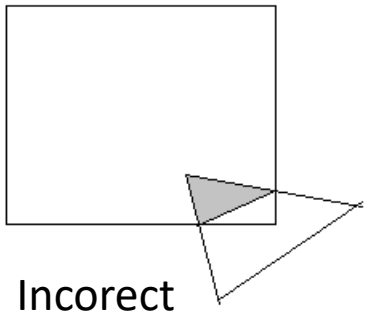
Decuparea poligoanelor

Prof.dr.ing Florica Moldoveanu
Sl.dr.ing. Victor Asavei
(victor.asavei@cs.pub.ro)

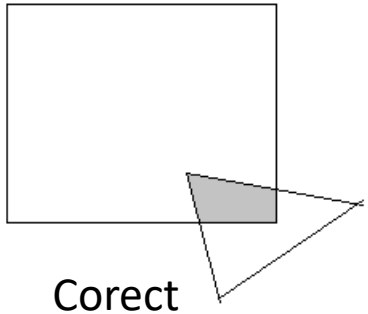
Curs Elemente de Grafică pe Calculator – UPB, Automatică şi Calculatoare
2021-2022

Decuparea poligoanelor 2D

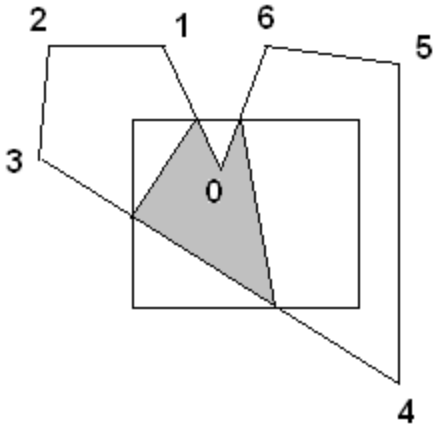


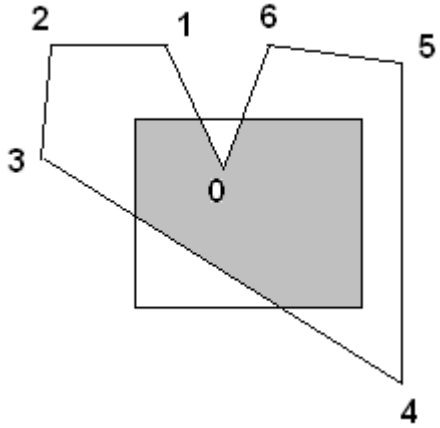


Incorect



Corect



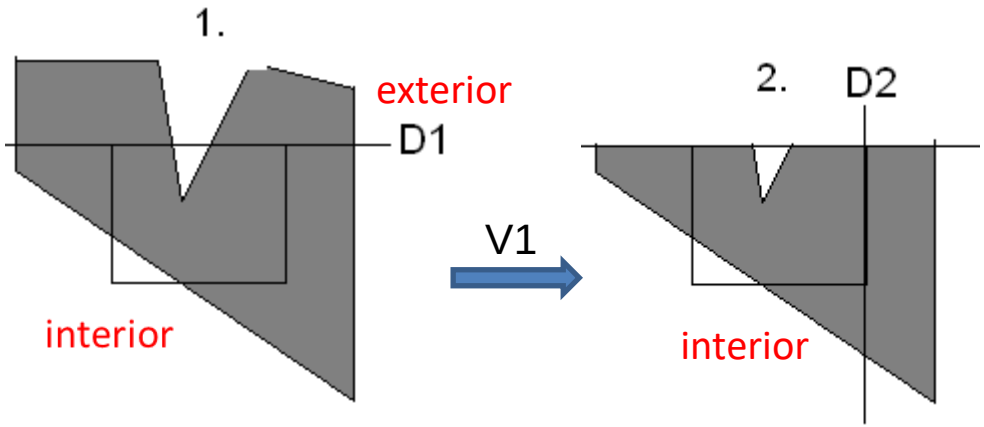


Rezultatul obținut dacă se decupează pe rând laturile poligonului folosind algoritmul de decupare vectori.

Rezultatul corect.

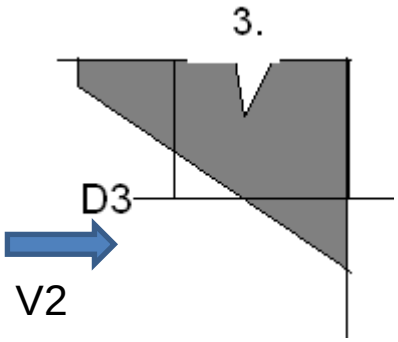
Algoritmul Sutherland-Hodgman (1)

Decuparea poligoanelor convexe față de o zona dreptunghiulară

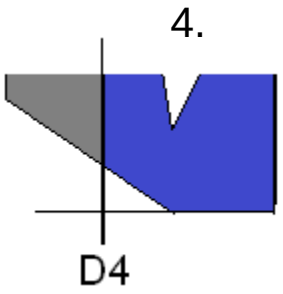


2. Se decupează poligonul cu vârfurile din lista V1 față de dreapta D2
   - Se crează lista vârfurilor poligonului rezultat ,V2

1. Se decupează laturile poligonului față de dreapta pe care se afla o latură a zonei de decupare, D1
   - Se crează lista vârfurilor poligonului rezultat (partea din poligonul inițial cuprinsă în semiplanul interior), V1



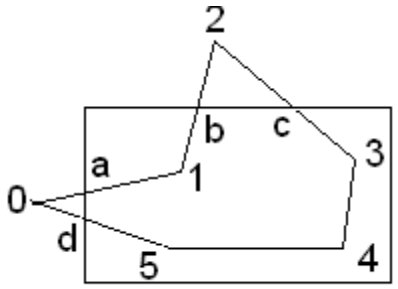
3. Se decupează poligonul cu varfurile din lista V2 față de dreapta D3
   - Se crează lista vârfurilor poligonului rezultat, V3



4. Se decupează poligonul cu vârfurile din lista V3 față de dreapta D4
   - Se crează lista varfurilor poligonului rezultat ,V4

Algoritmul Sutherland-Hodgman (2)

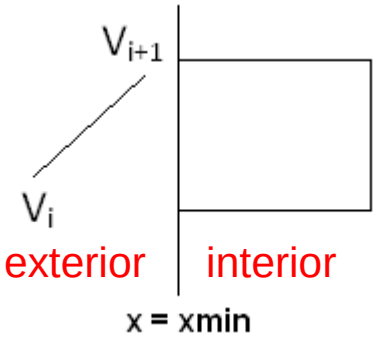
Calculul vârfurilor poligonului rezultat într-o etapă a decupării



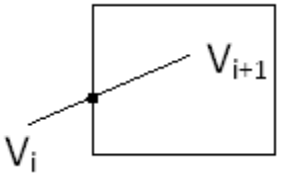
0-1: a, 1 --> a, 1
1-2: 1,b --> a, 1, 1, b
2-3: c,3 --> a, 1, 1, b, c, 3
3-4: 3,4 --> a,1, 1, b, c, 3, 3, 4
4-5: 4,5 --> a,1, 1, b, c, 3, 3, 4, 4, 5
5-0: 5,d --> a,1, 1, b, c, 3, 3, 4, 4, 5, 5, d

Lista corecta: a, 1, b, c, 3, 4, 5, d

Convenție:
- Fiecare latură a poligonului se considera un interval: [V_i -> V_{i+1})
- Numai vârful V_i aparține laturii

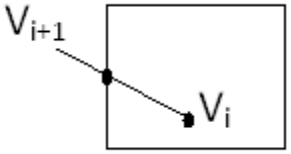


0 vărfuri



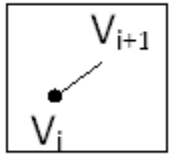
Latura intră în semiplanul interior

un vârf



Latura iese din semiplanul interior

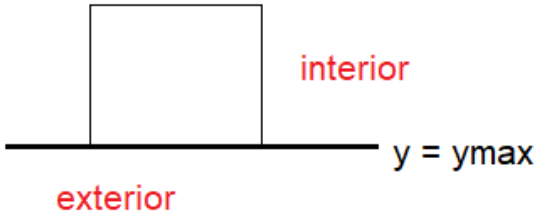
2 vârfuri



un vârf

Algoritmul Sutherland-Hodgman (3)

```
// decupează latura v1->v2, față de dreapta pe care se află o latura a zonei de
// decupare
// funcția întoarce numărul de vârfuri rezultate din decupare
int DecupLatura(P2D v1, P2D v2, P2d * v1d, P2D *v2d, int latura)
{
 switch (latura)
 {
   case 1: // dreapta y = ymax
     // latura în semiplanul exterior al zonei de decupare
     if (v1.y > ymax && v2.y > ymax) return 0;
     // latura în semiplanul interior
     if (v1.y <= ymax && v2.y <= ymax) { *v1d = v1; return 1; }
     // latura iese din semiplanul interior
     if (v1.y <= ymax && v2.y > ymax)
     {
       *v1d = v1; x_intersectie = …; v2d->x = x_intersectie; v2d->y = ymax;
       return 2;
     }
     // v1.y > ymax && v2.y <= ymax  deci latura intră în semiplanul interior
     else
     {
        x_intersectie = ….; v1d->x = x_intersectie; v1d->y = ymax; return 1;
     }
   case 2:  …….
 }
}
```



Algoritmul Sutherland-Hodgman (4)

```
// decupează toate laturile poligonului față de dreapta pe care se afla o
// latura a zonei de decupare
// vrf : lista vârfurilor poligonului de decupat
// nv  : numărul de vârfuri ale poligonului de decupat
// vrfd: lista vârfurilor poligonului rezultat din decupare
// nd  : numărul de vârfuri ale poligonului rezultat din decupare
void DecupLaturi(int nv, P2D *vrf, int *nd, P2D *vrfd, int latura)
{
  *nd = 0;
  for (int i = 0; i < nv; i++)
     *nd += DecupLatura(vrf[i], vrf[i + 1], &vrfd[*nd], &vrfd[*nd + 1], latura);

  *nd += DecupLatura(vrf[nv - 1], vrf[0], &vrfd[*nd], &vrfd[*nd + 1], latura);
}
```

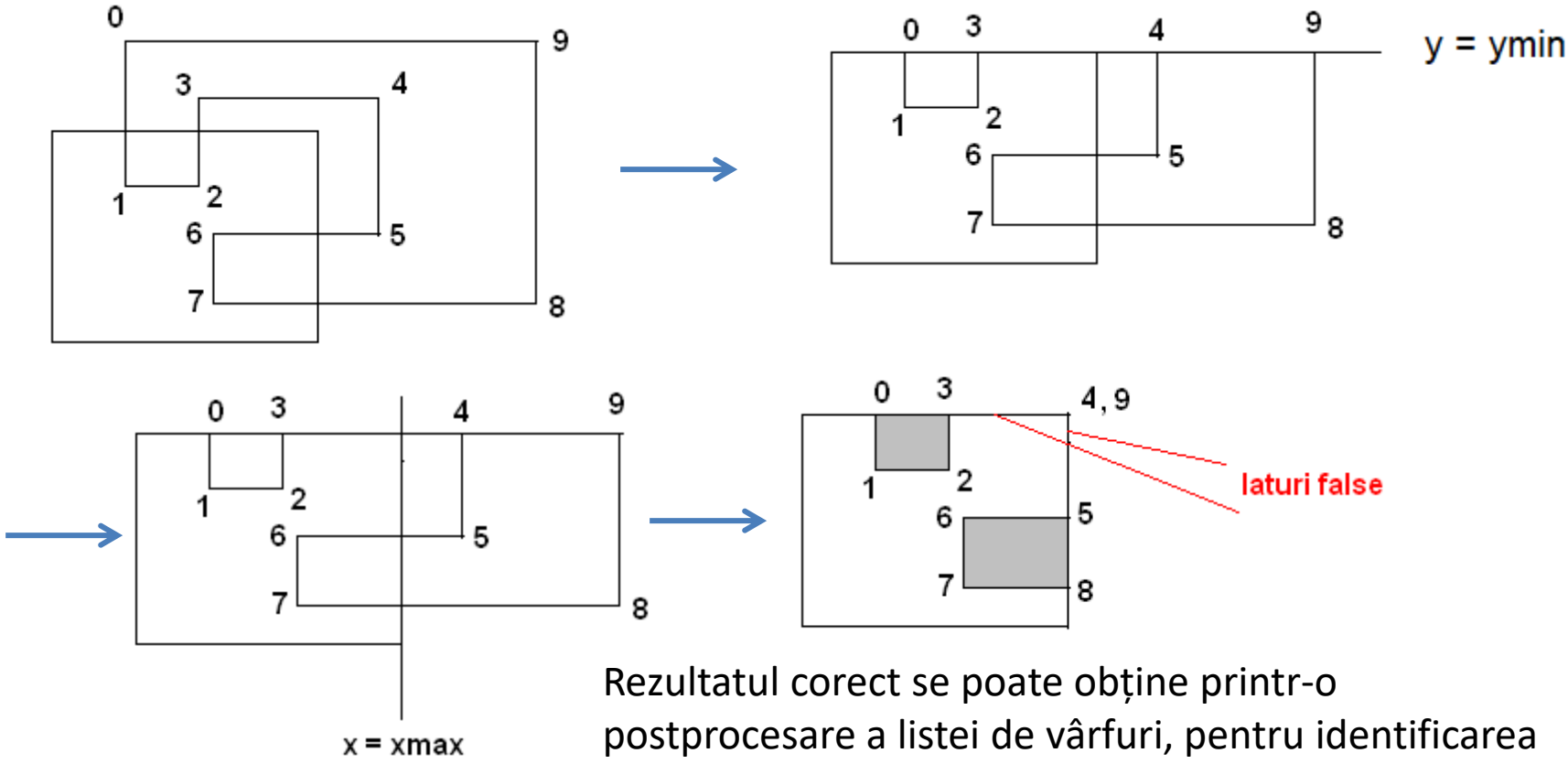
Algoritmul Sutherland-Hodgman (5)

```
// vrf : lista vârfurilor poligonului de decupat
// vrfd: lista vârfurilor poligonului rezultat din decupare
void DecupPoligon(int nv, P2D * vrf, int *nd, P2D **vrfd)
{
  // se folosesc 2 liste de vârfuri
  P2D * v1 = new P2D[2 * nv]; P2D * v2 = new P2D[2 * nv];
  memcpy(v1, vrf, nv * sizeof(P2D));
  // zona de decupare un dreptunghi
  for (int latura = 1; latura <= 4; latura + = 2)
  {
    DecupLaturi(nv, v1, nd, v2, latura);
    DecupLaturi(*nd, v2, &nv, v1, latura + 1);
  }
  *vrfd = new P2D[(*nd)];
  memcpy(*vrfd, v1, (*nd) * sizeof(P2D));
  delete v1; delete v2;
}
```

În cazul unei zone de decupare poligon oarecare, decuparea se efectuează faţă de fiecare latura a poligonului.

Algoritmul Sutherland-Hodgman (6)

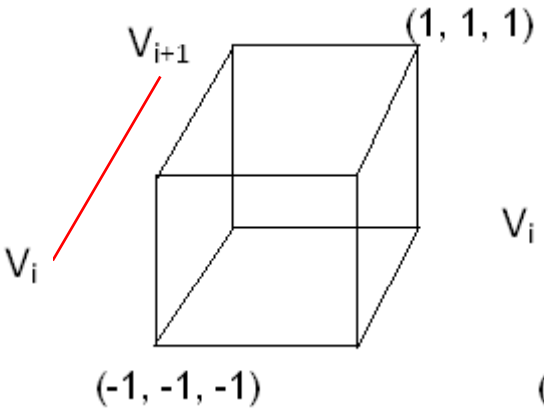
Limitarea algoritmului Sutherland-Hodgman: la decuparea poligoanelor concave, atunci când din decupare pot rezulta mai multe poligoane. Algoritmul produce o singură lista de vârfuri: un singur poligon, cu posibile laturi false.



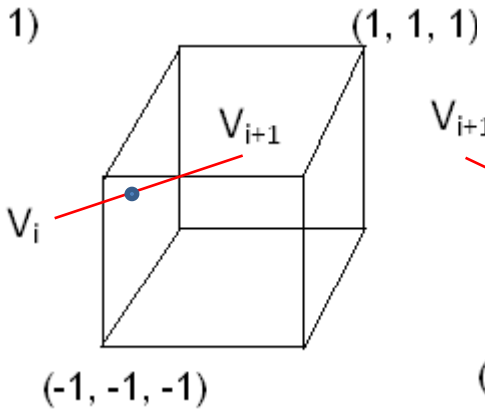
Rezultatul corect se poate obține printr-o postprocesare a listei de vârfuri, pentru identificarea ciclurilor de vârfuri.

Decuparea poligoanelor 3D prin algoritmul Sutherland-Hodgman (1)

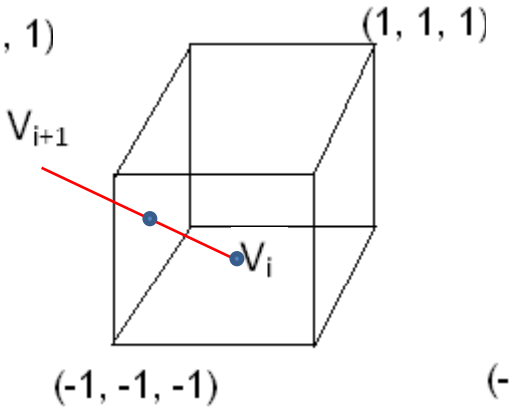
- Decuparea unui poligon 3D se efectuează la frontiera volumului canonic.
- Se intersectează poligonul, pe rând, **cu cele 6 planuri** care delimitează volumul.
- Fiecare plan de intersecție împarte spațiul în 2 semispații:
  - semispațiul interior – care conține volumul
  - semispațiul exterior
- Din fiecare intersecție pot rezulta noi vârfuri. Exemplu: intersecția cu planul x=-1



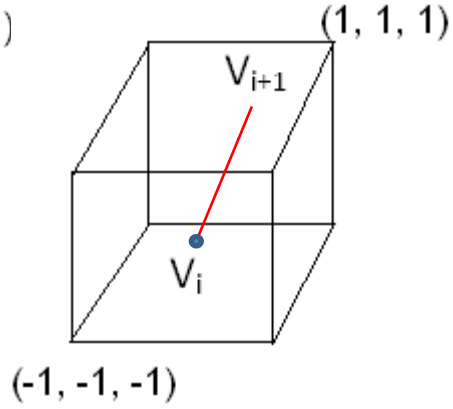
0 vârfuri



1 vârf: intersecția vectorului cu planul x=-1



2 vârfuri: V_i și intersecția vectorului cu planul x=-1



1 vârf: vârful V_i

Decuparea poligoanelor 3D prin algoritmul Sutherland-Hodgman (2)

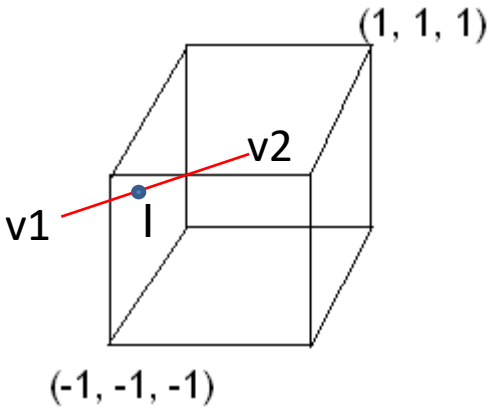
Intersecția unei laturi V1-V2 cu planul x=-1

Ec. parametrice ale laturii

$x = x1 + t(x2 - x1)$

$y = y1 + t(y2 - y1)$

$z = z1 + t(z2 - z1)$



Intersecția cu planul x=-1

$-1 = x1 + t(x2-x1)$ → $tinters = -(x1+1)/(x2-x1)$

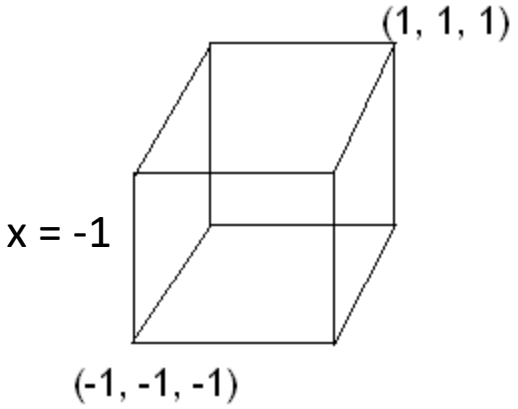
$xi = -1$

$yi = y1 + tinters * (y2-y1)$

$zi = z1 + tinters * (z2-z1)$

Decuparea poligoanelor 3D prin algoritmul Sutherland-Hodgman (3)

```
// decupează latura v1->v2 față de planul în care se află o față a volumului de decupare
// funcția întoarce numărul de vârfuri rezultate din decupare
int DecupLatura(P3D v1, P3D v2, P3d * v1d, P3D *v2d, int plan)
{
  float tint;
  switch (plan)
  {
   case 1: // planul x = -1
   // latura în semispațiul exterior
   if (v1.x < -1 && v2.x < -1) return 0;
   // latura în semispațiul interior
   if (v1.x >= -1 && v2.x >= -1) { *v1d = v1; return 1 ; }
   // latura intră în semispațiul interior
   if (v1.x < -1 && v2.x >= -1)
   {
     tint = -(v1.x + 1) / (v2.x - v1.x);  v1d->x = -1;
     v1d->y = v1.y + tint*(v2.y - v1.y);  v1d->z = v1.z + tint * (v2.z - v1.z);
     return 1;
   }
   else // v1.x >= -1 && v2.x  < -1 deci latura iese din semispatiul interior
   {
     *v1d = v1; tint = -(v1.x + 1) / (v2.x - v1.x);  v2d->x = -1;
     v2d->y = v1.y + tint * (v2.y - v1.y); v2d->z = v1.z + tint * (v2.z - v1.z);
     return 2;
   }
   case 2: ……..
  ……..
  }
}
```



Decuparea poligoanelor 3D prin algoritmul Sutherland-Hodgman (4)

```
// decupează toate laturile poligonului față de planul în care se află
// o față a volumului de decupare
// vrf : lista vârfurilor poligonului de decupat
// nv  : numărul de vârfuri ale poligonului de decupat
// vrfd: lista vârfurilor poligonului rezultat din decupare
// nd  : numărul de vârfuri ale poligonului rezultat din decupare
void DecupLaturi(int nv, P3D *vrf, int *nd, P3D *vrfd, int plan)
{
  *nd = 0;
  for (int i = 0; i < nv; i++)
    *nd += DecupLatura(vrf[i], vrf[i + 1], &vrfd[*nd], &vrfd[*nd + 1], plan);

  *nd += DecupLatura(vrf[nv - 1], vrf[0], &vrfd[*nd], &vrfd[*nd + 1], plan);
}
```

Decuparea poligoanelor 3D prin algoritmul Sutherland-Hodgman (5)

```
// vrf : lista vârfurilor poligonului de decupat
// vrfd: lista vârfurilor poligonului rezultat din decupare
void DecupPoligon(int nv, P3D * vrf, int *nd, P3D **vrfd)
{
  // se folosesc 2 liste de varfuri
  P3D * v1 = new P3D[2 * nv]; P3D * v2 = new P3D[2 * nv];
  memcpy(v1, vrf, nv * sizeof(P3D));
  // se decupeaza laturile poligonului față de cele 6
  // planuri în care se află fețele volumului de decupare
  for (int plan = 1; plan <= 6; plan + = 2)
  {
    DecupLaturi(nv, v1, nd, v2, plan);
    DecupLaturi(*nd, v2, &nv, v1, plan + 1);
  }
  *vrfd = new P3D[(*nd)];
  memcpy(*vrfd, v1, (*nd) * sizeof(P3D));
  delete v1; delete v2;
}
```